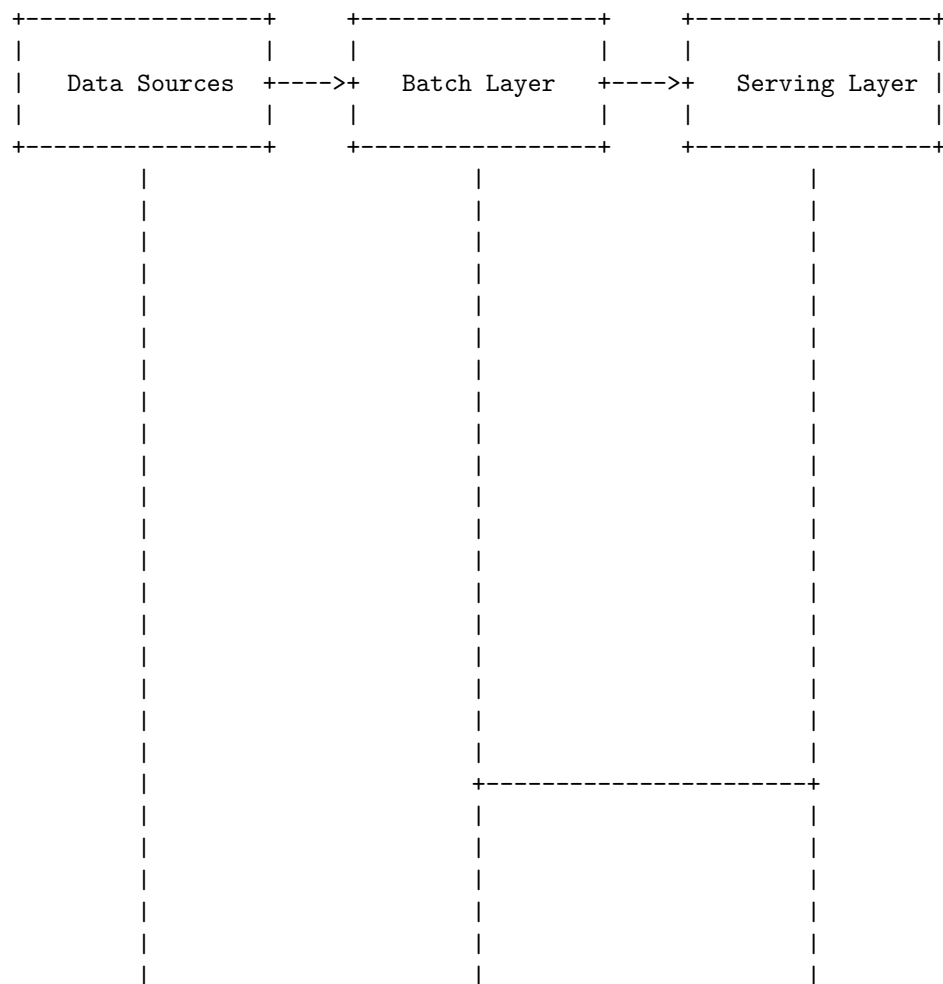


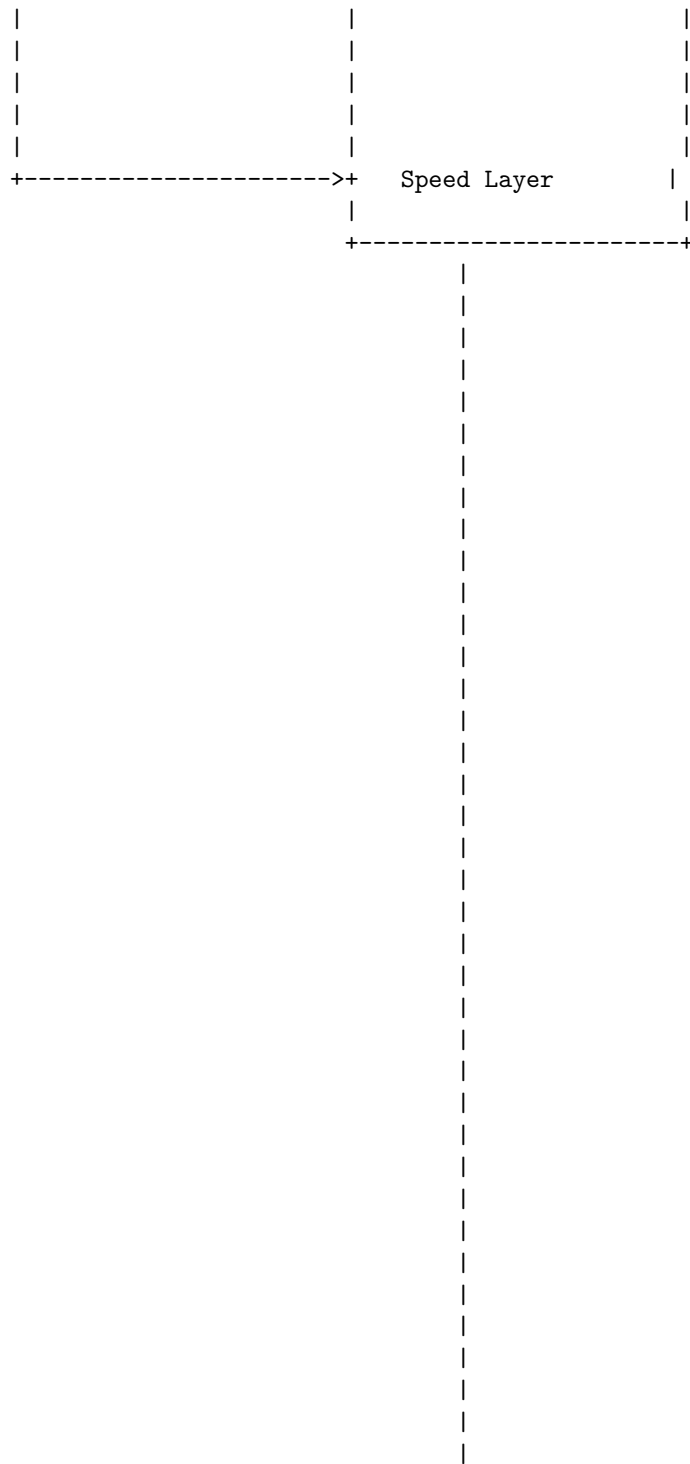
Big data is the term used to describe the large and complex datasets that are generated from various sources and require special techniques and technologies to store, process, and analyze. Big data architecture is the design of the system that handles the big data lifecycle, from ingestion to analysis.

There are different types of big data architectures, depending on the business logic and the requirements of the system. One of the common big data architectures is the lambda architecture, which combines batch and stream processing to handle both historical and real-time data.

The following is a possible ASCII diagram of the lambda architecture, based on the information from the web search results :

Big Data











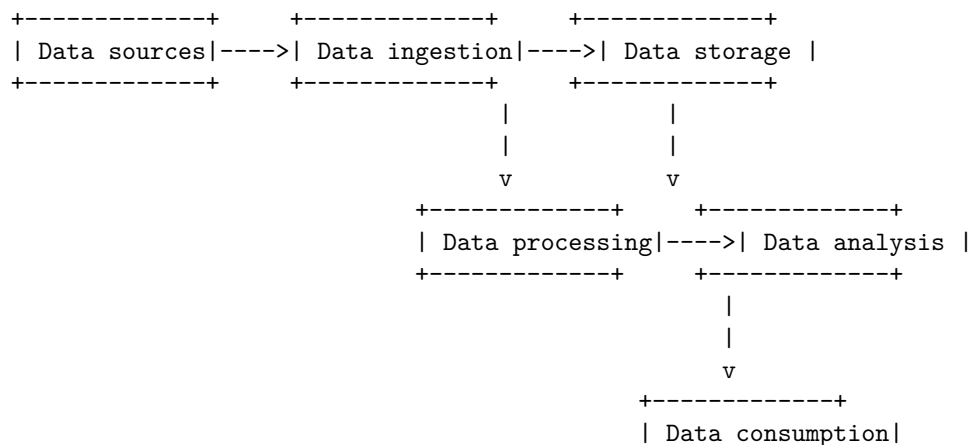
Unit 1 - Introduction to Big Data

Big data is the term used to describe the large and complex datasets that are generated from various sources and require special techniques and technologies to store, process, and analyze. Big data can be characterized by the 5 Vs: volume, velocity, variety, veracity, and value.

A big data architecture is the design of a system that can handle the ingestion, processing, and analysis of big data. A big data architecture typically consists of the following components:

- Data sources: These are the originators of the data, such as sensors, applications, web servers, social media, etc.
- Data ingestion: This is the process of capturing, validating, and transferring the data from the sources to a data store or a processing engine. Data ingestion can be done in batch mode (periodically) or in real-time (streaming).
- Data storage: This is the component that stores the raw or processed data in a scalable and reliable way. Data storage can be done using various technologies, such as relational databases, NoSQL databases, data warehouses, data lakes, etc.
- Data processing: This is the component that transforms, enriches, and analyzes the data using various techniques, such as ETL (extract, transform, load), ELT (extract, load, transform), MapReduce, Spark, etc.
- Data analysis: This is the component that provides insights and answers from the data using various tools, such as BI (business intelligence), ML (machine learning), AI (artificial intelligence), etc.
- Data consumption: This is the component that delivers the data or the insights to the end users or applications, such as dashboards, reports, APIs, etc.

The following diagram illustrates the basic architecture of a big data system using ASCII art:



+-----+

Big data is a term that refers to the large and complex datasets that are generated from various sources and require special techniques and tools to store, process, and analyze. Big data can be classified into three main types based on the format and structure of the data: structured, unstructured, and semi-structured.

Structured data is data that has a predefined schema and can be easily stored and queried in a relational database management system (RDBMS). Examples of structured data are transaction records, customer information, sensor readings, etc.

Unstructured data is data that has no fixed schema and cannot be easily organized or queried in a RDBMS. Examples of unstructured data are text documents, images, videos, audio files, social media posts, etc.

Semi-structured data is data that has some elements of structure, but not enough to fit into a RDBMS. Examples of semi-structured data are XML files, JSON files, log files, etc.

The following diagram illustrates the types of digital data in big data using ASCII characters:

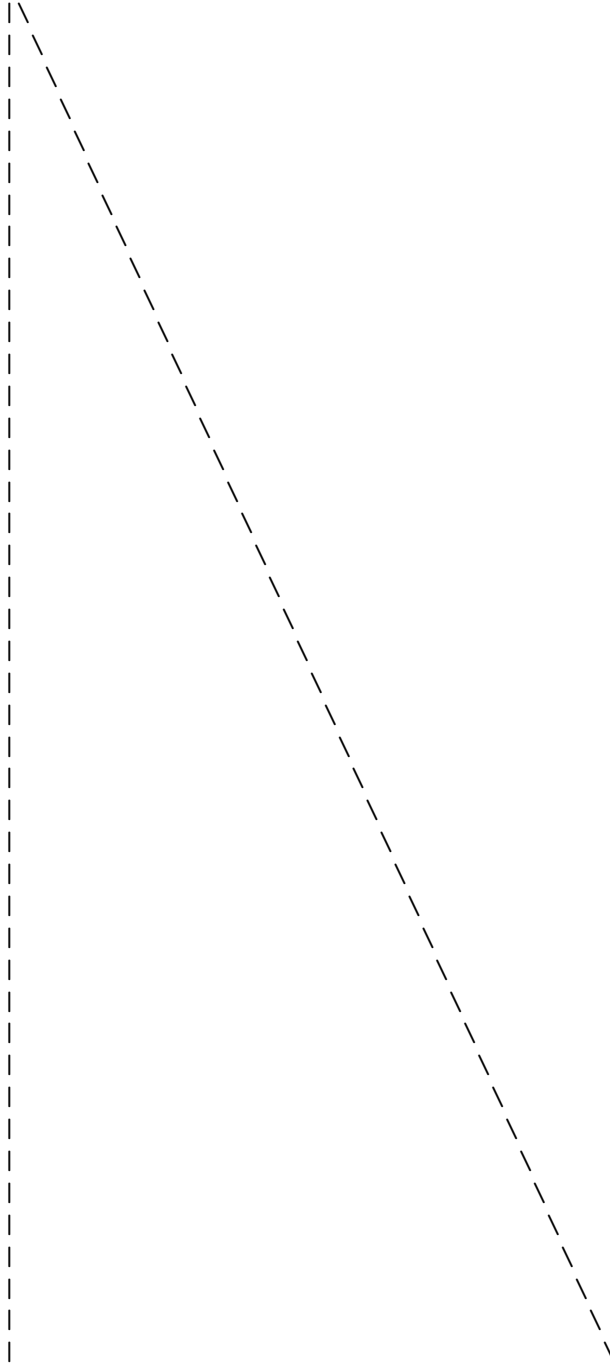
Structured	Unstructured	Semi-structured
- Transactions	- Text	- XML
- Customers	- Images	- JSON
- Sensors	- Videos	- Logs
- ...	- Audio	- ...
	- Social media	
	- ...	

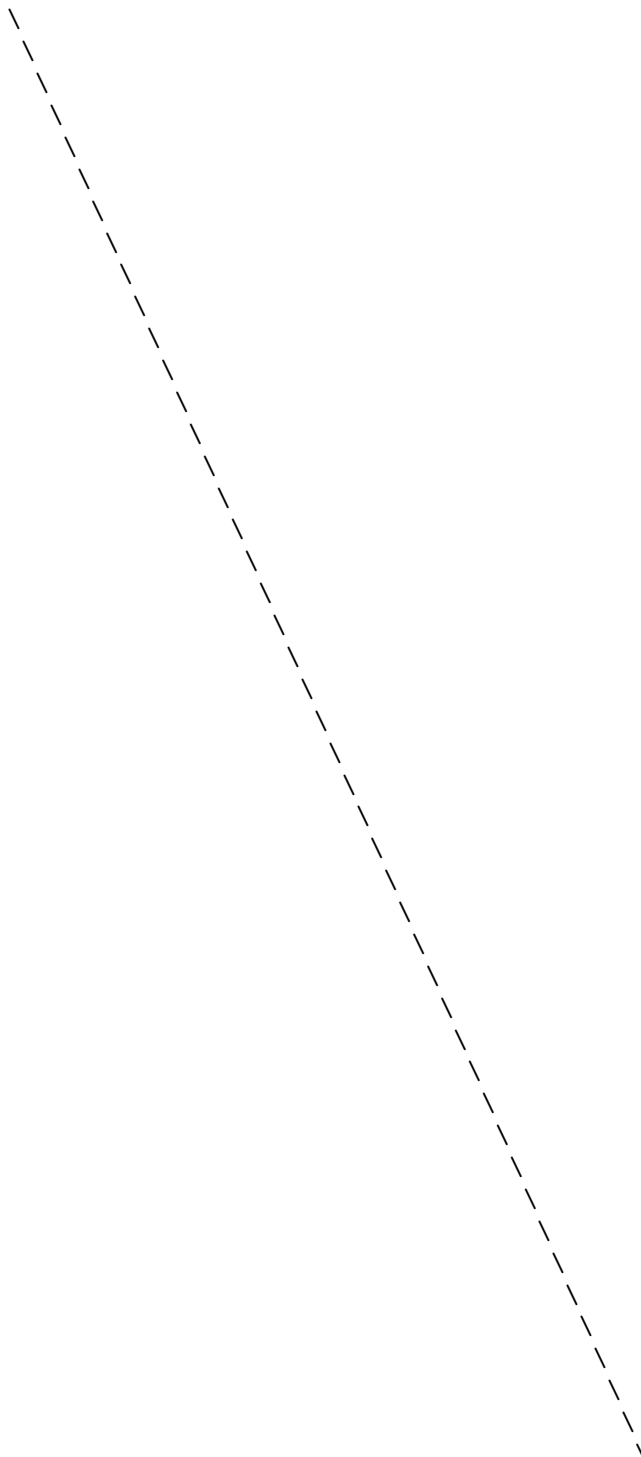
The history of Big Data innovation can be traced back to the 1960s and 1970s when the first data centers and relational databases were developed. The term Big Data itself was coined in 2005 by Roger Mougals from O'Reilly Media to describe the large and complex data sets that were difficult to process using traditional tools. Since then, Big Data has evolved with the emergence of new technologies and applications, such as cloud computing, Hadoop, MapReduce, NoSQL, machine learning, and data analytics.

The following diagram illustrates the history of Big Data innovation using ASCII art:

History of Big Data innovation

1960s-1970s: Data centers and relational databases





Introduction to Big Data Platform

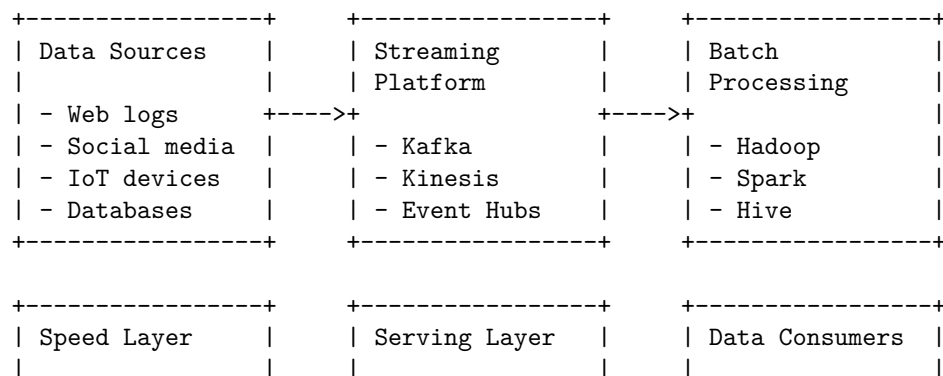
A big data platform is a system that supports the ingestion, processing, and analysis of data that is too large or complex for traditional database systems. A big data platform typically involves one or more of the following types of workload:

- Batch processing of big data sources at rest
- Real-time processing of big data in motion
- Interactive analysis of big data
- Machine learning and advanced analytics

A big data platform may use different technologies and architectures depending on the requirements and goals of the organization. Some common architectures are:

- Lambda architecture: A hybrid approach that combines batch and real-time processing of big data. The data is ingested into a batch layer and a speed layer. The batch layer performs historical analysis and creates a master dataset. The speed layer handles real-time data and provides incremental updates. The results from both layers are merged in a serving layer that provides a unified view of the data.
- Kappa architecture: A simplified version of the lambda architecture that only uses a speed layer for processing both historical and real-time data. The data is ingested into a streaming platform and processed by a stream processing engine. The results are stored in a serving layer that provides a unified view of the data.
- Data lake architecture: A flexible approach that stores raw data in a centralized repository, usually on a cloud platform. The data can be structured, semi-structured, or unstructured, and can be accessed by various applications and tools. The data can be processed and transformed on demand using different frameworks and engines.

The following diagram illustrates the basic architecture of a big data platform using ASCII art:

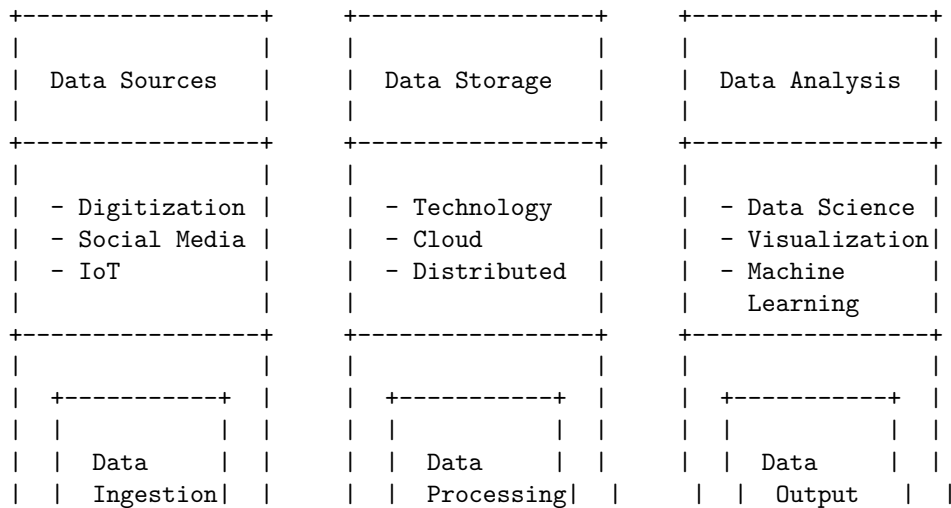


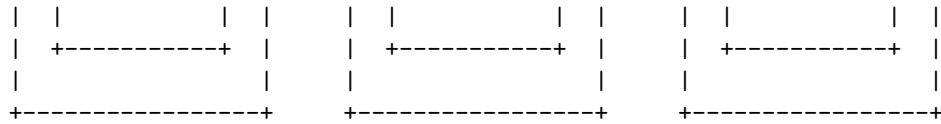
- Spark	+----->+ - HBase	+----->+ - Dashboards	
- Storm	- Cassandra	- Reports	
- Flink	- MongoDB	- Applications	
- Samza	- Elasticsearch	- ML models	
+-----+	+-----+	+-----+	

The drivers for Big Data are the factors that have contributed to the growth, use and exploitation of large and complex data sets in various domains. According to the sources , some of the main drivers for Big Data are:

- The digitization of society: The increasing use of digital devices and platforms to generate, store and share data in various formats, such as text, images, audio, video, etc.
- The drop in technology costs: The decreasing costs of hardware, software and cloud services that enable the collection, processing and analysis of Big Data.
- Connectivity through cloud computing: The availability of cloud-based platforms and services that provide scalable, flexible and cost-effective access to Big Data resources and applications.
- Increased knowledge about data science: The development of data science as a discipline that combines mathematics, statistics, computer science and domain knowledge to extract insights from Big Data.
- Social media applications: The popularity of social media platforms and applications that generate and consume large amounts of user-generated data, such as posts, comments, likes, shares, etc.
- The rise of Internet-of-Things (IoT): The emergence of IoT devices and sensors that collect and transmit data from various physical objects and environments, such as smart homes, smart cities, smart cars, etc.

The following diagram illustrates the basic architecture of a Big Data system that incorporates these drivers:



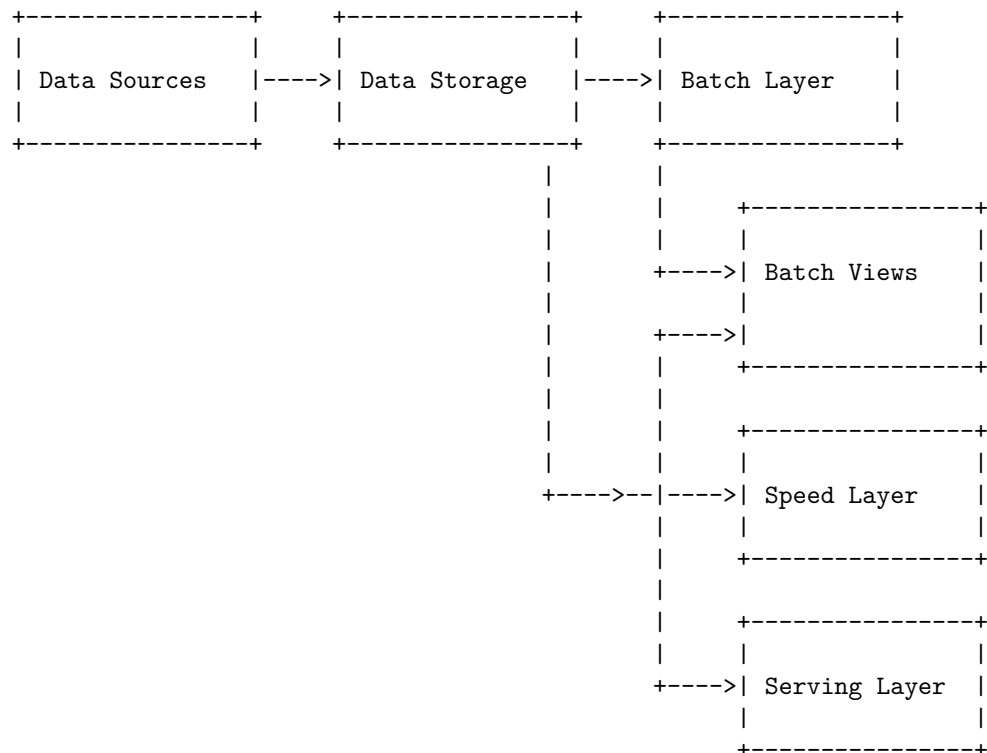


Big data architecture is the framework that defines the components, processes, and technologies needed to capture, store, process, and analyze big data. Big data refers to data sets that are too large or complex for traditional data processing systems to handle.

There are different types of big data architectures, depending on the requirements and goals of the data analysis. One of the most common types is the lambda architecture, which combines batch processing and stream processing to handle both historical and real-time data. Another type is the kappa architecture, which simplifies the lambda architecture by using only stream processing and treating all data as real-time.

The following diagram illustrates the basic architecture of a lambda-based big data system using ASCII art:

Big Data Architecture



The components of this architecture are:

- **Data sources:** These are the inputs that generate data in various formats, such as structured, semi-structured, or unstructured. Examples of data sources are web logs, social media, sensors, transactions, etc.
- **Data storage:** This is the layer that ingests data from the data sources and stores it in a scalable and fault-tolerant way. It also converts unstructured or semi-structured data into a structured format that can be processed by the batch layer. Examples of data storage technologies are Hadoop Distributed File System (HDFS), Amazon S3, Azure Blob Storage, etc.
- **Batch layer:** This is the layer that performs batch processing on the historical data stored in the data storage layer. It applies complex algorithms and transformations to the data and generates batch views, which are pre-computed results that can be queried by the serving layer. Examples of batch processing technologies are MapReduce, Spark, Hive, Pig, etc.
- **Batch views:** These are the outputs of the batch layer that contain the aggregated and summarized data that can answer analytical queries. They are stored in a read-optimized format that can be accessed by the serving layer. Examples of batch view technologies are HBase, Cassandra, MongoDB, etc.
- **Speed layer:** This is the layer that performs stream processing on the real-time data that arrives from the data sources. It applies simple algorithms and transformations to the data and generates real-time views, which are incremental updates that can be merged with the batch views by the serving layer. Examples of stream processing technologies are Storm, Spark Streaming, Kafka, etc.
- **Real-time views:** These are the outputs of the speed layer that contain the latest data that can answer analytical queries. They are stored in a write-optimized format that can be accessed by the serving layer. Examples of real-time view technologies are Redis, Memcached, etc.
- **Serving layer:** This is the layer that provides a unified view of the data to the end users or applications. It combines the batch views and the real-time views and exposes a query interface that can answer analytical queries. Examples of serving layer technologies are Druid, Impala, Presto, etc.

Big data is data that contains greater variety, arriving in increasing volumes and with more velocity. This is also known as the three Vs. However, some sources also include two more Vs: value and veracity . Value refers to the usefulness and relevance of the data, and veracity refers to the quality and accuracy of the data.

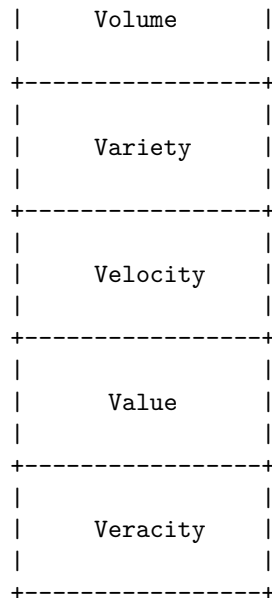
Big data characteristics

The following diagram illustrates the five characteristics of big data using ASCII art:

```

+-----+
|       |

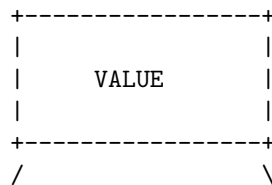
```

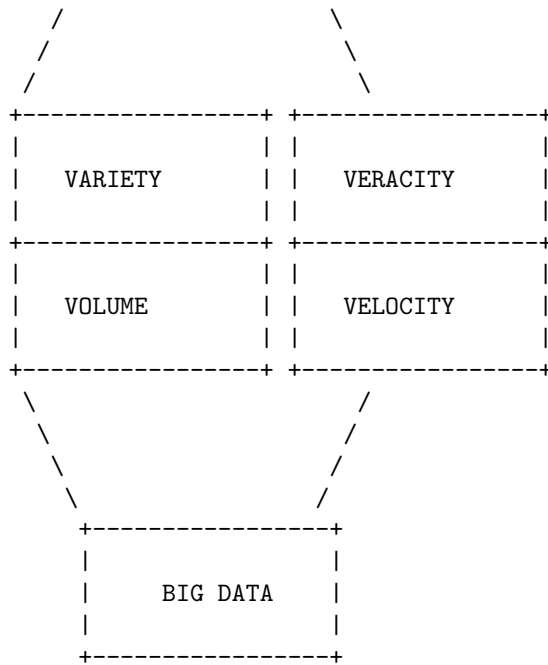


The 5 Vs of Big Data are the five main and innate characteristics of big data that describe its challenges and opportunities. They are:

- **Volume:** The amount of data generated and stored. Big data usually deals with data sets that are too large or complex for traditional data processing systems.
- **Velocity:** The speed at which data is created, collected, and analyzed. Big data often involves real-time or near-real-time data streams that require fast processing and decision making.
- **Variety:** The diversity of data sources, formats, and types. Big data can include structured, semi-structured, or unstructured data from various domains and applications.
- **Veracity:** The quality, accuracy, and reliability of data. Big data can be affected by noise, inconsistency, incompleteness, or ambiguity, which can reduce its usefulness and trustworthiness.
- **Value:** The potential benefit and usefulness of data for an organization or a user. Big data can provide valuable insights, patterns, trends, or predictions that can enhance decision making, innovation, or customer satisfaction.

The following diagram illustrates the 5 Vs of Big Data using ASCII art:



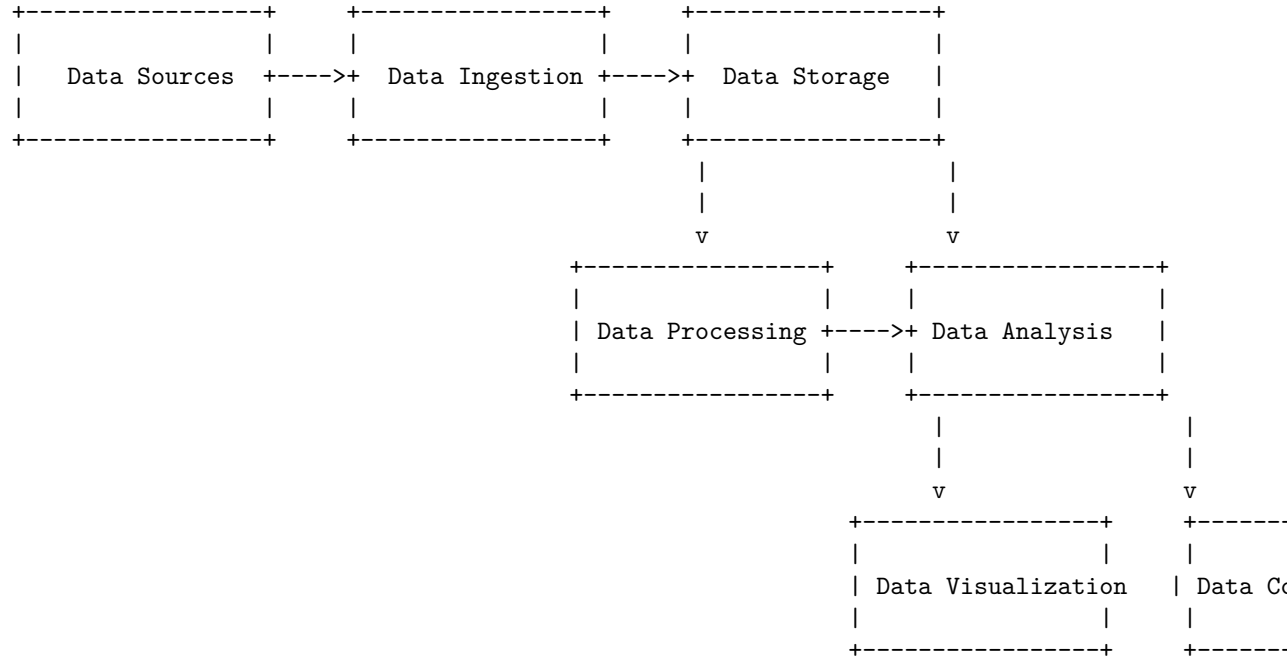


Big Data technology components are the various tools and techniques that are used to collect, store, process, analyze and visualize large and complex datasets. There are different types of Big Data technology components, depending on the purpose and function of the data analysis. Some of the common components are:

- Data sources: These are the origin of the data, such as application data stores, static files, streaming data, social media, sensors, etc.
- Data ingestion: This is the process of moving the data from the sources to the data storage or processing system, such as data pipelines, ETL tools, message queues, etc.
- Data storage: This is the system that stores the data in a structured, semi-structured or unstructured format, such as data lakes, data warehouses, databases, file systems, etc.
- Data processing: This is the system that performs computations on the data, such as batch processing, stream processing, machine learning, etc.
- Data analysis: This is the system that applies various techniques to extract insights from the data, such as business intelligence, data mining, natural language processing, etc.
- Data visualization: This is the system that presents the data in a graphical or interactive way, such as dashboards, charts, reports, etc.

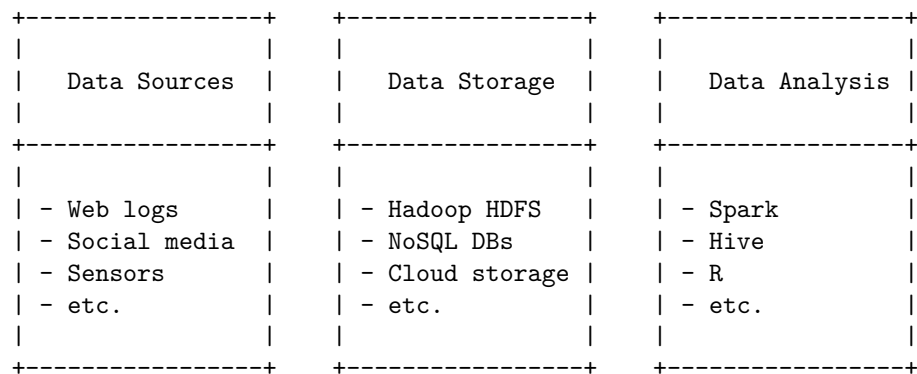
Big Data technology components

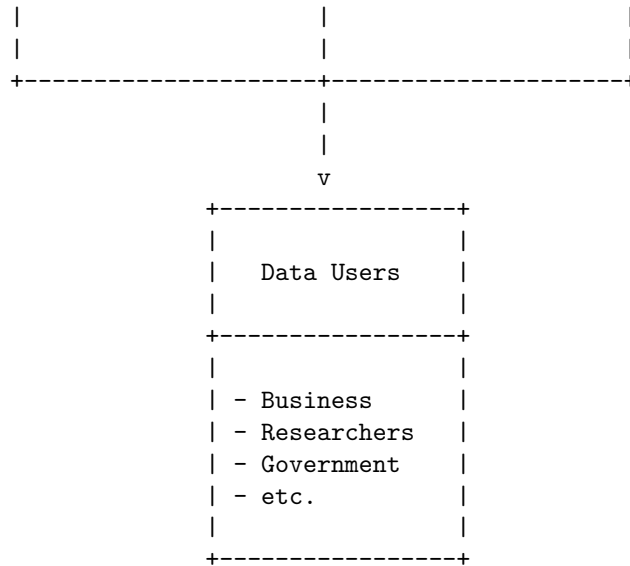
The following diagram illustrates the basic architecture of a Big Data technology system using ASCII art:



Big data is the term used to describe the large and complex datasets that are generated from various sources and in various formats. Big data is important because it can help businesses and organizations to gain valuable insights, improve decision making, enhance customer experience, reduce costs, and drive innovation.

The following diagram illustrates the basic architecture of a big data system using ASCII art:





The diagram shows that data sources generate big data that is stored in various data storage systems. The data storage systems can be distributed, scalable, and fault-tolerant. The data analysis layer performs various tasks such as querying, processing, mining, and visualizing the data using different tools and frameworks. The data analysis layer can also leverage parallel and distributed computing to handle large and complex data. The data users are the end-users who consume the data and use it for various purposes such as business intelligence, research, policy making, and so on.

Big Data applications are software solutions that deal with large and complex data sets that cannot be handled by traditional database systems. They typically involve one or more of the following types of workload:

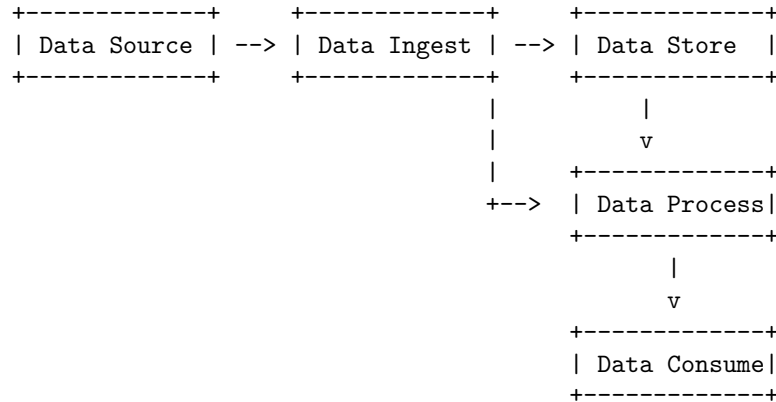
- Batch processing of big data sources at rest.
- Real-time processing of big data in motion.
- Interactive analysis of big data sources.
- Machine learning and advanced analytics on big data sources.

A common big data architecture consists of the following layers:

- Data sources: The origin of the data, such as sensors, web logs, social media, etc.
- Data ingestion: The process of collecting, validating, and moving the data from the sources to a data store or a processing engine.
- Data storage: The repository where the data is persisted, such as a distributed file system, a data lake, a data warehouse, etc.
- Data processing: The engine that performs the transformation, analysis, and computation on the data, such as MapReduce, Spark, Flink, etc.
- Data consumption: The interface that allows the users to access, visualize,

and utilize the data, such as dashboards, reports, APIs, etc.

The following diagram illustrates the basic architecture of a big data application using ASCII art:

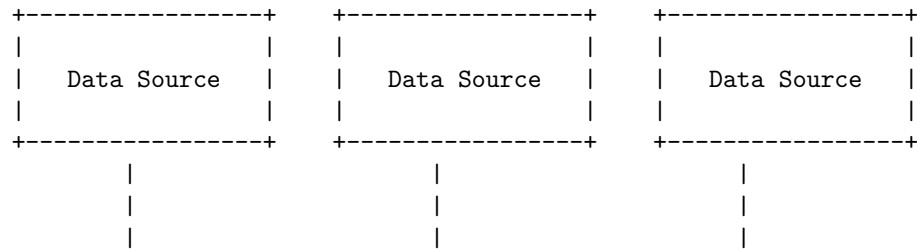


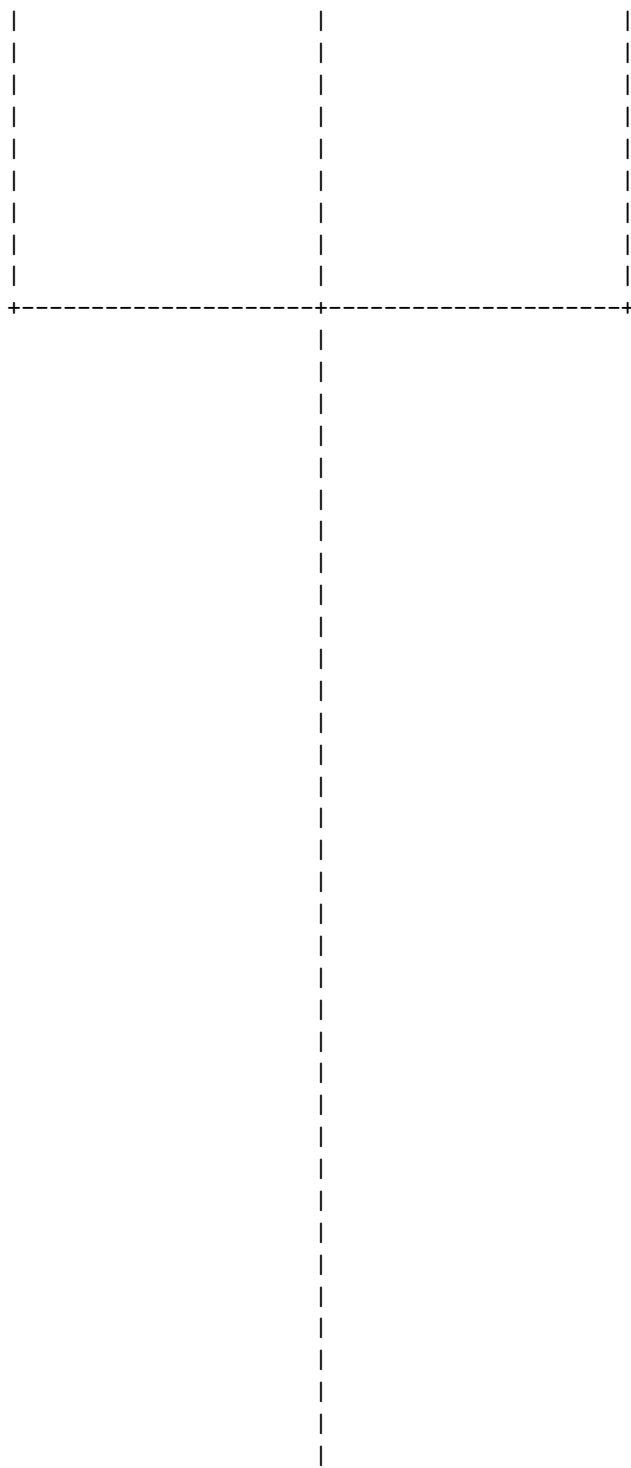
Big Data features – security, compliance, auditing and protection

Big data security is the process of ensuring the confidentiality, integrity, and availability of large volumes of data that are collected, stored, processed, and analyzed by various systems and applications. Big data security involves the following aspects:

- Security: preventing unauthorized access, modification, or deletion of data by using firewalls, encryption, authentication, authorization, and monitoring techniques.
- Compliance: adhering to the legal and regulatory requirements that govern the collection, use, and disclosure of data, such as privacy laws, data protection laws, and industry standards.
- Auditing: tracking and recording the activities and events that occur on the data and the systems that handle it, such as data access, data modification, data transfer, and data breach.
- Protection: safeguarding the data from loss, corruption, or damage by using backup, recovery, replication, and fault tolerance techniques.

The following diagram illustrates the basic architecture of a big data security system using ASCII art:



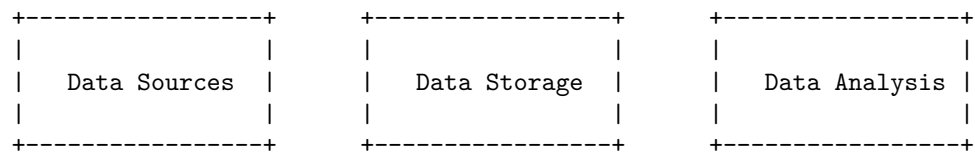




Security of big data is the process of implementing safeguards to protect an enterprise's big data from unauthorized access or breaches throughout the entirety of its lifecycle. Big data security involves various technologies and practices, such as encryption, centralized key management, user access control, data masking, auditing, and monitoring.

Security of Big Data

The following diagram illustrates the basic architecture of a big data security system using ASCII characters:



	- Web			- Hadoop	
	- IoT			- NoSQL	
	- Social Media			- Cloud	
	- Logs			- File System	
+-----+		+-----+		+-----+	
	- Encryption			- Encryption	
	- Authentication			- Key Management	
	- Authorization			- Access Control	
	- Firewall			- Data Masking	
				- Auditing	
+-----+		+-----+		+-----+	

The diagram shows the three main stages of big data lifecycle: data sources, data storage, and data analysis. Each stage has different types of data and technologies, and requires different security measures to protect the data from threats. Some of the common security measures are encryption, key management, access control, data masking, auditing, and firewall. Encryption protects the data from unauthorized access or modification, key management ensures the proper management and distribution of encryption keys, access control regulates who can access the data and what they can do with it, data masking obscures sensitive data from unauthorized users, auditing records the activities and events related to the data, and firewall blocks unwanted network traffic. These security measures can be applied at different levels of the big data system, such as the data itself, the network, the application, or the user.

Compliance of Big Data refers to the process of ensuring that the collection, processing, storage, and use of data adhere to the relevant laws and regulations, such as data privacy, security, and governance. Compliance of Big Data involves implementing appropriate technical and organizational measures to protect data from unauthorized access, modification, or disclosure, as well as to respect the rights and interests of data subjects, such as transparency, consent, access, and erasure.

Compliance of Big Data

+-----+	+-----+	+-----+	+-----+
Data Collection	Data Processing	Data Storage	Data Use
+-----+	+-----+	+-----+	+-----+
- Obtain	- Apply	- Encrypt	- Analyze
consent	data	data	data
- Inform	minimization	- Backup	- Report
data	principles	data	data
subjects	- Anonymize	- Secure	- Delete

The diagram shows the three main components of a big data system: data sources, data storage, and data analysis. Each component has a corresponding audit area: data quality, data security, and data accuracy. The audit areas are connected to the components by arrows, indicating the direction of the audit flow. The audit areas are also connected to each other by arrows, indicating the interdependence of the audit objectives. For example, data quality audit depends on data security audit, and data accuracy audit depends on data quality audit. The diagram illustrates the basic architecture of a big data audit.

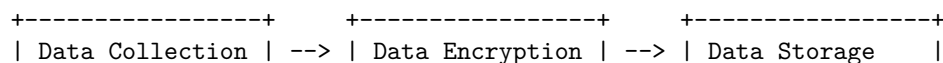
Protection of Big Data

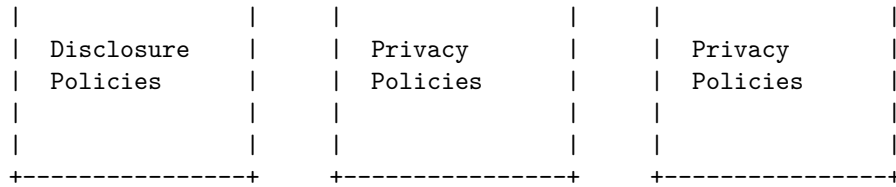
Big data is the term used to describe large and complex datasets that are difficult to process, store, and analyze using traditional methods. Big data can provide valuable insights for various domains, such as business, health, education, and security. However, big data also poses significant challenges for data privacy, as it may contain sensitive information about individuals or organizations that can be exploited by unauthorized parties.

To protect the privacy of big data, several measures need to be taken at different stages of the data lifecycle, such as data collection, retention, archiving, use, and disclosure. Some of the common techniques for protecting big data privacy are:

- Data anonymization: This is the process of removing or modifying personally identifiable information (PII) from the data, such as names, addresses, phone numbers, etc. Data anonymization can reduce the risk of re-identification of individuals or groups from the data, but it may also affect the utility and quality of the data for analysis.
- Data encryption: This is the process of transforming the data into an unreadable form using a secret key, such that only authorized parties can decrypt and access the data. Data encryption can protect the data from unauthorized access or modification, but it may also increase the computational and storage costs of the data.
- Data masking: This is the process of replacing or hiding sensitive data elements with fictitious or random values, such that the data can still be used for testing, development, or analysis purposes, without revealing the original data. Data masking can preserve the structure and format of the data, but it may also introduce errors or biases in the data.
- Data minimization: This is the principle of collecting, retaining, and using only the minimum amount of data that is necessary and relevant for a specific purpose, and deleting or anonymizing the data when it is no longer needed. Data minimization can reduce the exposure and storage of the data, but it may also limit the potential value and insights of the data.

The following diagram illustrates the basic architecture of a big data privacy protection system, using the above techniques:



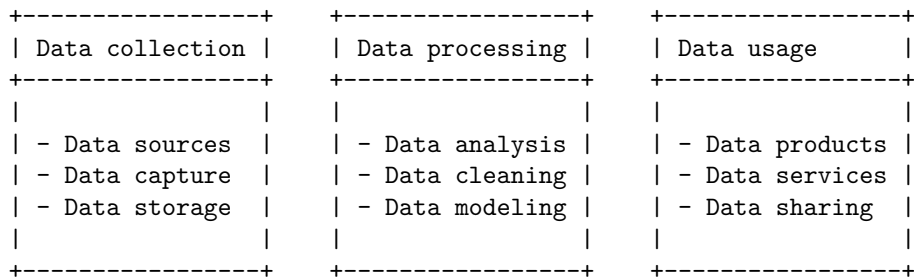


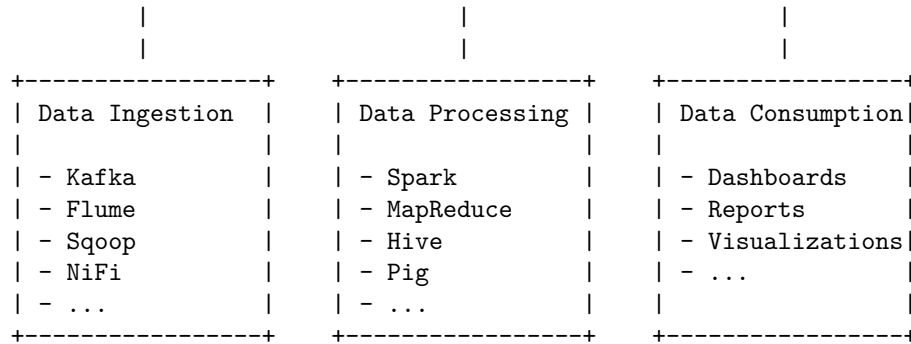
The following diagram illustrates the basic architecture of Big Data privacy. The data collection stage involves gathering data from various sources, such as sensors, web logs, social media, etc. The data retention and archiving stage involves storing and preserving the data for future use, such as analysis, reporting, or compliance. The data use stage involves processing and analyzing the data for various purposes, such as business intelligence, machine learning, or decision making. The disclosure policies stage involves defining and enforcing the rules and regulations for sharing and disclosing the data to third parties, such as customers, partners, or regulators. The privacy policies stage involves defining and enforcing the rules and regulations for protecting the privacy of the data subjects, such as consent, anonymization, encryption, or deletion. The privacy policies should be aligned with the disclosure policies, and both should comply with the relevant laws and ethical standards.

Big Data ethics refers to systemizing, defending, and recommending concepts of right and wrong conduct in relation to data, in particular personal data. It is concerned with the following principles :

- Ownership: Individuals own their own data
- Transaction transparency: If an individual's personal data is used, they should have transparent access to the purpose, methods, and outcomes of the data processing
- Consent: If an individual or legal entity would like to use personal data, one must obtain the explicit and informed consent of the data owner
- Privacy: Individuals have the right to control the access and use of their personal data
- Security: Data collectors and disseminators must ensure the protection and integrity of personal data from unauthorized access, misuse, or harm

The following diagram illustrates the basic architecture of a Big Data ethics framework using ASCII characters:





The diagram shows the three main layers of a big data analytics system: data sources, data storage, and data analysis. Each layer has different components that perform specific functions.

- Data sources are the origin of the data, such as web logs, social media, IoT devices, sensors, etc. They generate data in various formats and velocities.
- Data storage is the layer that stores the data in different formats and structures, such as data lake, data warehouse, NoSQL database, HDFS, etc. They provide scalability, reliability, and accessibility for the data.
- Data analysis is the layer that applies advanced analytic techniques to the data, such as machine learning, BI tools, SQL queries, etc. They provide insights, predictions, and recommendations for the data.

The diagram also shows the three supporting layers of a big data analytics system: data ingestion, data processing, and data consumption. Each layer has different components that enable the flow and transformation of the data.

- Data ingestion is the layer that collects and transfers the data from the data sources to the data storage, such as Kafka, Flume, Sqoop, NiFi, etc. They handle the challenges of data volume, variety, and velocity.
- Data processing is the layer that processes and transforms the data in the data storage, such as Spark, MapReduce, Hive, Pig, etc. They handle the challenges of data quality, consistency, and complexity.
- Data consumption is the layer that consumes and presents the data from the data analysis, such as dashboards, reports, visualizations, etc. They handle the challenges of data interpretation, communication, and action.

Challenges of conventional systems compared to Big Data

Conventional systems are data processing and storage systems that use traditional database architectures and analytical tools. They are designed for structured and relatively small data sets that can be easily managed and queried. Some examples of conventional systems are relational databases, data warehouses, and business intelligence tools.

Big Data, on the other hand, is data that exceeds the processing capacity of

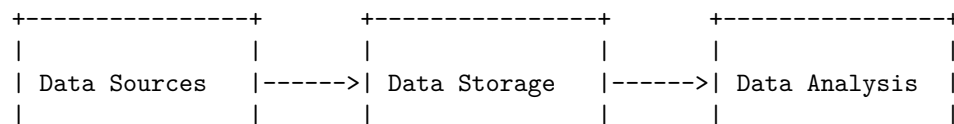
conventional systems. It is characterized by the 3Vs: volume, velocity, and variety. Big Data is large, fast, and diverse. It can be structured, unstructured, or semi-structured. It can come from various sources, such as social media, sensors, web logs, or streaming data. Some examples of Big Data systems are Hadoop, Spark, NoSQL databases, and cloud computing platforms.

The following diagram illustrates the main challenges of conventional systems compared to Big Data:

	Conventional Systems	Big Data Systems
Data Volume	Limited by storage and processing capacity	Scalable by parallel and distributed computing
Data Velocity	Batch-oriented and periodic processing	Stream-oriented and real-time processing
Data Variety	Structured and homogeneous data	Structured, unstructured, and heterogeneous data
Data Quality	High and consistent data quality	Low and variable data quality
Data Analysis	Predefined and descriptive analysis	Exploratory and predictive analysis
Data Security	Well-defined and centralized security policies	Complex and decentralized security policies

Intelligent data analysis in Big Data is the process of applying advanced analytical techniques, such as data mining, statistical analysis, predictive modeling, and deep learning, to large and complex datasets, in order to extract meaningful insights and patterns that can support decision making and problem solving.

The following diagram illustrates the basic architecture of a typical intelligent data analysis system in Big Data:



+-----+	+-----+	+-----+
- Web	- Hadoop	- Data Mining
- IoT	- NoSQL	- Predictive
- Social Media	- Cloud	Analytics
- Sensors		- Deep Learning
- ...		- ...
+-----+	+-----+	+-----+

The data sources are the various types of data that are generated and collected from different domains and applications, such as web, internet of things, social media, sensors, etc. The data sources can be structured, semi-structured, or unstructured, and can have different formats, sizes, and velocities.

The data storage is the layer that stores and manages the data from the data sources, using technologies that can handle the volume, variety, and velocity of Big Data, such as Hadoop, NoSQL, and cloud computing. The data storage layer provides scalability, reliability, and availability of the data, as well as data preprocessing and integration capabilities.

The data analysis is the layer that performs the intelligent data analysis on the data stored in the data storage layer, using techniques that can discover patterns, relationships, and trends from the data, such as data mining, predictive analytics, and deep learning. The data analysis layer provides actionable insights and knowledge that can help the users and stakeholders to make informed decisions and solve problems.

The nature of data in Big Data refers to the different types, formats and sources of data that are collected and processed by organizations for various purposes. Big Data can be classified into three main types: structured, unstructured and semistructured data .

Structured data is data that has a predefined schema and can be easily stored and queried in relational databases. Examples of structured data are transactions, financial records, customer information, etc.

Unstructured data is data that has no fixed format or structure and cannot be easily organized or analyzed by conventional methods. Examples of unstructured data are text, documents, images, videos, audio, social media posts, etc.

Semistructured data is data that has some elements of structure, such as tags, labels or metadata, but does not conform to a rigid schema. Examples of semistructured data are web server logs, streaming data from sensors, XML files, JSON files, etc.

The following diagram illustrates the nature of data in Big Data using ASCII characters:

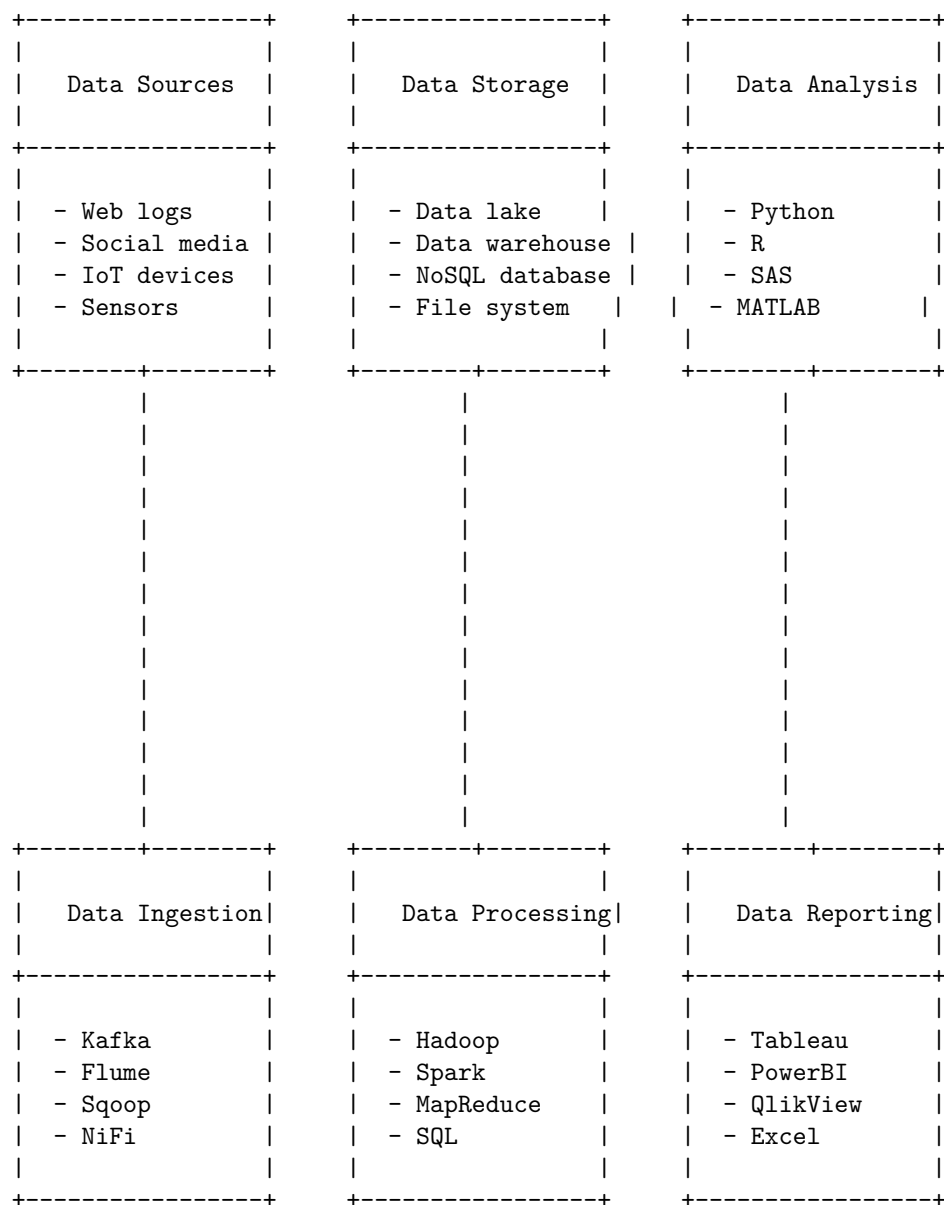
```
+-----+
|           |           |           |
+-----+
```

Structured	Unstructured	Semistructured
Transactions	Text	Web logs
Financial records	Documents	Streaming data
Customer information	Images	XML files
	Videos	JSON files
	Audio	
	Social media posts	

Analytic processes and tools for Big Data are the methods and technologies that enable the extraction of insights from large and complex data sets. Some of the common analytic processes and tools for Big Data are:

- **Data ingestion:** The process of collecting, importing, and processing data from various sources into a data storage system, such as a data lake or a data warehouse. Data ingestion can be done in batch mode, where data is loaded periodically, or in stream mode, where data is processed in real-time as it arrives.
- **Data storage:** The process of organizing, managing, and securing data in a data storage system, such as a data lake or a data warehouse. Data storage can be done using different formats and structures, such as relational, non-relational, or file-based. Data storage can also involve data compression, encryption, partitioning, and replication for performance and security purposes.
- **Data processing:** The process of transforming, filtering, aggregating, and enriching data using various tools and frameworks, such as Hadoop, Spark, MapReduce, or SQL. Data processing can be done in batch mode, where data is processed periodically, or in stream mode, where data is processed in real-time as it arrives.
- **Data analysis:** The process of applying statistical, machine learning, or artificial intelligence techniques to data to discover patterns, trends, correlations, and anomalies. Data analysis can be done using various tools and languages, such as Python, R, SAS, or MATLAB. Data analysis can also involve data visualization, which is the process of presenting data in graphical or interactive forms, such as charts, graphs, dashboards, or maps.
- **Data reporting:** The process of communicating the results and insights of data analysis to various stakeholders, such as business users, managers, or customers. Data reporting can be done using various tools and platforms, such as Tableau, PowerBI, QlikView, or Excel. Data reporting can also involve data storytelling, which is the process of using narratives, context, and emotions to convey the meaning and value of data.

The following diagram illustrates the basic architecture of a Big Data analytics system using ASCII art:

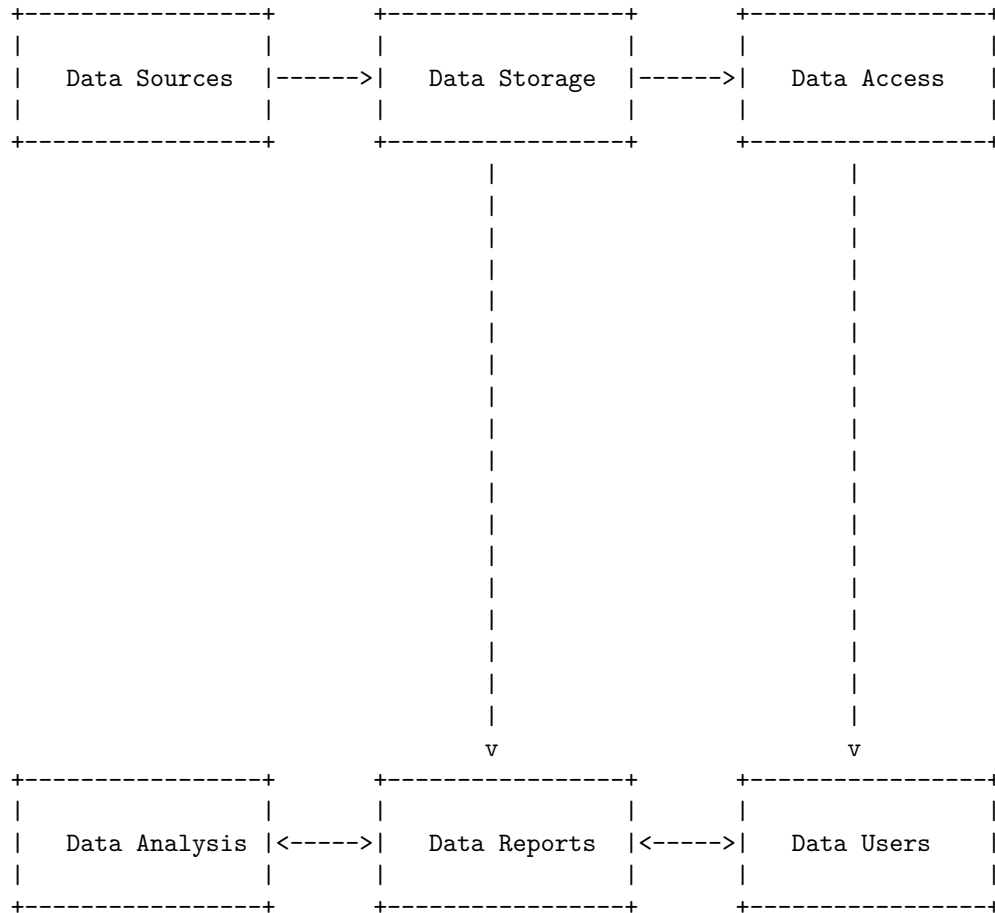


I am Sydney, your AI assistant. I will draw a detailed ASCII diagram for analysis vs reporting in Big Data.

Analysis vs Reporting in Big Data

Analysis and reporting are two different processes that involve working with data. Analysis is the process of exploring data and reports in order to extract meaningful insights, which can be used to better understand and improve business performance. Reporting is the process of organizing and summarizing data in a digestible manner, such as charts, tables, or dashboards.

The following diagram illustrates the basic architecture of a Big Data system that supports both analysis and reporting:



- **Data Sources:** These are the various sources of data that are collected and processed by the Big Data system, such as web logs, social media, sensors, transactions, etc.
- **Data Storage:** This is the layer where the data is stored and managed, such as Hadoop Distributed File System (HDFS), NoSQL databases, data warehouses, etc.

- **Data Access:** This is the layer where the data is accessed and queried, such as MapReduce, Spark, Hive, Pig, SQL, etc.
- **Data Analysis:** This is the layer where the data is analyzed and explored, such as machine learning, statistics, data mining, etc.
- **Data Reports:** This is the layer where the data is reported and visualized, such as charts, tables, dashboards, etc.
- **Data Users:** These are the end users who consume the data analysis and reports, such as business analysts, managers, customers, etc.

Modern data analytic tools for Big Data are software applications that can process, analyze, and visualize large and complex datasets. Some of the common features of these tools are:

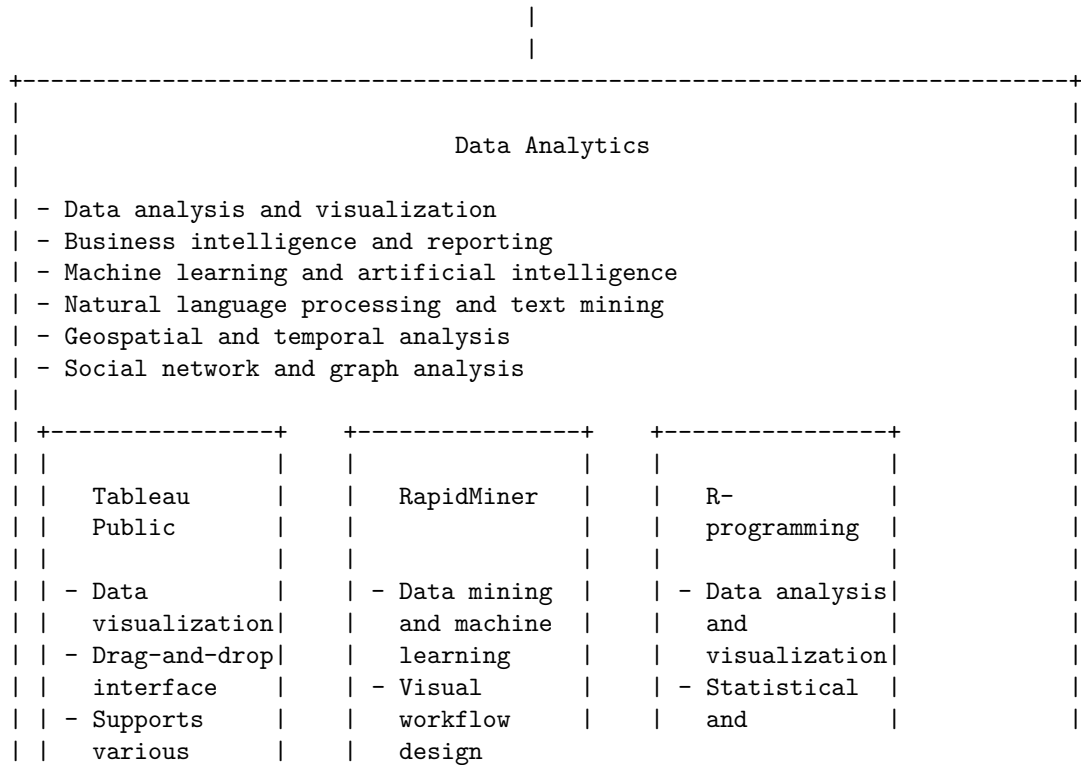
- They can handle structured, semi-structured, and unstructured data from various sources.
- They can perform various types of analysis, such as descriptive, diagnostic, predictive, and prescriptive.
- They can leverage advanced techniques, such as machine learning, artificial intelligence, and natural language processing.
- They can provide interactive and intuitive dashboards, charts, graphs, and maps to present the insights.

Modern data analytic tools for Big Data

The following diagram illustrates some of the popular modern data analytic tools for Big Data and their functionalities:

Apache Hadoop	KNIME	OpenRefine	Orange
- Distributed storage and processing	- Data mining and machine learning	- Data cleaning and transformation	- Data mining and machine learning
- Scalable and fault-tolerant	- Visual programming	- Web-based interface	- Visual programming
- Supports various languages and tools	- Supports various formats and databases	- Supports various formats and databases	- Supports various formats and databases

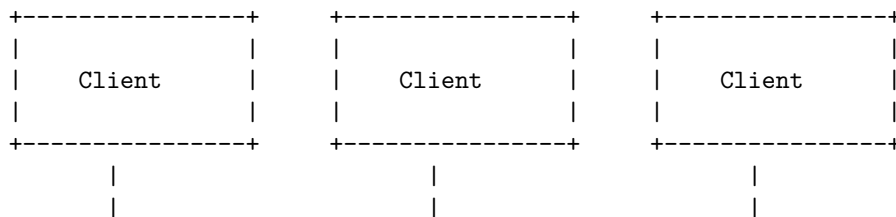


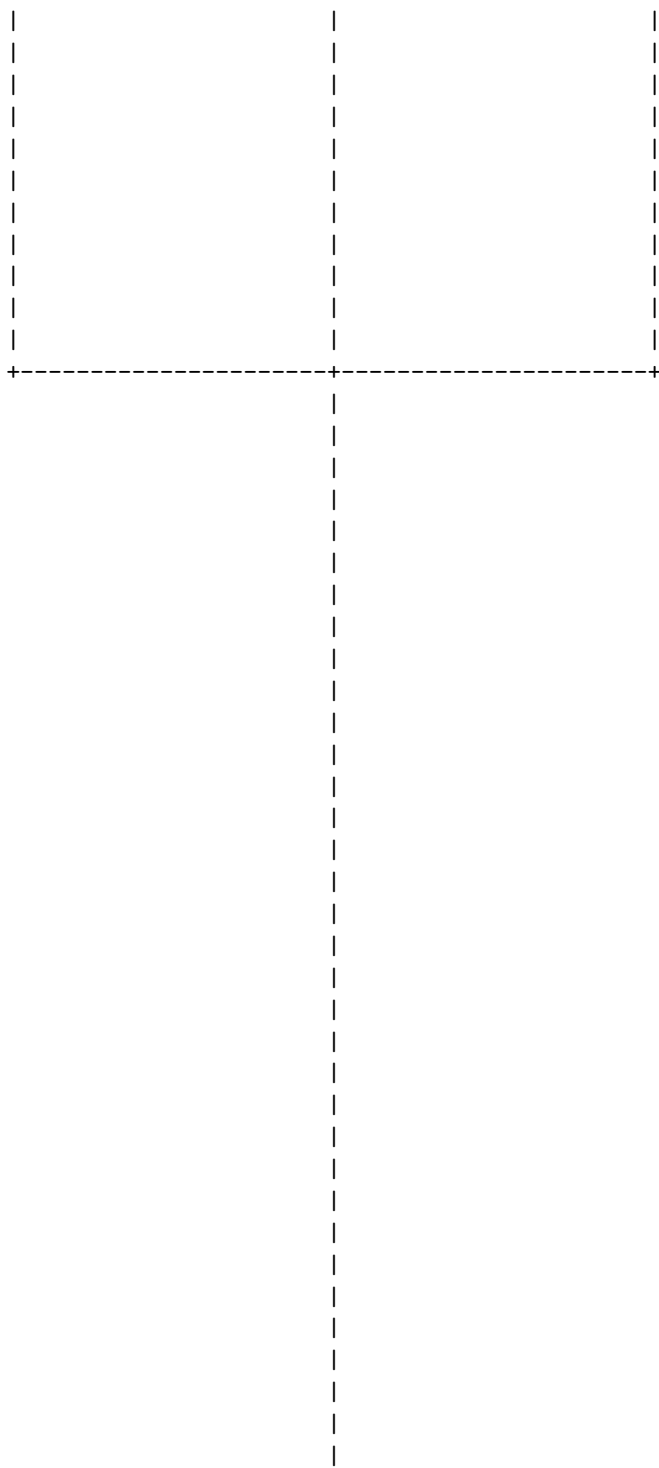


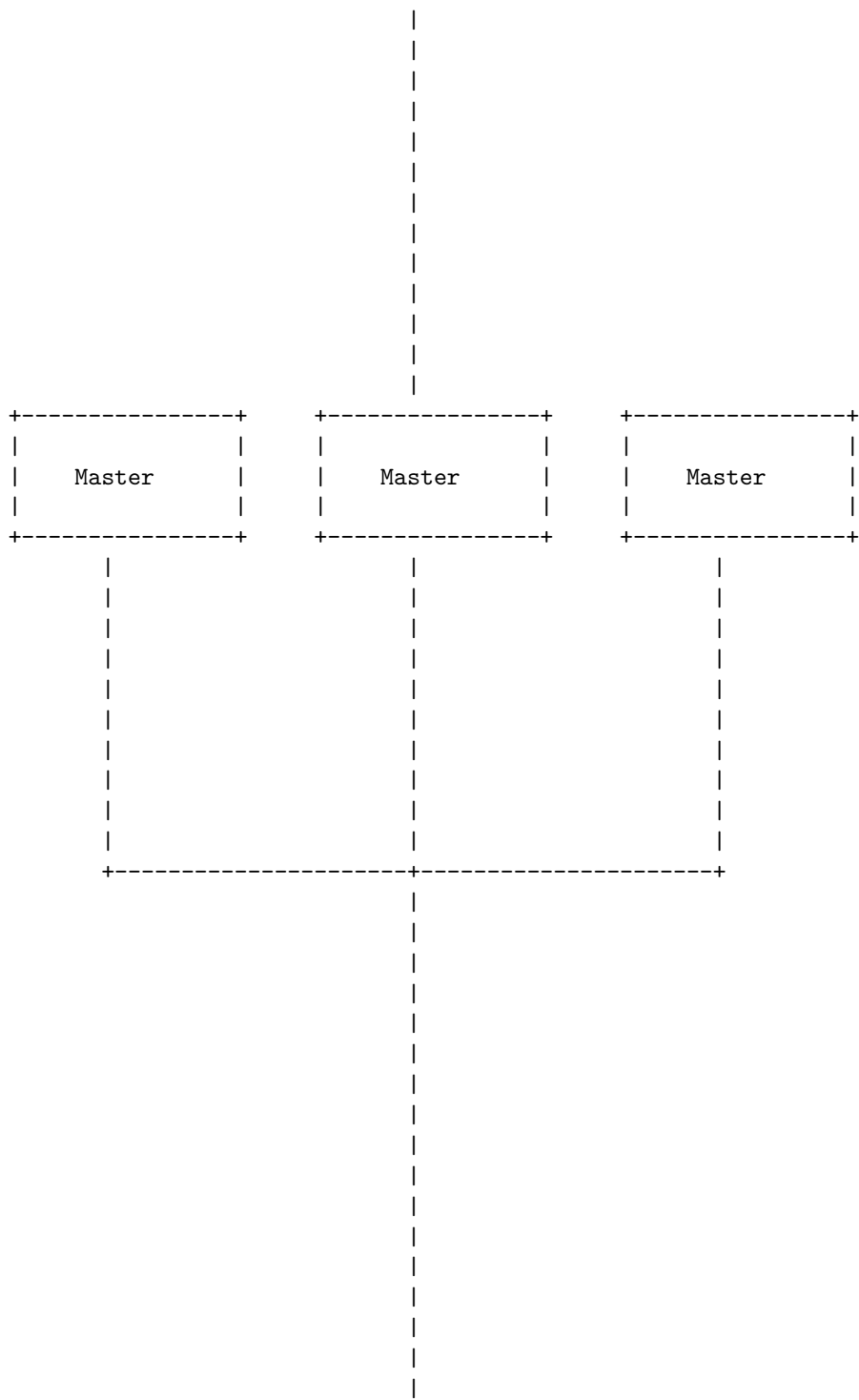
Unit 2 - Hadoop and Map Reduce

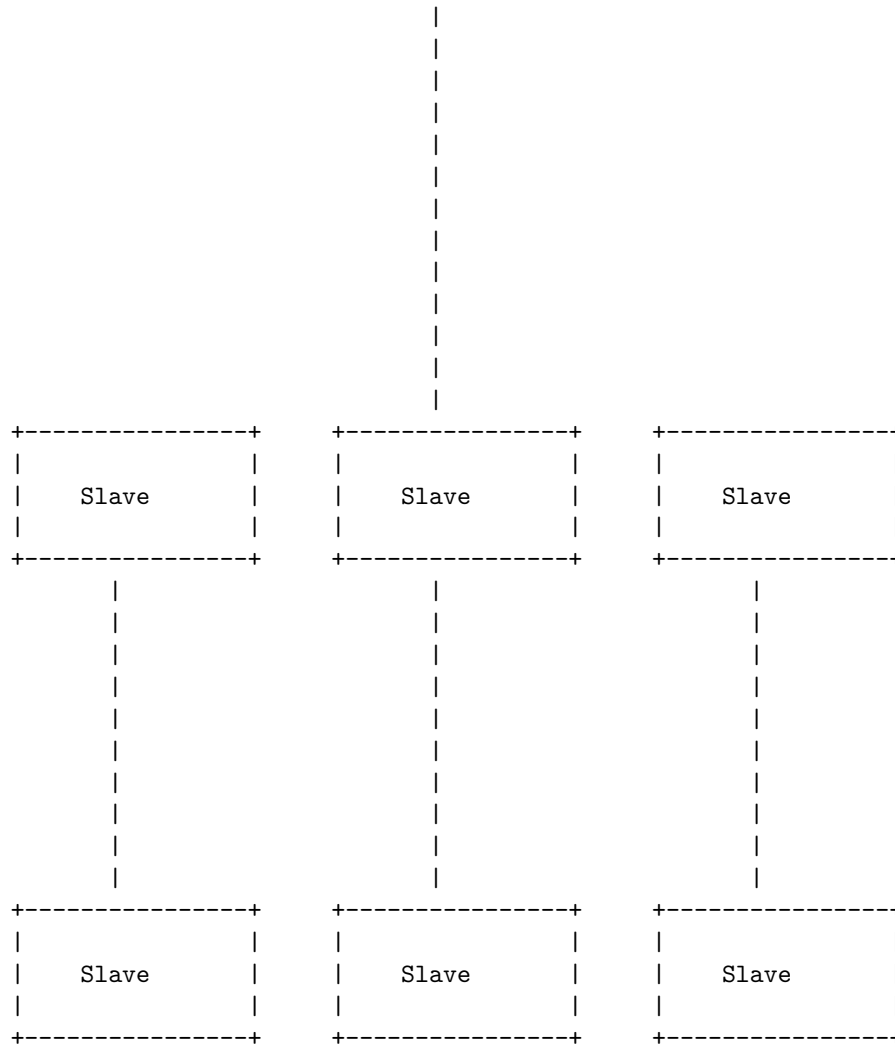
Hadoop and Map Reduce are two components of the Hadoop ecosystem that enable parallel processing of large data sets in a distributed manner. Hadoop consists of a distributed file system called HDFS, which stores the data across multiple nodes, and a resource management layer called YARN, which allocates the resources for the applications. Map Reduce is a programming model that divides the data processing into two phases: map and reduce. The map phase applies a user-defined function to each input record and produces intermediate key-value pairs. The reduce phase aggregates the intermediate values associated with the same key and produces the final output.

The following diagram illustrates the basic architecture of a Hadoop and Map Reduce system using ASCII art:









The master nodes are responsible for coordinating the tasks and managing the metadata. The slave nodes are responsible for executing the tasks and storing the data. The client nodes are the ones that submit the applications and interact with the system. The master nodes include:

- NameNode: The master node of HDFS, which maintains the namespace and the block locations of the files.
- Secondary NameNode: A backup node of HDFS, which periodically merges the namespace image and the edit log of the NameNode.
- ResourceManager: The master node of YARN, which allocates the resources and schedules the applications across the cluster.
- JobTracker: The master node of Map Reduce, which assigns the map and reduce tasks to the slave nodes and monitors their progress.

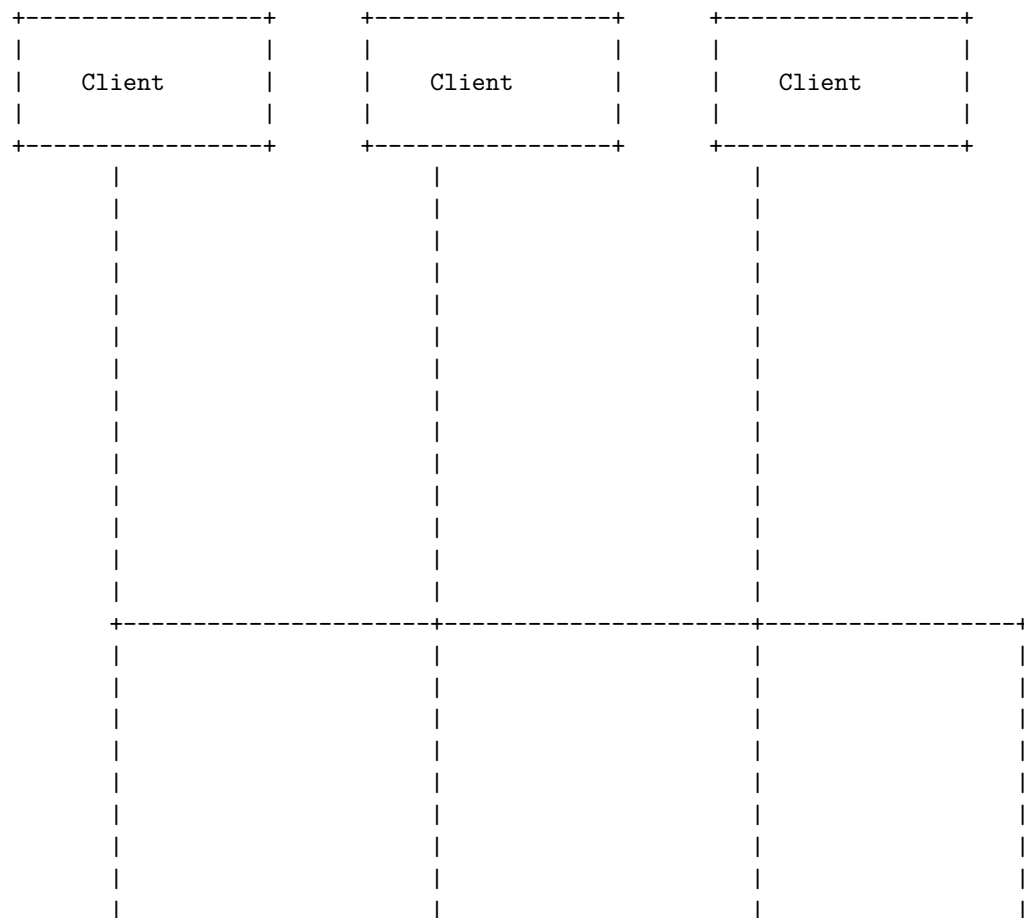
The slave nodes include:

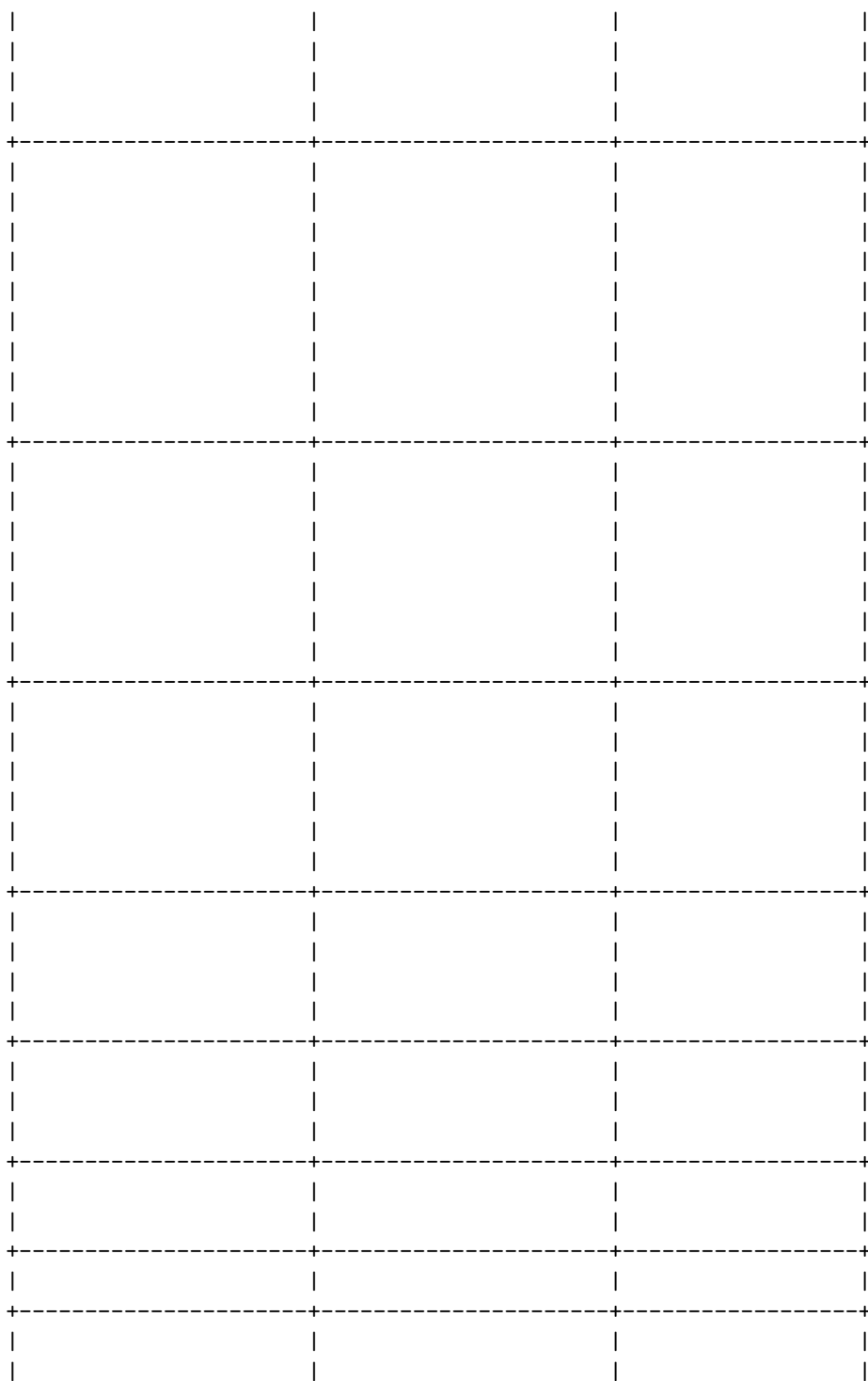
- DataNode: The slave node of HDFS, which stores the data blocks and communicates with the NameNode.
- NodeManager: The slave

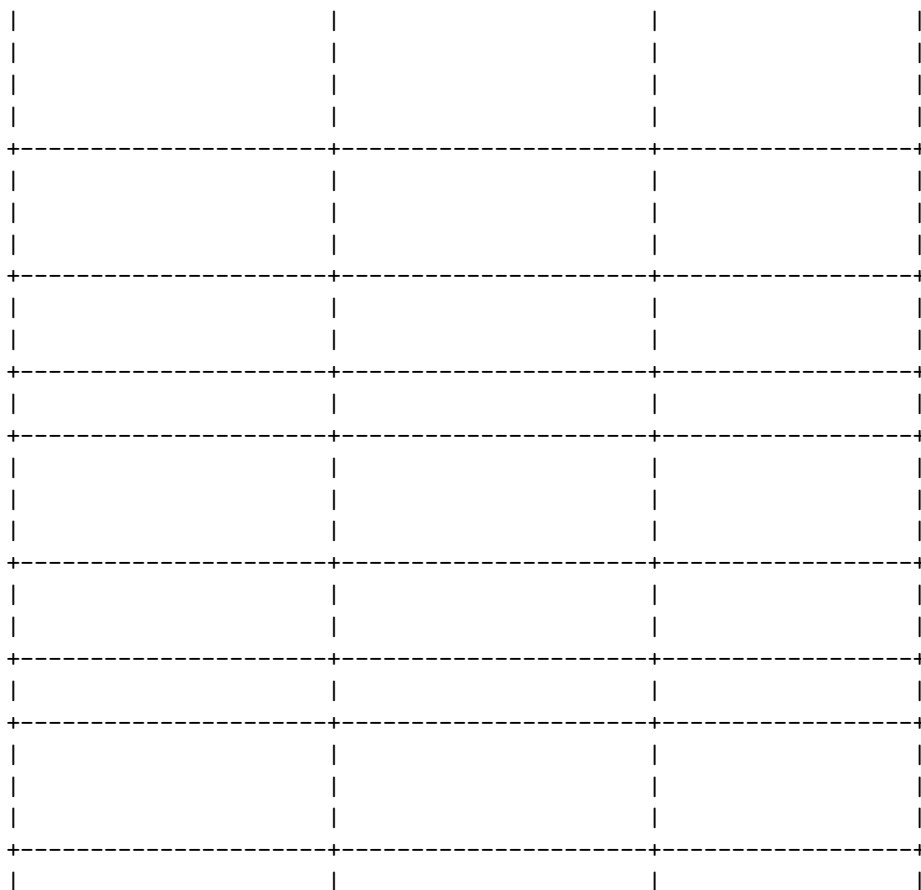
Hadoop is a big data solution that provides distributed storage and processing of large datasets using commodity hardware. Hadoop has three core components: HDFS, YARN and MapReduce. HDFS is the Hadoop Distributed File System that stores data in blocks across multiple nodes. YARN is the Yet Another Resource Negotiator that manages the resources and scheduling of tasks. MapReduce is the programming model that allows parallel processing of data using mapper and reducer functions.

The following is a detailed ASCII diagram for Hadoop:

Hadoop







The history of Hadoop can be traced back to the year 2002, when Doug Cutting and Mike Cafarella started working on the Apache Nutch project, which aimed to build a search engine system that can index 1 billion pages. They faced challenges in scaling up the system and processing large amounts of data. In 2003, Google published a white paper that described the Google File System (GFS), a distributed file system that can store and manage huge amounts of data across multiple machines. Inspired by this, Cutting and Cafarella implemented a similar file system for Nutch, called Nutch Distributed File System (NDFS). In 2004, Google published another white paper that introduced the MapReduce programming model, which allows parallel processing of large data sets using a simple map and reduce functions. Cutting and Cafarella adopted this model for Nutch as well, and created a prototype of a distributed computing framework.

In 2006, Cutting joined Yahoo, and the Nutch project was divided into two subprojects: Nutch, the web crawler, and Hadoop, the distributed computing framework. Hadoop was named after Cutting's son's toy elephant. Hadoop became an open-source project under the Apache Software Foundation, and attracted more contributors and committers. In 2008, Hadoop set a world record

by sorting 1 terabyte of data in 209 seconds, beating the previous record held by a supercomputer. Hadoop also became the core platform for Yahoo's web search and advertising businesses.

Since then, Hadoop has evolved and expanded into a large ecosystem of projects that provide various tools and services for big data analytics. Some of the major projects in the Hadoop ecosystem are:

- Hadoop Common: The common utilities and libraries that support other Hadoop modules.
- Hadoop Distributed File System (HDFS): The distributed file system that stores data across multiple machines and provides high availability and fault tolerance.
- Hadoop MapReduce: The programming model and software framework for parallel processing of large data sets using map and reduce functions.
- Hadoop YARN: The resource management and scheduling system that allocates and manages resources for Hadoop applications.
- Hadoop Ozone: The scalable, distributed object store for Hadoop that can handle billions of files and objects.
- Apache HBase: The distributed, column-oriented database that provides random access and consistent read/write operations for large data sets.
- Apache Hive: The data warehouse system that provides data summarization, query, and analysis using a SQL-like language called HiveQL.
- Apache Pig: The high-level scripting language and platform that allows users to write complex data transformations and analysis using a simple syntax.
- Apache Spark: The fast and general engine for large-scale data processing that supports batch, streaming, SQL, machine learning, and graph processing.
- Apache Flume: The service that collects, aggregates, and moves large amounts of log data from various sources to HDFS or other destinations.
- Apache Sqoop: The tool that transfers data between Hadoop and relational databases or data warehouses.
- Apache Oozie: The workflow scheduler that manages and coordinates Hadoop jobs and tasks.
- Apache ZooKeeper: The service that provides distributed coordination and configuration management for Hadoop clusters and applications.
- Apache Mahout: The library that provides scalable machine learning and data mining algorithms for Hadoop.
- Apache Cassandra: The distributed, wide-column store that provides high availability and scalability for large data sets.
- Apache Kafka: The distributed, publish-subscribe messaging system that handles real-time data streams.
- Apache Storm: The distributed, real-time computation system that processes data streams using a directed acyclic graph (DAG) of spouts and bolts.
- Apache Flink: The distributed, stream and batch processing system that

provides high throughput, low latency, and stateful computations.

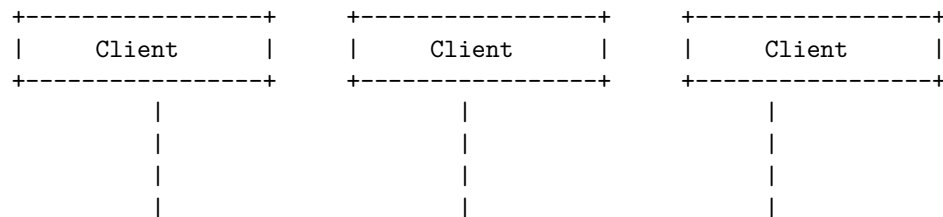
- Apache Samza: The distributed, stream processing framework that integrates with Kafka and provides a simple API for writing stateful applications.
- Apache Nifi: The data flow automation system that enables users to capture, process, and distribute data from various sources and destinations.
- Apache Ambari: The web-based tool that simplifies the provisioning, management, and monitoring of Hadoop clusters.
- Apache Ranger: The security framework that provides centralized access control and auditing for Hadoop resources and services.
- Apache Knox: The gateway service that provides a single point of authentication and access for Hadoop REST APIs and UIs.
- Apache Tez: The application framework that allows users to express complex data processing logic as a DAG of tasks, and optimizes the execution on YARN.
- Apache Phoenix: The SQL query engine that provides low-latency, high-performance queries over H

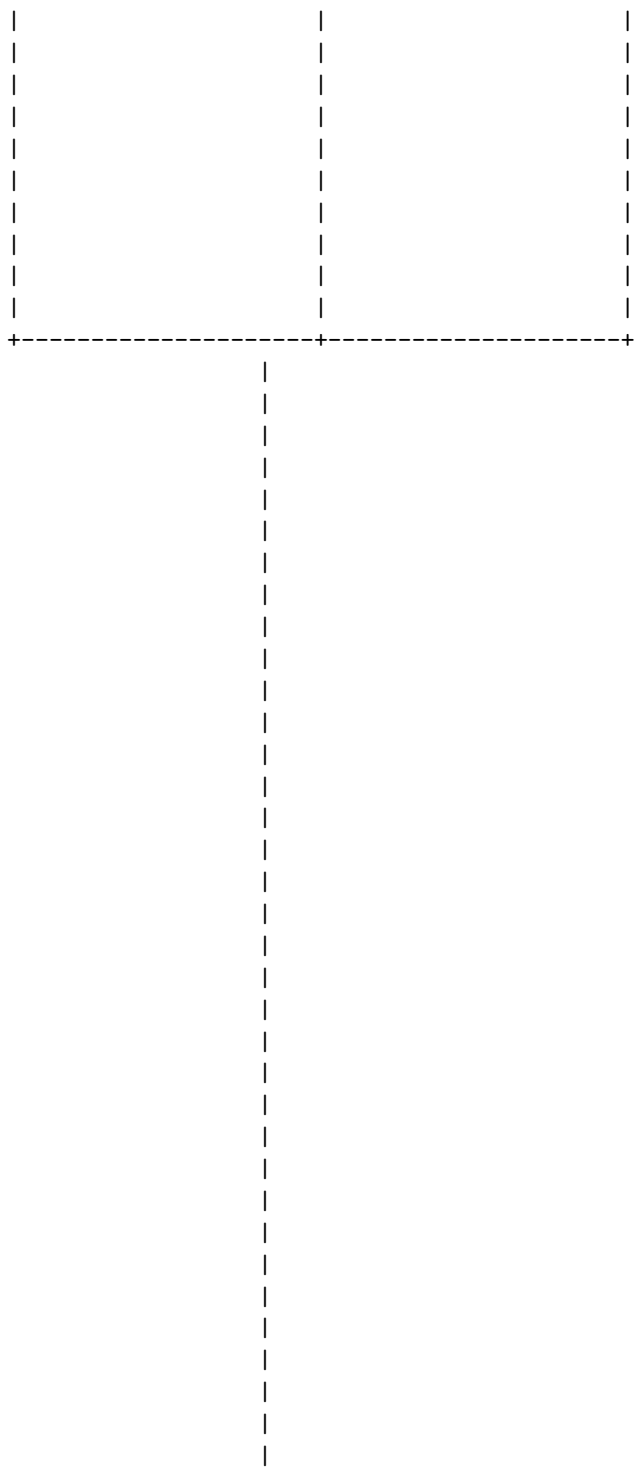
Apache Hadoop is a software framework for storing and processing large datasets of varying sizes and formats across clusters of computers. It follows the master-slave architecture, where the master nodes assign tasks to the slave nodes and monitor their progress. Hadoop consists of two main components: HDFS and MapReduce.

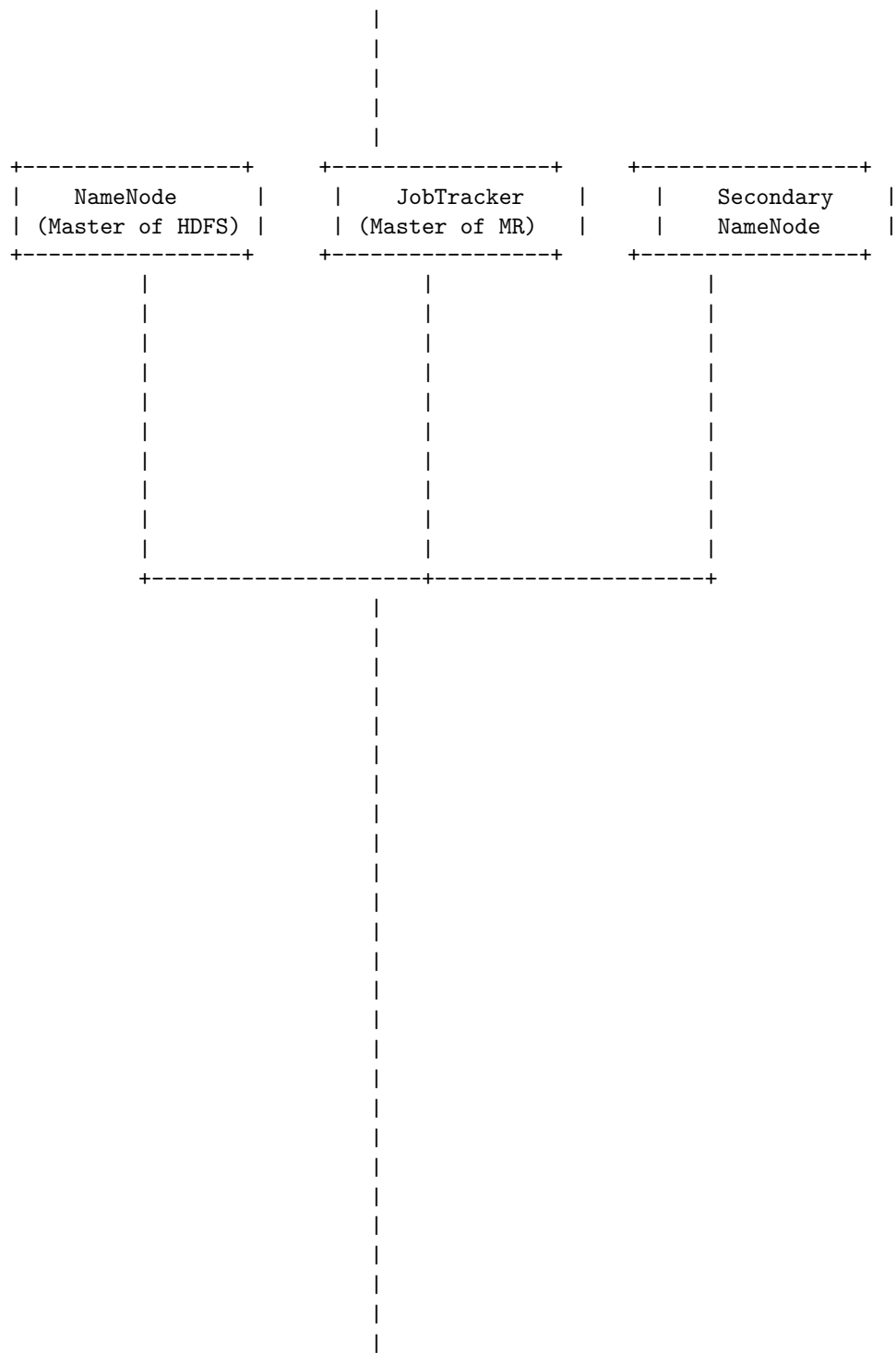
HDFS stands for Hadoop Distributed File System, which is responsible for storing the data blocks across the cluster nodes. HDFS provides fault tolerance, high availability, scalability, and reliability. HDFS has two types of nodes: NameNode and DataNode. NameNode is the master node that manages the metadata of the file system, such as the location of the data blocks, the file permissions, the replication factor, etc. DataNode is the slave node that stores the actual data blocks and communicates with the NameNode.

MapReduce is the programming model for processing the data blocks in parallel. MapReduce has two types of nodes: JobTracker and TaskTracker. JobTracker is the master node that coordinates the execution of the MapReduce jobs, such as splitting the input data, assigning the map and reduce tasks, monitoring the task status, etc. TaskTracker is the slave node that runs the map and reduce tasks on the data blocks and reports to the JobTracker.

The following diagram illustrates the basic architecture of Apache Hadoop:

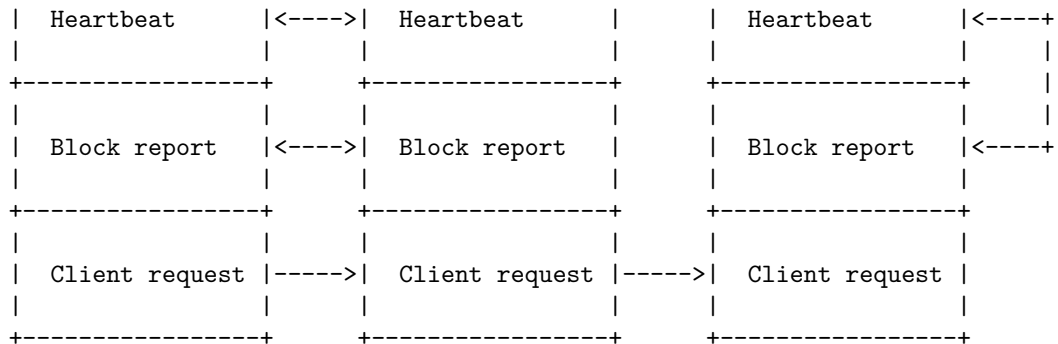








Hadoop Distributed File System

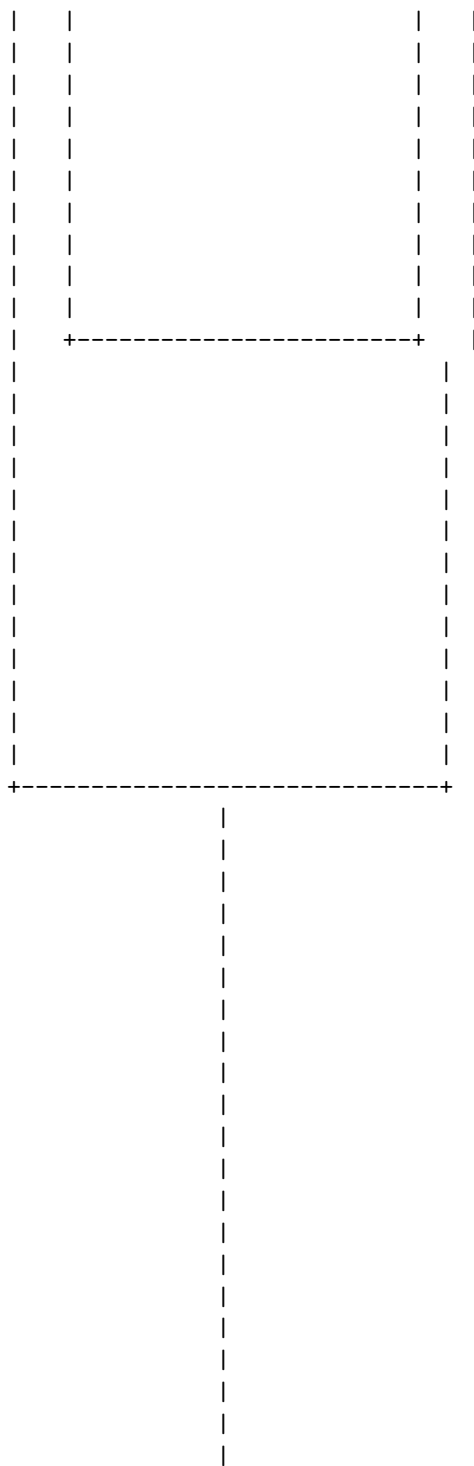


Hadoop is a framework for distributed storage and processing of large-scale data sets. It consists of several components that work together to achieve this goal. The main components of Hadoop are:

- Hadoop Distributed File System (HDFS): This is the storage layer of Hadoop that stores data across multiple nodes in a cluster. It splits the data into blocks and replicates them for fault tolerance. It also provides a namespace and a file system interface for accessing the data. HDFS has two types of nodes: NameNode and DataNode. NameNode is the master node that manages the metadata and the namespace of the file system. DataNode is the worker node that stores and serves the data blocks.
- Hadoop MapReduce: This is the processing layer of Hadoop that implements a programming model for parallel processing of data. It consists of two phases: Map and Reduce. Map phase takes the input data and transforms it into key-value pairs. Reduce phase takes the output of the Map phase and aggregates it based on the keys. MapReduce has two types of nodes: JobTracker and TaskTracker. JobTracker is the master node that coordinates the execution of the MapReduce jobs. TaskTracker is the worker node that runs the map and reduce tasks assigned by the JobTracker.
- Hadoop YARN: This is the resource management layer of Hadoop that allocates and manages the resources (CPU, memory, disk, network) for the applications running on the cluster. It consists of two components: ResourceManager and NodeManager. ResourceManager is the master node that oversees the resource allocation and scheduling of the applications. NodeManager is the worker node that monitors and reports the resource usage and status of the node.

The following diagram illustrates the basic architecture of Hadoop using ASCII characters:





|

|

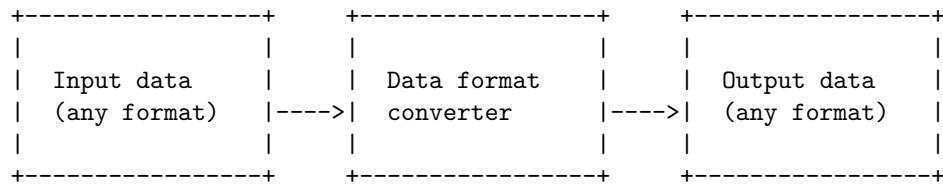
A data format is a way of encoding data so that it can be read, recognized, and used by different applications and programs. There are different types of data formats, such as text, image, audio, video, database, etc. Each data format may

have different specifications, standards, and extensions .

A data format co diagram is a type of data flow diagram (DFD) that shows the flow of information for a process or system that involves data formats. It uses defined symbols like rectangles, circles, and arrows, plus short text labels, to show data inputs, outputs, storage points, and the routes between each destination .

The following diagram illustrates the basic architecture of a data format co system that converts different data formats:

Data format co diagram



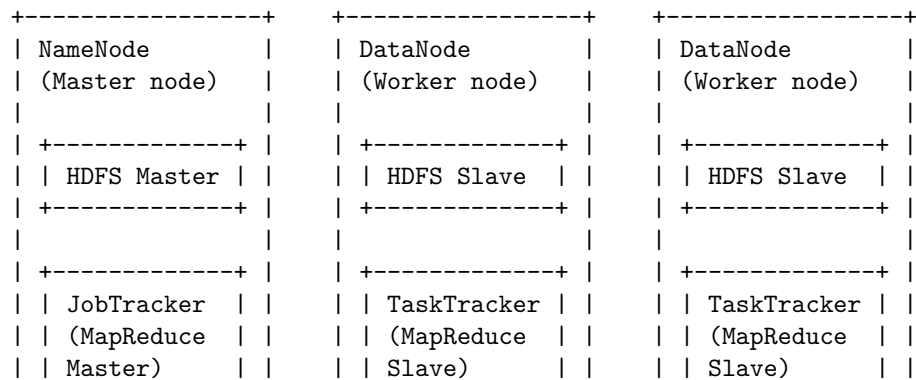
Analyzing data with Hadoop

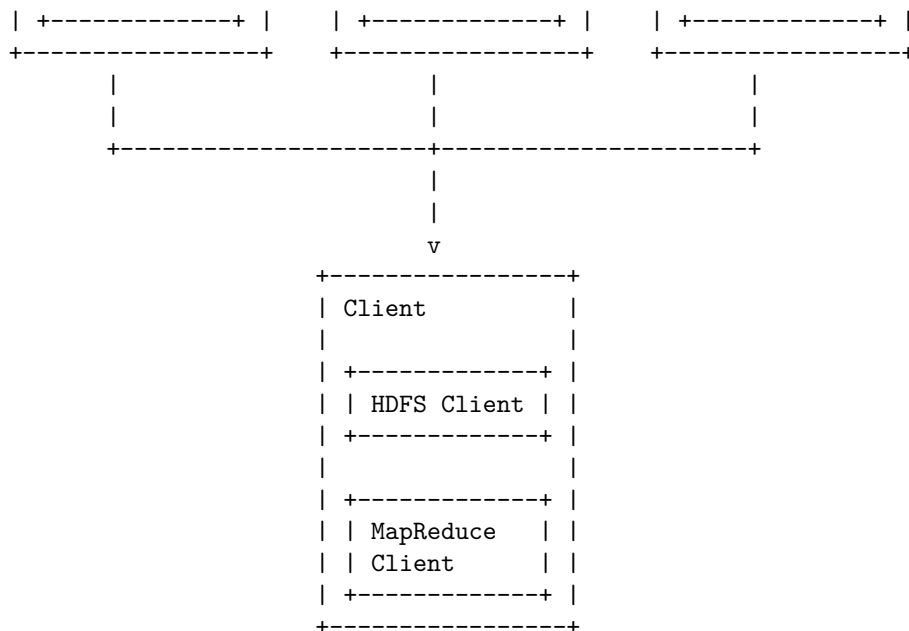
Hadoop is a framework for distributed processing of large-scale data sets using a cluster of computers. Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and MapReduce.

HDFS is a distributed file system that stores data across multiple nodes in a cluster. HDFS provides high availability, fault tolerance, scalability and parallelism.

MapReduce is a programming model that allows users to write applications that process large amounts of data in parallel on a cluster. MapReduce consists of two phases: map and reduce. The map phase takes an input data set and transforms it into a set of key-value pairs. The reduce phase takes the output of the map phase and combines the values associated with the same key.

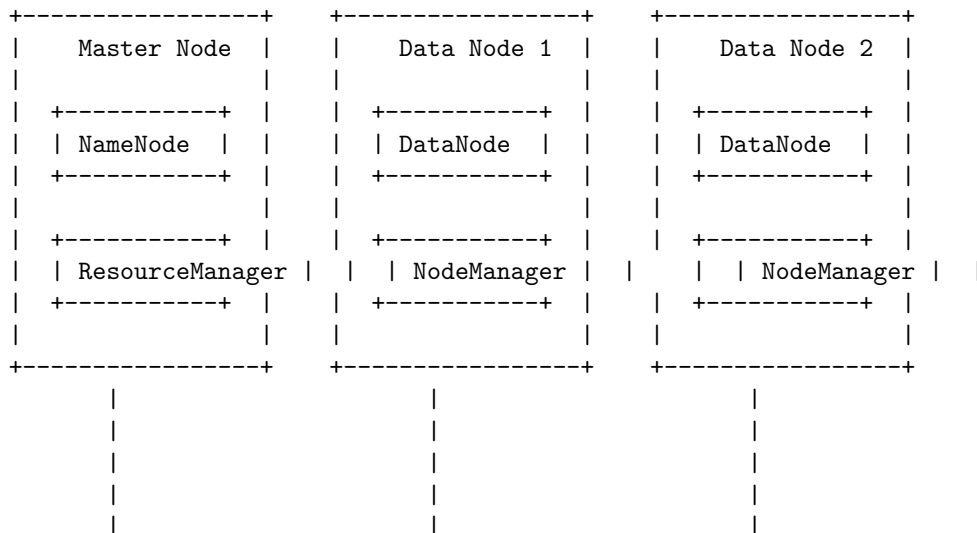
The following diagram illustrates the basic architecture of a Hadoop cluster:

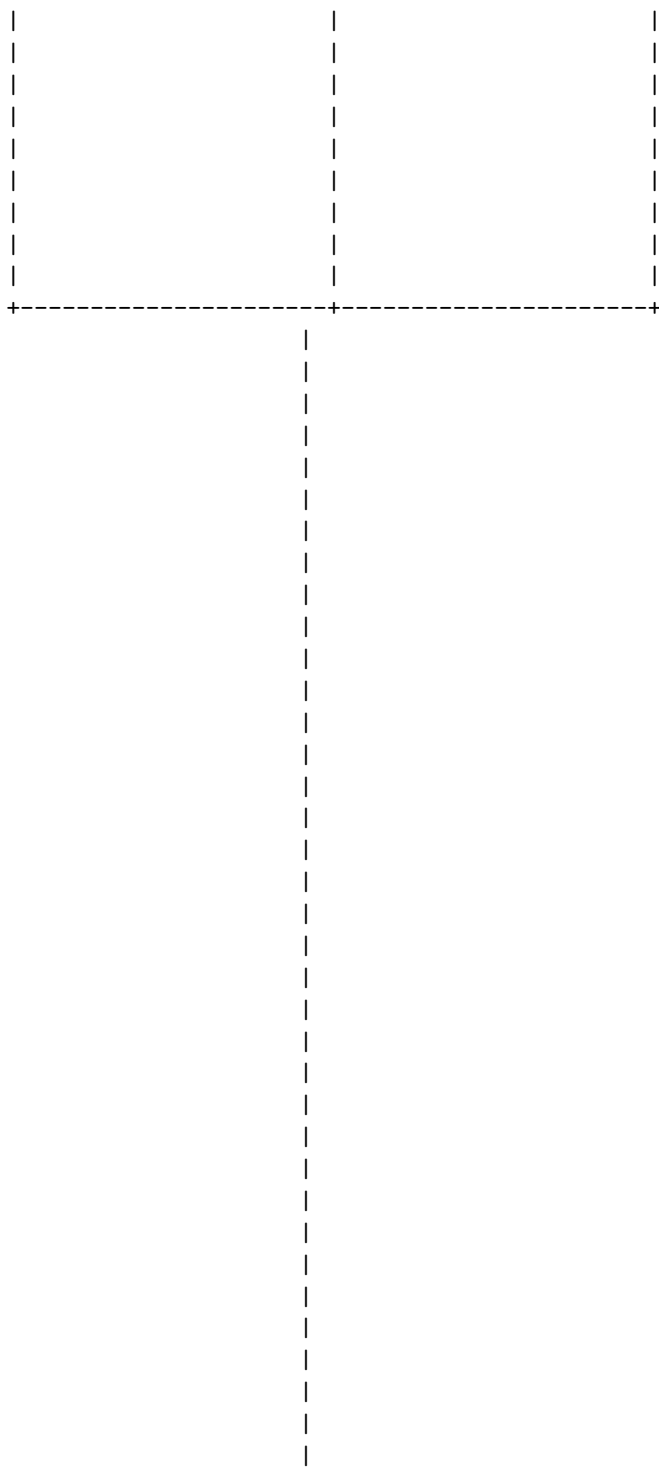




Scaling out with Hadoop means using a distributed file system (such as HDFS) and a resource management system (such as YARN) to store and process large data sets across multiple machines in a cluster. Hadoop can move the computation to the data nodes, rather than moving the data to the computation nodes, which improves performance and scalability.

Scaling out with Hadoop





|

|

|

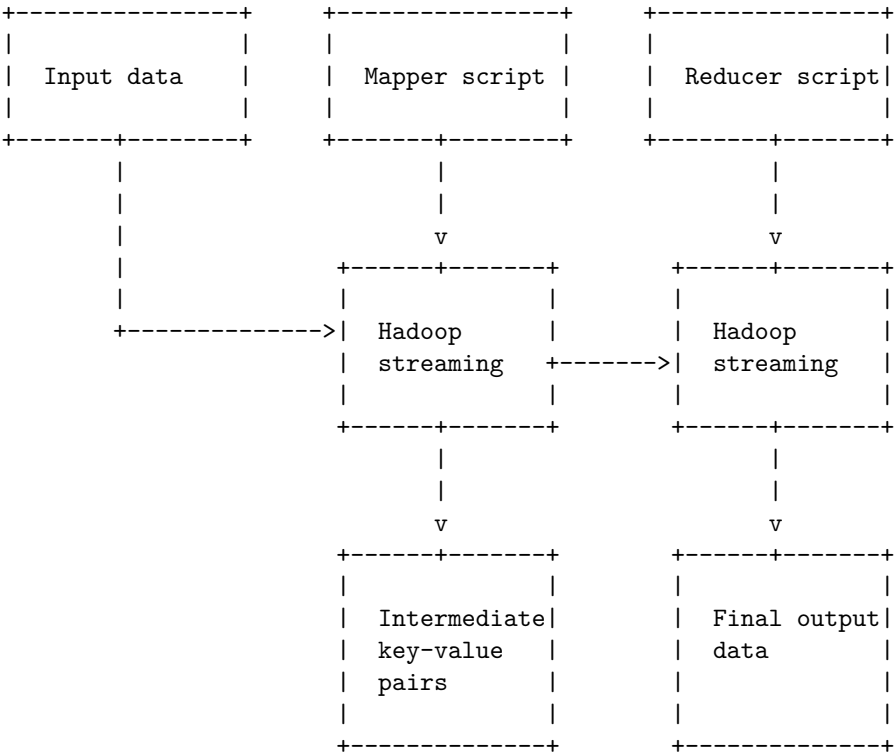
|

|

|
|
|
|
|
|
|

Hadoop streaming is a utility that allows you to create and run MapReduce jobs with any executable or script as the mapper and/or the reducer. It works by passing the input data to the mapper script as standard input, and collecting the output data from the standard output. Similarly, the reducer script receives the intermediate key-value pairs from the standard input, and writes the final output to the standard output. Hadoop streaming handles the communication between the nodes and the partitioning of the data.

Hadoop streaming diagram

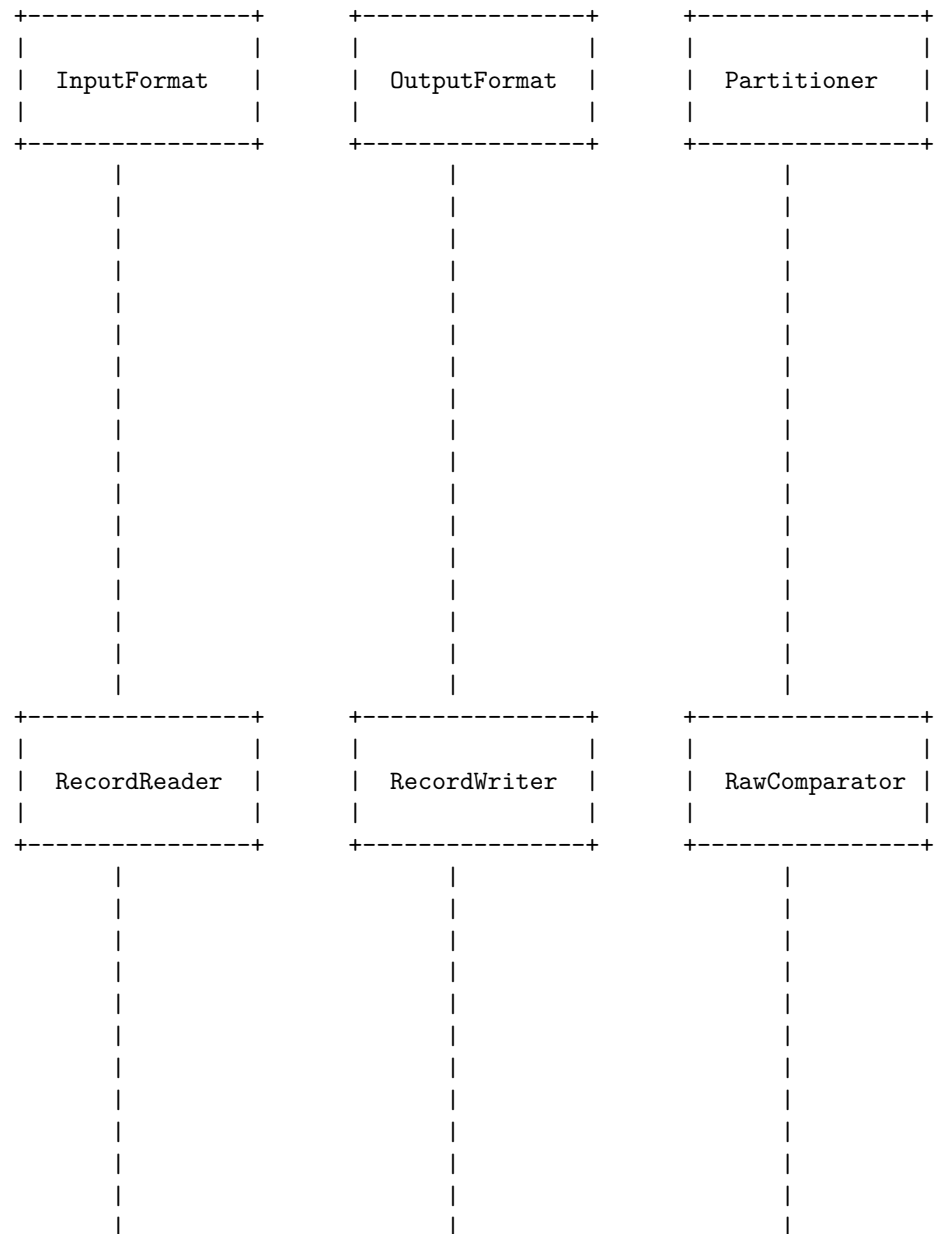


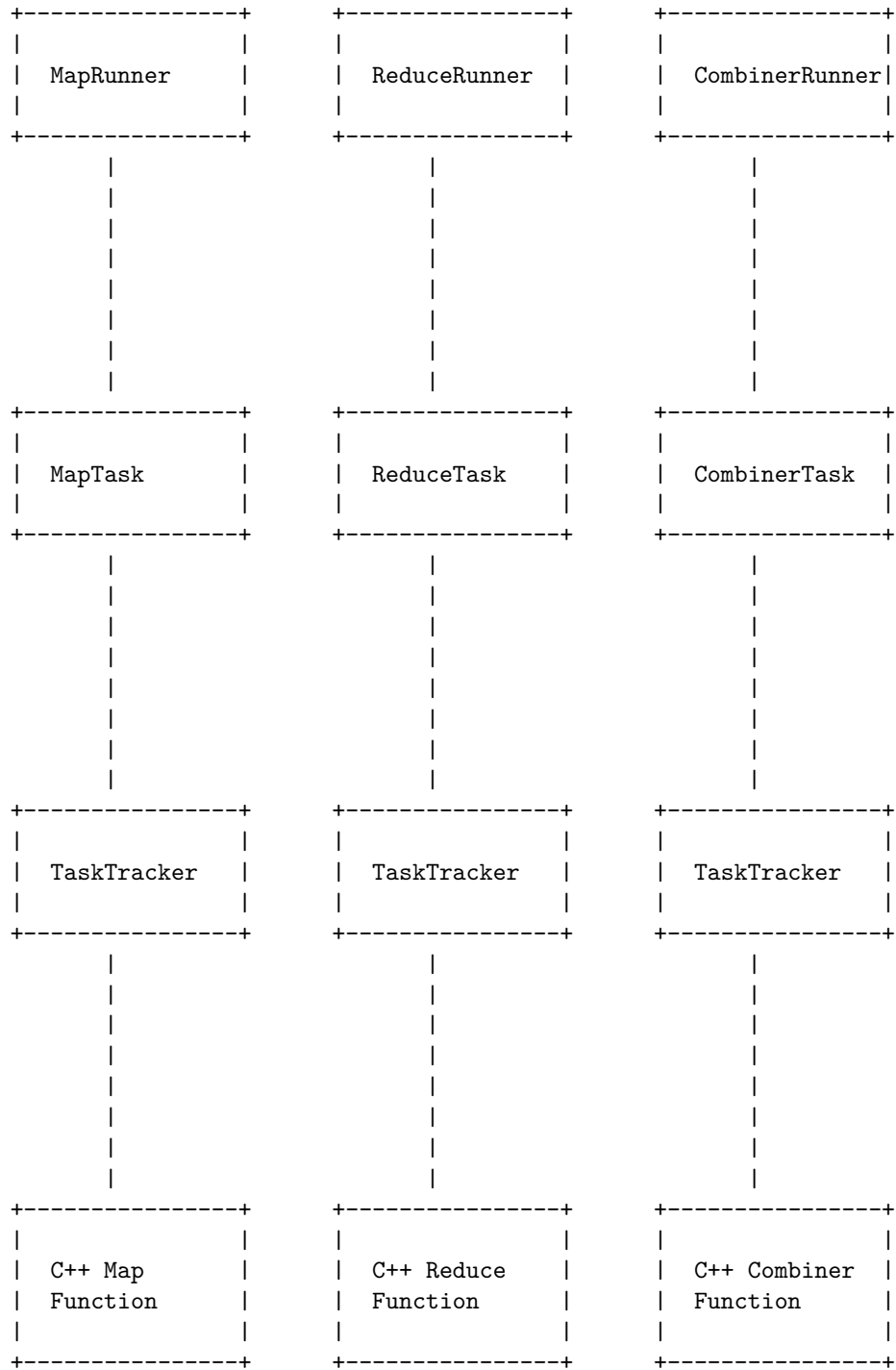
Hadoop Pipes is the name of the C++ interface to Hadoop MapReduce. It allows writing map and reduce functions in C++ and communicating with the Hadoop framework using sockets. Hadoop Pipes uses a protocol buffer-based

wire protocol to exchange messages between the C++ process and the Java task tracker.

Hadoop Pipes

The following is a simplified ASCII diagram of the Hadoop Pipes architecture:



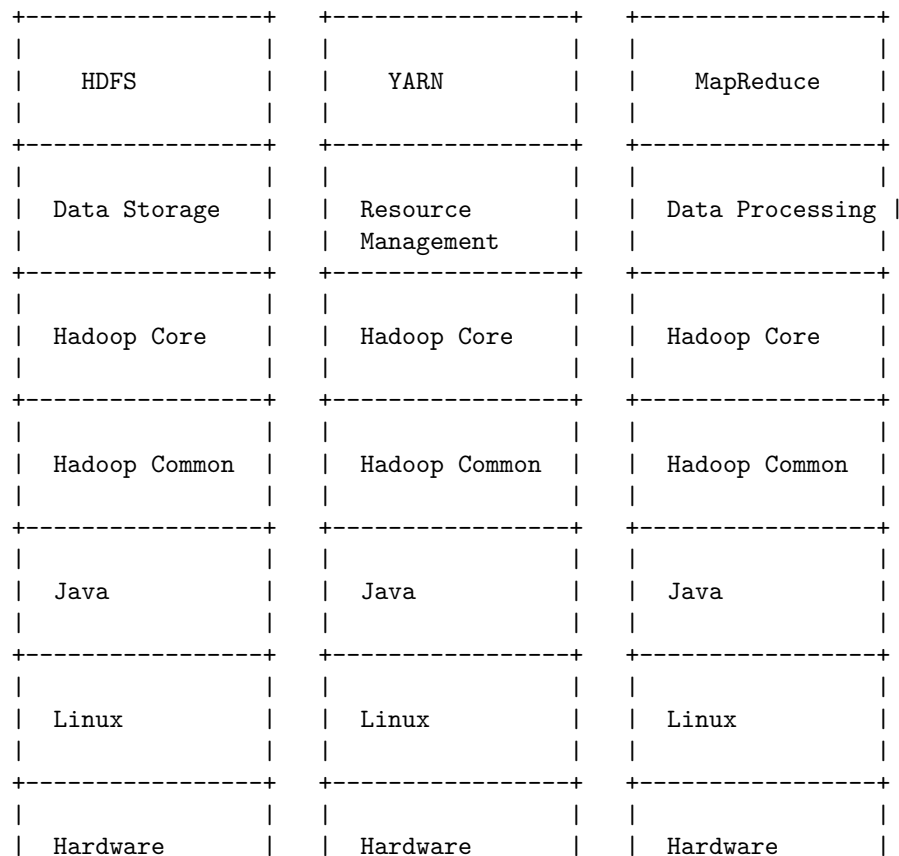


The diagram shows the main components involved in a Hadoop Pipes job. The InputFormat, OutputFormat, Partitioner, RecordReader, RecordWriter, and RawComparator are Java classes that define how the input and output data are formatted, partitioned, read, written, and sorted. The MapRunner, ReduceRunner, and CombinerRunner are Java classes that run the map, reduce, and combiner tasks respectively. The MapTask, ReduceTask, and CombinerTask are Java classes that represent the tasks assigned by the JobTracker to the TaskTrackers. The TaskTrackers are Java processes that run on the cluster nodes and execute the tasks. The C++ Map

Hadoop Ecosystem is a collection of tools, libraries, and frameworks that help you build applications on top of Apache Hadoop. Hadoop provides massive parallelism with low latency and high throughput, which makes it well-suited for big data problems.

The following is an ASCII diagram that illustrates the basic architecture of a Hadoop Ecosystem:

Hadoop Ecosystem

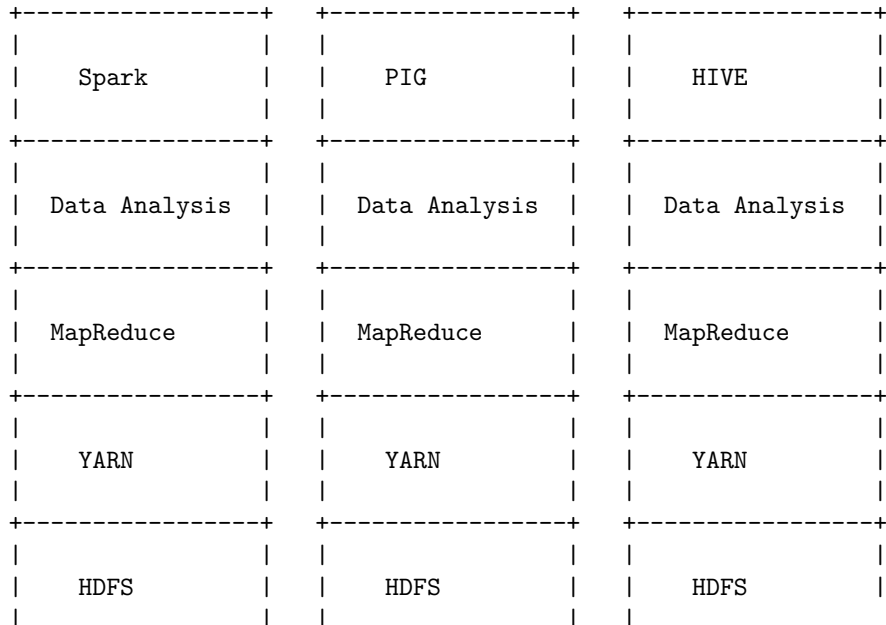


--	--	--	--

On top of the Hadoop Core, there are various tools and frameworks that provide different functionalities and features for data analysis, such as:

- Spark: In-Memory data processing
- PIG, HIVE: Query based processing of data services
- HBase: NoSQL Database
- Sqoop: Data transfer between Hadoop and relational databases
- Flume: Data ingestion from various sources to Hadoop
- Kafka: Distributed messaging system
- Oozie: Workflow scheduler for Hadoop jobs
- Zookeeper: Distributed coordination service
- Mahout: Machine learning library
- Impala: SQL engine for Hadoop

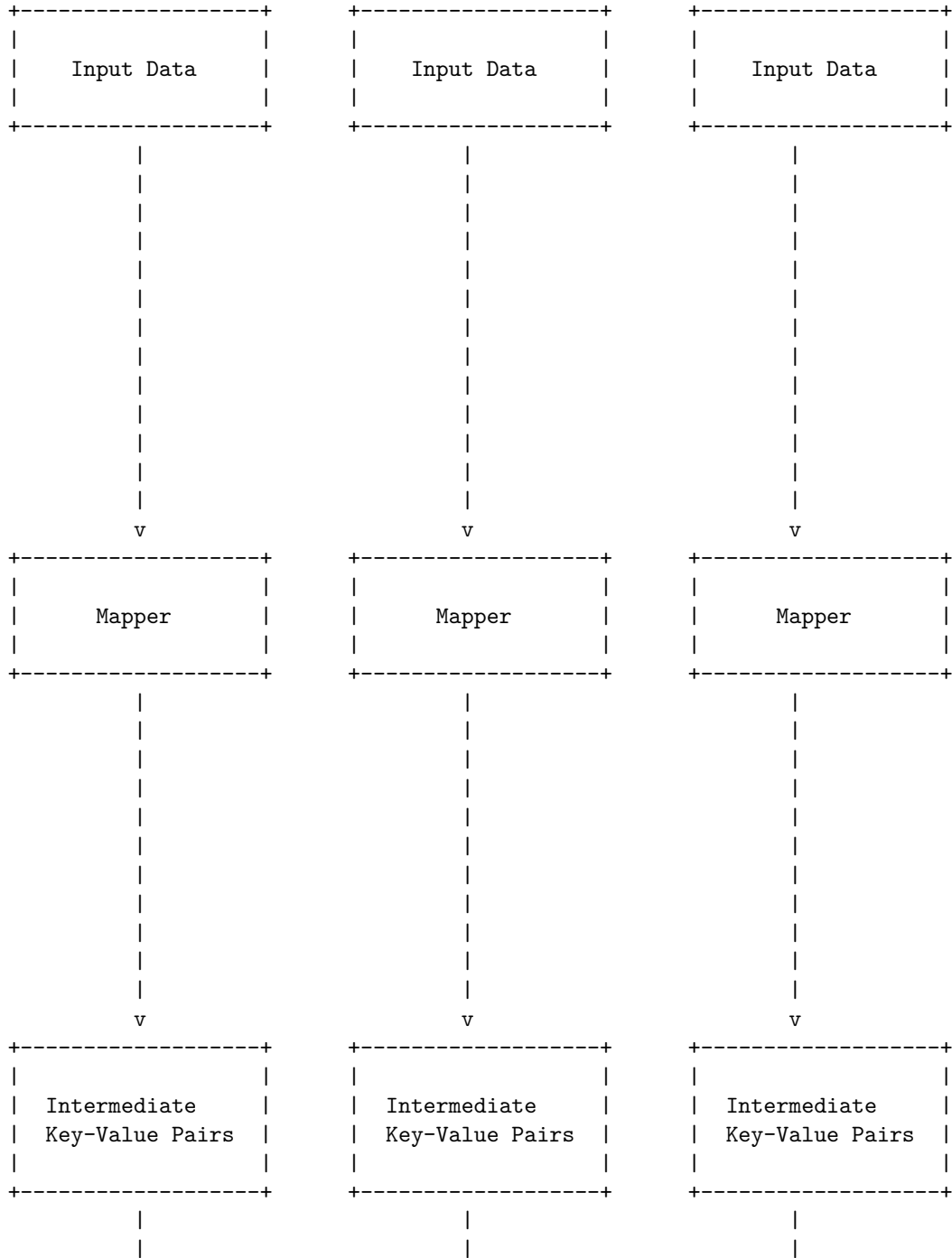
The following is an ASCII diagram that illustrates the relationship between some of these tools and frameworks in the Hadoop Ecosystem:

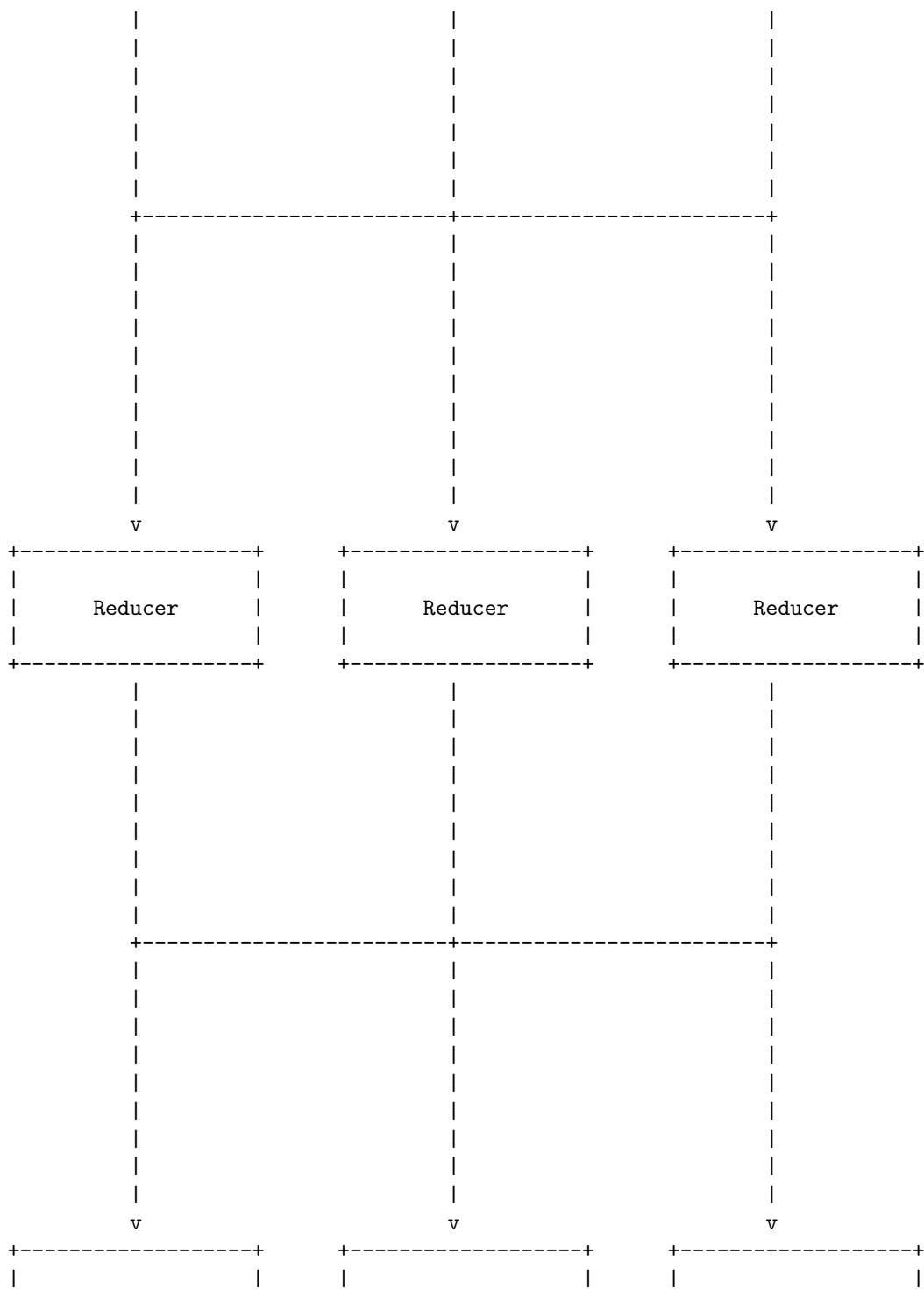


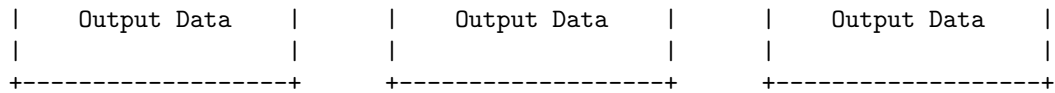
MapReduce is a programming model and a software framework for processing large amounts of data in parallel across a cluster of nodes. It consists of two phases: Map and Reduce. The Map phase applies a user-defined function to each input record and produces a set of intermediate key-value pairs. The Reduce phase aggregates the intermediate values associated with the same key and produces the final output.

The following diagram illustrates the basic architecture of a MapReduce job:

Map Reduce





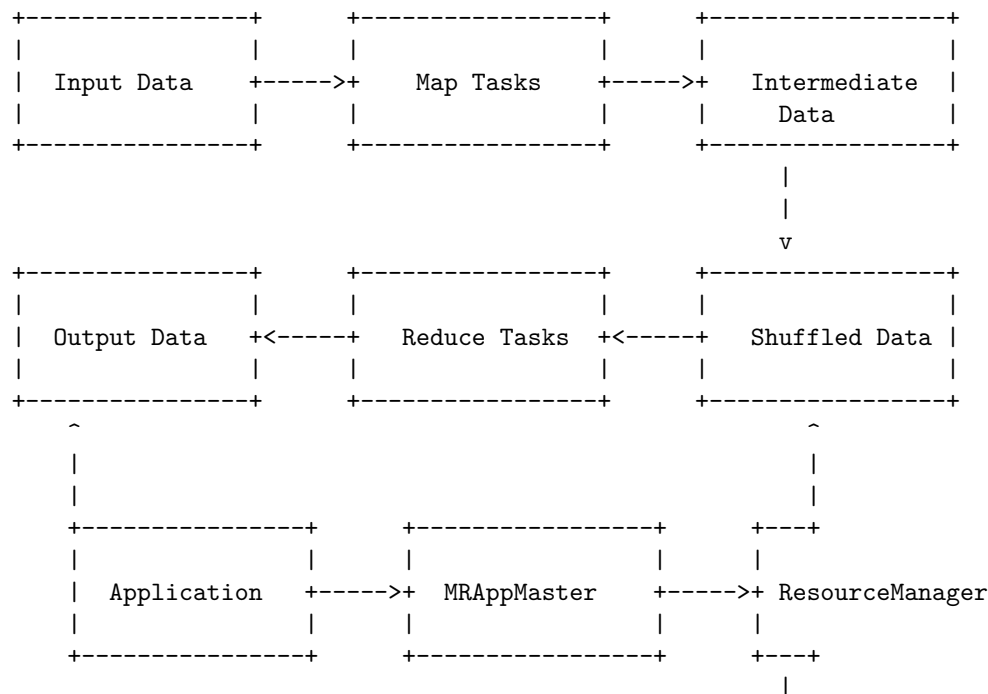


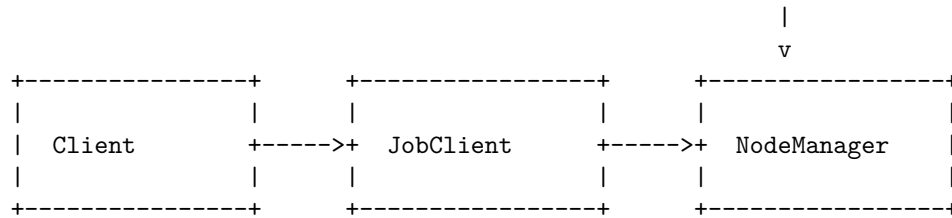
The MapReduce framework consists of a single master node called the JobTracker and multiple slave nodes called the TaskTrackers. The JobTracker is responsible for scheduling the jobs' component tasks on the TaskTrackers, monitoring them and re-executing the failed tasks. The TaskTrackers execute the tasks as directed by the JobTracker. [^5]

MapReduce is a software framework and programming model used for processing huge amounts of data in a distributed and parallel fashion over a cluster of machines . MapReduce program work in two phases, namely, Map and Reduce. Map tasks deal with splitting and mapping of data while Reduce tasks shuffle and reduce the data .

The MapReduce framework consists of a single master ResourceManager, one worker NodeManager per cluster-node, and MRAppMaster per application. The ResourceManager is responsible for allocating resources and scheduling tasks. The NodeManager is responsible for launching and monitoring the tasks on each node. The MRAppMaster is responsible for coordinating the execution of a MapReduce job.

Map Reduce framework and basics





: <https://www.guru99.com/introduction-to-mapreduce.html>

<https://hadoop.apache.org/docs/current/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>

<https://www.talend.com/resources/what-is-mapreduce/>

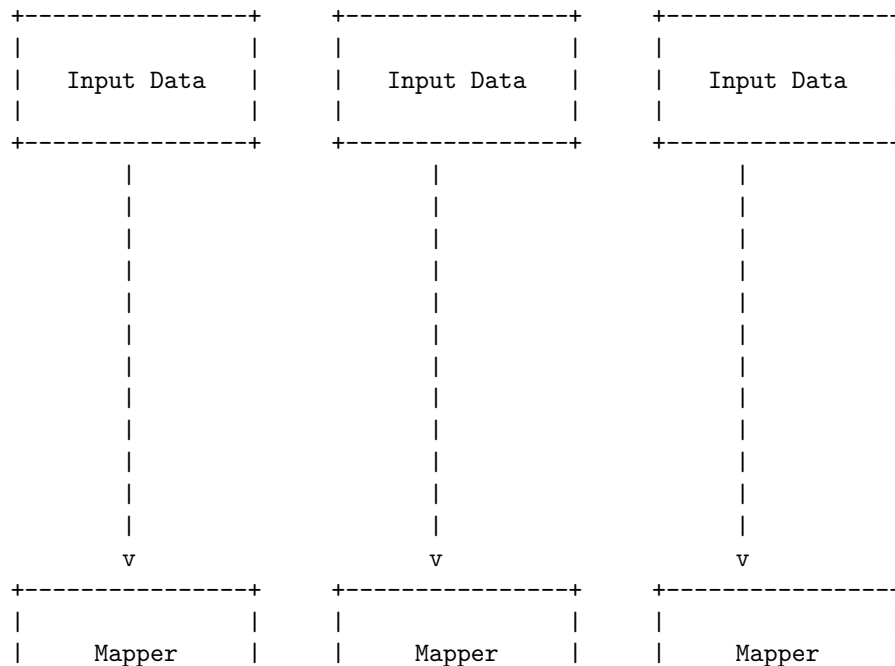
<https://www.edureka.co/blog/mapreduce-tutorial/>

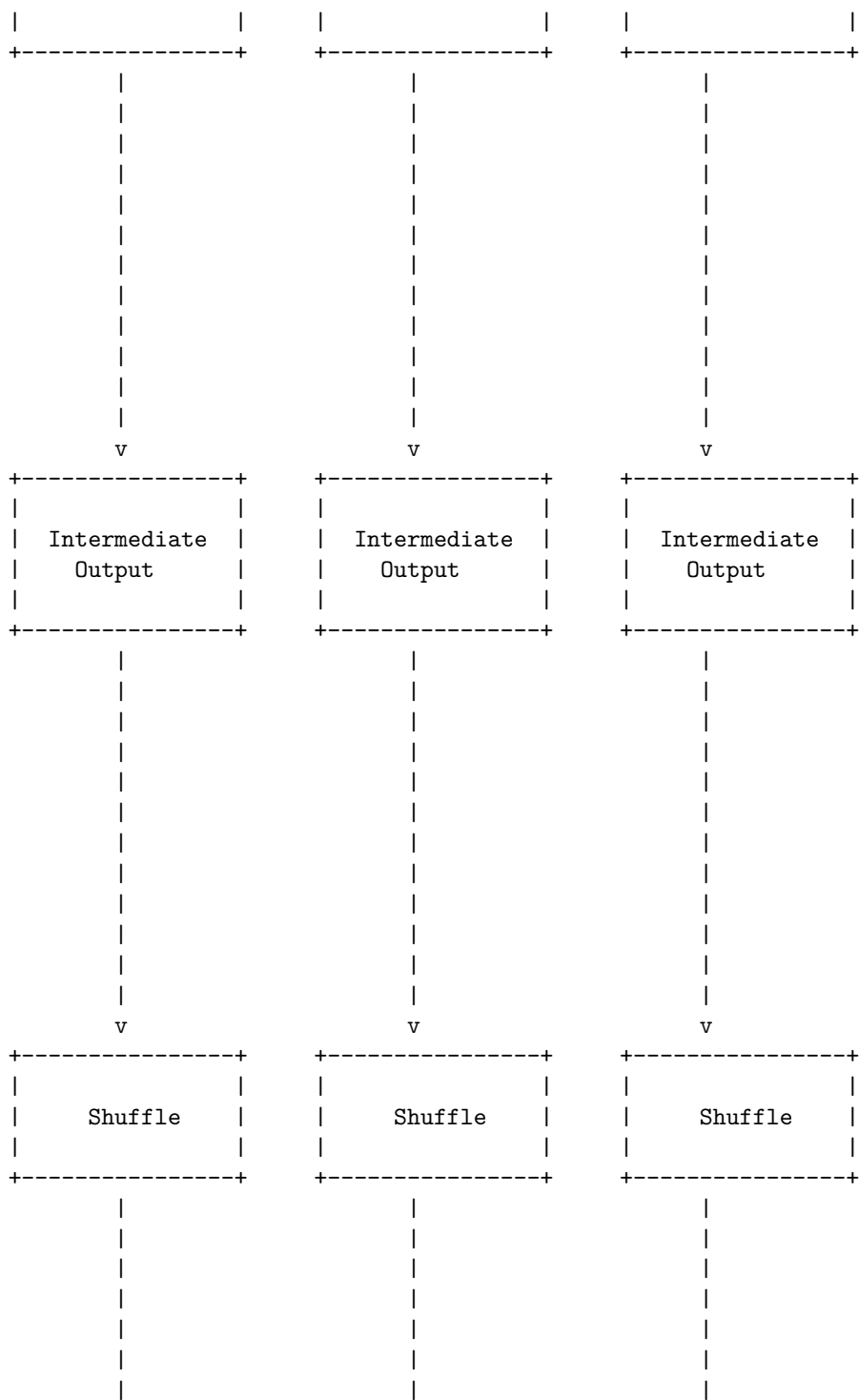
https://hci.stanford.edu/courses/cs448g/a2/files/map_reduce_tutorial.pdf

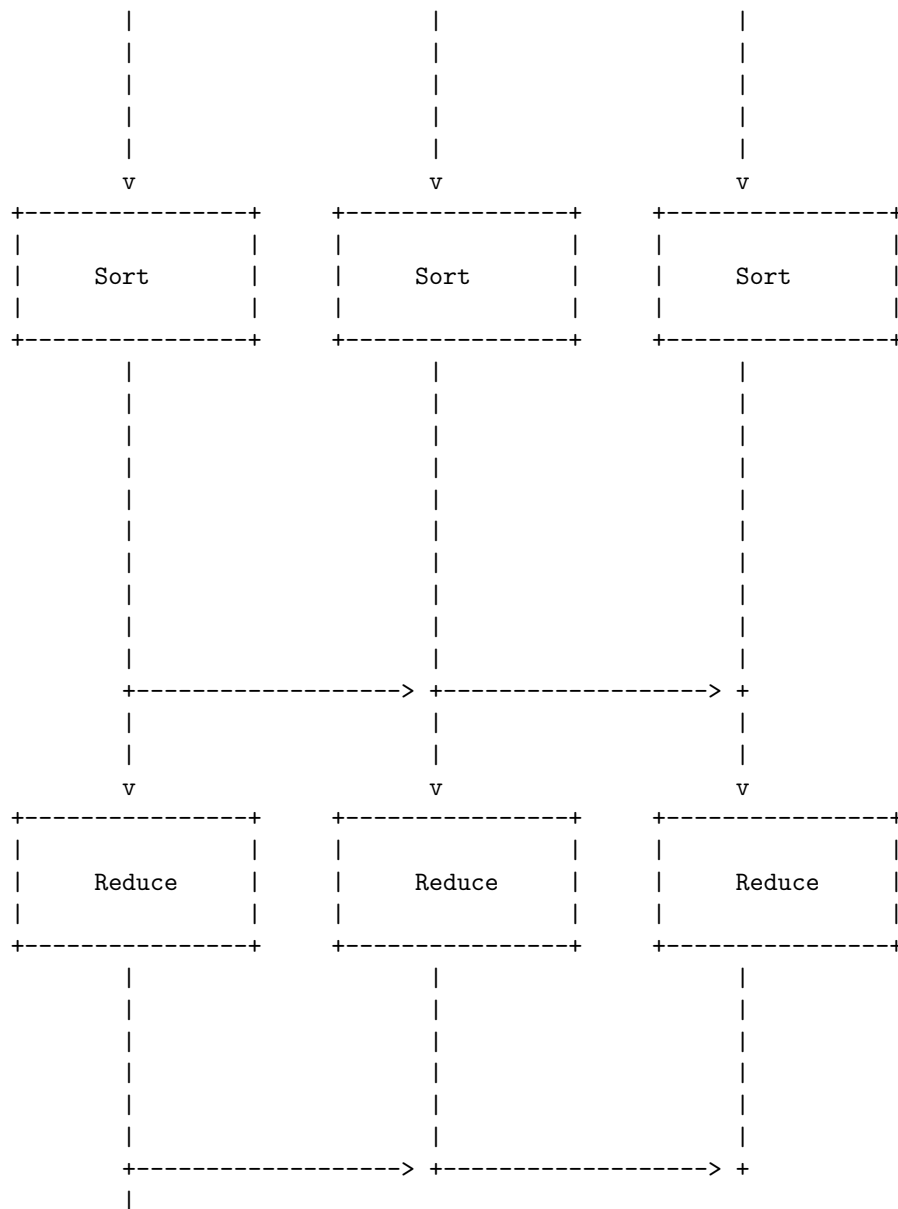
MapReduce is a programming model and an associated implementation for processing and generating big data sets with a parallel, distributed algorithm on a cluster. It consists of two phases: Map and Reduce. The Map phase takes a set of data and converts it into another set of data, where individual elements are broken down into key-value pairs. The Reduce phase takes the output from the Map phase and merges those data tuples into a smaller set of tuples.

How MapReduce works

The following is a detailed ASCII diagram for how MapReduce works:



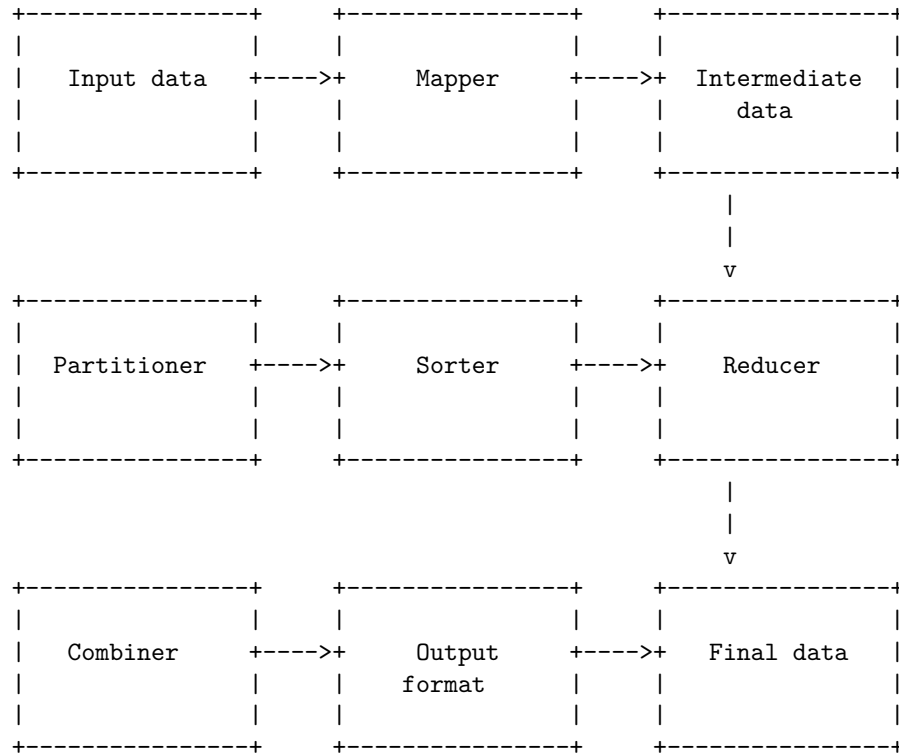




Developing a Map Reduce application

A Map Reduce application consists of two main components: a mapper and a reducer. The mapper takes an input key-value pair and produces a set of intermediate key-value pairs. The reducer takes the intermediate key-value pairs with the same key and combines them into a final output value.

The following diagram illustrates the basic architecture of a Map Reduce application:



The input data is split into chunks and distributed across multiple nodes in a cluster. Each node runs a mapper function on its chunk of data and emits intermediate key-value pairs. The partitioner function determines which node will receive which intermediate key-value pairs based on the key. The sorter function sorts the intermediate key-value pairs by key. The reducer function takes the sorted intermediate key-value pairs with the same key and applies a user-defined function to produce the final output value. The combiner function is an optional optimization that can reduce the amount of data transferred between the mapper and the reducer by performing some local aggregation. The output format function specifies how the final output value will be stored or displayed. The final data is the result of the Map Reduce application.

Unit tests with MR unit are a way of testing Hadoop MapReduce jobs using a JUnit-based Java library called MRUnit. MRUnit allows you to create test input, run it through your mapper and/or reducer, and verify the output all in a JUnit test. This helps you to debug your code and ensure its correctness.

The following diagram illustrates the basic architecture of a unit test with MR unit:

Test Input Data	-->	Mapper/Reducer	-->	Expected Output
v		v		v
InputSplit	-->	MapDriver	-->	OutputCollector
		or		
RecordReader	-->	ReduceDriver	-->	OutputVerifier
		or		
InputFormat	-->	MapReduceDriver	-->	JUnit Assert

The test input data is a set of key-value pairs that represent the input to the mapper or reducer. The expected output is another set of key-value pairs that represent the expected output from the mapper or reducer. The input split, record reader, and input format are classes that handle the reading and parsing of the input data. The map driver, reduce driver, or map reduce driver are classes that simulate the execution of the mapper, reducer, or both. The output collector, output verifier, and JUnit assert are classes that collect, verify, and assert the output of the mapper or reducer. The MRUnit library provides these classes and methods to make it easy to write unit tests for Hadoop MapReduce jobs.

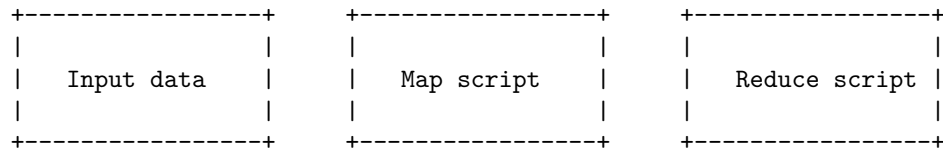
Test data and local tests in map reduce

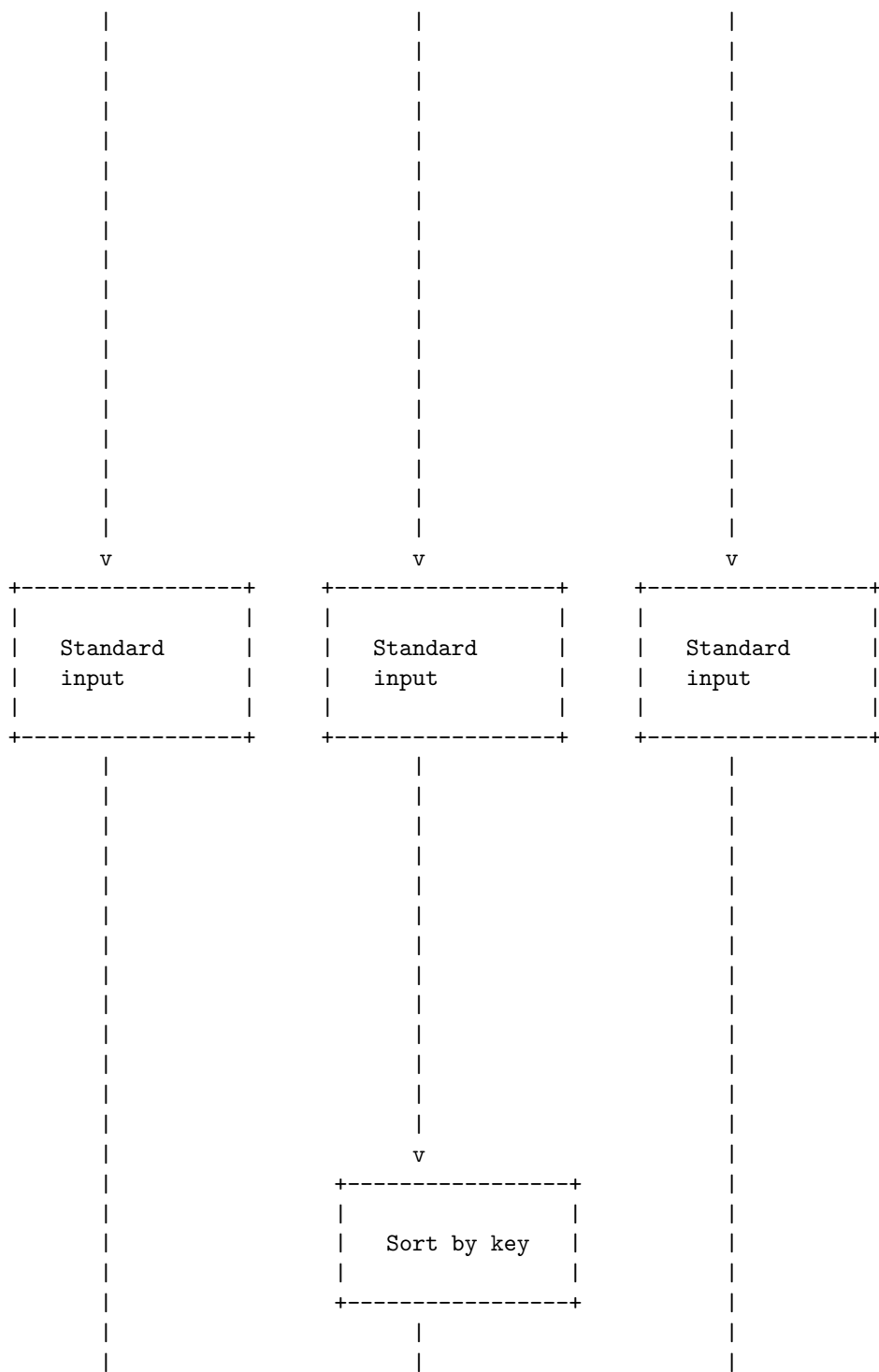
One way to test map and reduce code locally is to use hadoop streaming, which allows you to write map and reduce scripts in any language that can read from standard input and write to standard output. For example, if you have a map.py and a reduce.py script in Python, you can test them locally by running the following command:

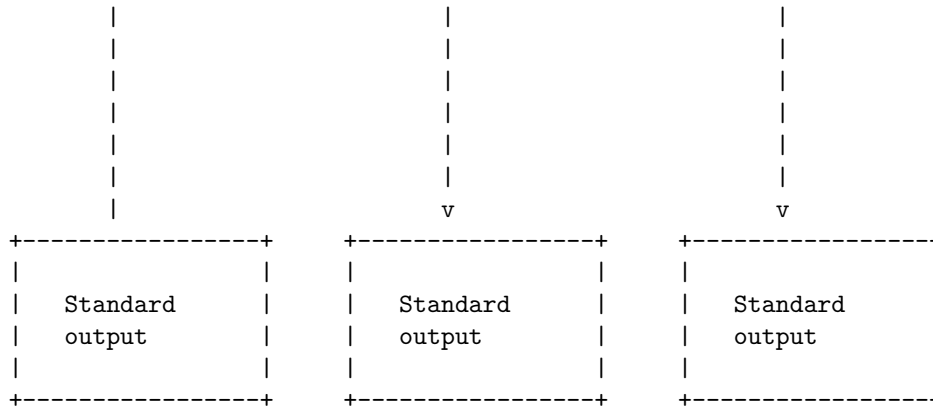
```
cat *.csv | map.py | sort -k1,1 | reduce.py
```

This will simulate the map and reduce phases of a map reduce job, using the csv files in the current directory as the input data. The sort command is necessary to group the key-value pairs by key before passing them to the reducer. The output of the reduce script will be printed to the standard output.

The following diagram illustrates the basic architecture of a map reduce job using hadoop streaming:



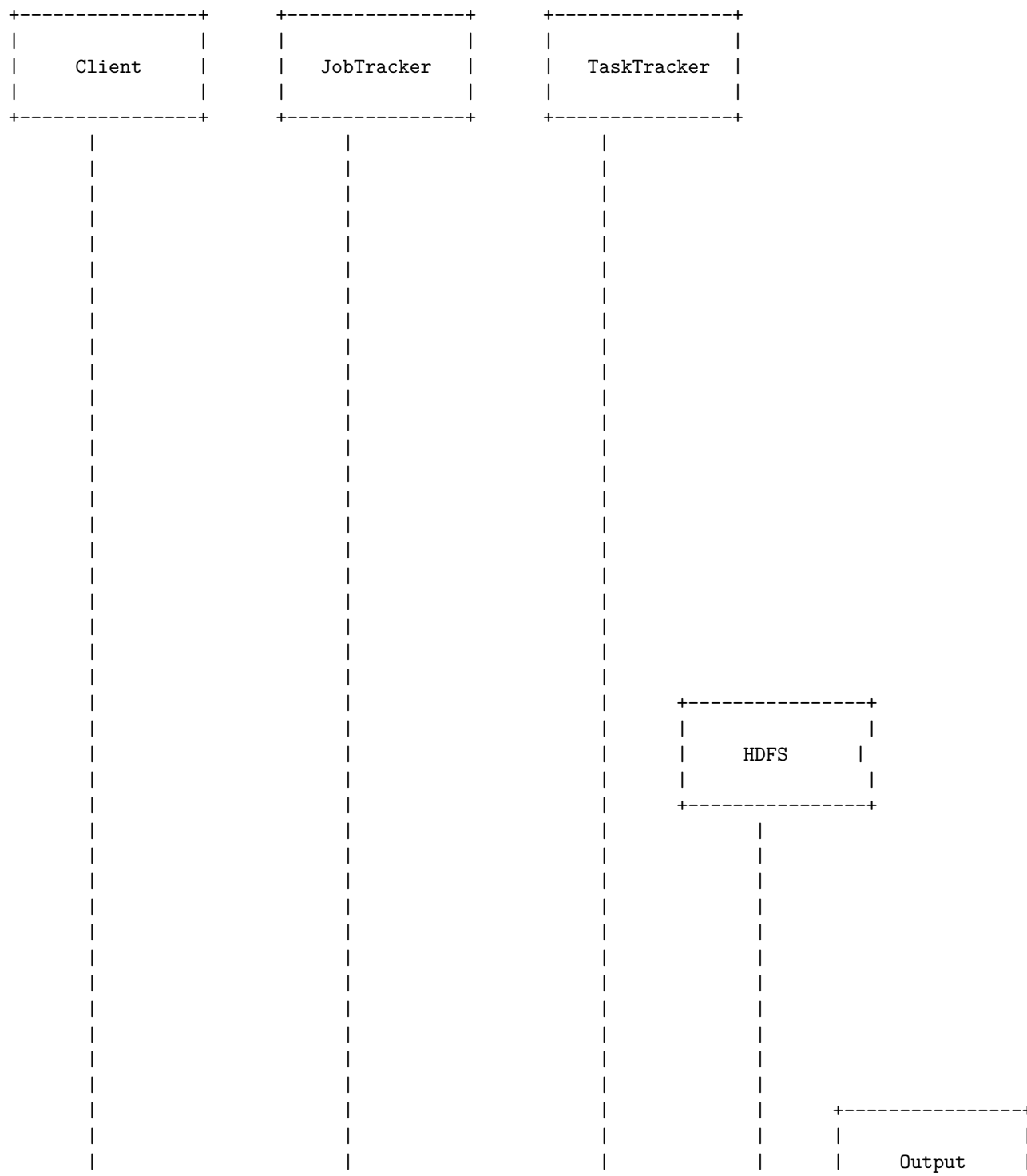


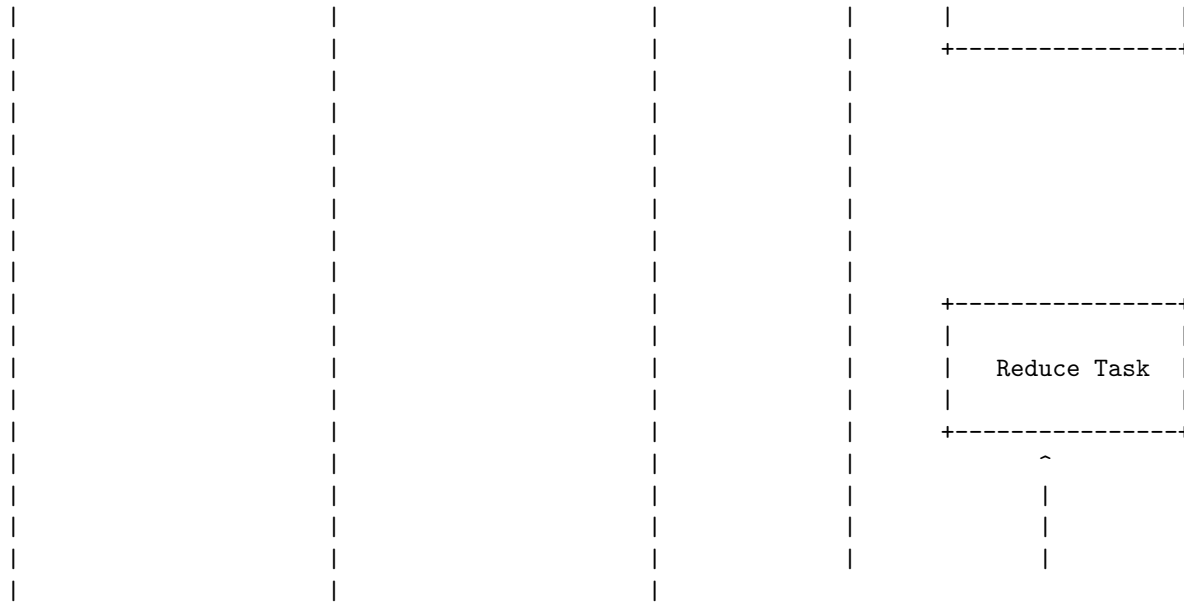


A MapReduce job run consists of the following steps:

1. The client submits the job to the JobTracker, which is the master node that coordinates the execution of the job.
2. The JobTracker splits the input data into fixed-size chunks called input splits, and assigns a map task to each split. The input splits are stored in the Hadoop Distributed File System (HDFS), which is a distributed file system that replicates the data across multiple nodes for fault tolerance and high availability.
3. The map tasks read the input splits and apply a user-defined map function to each record. The map function transforms the input records into intermediate key-value pairs, which are written to a local disk on the same node where the map task is running.
4. The map tasks periodically report their progress and status to the JobTracker, which monitors the health and availability of the map tasks. If a map task fails or times out, the JobTracker can reassign the task to another node.
5. The JobTracker also partitions the intermediate key-value pairs into a fixed number of reduce tasks, based on a user-defined partitioning function. The partitioning function determines which reduce task is responsible for processing a given key.
6. The reduce tasks fetch the intermediate key-value pairs from the local disks of the map tasks, using a process called shuffle. The shuffle involves transferring data over the network, sorting and merging the data by key, and storing the data in the memory or disk of the reduce task node.
7. The reduce tasks apply a user-defined reduce function to each group of values that share the same key. The reduce function aggregates, filters, or transforms the values into a final output, which is written to the HDFS.
8. The JobTracker notifies the client when the job is completed, and the client can retrieve the output from the HDFS.

Anatomy of a MapReduce job run





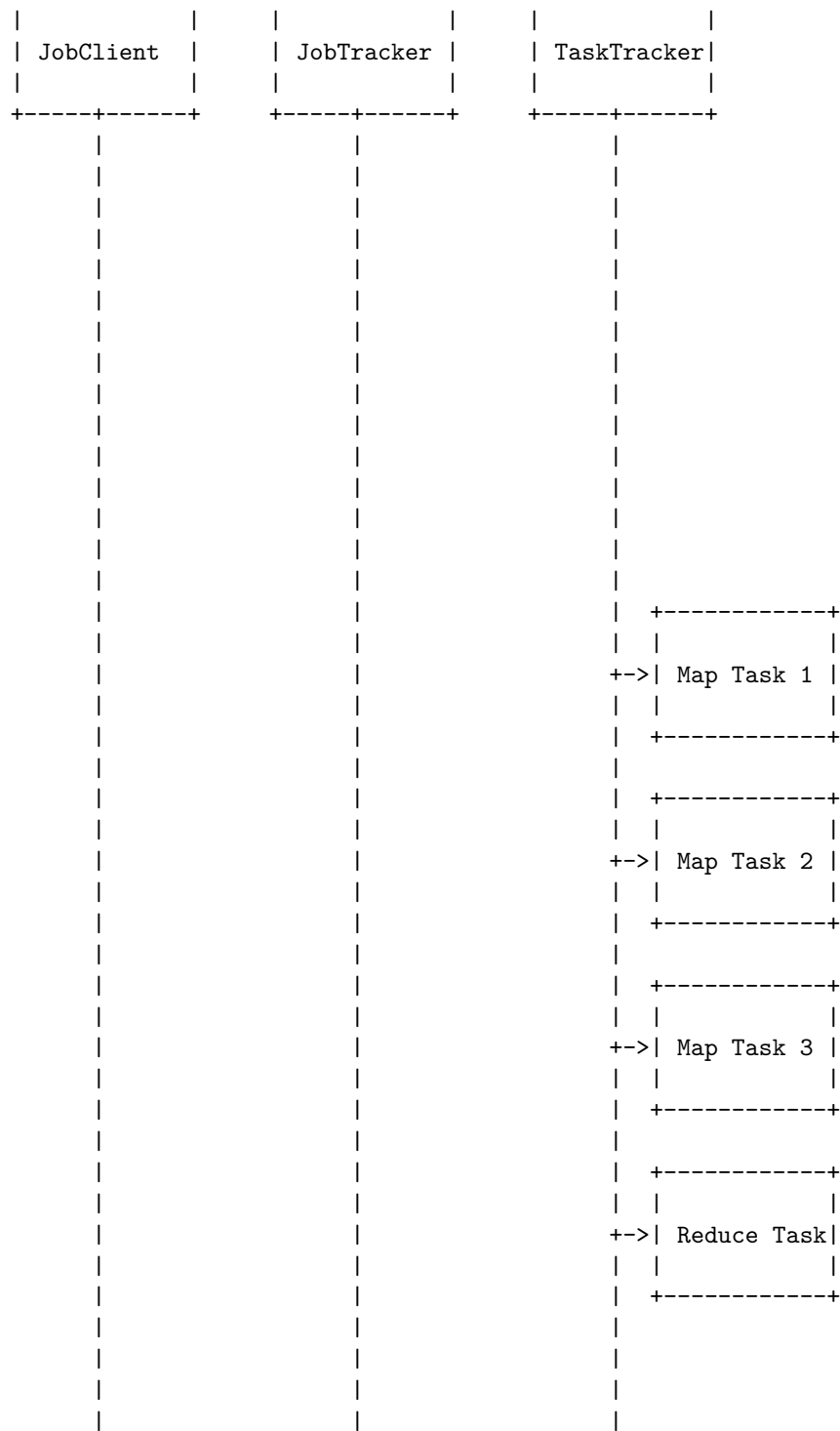
MapReduce is a programming model and framework for processing large-scale data sets in parallel using a cluster of commodity machines. MapReduce consists of two phases: map and reduce. In the map phase, each input data split is assigned to a map task that transforms it into a set of intermediate key-value pairs. In the reduce phase, the intermediate key-value pairs are shuffled and sorted by key, and then assigned to a reduce task that aggregates them by key and produces the final output.

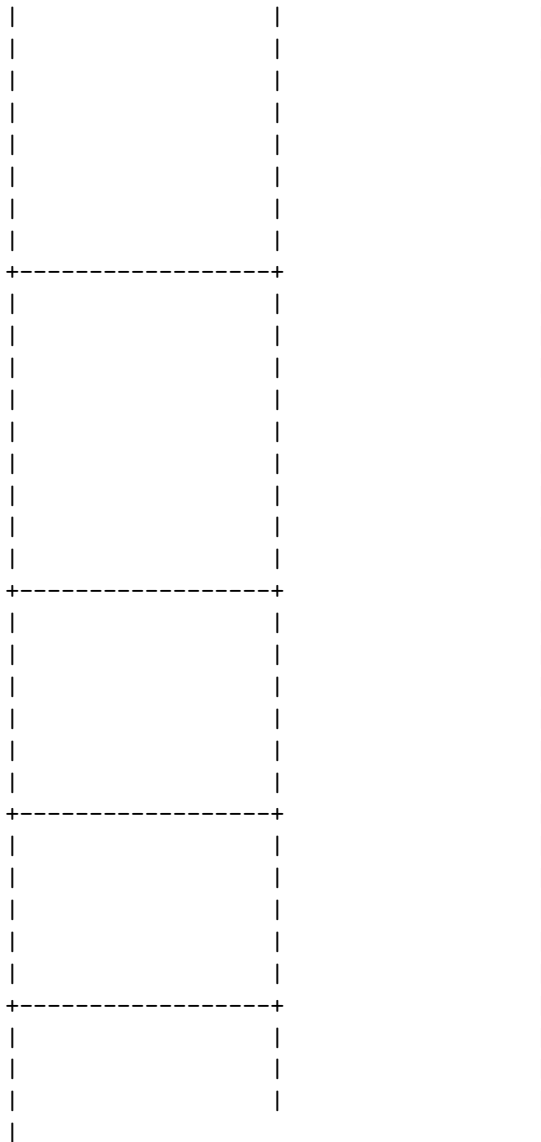
Failures in MapReduce can occur at different levels: task, node, and cluster. Task failures are caused by errors or exceptions in the user code, such as bad records, divide by zero, null pointer, etc. Node failures are caused by hardware or software failures, such as CPU/memory/disk failure, network partition, power outage, etc. Cluster failures are caused by catastrophic events, such as natural disasters, fire, etc.

MapReduce has a built-in mechanism to handle failures gracefully and transparently. The master node, called the JobTracker, monitors the status of the worker nodes, called the TaskTrackers, and the progress of the map and reduce tasks. If a task fails, the JobTracker will retry the task on another TaskTracker, up to a maximum number of attempts (default is 4). If a TaskTracker fails, the JobTracker will reassign all the tasks that were running or completed on that node to other TaskTrackers. If the JobTracker fails, the whole job will fail and need to be restarted manually.

The following diagram illustrates the basic architecture of a MapReduce job and the possible failure scenarios:

+-----+ +-----+ +-----+





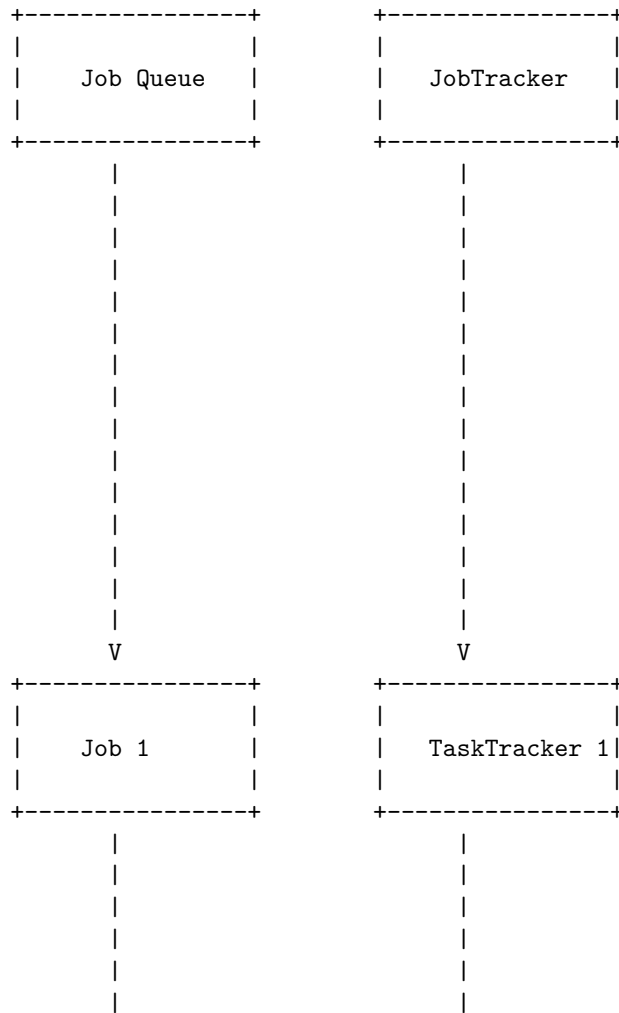
Job scheduling in MapReduce is the process of assigning tasks to different nodes in a cluster, based on the availability of resources and the compatibility of tasks. The basic steps of job scheduling in MapReduce are:

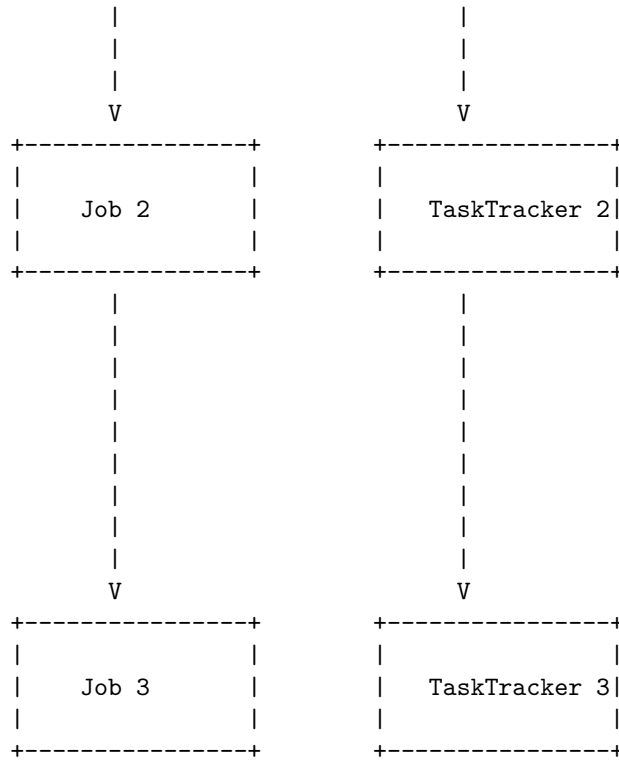
- The user submits a job to a queue, which contains a set of map and reduce tasks that operate on the input data.
- The master node, also known as the JobTracker, distributes the tasks to different worker nodes, also known as the TaskTrackers, based on their availability and capacity.
- The map tasks read the data splits from the input file system, and run

the map function on each record. The map function produces a set of intermediate key-value pairs, which are stored in the local file system of the map node.

- The reduce tasks are assigned to different nodes based on the partitioning of the intermediate keys. The reduce tasks fetch the intermediate values from the map nodes, and run the reduce function on each key-value pair. The reduce function produces the final output, which is stored in the output file system.
- The JobTracker monitors the progress of the tasks, and re-executes the failed or slow tasks on different nodes if necessary. The JobTracker also notifies the user about the status of the job.

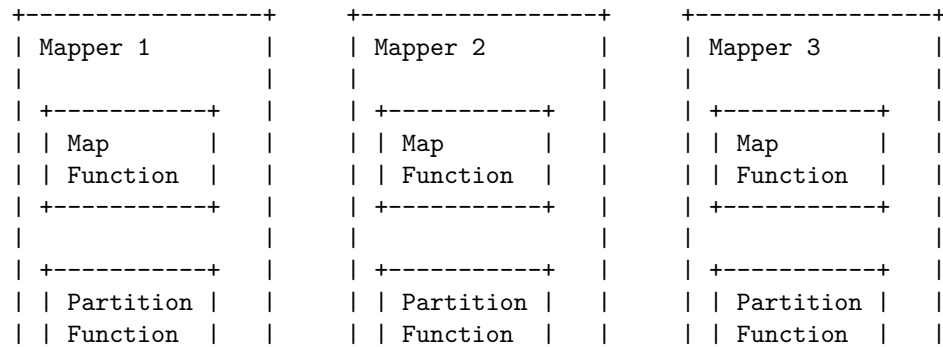
The following diagram illustrates the basic architecture of a job scheduling in MapReduce:

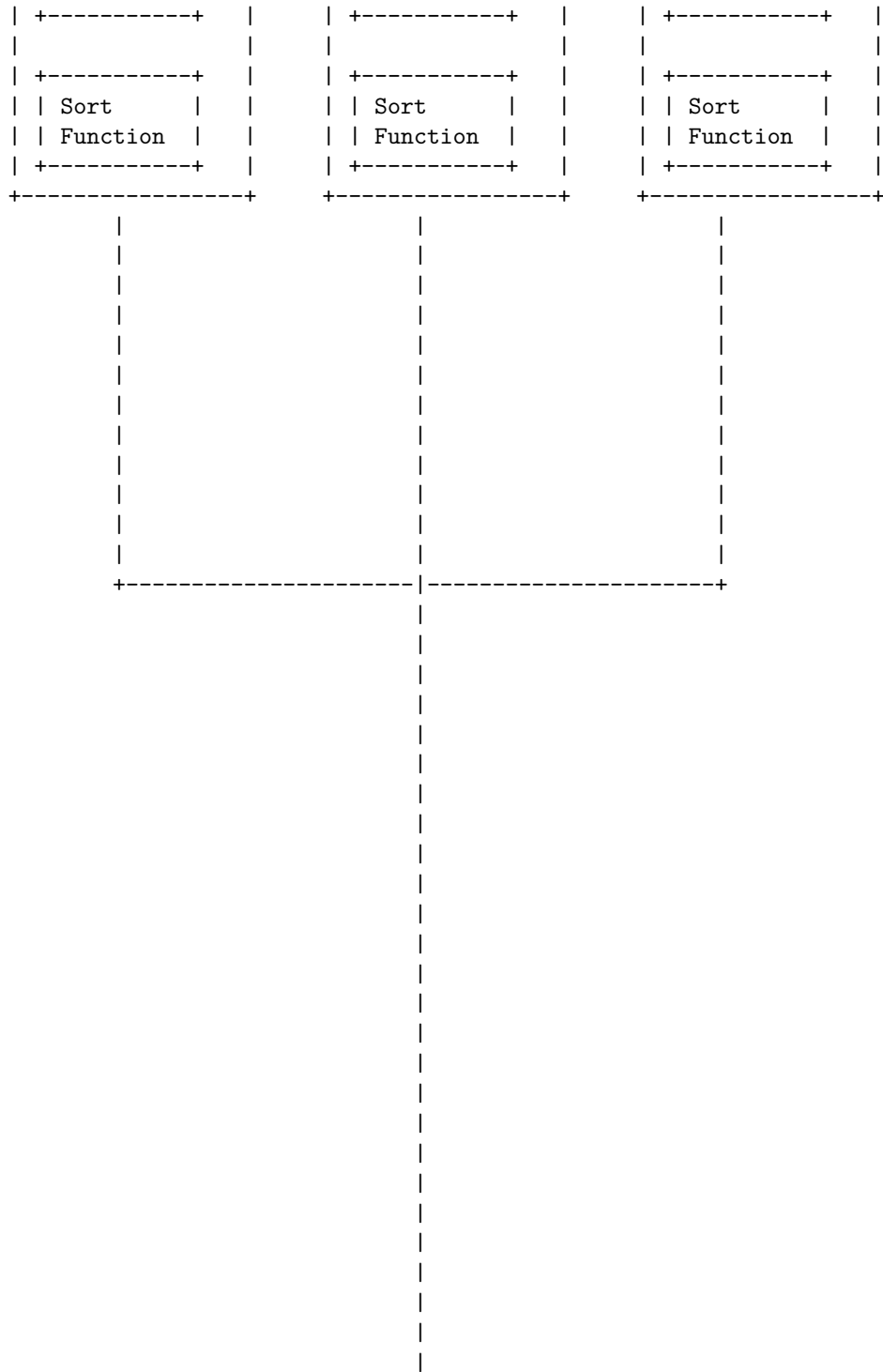


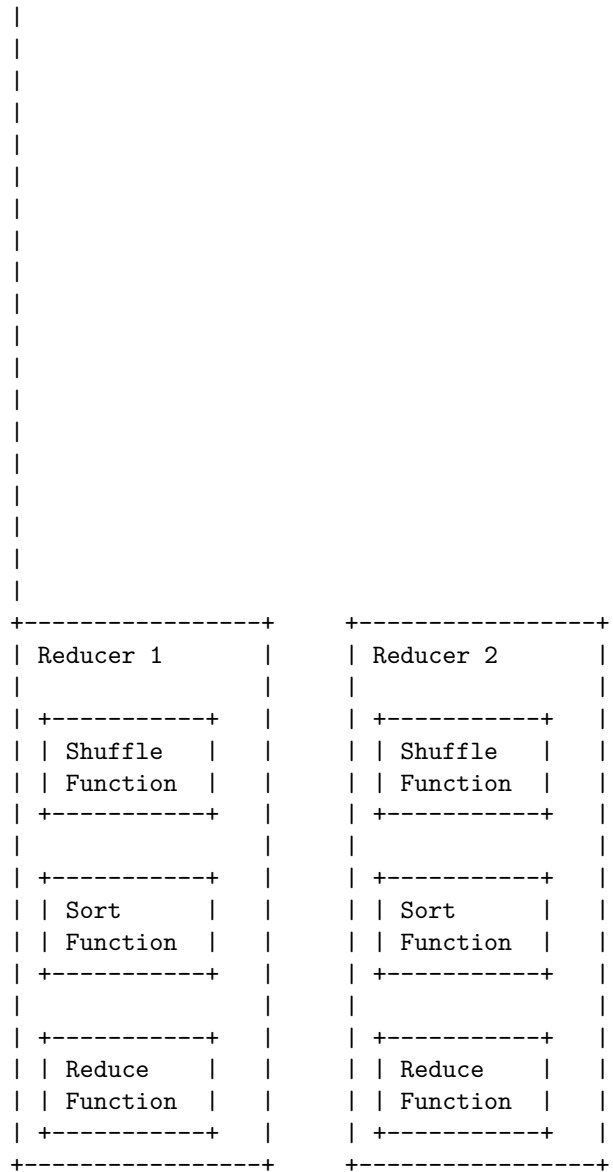


Each TaskTracker can run multiple map and reduce tasks in parallel, depending on the number of slots available. The JobTracker can also use different scheduling algorithms to prioritize the jobs in the queue, such as FIFO, Fair Scheduler, or Capacity Scheduler.

Shuffle and sort are two phases in the MapReduce framework that occur between the map and reduce tasks. Shuffle is the process of transferring the intermediate data from the mappers to the reducers, while sort is the process of grouping and ordering the intermediate data by key. The following diagram illustrates the basic architecture of shuffle and sort in MapReduce using ASCII characters:







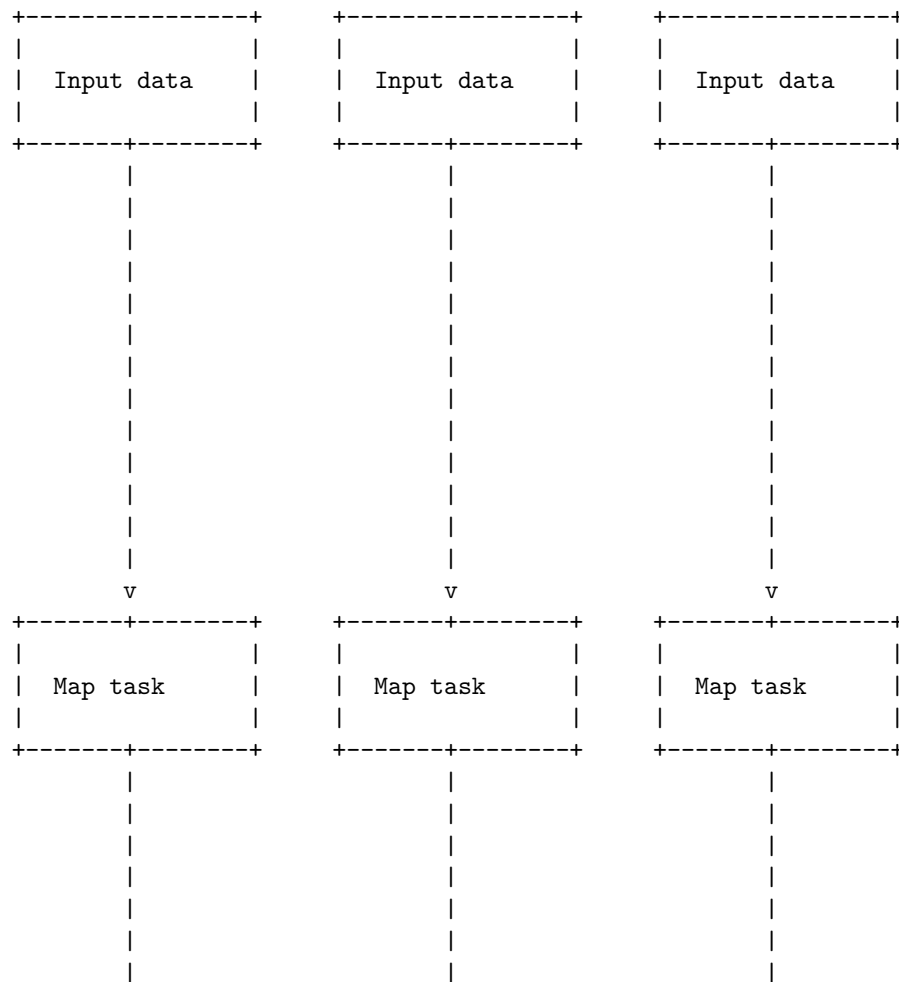
The diagram shows the following steps:

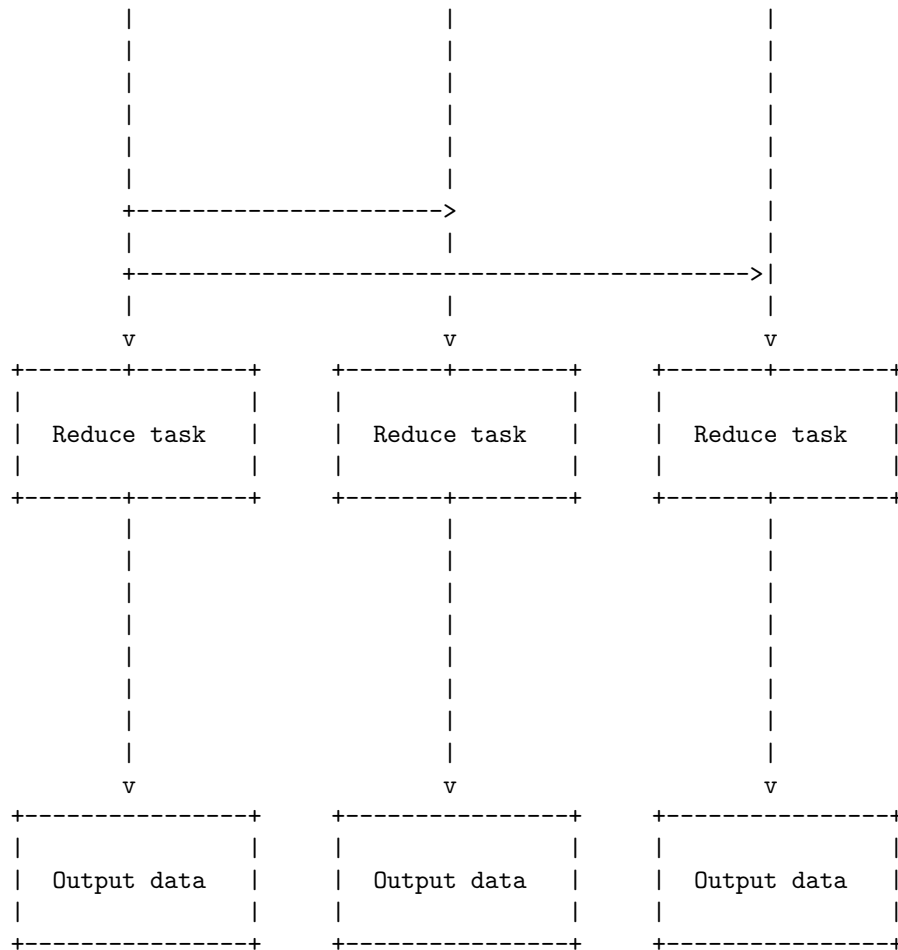
- The map function takes the input data and produces key-value pairs as intermediate output.
- The partition function assigns each key-value pair to a reducer based on a hash function.
- The sort function sorts the key-value pairs by key within each mapper.
- The shuffle function transfers the key-value pairs from the mappers to the reducers over the network, using HTTP requests.

- The sort function sorts the key-value pairs by key within each reducer, merging the data from different mappers.
- The reduce function takes the sorted key-value pairs and performs some aggregation or computation on the values for each key

Task execution in map reduce is the process of running a map reduce job on a cluster of nodes. A map reduce job consists of a map function and a reduce function, which are applied to a set of input data to produce a set of output data. The map function takes a key-value pair as input and produces a list of intermediate key-value pairs as output. The reduce function takes a key and a list of values associated with that key as input and produces a list of output values for that key.

The following diagram illustrates the basic architecture of a map reduce job using ASCII art:





The map reduce framework takes care of scheduling tasks, monitoring them and re-executing the failed tasks. The framework also sorts the outputs of the map tasks, which are then input to the reduce tasks. Typically both the input and the output of the job are stored in a file-system .

MapReduce is a programming model for processing large-scale data sets in parallel and distributed manner. It consists of two phases: map and reduce. The map phase takes a set of input key-value pairs and transforms them into a set of intermediate key-value pairs. The reduce phase takes the intermediate key-value pairs with the same key and combines them into a smaller set of output key-value pairs.

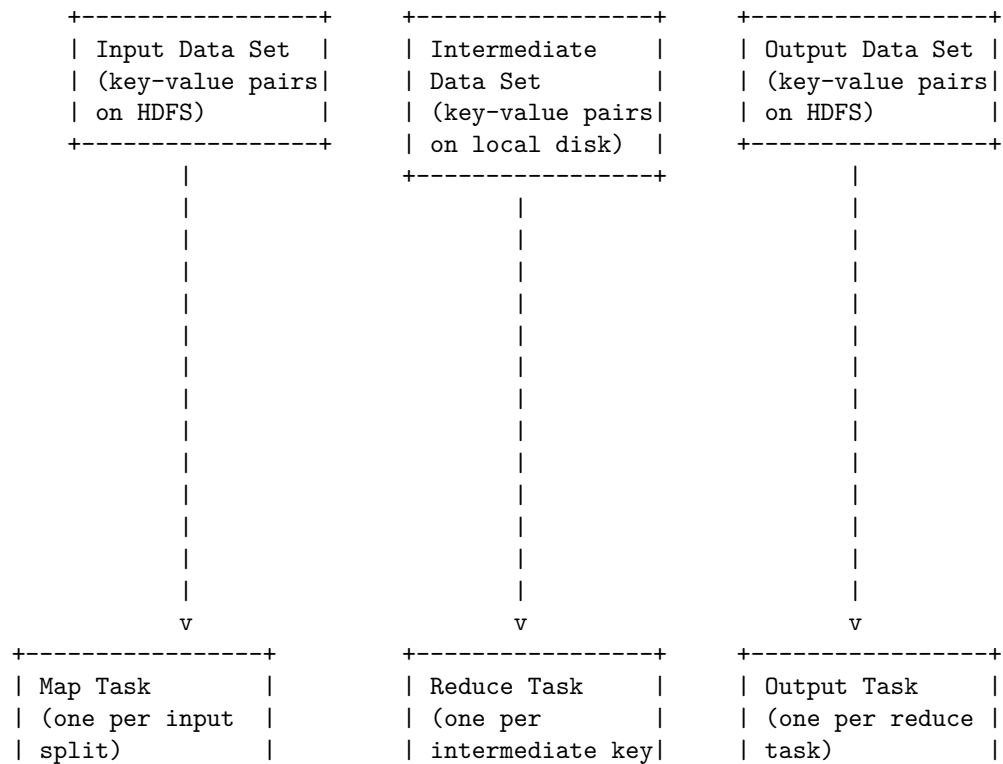
There are different types of MapReduce depending on the input and output formats of the map and reduce functions. The following are some common types of MapReduce:

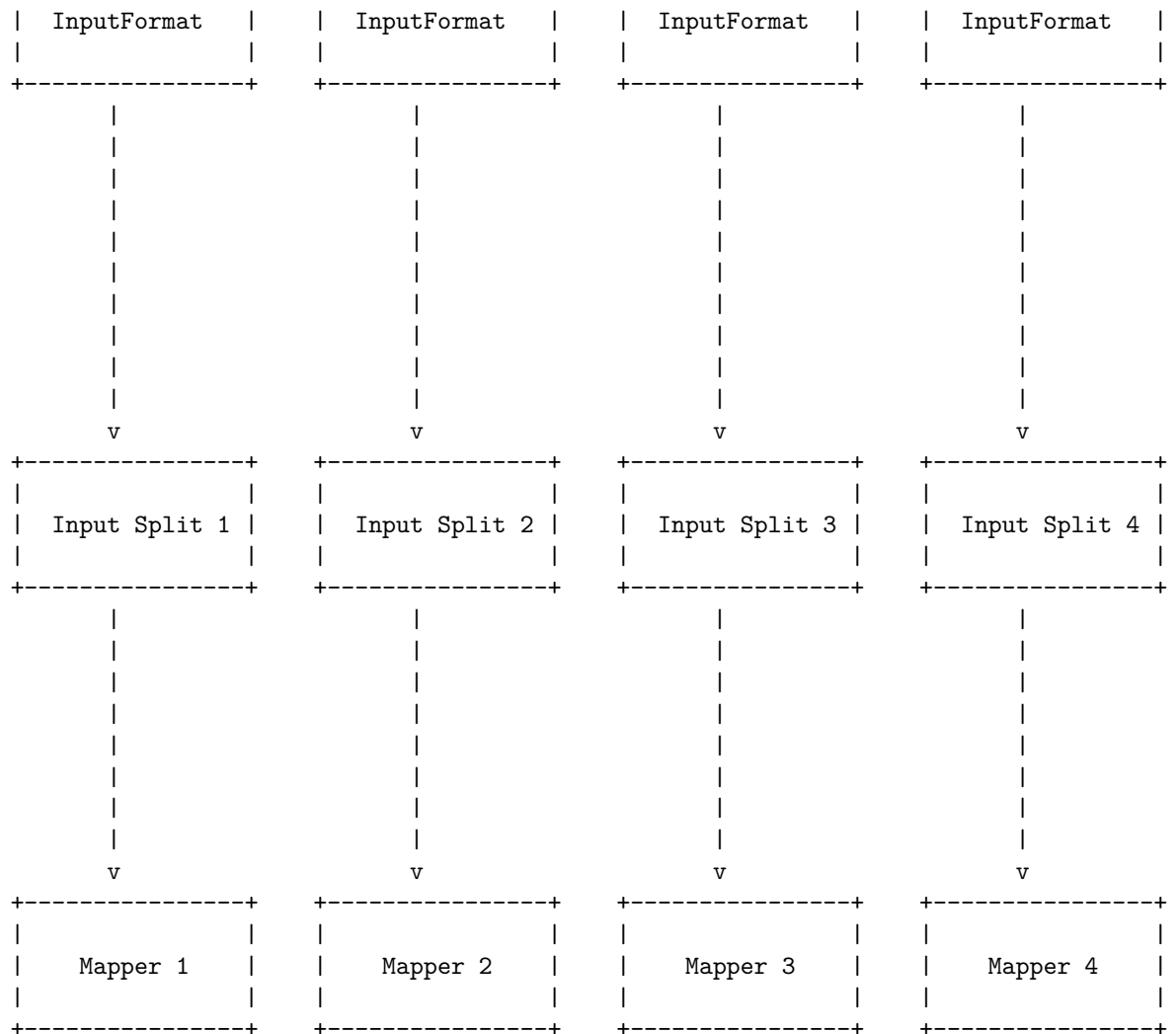
- WordCount: This type of MapReduce counts the frequency of each word

in a text file. The input format is a text file with each line as a key-value pair, where the key is the line number and the value is the line content. The output format is a text file with each line as a key-value pair, where the key is a word and the value is its frequency.

- **InvertedIndex:** This type of MapReduce builds an inverted index for a collection of documents. The input format is a set of text files, where each file is a document. The output format is a text file with each line as a key-value pair, where the key is a word and the value is a list of document IDs that contain the word.
- **Join:** This type of MapReduce performs a join operation on two data sets. The input format is two sets of key-value pairs, where the key is a common attribute and the value is the rest of the record. The output format is a set of key-value pairs, where the key is the common attribute and the value is the joined record.
- **MatrixMultiplication:** This type of MapReduce performs a matrix multiplication on two matrices. The input format is two sets of key-value pairs, where the key is a matrix identifier and a row or column index, and the value is a vector of elements. The output format is a set of key-value pairs, where the key is a row and column index, and the value is the product of the corresponding elements.

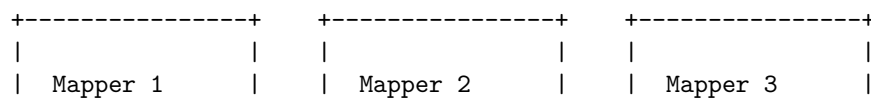
The following diagram illustrates the basic architecture of a MapReduce job:

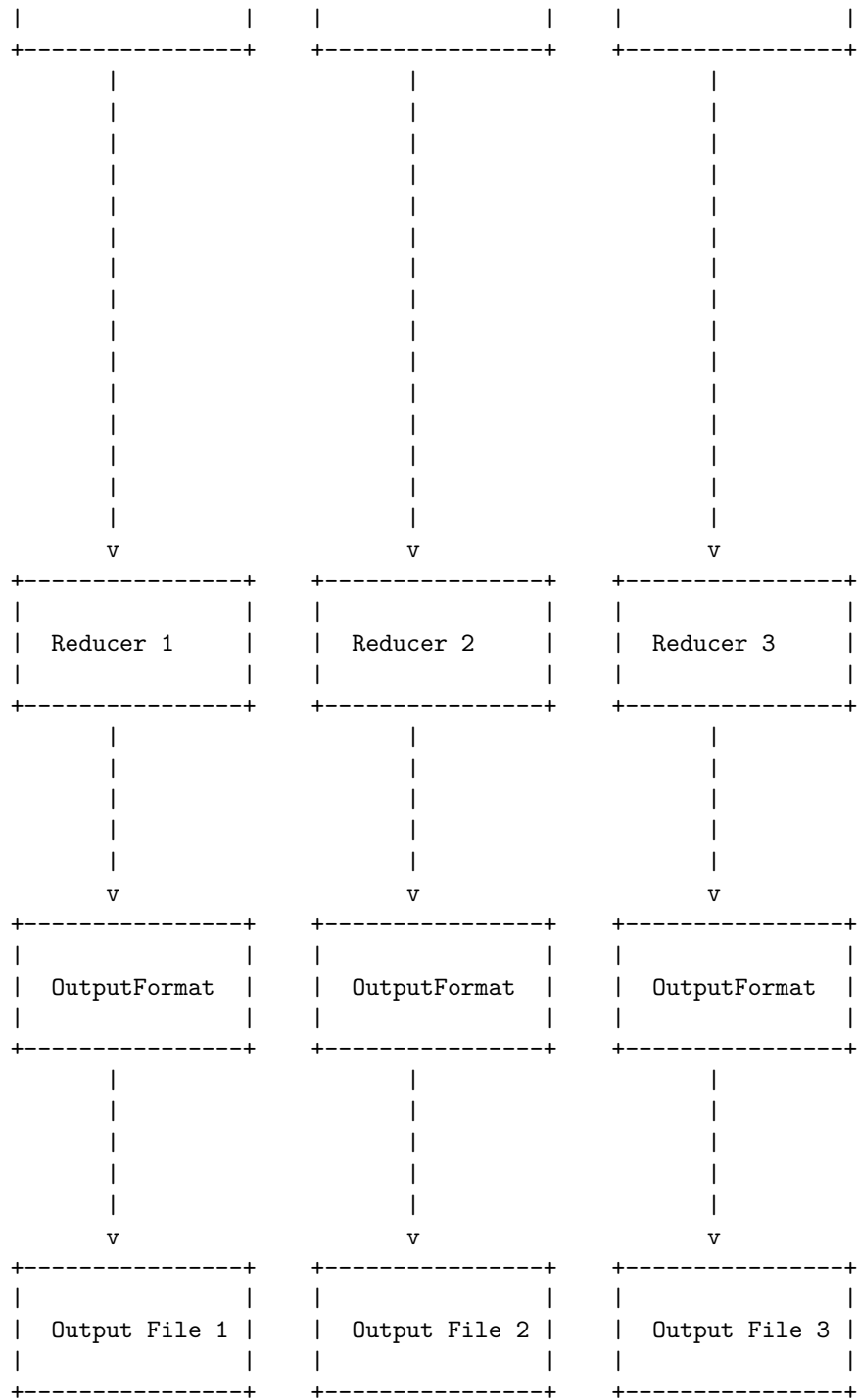




OutputFormat is an interface that describes the output-specification for a MapReduce job. It provides the RecordWriter implementation to write the output files of the job to a FileSystem. There are different types of OutputFormat in MapReduce, such as TextOutputFormat, SequenceFileOutputFormat, MapFileOutputFormat, DBOutputFormat, etc. Each type has its own advantages and disadvantages depending on the use case and the data format.

The following diagram illustrates the basic architecture of a MapReduce job with different OutputFormat types:





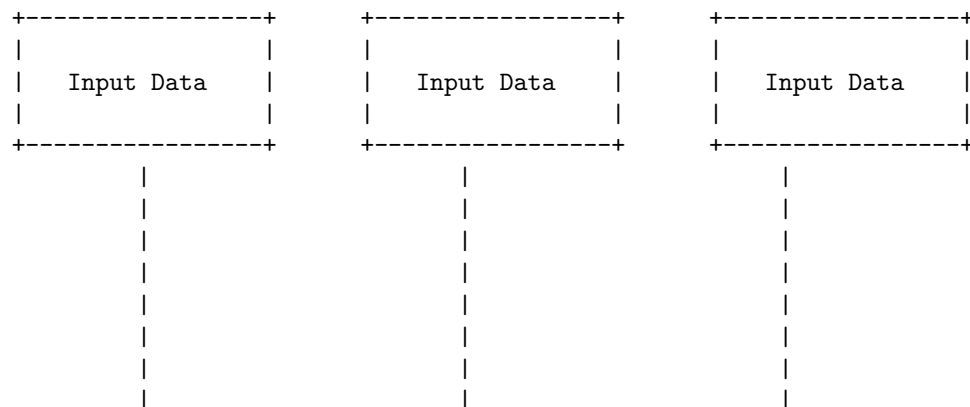
MapReduce is a programming model and a framework for processing large-scale data sets in parallel. It consists of two phases: map and reduce. The map phase applies a user-defined function to each input record and produces a set of intermediate key-value pairs. The reduce phase aggregates the intermediate values associated with the same key and produces the final output.

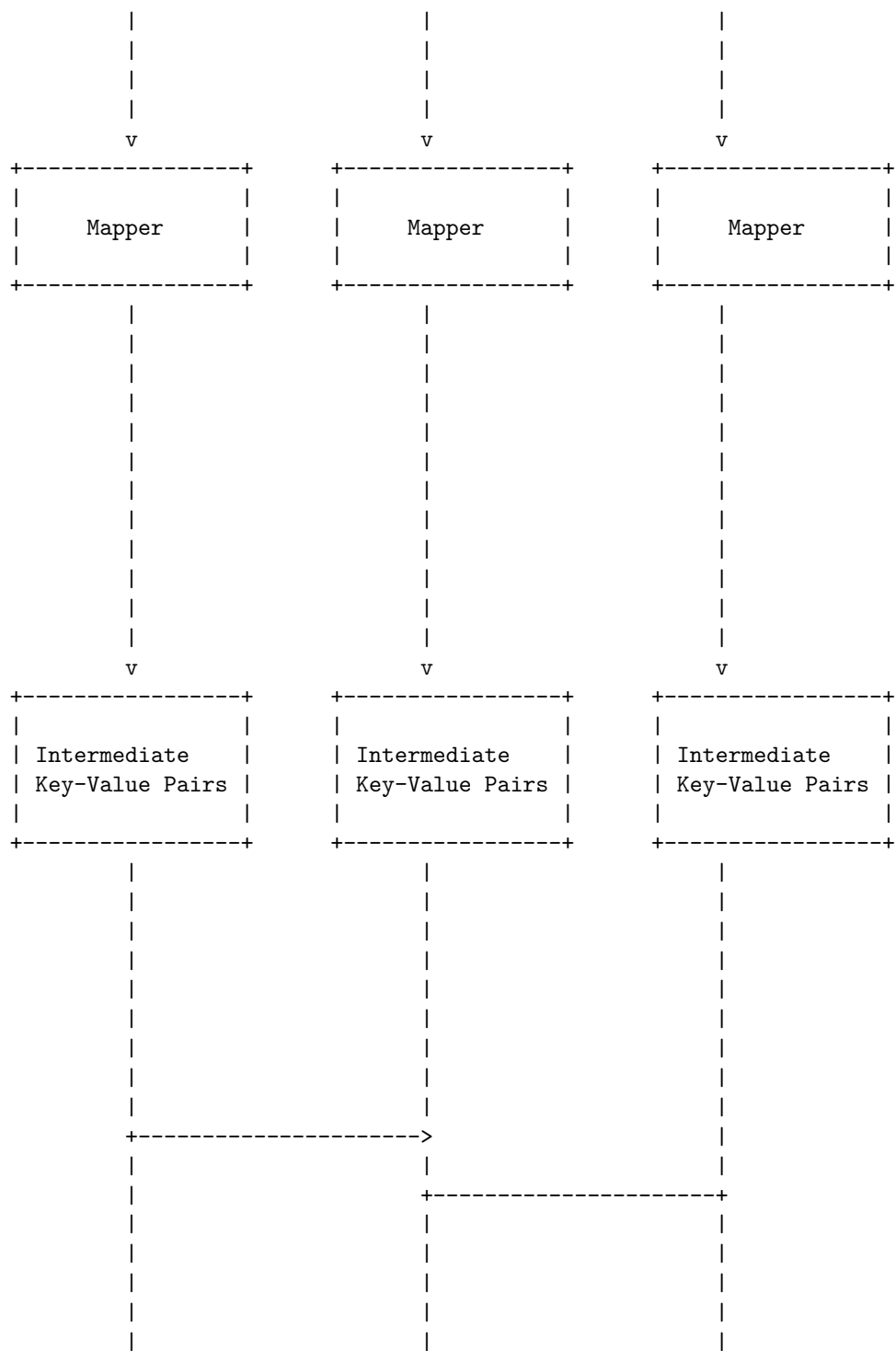
Some of the features of MapReduce are:

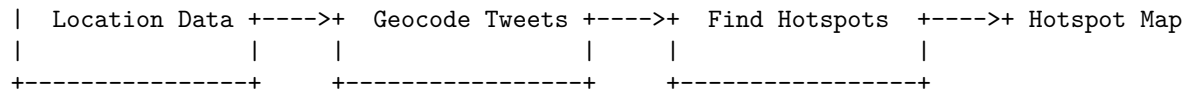
- Scalability: MapReduce can handle huge amounts of data by distributing the work across multiple nodes in a cluster.
- Flexibility: MapReduce can process different types of data, such as structured, unstructured, or semi-structured, and support various formats, such as text, binary, or XML.
- Security and Authentication: MapReduce can use Kerberos to authenticate the users and nodes, and encrypt the data in transit and at rest.
- Cost-effectiveness: MapReduce can run on commodity hardware, which reduces the cost of infrastructure and maintenance.
- Speed: MapReduce can leverage the parallelism and locality of data to speed up the processing.
- Simplicity: MapReduce provides a simple and intuitive programming model that abstracts the details of distributed computing, such as network communication, fault tolerance, and load balancing.
- Parallelism: MapReduce can execute multiple map and reduce tasks concurrently on different nodes, and use a master node to coordinate the work and handle failures.
- Availability and Resilience: MapReduce can tolerate node failures and data loss by replicating the data across the cluster and re-executing the failed tasks on other nodes.

MapReduce features

The following diagram illustrates the basic architecture of a MapReduce system using ASCII characters:







In this diagram, each row represents a different MapReduce job that takes a different input data and produces a different output data. Each job has a Map phase and a Reduce phase, which are composed of multiple tasks that run in parallel on different nodes. The tasks communicate with each other through an intermediate data format, which is usually a key-value pair. For example, the Map phase of the word count job splits the tweets into words and emits a key-value pair for each word, where the key is the word and the value is 1. The Reduce phase of the word count job sums up the values for each word and emits a key-value pair for each word, where the key is the word and the value is the total count. The output data of the word count job is a list of words and their frequencies in the tweets.

Unit 4 - HDFS (Hadoop Distributed File System) and Hadoop Environment

HDFS is a distributed file system that runs on commodity hardware and provides high availability, fault tolerance, scalability, and high throughput access to large data sets. HDFS is designed to store and process data in a parallel and distributed manner using the MapReduce framework.

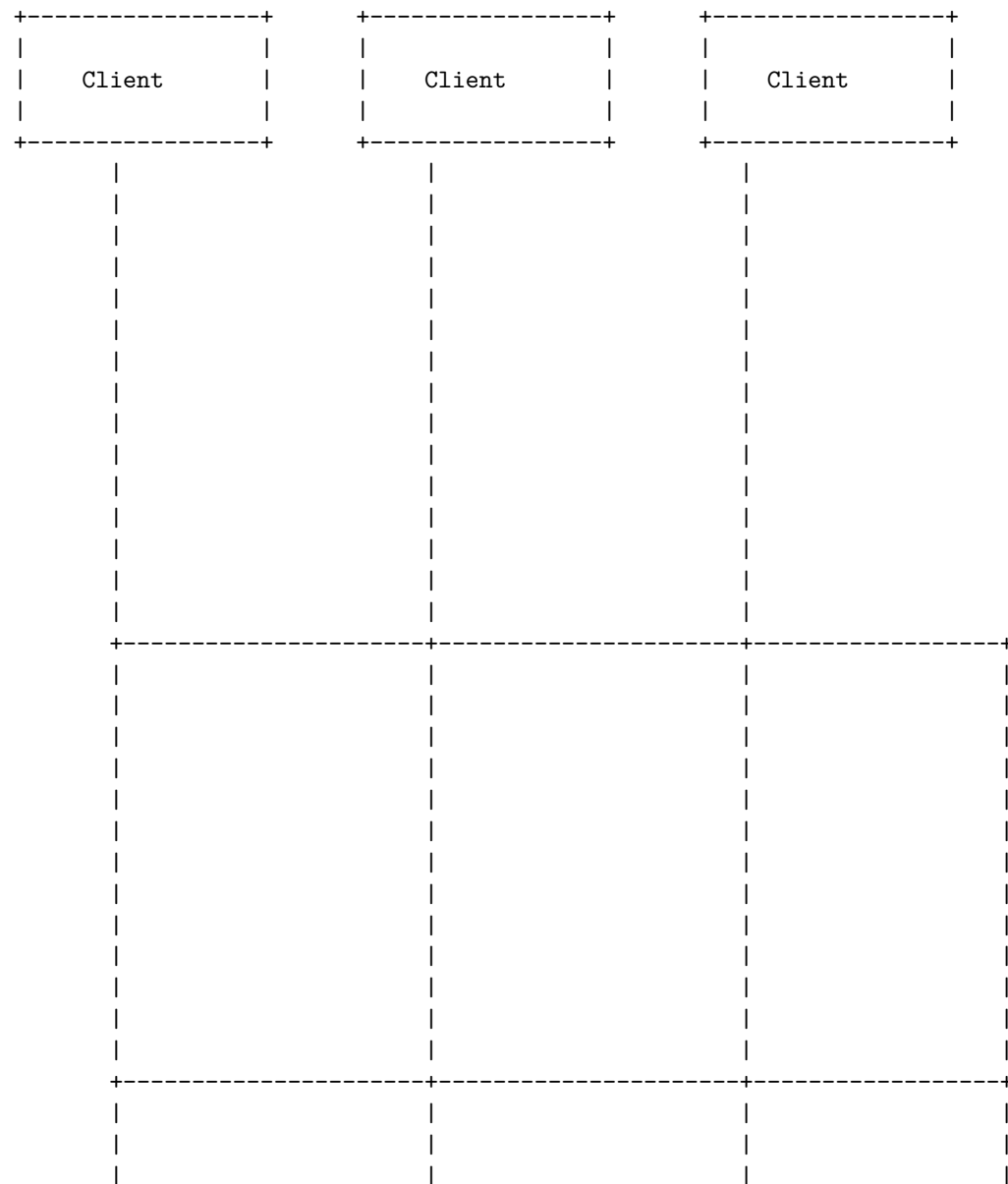
The basic architecture of HDFS consists of the following components:

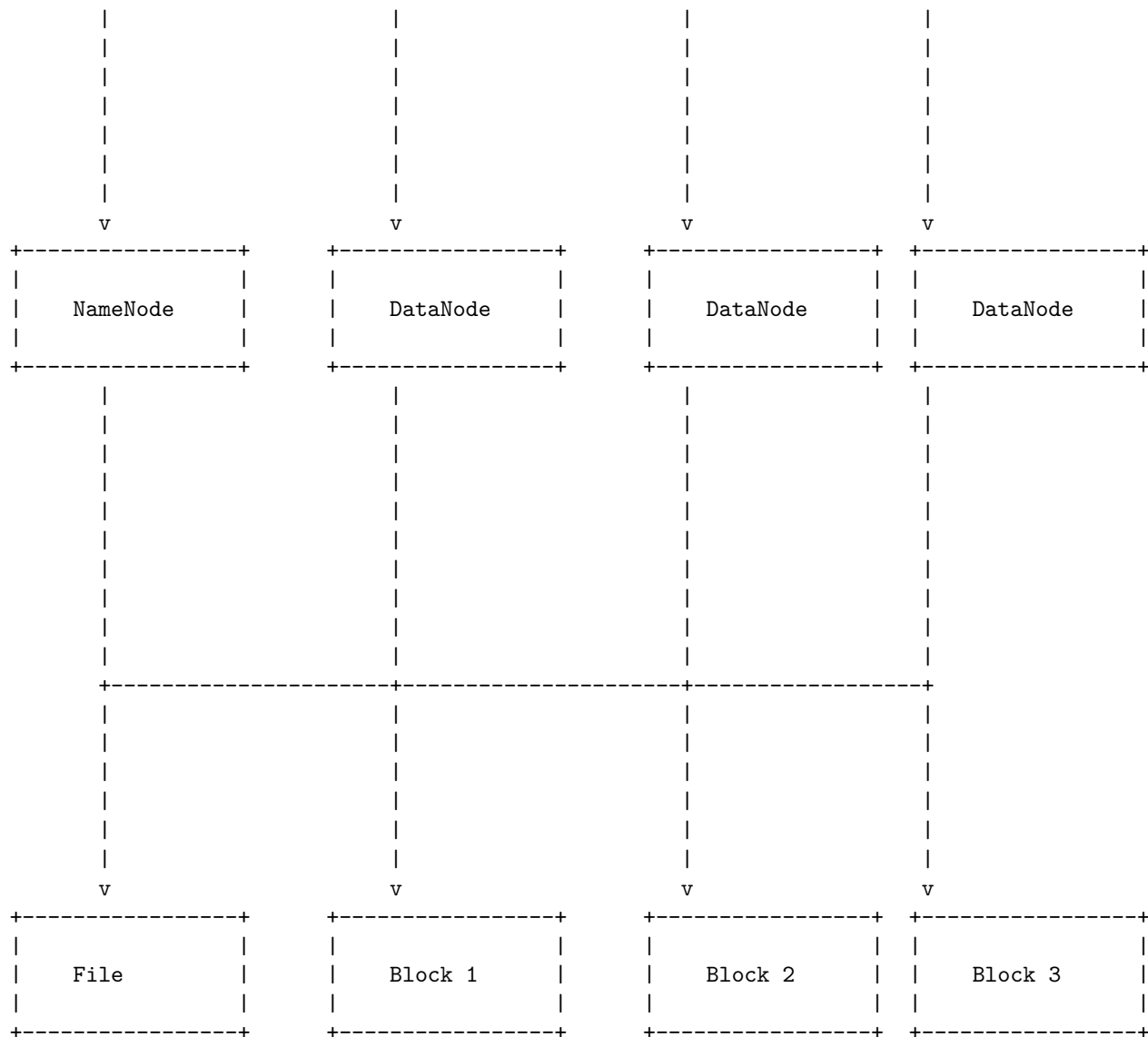
- **NameNode:** The master node that manages the file system namespace and regulates access to files by clients. It also maintains the metadata of the file system, such as the file and directory structure, the permissions, and the locations of the blocks that make up the files. The NameNode is a single point of failure in HDFS, and it is usually configured with a backup node or a secondary node for recovery purposes.
- **DataNode:** The slave node that stores the actual data blocks of the files in the local disks. Each block is typically 64 MB or 128 MB in size, and a file can have one or more blocks. The DataNodes are responsible for serving read and write requests from the clients, and also perform block creation, deletion, and replication upon instruction from the NameNode.
- **Secondary NameNode:** An optional node that periodically merges the namespace image and the edit log from the NameNode, and sends the updated image back to the NameNode. This reduces the startup time of the NameNode and the size of the edit log. The secondary NameNode is not a backup for the NameNode, as it does not store the entire file system state.
- **Checkpoint Node:** An alternative to the secondary NameNode that creates checkpoints of the namespace by downloading the edit log from the NameNode and applying it to a local copy of the namespace image. The checkpoint node then uploads the new image back to the NameNode,

Backup Node: Another alternative to the secondary NameNode that provides a backup for the NameNode. The backup node maintains an in-memory copy of the file system namespace, which is always synchronized with the NameNode. The backup node can also create checkpoints of the namespace, and can take over the role of the NameNode in case of a failure.

designed to run on commodity hardware. It has a master/slave architecture, where a single NameNode manages the file system namespace and regulates access to files by clients, and a number of DataNodes store the data blocks on the nodes that they run on. HDFS is highly fault-tolerant and scalable, and supports replication of data blocks across multiple DataNodes.

HDFS



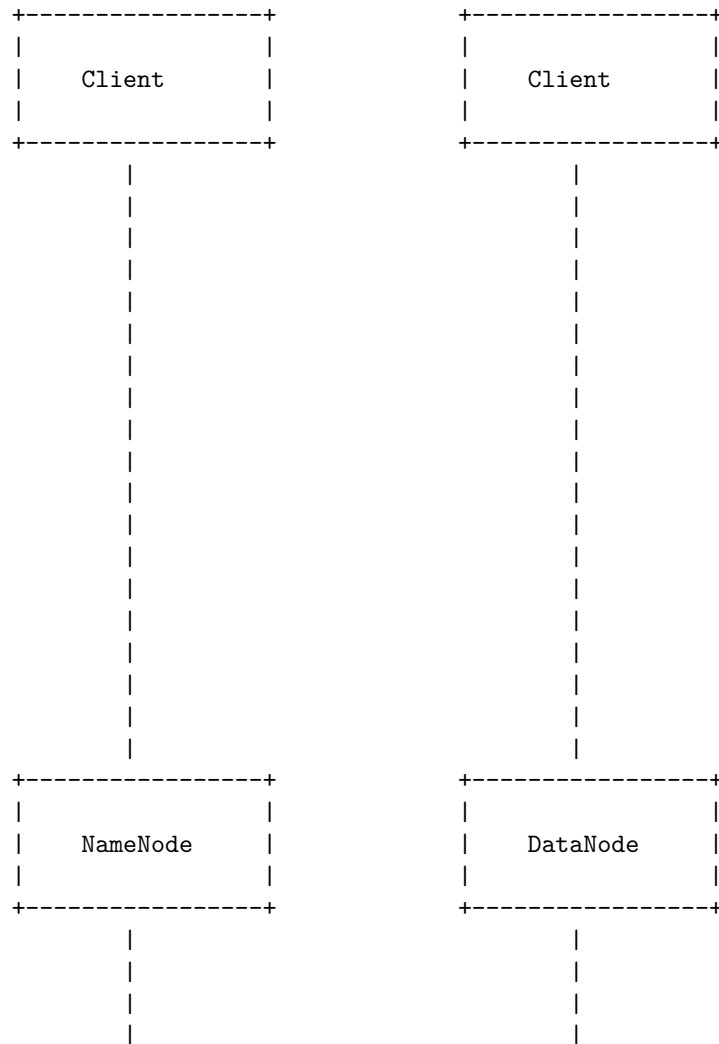


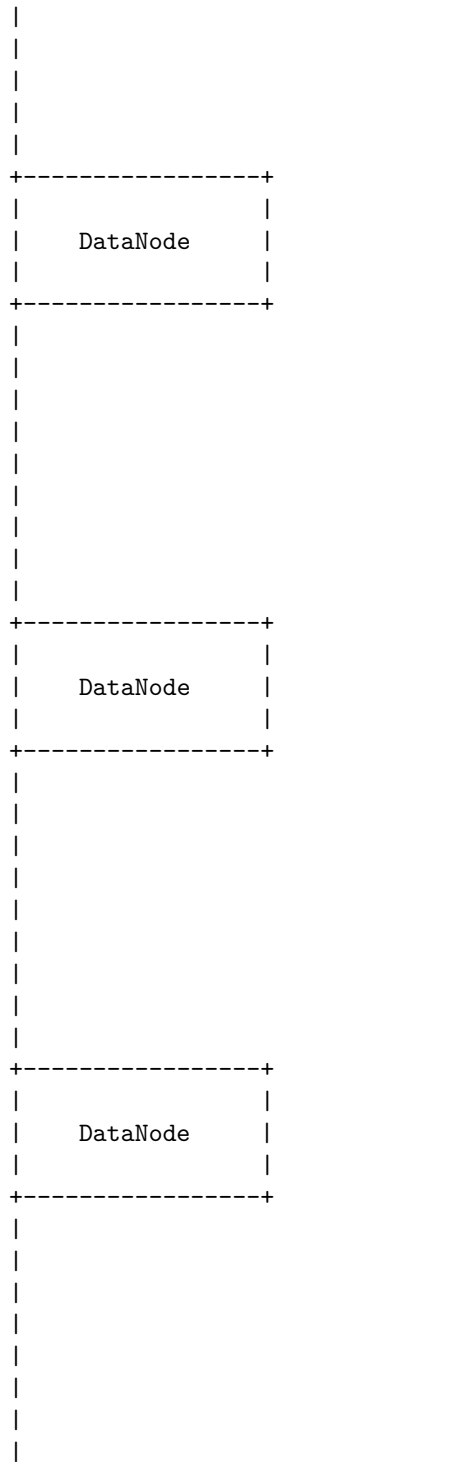
The above diagram illustrates the basic architecture of HDFS. A file is split into one or more blocks, and each block is stored on one or more DataNodes. The NameNode maintains the metadata of the file system, such as the file names, directories, permissions, and the locations of the blocks. The clients communicate with the NameNode to perform operations on the file system, such as creating, deleting, reading, or writing files. The clients also communicate with the DataNodes to read or write the data blocks. The NameNode and the DataNodes periodically exchange heartbeat and block report messages to monitor the health and status of the cluster.

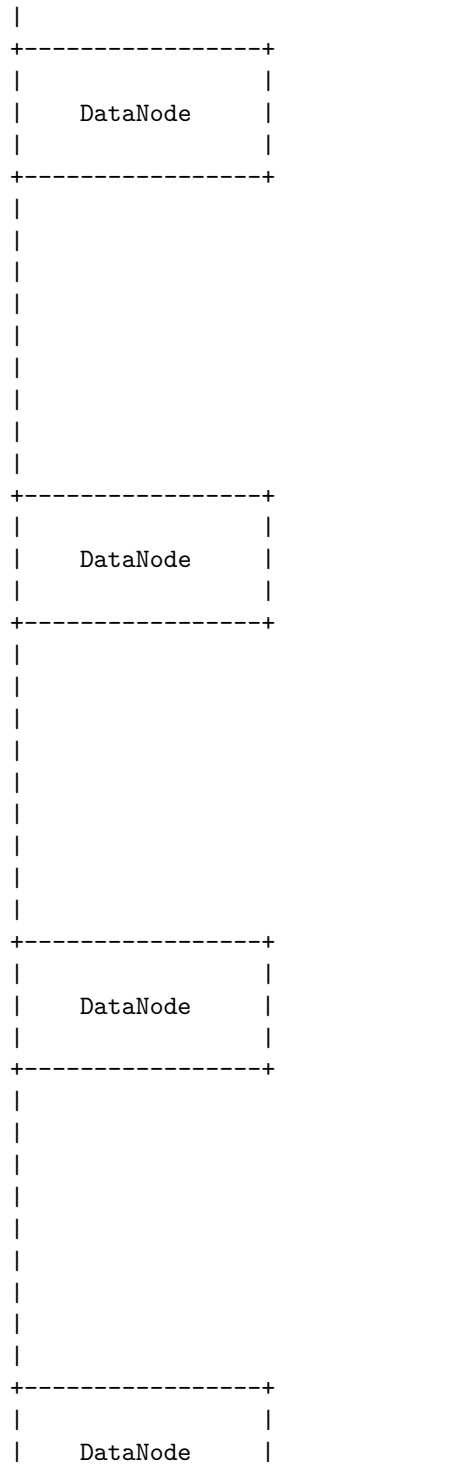
HDFS is a distributed file system that runs on clusters of commodity hardware and is designed for storing very large files with streaming data access patterns . It is based on the Google File System and is a member of the Hadoop Ecosystem. HDFS has a master-slave architecture, where a single NameNode manages the namespace and metadata of the file system, and multiple DataNodes store the actual data blocks of the files . HDFS provides high throughput, fault tolerance, scalability, and data locality for applications that process large amounts of data.

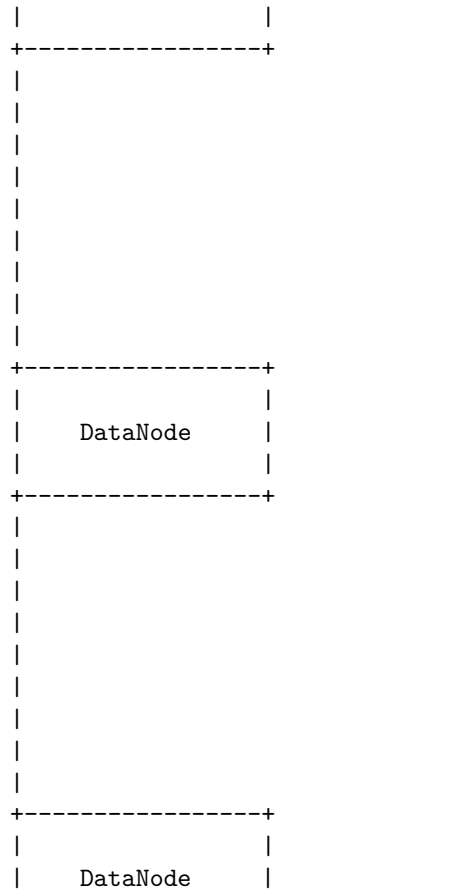
Design of HDFS

The following diagram illustrates the basic architecture of HDFS using ASCII characters:









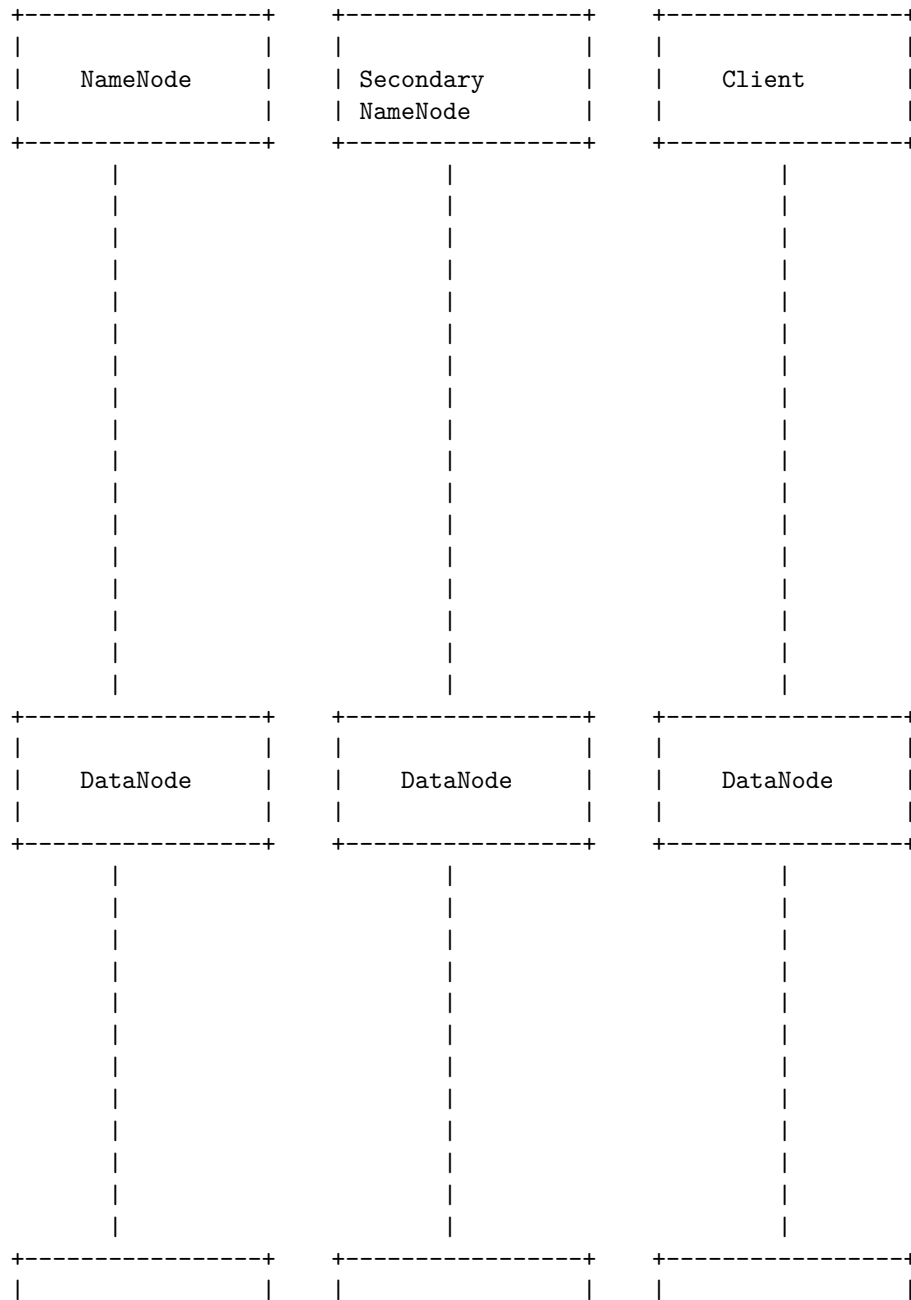
HDFS stands for Hadoop Distributed File System. It is a distributed file system that handles large data sets running on commodity hardware. It is one of the major components of Apache Hadoop, the others being MapReduce and YARN.

HDFS has the following design concepts:

- **Blocks:** HDFS is a block-structured file system. Each file is broken into blocks of fixed size, usually 128 MB, which are stored across various data nodes on the cluster. Each block is replicated multiple times, by default three times, for fault tolerance.
- **NameNode:** NameNode is the master node that manages the metadata of the file system, such as the file names, directories, permissions, and locations of the blocks. NameNode also performs operations such as opening, closing, and renaming files and directories. NameNode is a single point of failure in HDFS, so it is usually configured with a secondary or standby NameNode for backup and recovery.
- **DataNodes:** DataNodes are the worker nodes that store and serve the blocks of data. DataNodes also perform tasks such as block creation,

deletion, replication, and verification. DataNodes communicate with the NameNode and report the status of the blocks they are holding.

HDFS concepts



Block 1	Block 1	Block 1
Block 2	Block 2	Block 3
Block 3	Block 4	Block 4

HDFS stands for Hadoop Distributed File System. It is a file system that stores large amounts of data across multiple nodes in a cluster. HDFS provides high availability, scalability, fault tolerance, and performance for big data applications.

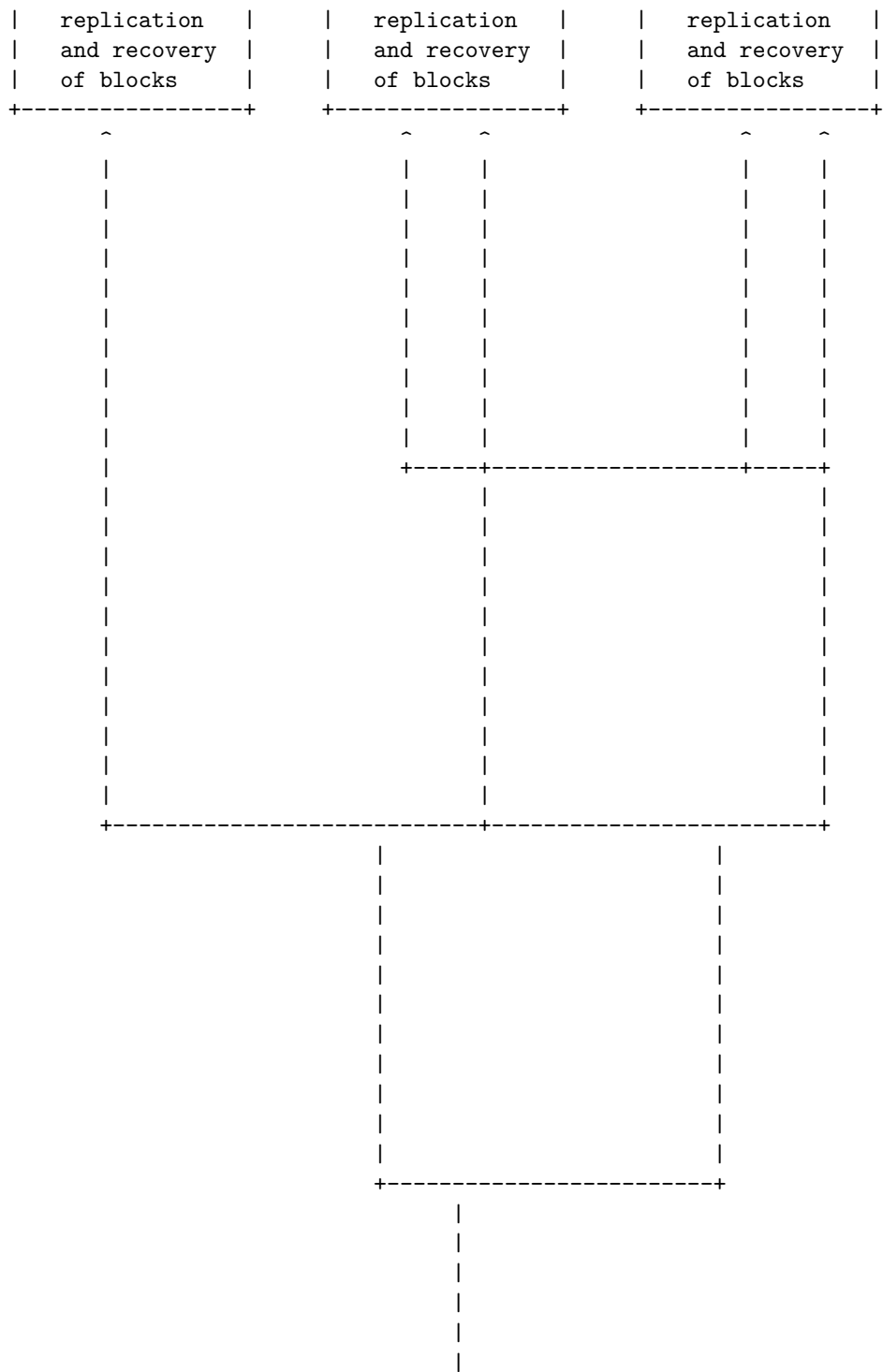
Some of the benefits of HDFS are:

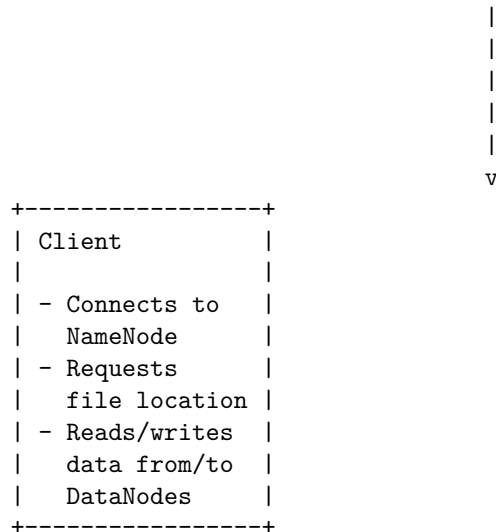
- It is fast. It can deliver more than 2 GB of data per second thanks to its cluster architecture .
- It is free. HDFS is an open-source software that comes with no licensing or support cost.
- It is reliable. The file system stores multiple copies of data in separate systems to ensure it is always accessible .
- It is scalable. HDFS can store petabytes of data and handle thousands of concurrent users by adding more nodes to the cluster .
- It is distributed. HDFS splits large files into smaller blocks and distributes them across the cluster, allowing parallel processing and load balancing .

Benefits of HDFS

The following diagram illustrates the basic architecture of HDFS and how it provides the benefits mentioned above:

NameNode (Master Node)	DataNode (Worker Node)	DataNode (Worker Node)
- Stores metadata	- Stores data blocks	- Stores data blocks
- Manages cluster configuration	- Reports block status to NameNode	- Reports block status to NameNode
- Handles client requests	- Serves read/write requests	- Serves read/write requests
- Performs	- Performs	- Performs





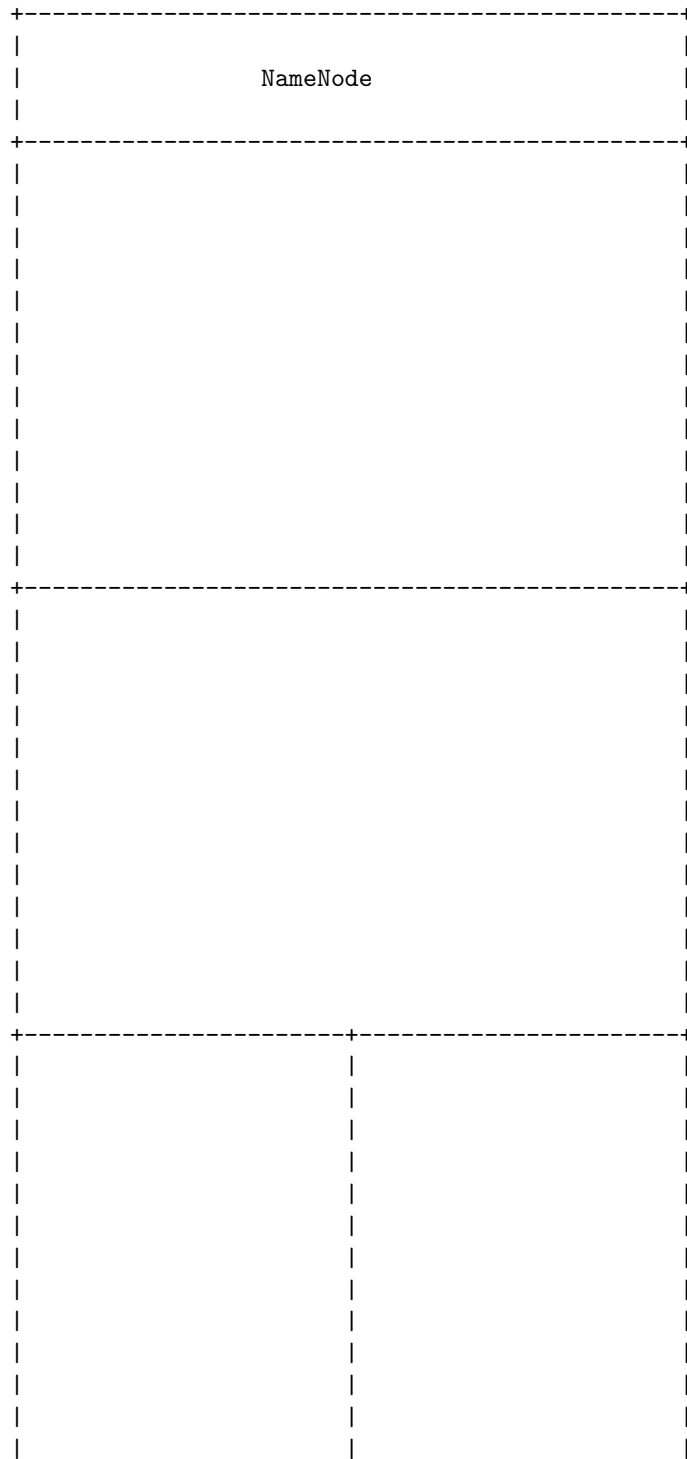
HDFS stands for Hadoop Distributed File System, which is a scalable and fault-tolerant storage system for big data processing. HDFS splits large files into fixed-size blocks and distributes them across multiple nodes in a cluster. HDFS also replicates each block to ensure data availability and reliability.

However, HDFS also faces some challenges, such as:

- Issues with small files: HDFS is not suitable for storing and processing small files, as each file occupies a block regardless of its size. This leads to inefficient use of disk space and memory, as well as increased network traffic and metadata overhead.
- Slow processing speed: HDFS relies on MapReduce, a batch processing framework, to process the data stored in it. MapReduce has high latency and is not suitable for real-time or interactive applications. MapReduce also requires multiple disk I/O and network transfers, which slows down the performance.
- Support for batch processing only: HDFS does not support streaming or transactional data processing, as it is designed for batch processing of large and static datasets. HDFS also does not support random access or updates to the data, as it is optimized for sequential reads and writes.
- No real-time processing: HDFS cannot handle real-time data analysis, as it depends on MapReduce, which has high latency and is not suitable for streaming or interactive applications. HDFS also lacks the ability to support complex queries or joins, as it is a file system and not a database.
- Iterative processing: HDFS is not efficient for iterative processing, as it requires multiple MapReduce jobs to run sequentially, each with its own disk I/O and network transfers. HDFS also does not support caching or in-memory processing, which can improve the performance of iterative algorithms.

- The following diagram illustrates the basic architecture of HDFS using ASCII characters:

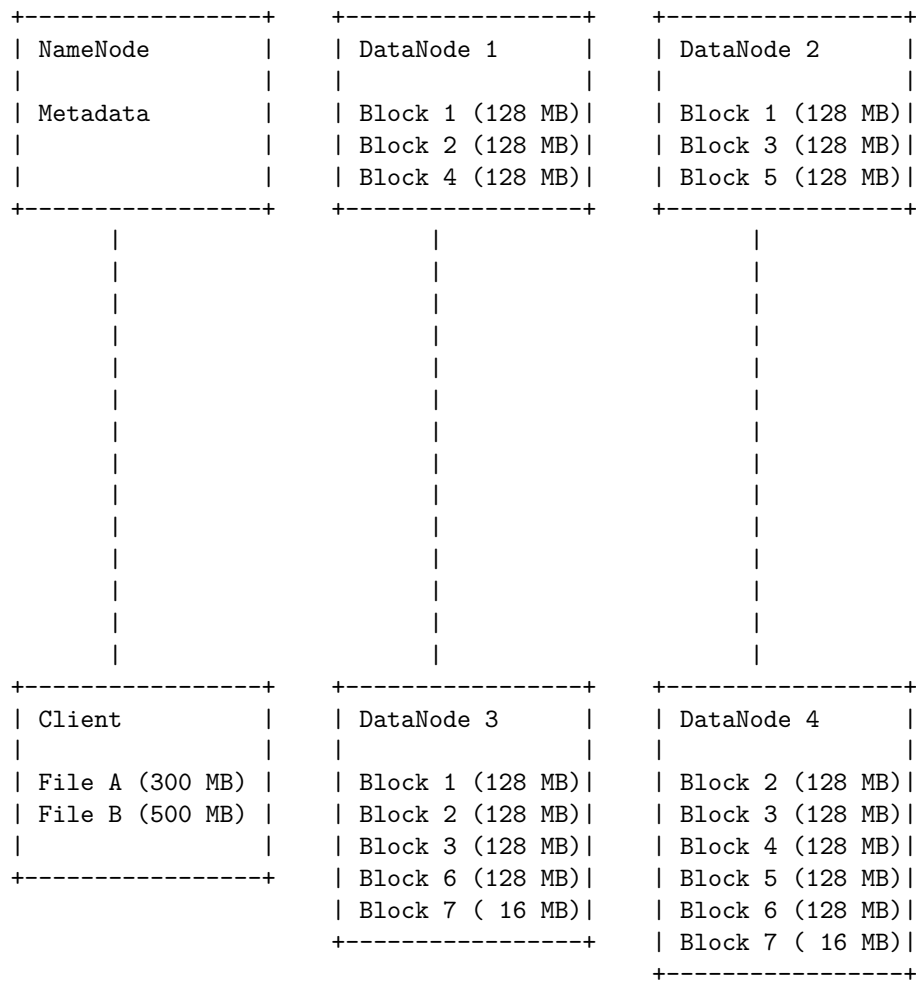




File sizes in HDFS

HDFS is a distributed file system that stores large files across multiple nodes in a cluster. HDFS breaks down files into fixed-size blocks, which are stored as independent units. The default block size in HDFS is 128 MB, but it can be configured manually. HDFS also replicates each block across multiple nodes to ensure fault tolerance and high availability.

The following ASCII diagram illustrates the basic architecture of HDFS and how files are divided into blocks and replicated across nodes:



In this diagram, the client has two files: File A (300 MB) and File B (500 MB). File A is divided into three blocks: Block 1, Block 2, and Block 3. File B is divided into four blocks: Block 4, Block 5, Block 6, and Block 7. The NameNode stores the metadata of the files and blocks, such as their names, locations, sizes, and replication factors. The DataNodes store the actual blocks of the files.

and communicate with the NameNode and the client. The client can read or write files by contacting the NameNode and getting the list of DataNodes that store the blocks of the files. The client then communicates directly with the DataNodes to perform the read or write operations.

The diagram also shows that each block is replicated across two or more DataNodes, depending on the replication factor. The default replication factor in HDFS is 3, but it can be configured manually. The replication ensures that the data is available even if some DataNodes fail or become unavailable. The NameNode is responsible for managing the replication of the blocks and balancing the load across the DataNodes. The NameNode also performs periodic checks on the DataNodes to ensure their health and status.

To find the size of a file or a directory in HDFS, the client can use the `hdfs dfs -du` command. This command shows the base size of the file or directory before replication. For example, to find the size of File A, the client can run:

```
hdfs dfs -du /user/client/FileA
```

This will show the output:

```
300 MB /user/client/FileA
```

To find the size of a directory, the client can run:

```
hdfs dfs -du /user/client
```

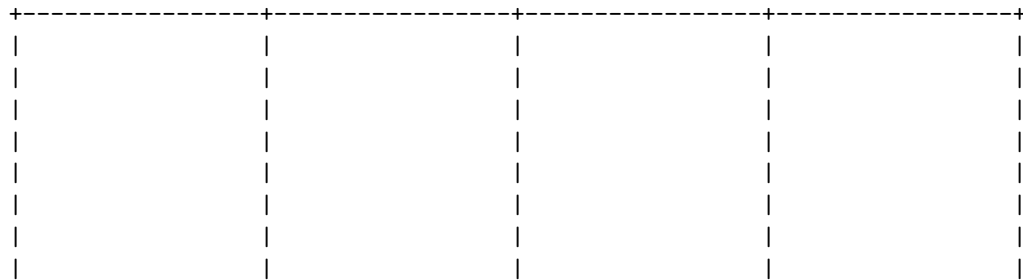
This will show the output:

```
800 MB /user/client
```

This is the sum of the sizes of File A and File B. To find the size of a directory with the replication factor, the client can use the `-s` option. For example, to find the size of the directory with a replication factor of 3, the client can

A block in HDFS is a fixed-size unit of data that is stored on one or more nodes in a cluster. The default block size in HDFS is 128 MB, but it can be configured manually by changing the `dfs.block.size` property in `hdfs-site.xml`. The advantage of using large blocks in HDFS is that it reduces the number of disk seeks and network transfers, and improves the throughput of data processing.

The following diagram illustrates the basic architecture of a block in HDFS:




```

+-----+      +-----+      +-----+

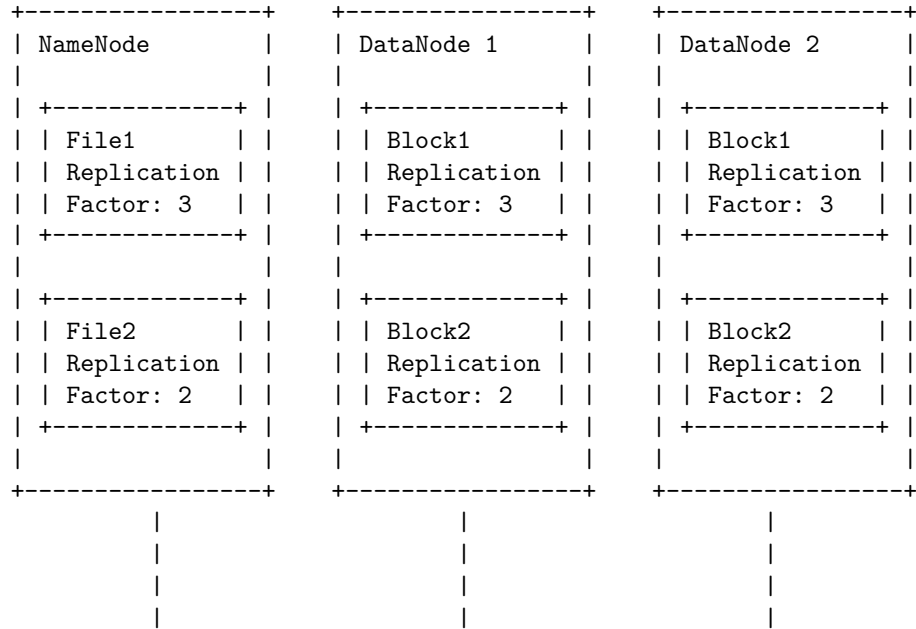
```

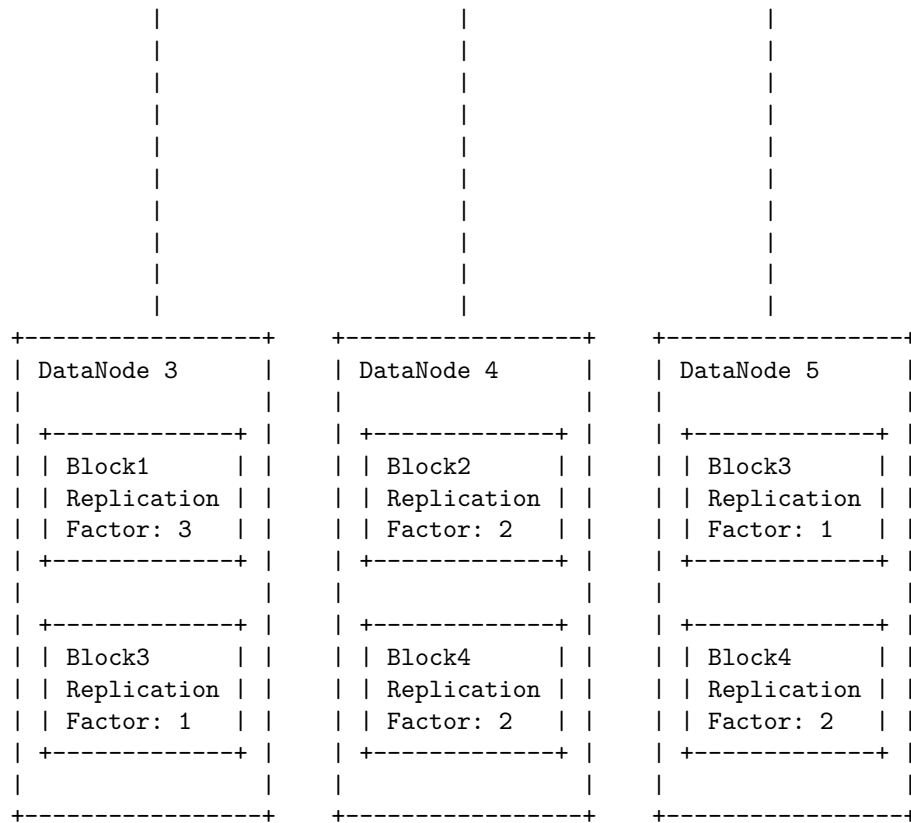
In this diagram, the NameNode is the master node that manages the file system namespace and the metadata of the files and blocks. The DataNodes are the slave nodes that store the actual data blocks and report to the NameNode periodically. The NameNode keeps track of which blocks belong to which file and where they are located in the cluster. The DataNodes are responsible for serving read and write requests from the clients and performing block creation, deletion, and replication as instructed by the NameNode.

As an example, File1 is a 128 MB file that is split into two 64 MB blocks, Block1 and Block2. These blocks are stored in DataNode 1 and DataNode 2 respectively. File2 is a 64 MB file that is stored as a single block, Block3, in DataNode 1. File3 is a 64 MB file that is split into three blocks, Block4, Block5, and Block6, each of 32 MB. These blocks are stored in DataNode 2, DataNode 1, and DataNode 2 respectively. The NameNode knows the location and size of each block and file, and can direct the clients to the appropriate DataNodes for accessing the data.

Data replication in HDFS is the process of copying the data blocks of a file from one node to another in a cluster for fault tolerance and high availability. The number of copies of each block is determined by the replication factor, which can be configured globally or per file. The default replication factor is 3, which means that each block has 3 replicas on different nodes.

The following diagram illustrates the basic architecture of data replication in HDFS using ASCII characters:



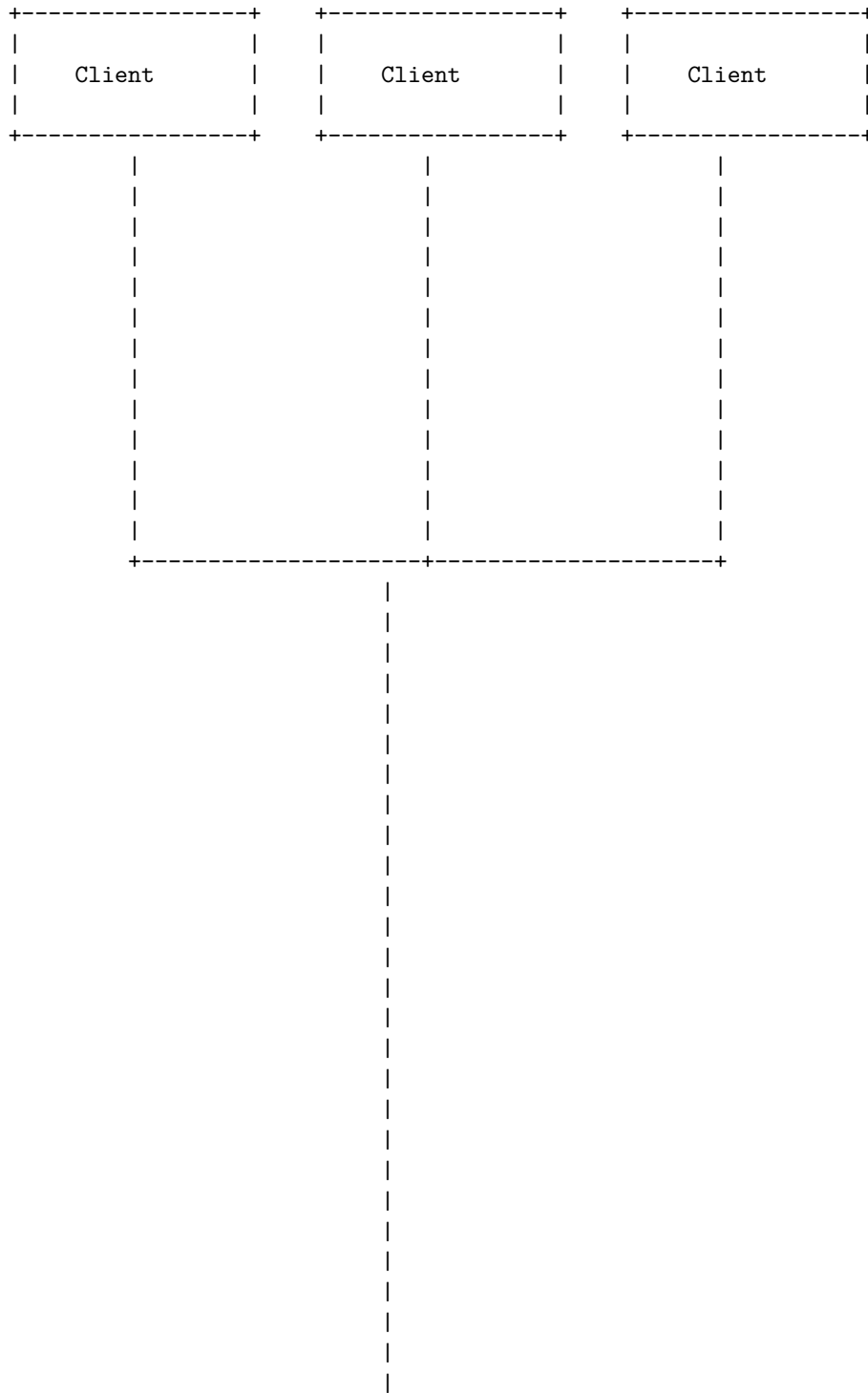


The NameNode is the master node that manages the metadata of the files and blocks, such as their locations, sizes, permissions, and replication factors. The DataNodes are the slave nodes that store the actual data blocks and report to the NameNode periodically. The NameNode is responsible for assigning the blocks to the DataNodes and balancing the load among them. The NameNode also handles the replication of the blocks according to the replication factor and the rack awareness policy, which tries to place the replicas on different racks for better reliability and performance. The NameNode also handles the failure of the DataNodes and the recovery of the blocks.

The diagram shows an example of how the files and blocks are distributed and replicated across the DataNodes. File1 has a

HDFS stands for Hadoop Distributed File System. It is a file system that stores data in a distributed manner across multiple nodes in a cluster. HDFS divides the data into fixed-size blocks (default 128 MB) and replicates each block on different DataNodes for fault tolerance. HDFS also maintains a NameNode that manages the file system namespace and the metadata of the blocks.

The following diagram illustrates the basic architecture of HDFS using ASCII characters:



|

|

|

|

|

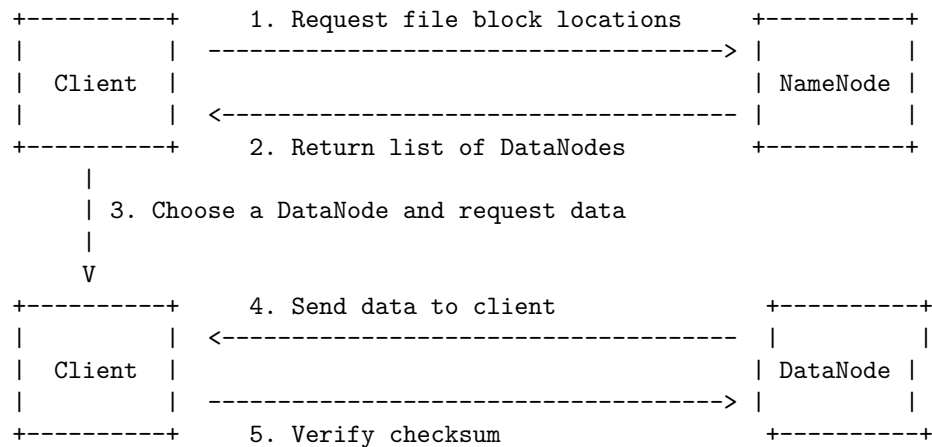
|

|

Read operations in HDFS

To read a file from HDFS, a client needs to interact with the NameNode, which stores the metadata about the file blocks and their locations on the DataNodes. The NameNode returns a list of DataNodes that have a copy of the requested file block. The client then contacts one of the DataNodes directly and reads the data from it. The client can also perform checksum verification to ensure data integrity.

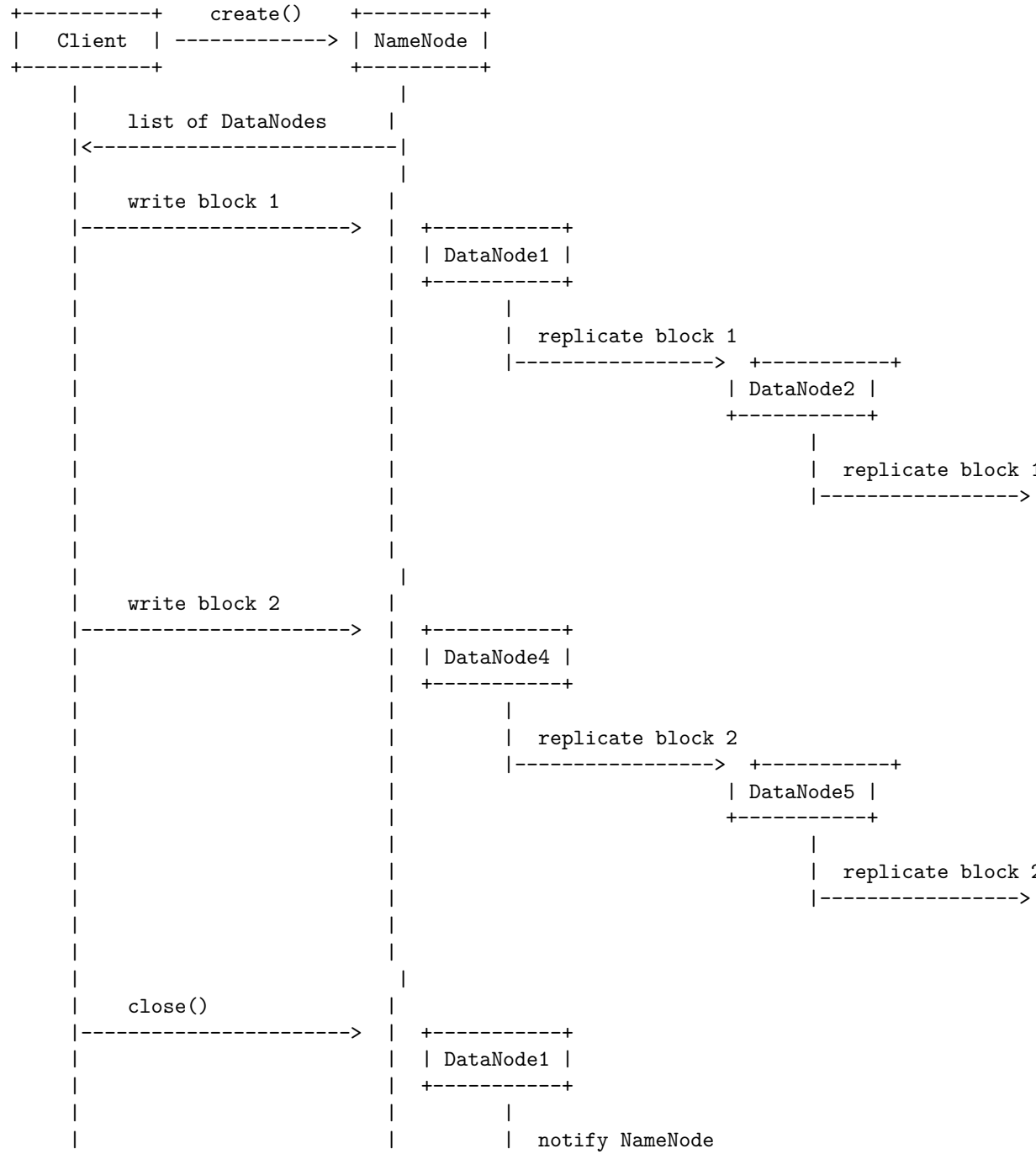
The following diagram illustrates the basic steps of a read operation in HDFS using ASCII art:

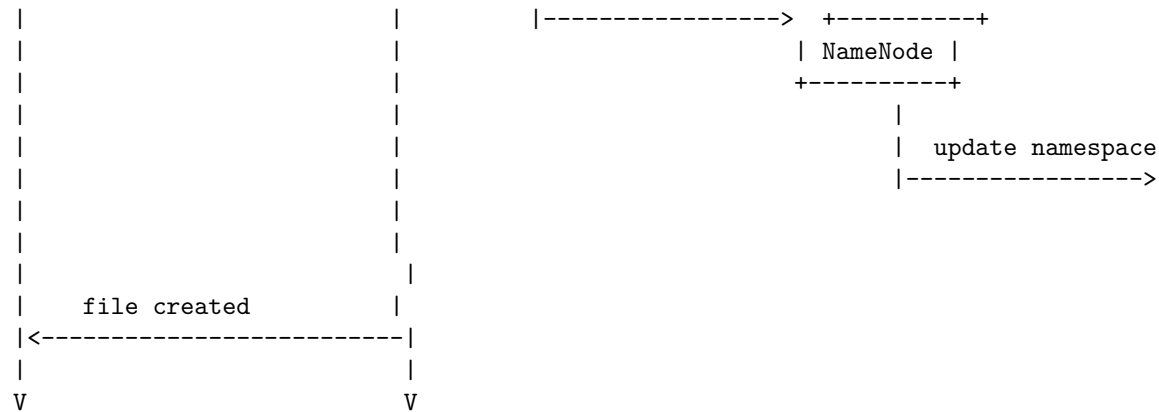


Write operations in HDFS involve the following steps:

1. The client contacts the NameNode and requests to create a new file in the HDFS namespace.
2. The NameNode checks if the file already exists or if the client has the permission to write the file. If not, it throws an exception to the client.
3. The NameNode returns a list of DataNodes that can host the replicas of the first block of the file to the client.
4. The client caches the list of DataNodes and writes the first block of the file to the first DataNode in the list using a TCP connection.
5. The first DataNode starts replicating the block to the second DataNode in the list, which in turn replicates it to the third DataNode, and so on. This forms a pipeline of DataNodes for each block.
6. When the first block is written, the client requests the NameNode for a new list of DataNodes for the second block, and repeats the same process until all the blocks of the file are written.
7. The client finalizes the file creation by calling `close()` on the output stream, which flushes the remaining packets to the DataNodes and notifies the NameNode.

The following diagram illustrates the basic architecture of a write operation in HDFS:





Java interfaces to HDFS

HDFS is a distributed file system that can be accessed by applications using the Java API. The Java API provides various classes and methods to perform operations on HDFS, such as creating, reading, writing, deleting, and copying files and directories.

The main class that represents the HDFS file system is the `org.apache.hadoop.fs.FileSystem` class, which is an abstract class that defines the common interface for all file systems supported by Hadoop. The `FileSystem` class has a static method called `get()` that returns an instance of a concrete subclass of `FileSystem` based on the configuration and the URI of the file system. For example, to get an instance of the HDFS file system, one can use the following code:

```
Configuration conf = new Configuration();
FileSystem fs = FileSystem.get(new URI("hdfs://namenode:8020"), conf);
```

The `FileSystem` instance can then be used to perform various operations on HDFS, such as creating a file, writing data to a file, reading data from a file, deleting a file, etc. For example, to create a file called `test.txt` in HDFS and write some data to it, one can use the following code:

```
Path path = new Path("/test.txt");
FSDataOutputStream out = fs.create(path);
out.writeUTF("Hello, HDFS!");
out.close();
```

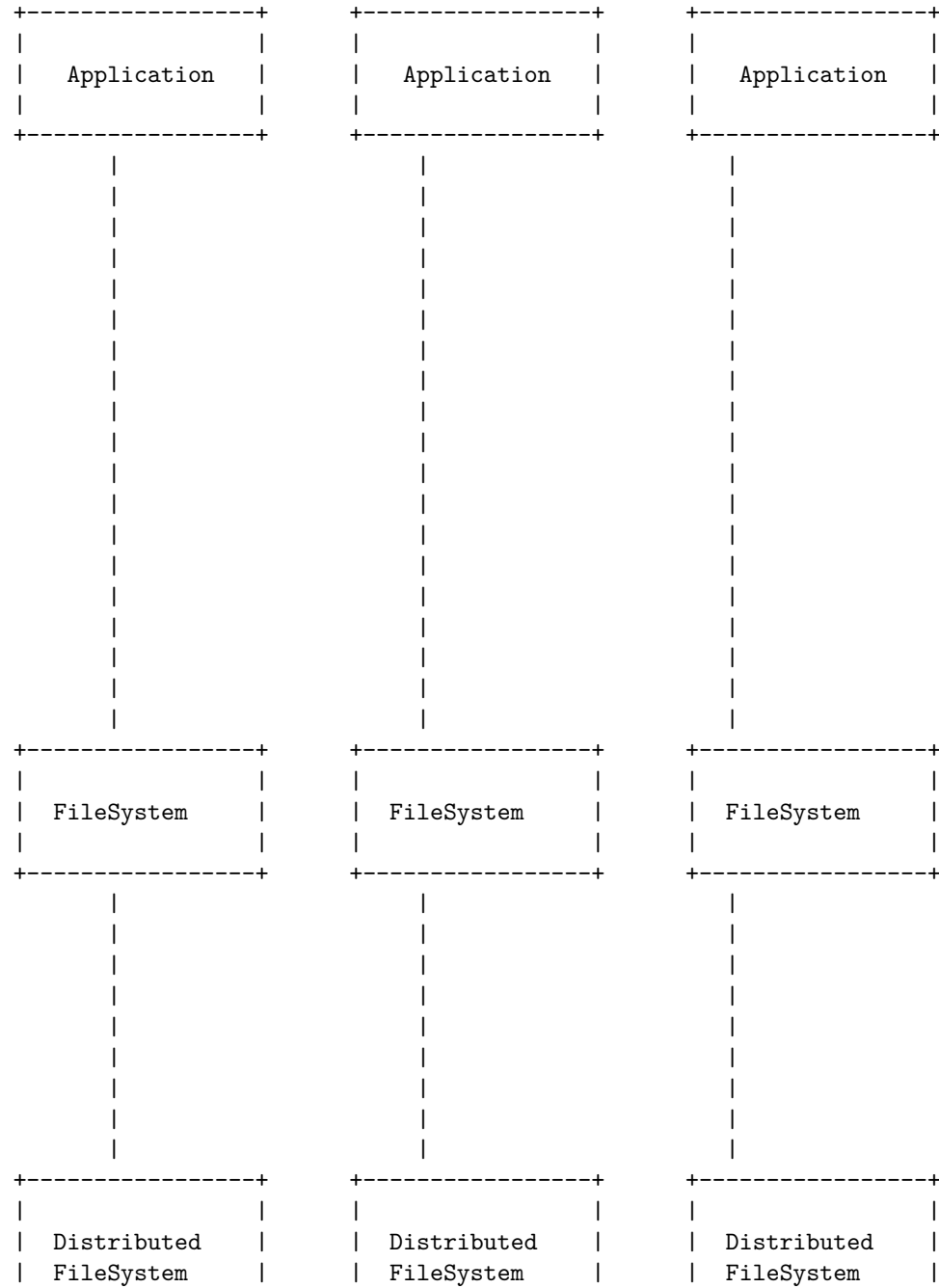
To read the data from the file, one can use the following code:

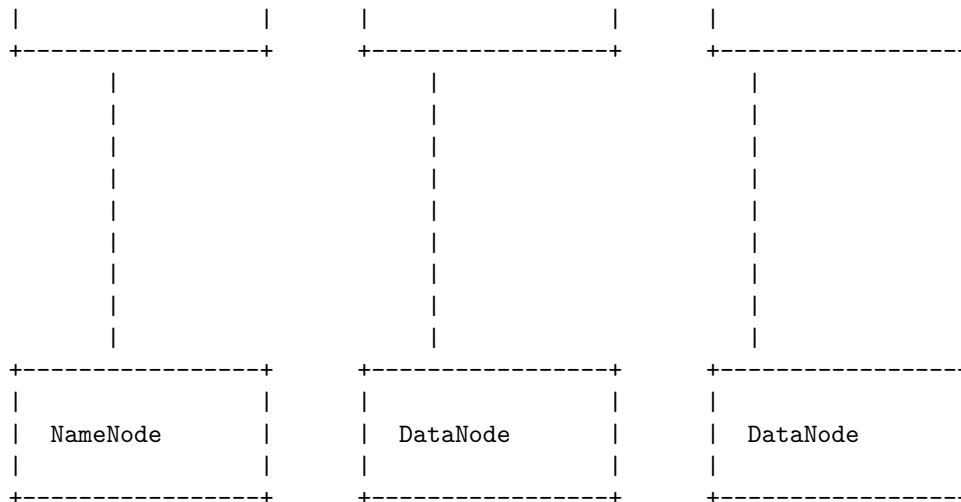
```
FSDataInputStream in = fs.open(path);
String data = in.readUTF();
System.out.println(data);
in.close();
```

To delete the file, one can use the following code:

```
fs.delete(path, false);
```

The following diagram illustrates the basic architecture of the Java interface to HDFS:



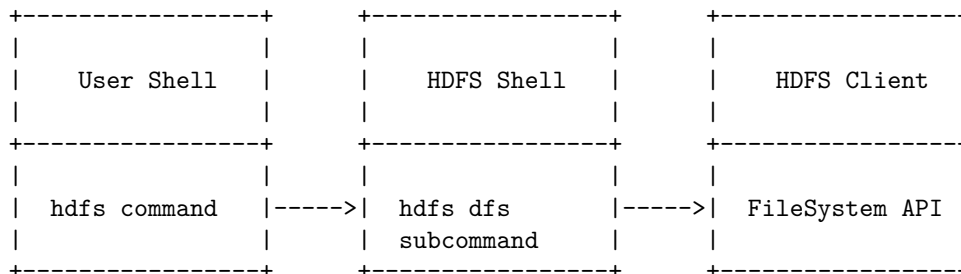


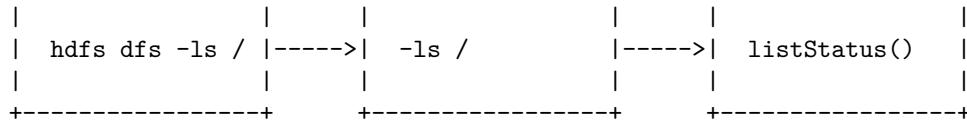
The `FileSystem` class is a client-side abstraction that communicates with the `NameNode` and the `DataNodes` to perform the file system operations. The `NameNode` is the master node that manages the metadata of the file system, such as the file names, locations, permissions, etc. The `DataNodes` are the worker nodes that store the actual data blocks of the files. The Distributed

Command line interface to HDFS

The command line interface (CLI) is one of the simplest ways to interact with HDFS. The CLI has support for filesystem operations like reading the file, creating directories, moving files, deleting data, and listing directories. The CLI can be accessed by using the `hdfs dfs` command, which is a subcommand of the `hdfs` command. The `hdfs` command is a part of the Hadoop distribution and can be found in the `$HADOOP_HOME/bin` directory. The `hdfs dfs` command takes various options and arguments to perform different operations on HDFS. For example, to list the files and directories in the root directory of HDFS, one can use the command `hdfs dfs -ls /`.

The following diagram illustrates the basic architecture of the command line interface to HDFS:



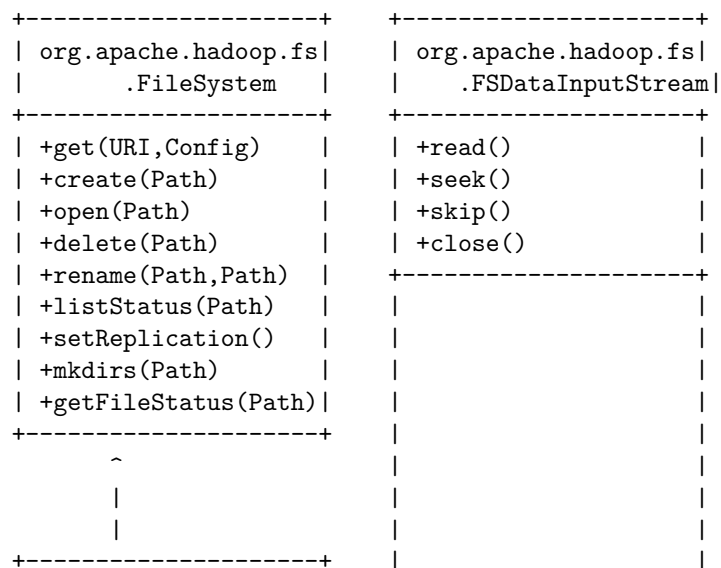


The user shell is the terminal where the user enters the `hdfs` command. The HDFS shell is the component that parses the `hdfs` command and invokes the appropriate subcommand. The HDFS client is the component that communicates with the HDFS cluster using the `FileSystem` API. The `FileSystem` API is a Java interface that abstracts the details of the underlying file system. The HDFS client implements the `FileSystem` API for HDFS and provides methods for performing various operations on HDFS. The HDFS client interacts with the `NameNode` and the `DataNodes` of the HDFS cluster to perform the operations. For example, when the user executes the `hdfs dfs -ls /` command, the HDFS client calls the `listStatus()` method of the `FileSystem` API, which returns the metadata of the files and directories in the root directory of HDFS. The HDFS shell then displays the output of the command to the user shell.

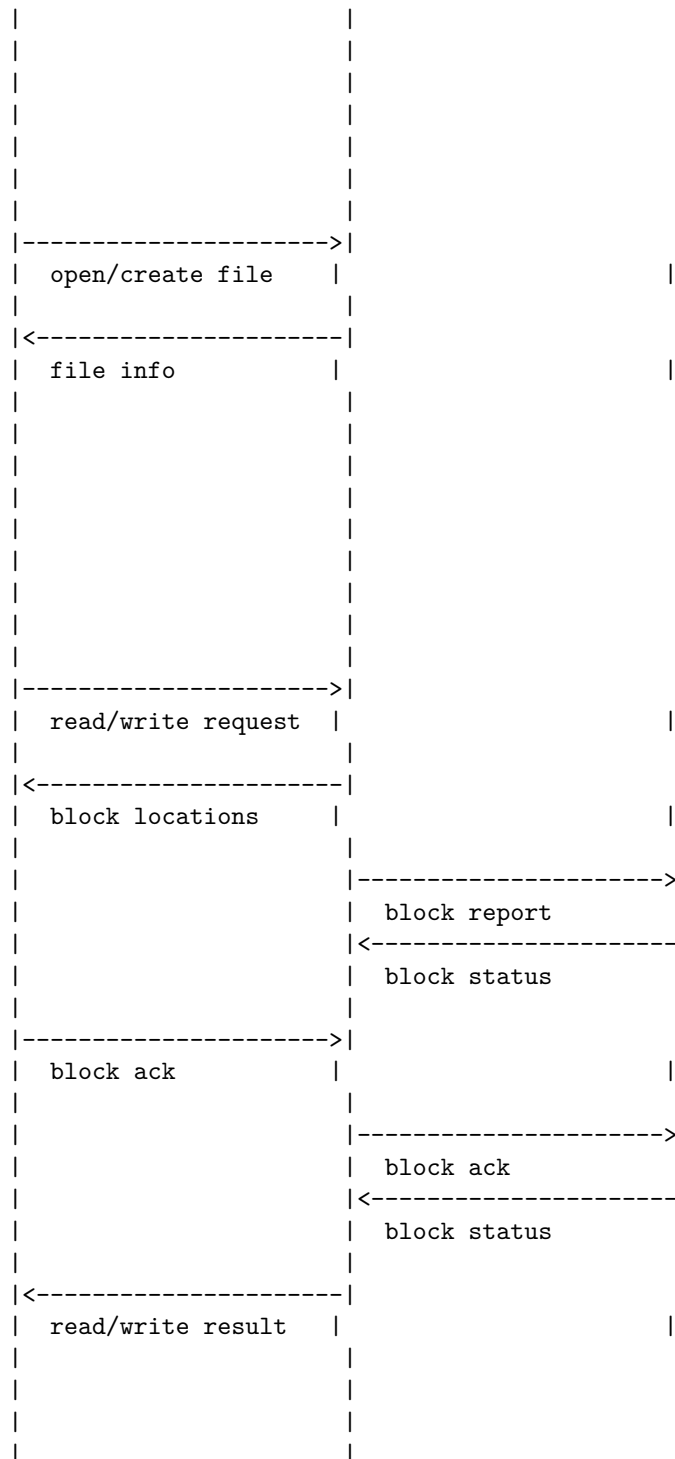
Hadoop file system interfaces are the Java abstract classes and interfaces that represent the client interface to a file system in Hadoop. There are several concrete implementations of these interfaces, such as HDFS, S3, FTP, etc. Hadoop uses the URI scheme to select the appropriate file system instance to communicate with.

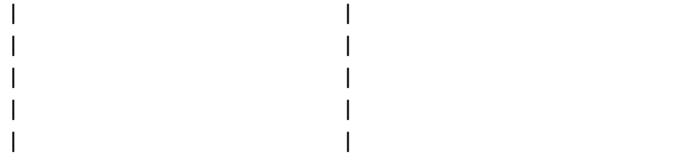
Hadoop file system interfaces

The following is a simplified ASCII diagram of the Hadoop file system interfaces and some of their implementations:



org.apache.hadoop.fs		
.FilterFileSystem		
+-----+		
+create(Path)		
+open(Path)		
+delete(Path)		
+rename(Path,Path)		
+listStatus(Path)		
+setReplication()		
+mkdirs(Path)		
+getFileStatus(Path)		
+-----+		
^		
+-----+		
org.apache.hadoop.fs		
.LocalFileSystem		
+-----+		
+create(Path)		
+open(Path)		
+delete(Path)		
+rename(Path,Path)		
+listStatus(Path)		
+setReplication()		
+mkdirs(Path)		
+getFileStatus(Path)		
+-----+		
^		
+-----+		
org.apache.hadoop.fs		
.RawLocalFileSystem		
+-----+		
+create(Path)		
+open(Path)		
+delete(Path)		
+rename(Path,Path)		
+listStatus(Path)		
+setReplication()		
+mkdirs(Path)		
+getFileStatus(Path)		
+-----+		
^		





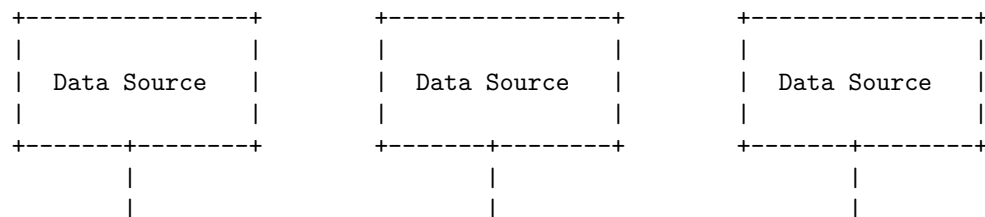
The diagram shows the following steps for data flow in HDFS:

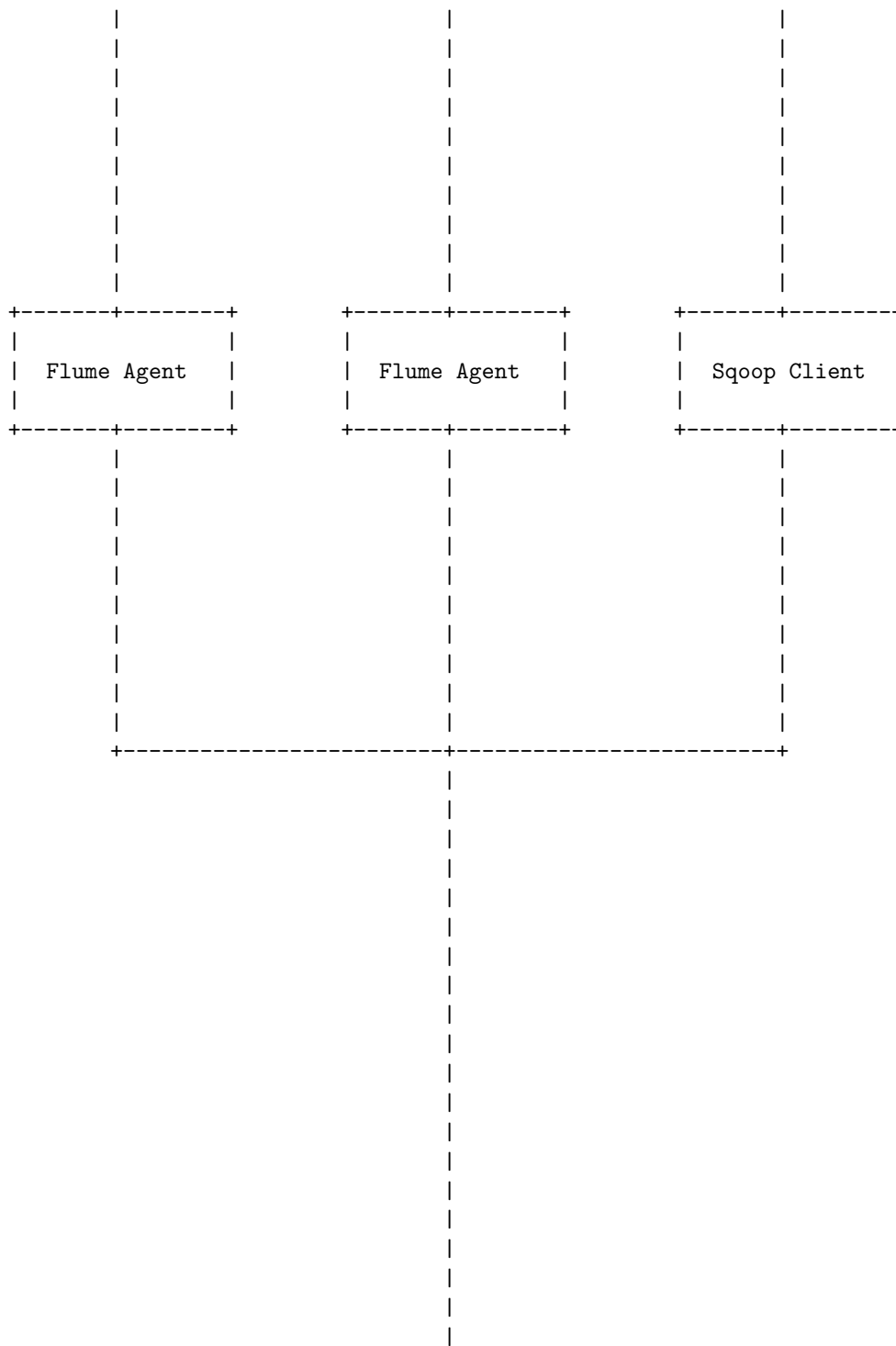
- The client opens or creates a file by calling the DistributedFileSystem (DFS) object, which is an instance of HDFS.
- The DFS object makes a remote procedure call (RPC) to the name node to get the file information, such as the file name, size, permissions, and block locations.
- The name node returns the file information to the client, or creates a new file in the file system namespace if the file does not exist.
- The client sends a read or write request to the name node, specifying the file name and the block number.
- The name node returns the block locations to the client, which are the data nodes that store the replicas of the block.
- The client contacts one of the data nodes directly and reads or writes the data from or to the block.
- The data node sends a block report to the name node, indicating the status of the block, such as whether it is corrupted, under-replicated, or over-replicated.
- The name node sends a block ack to the client, confirming the completion of the read or write operation.
- The client sends a block ack to the name node, acknowledging the receipt of the block ack.
- The name node updates the file system metadata and the block status accordingly.

Data ingest with Flume and Sqoop in HDFS

Flume and Sqoop are two tools that can be used to ingest data from different sources into HDFS, the distributed file system of Hadoop. Flume is designed for streaming data, such as log files, web server logs, social media data, etc. Sqoop is designed for bulk data, such as relational databases, data warehouses, etc.

The following ASCII diagram illustrates the basic architecture of data ingest with Flume and Sqoop in HDFS:





|

|

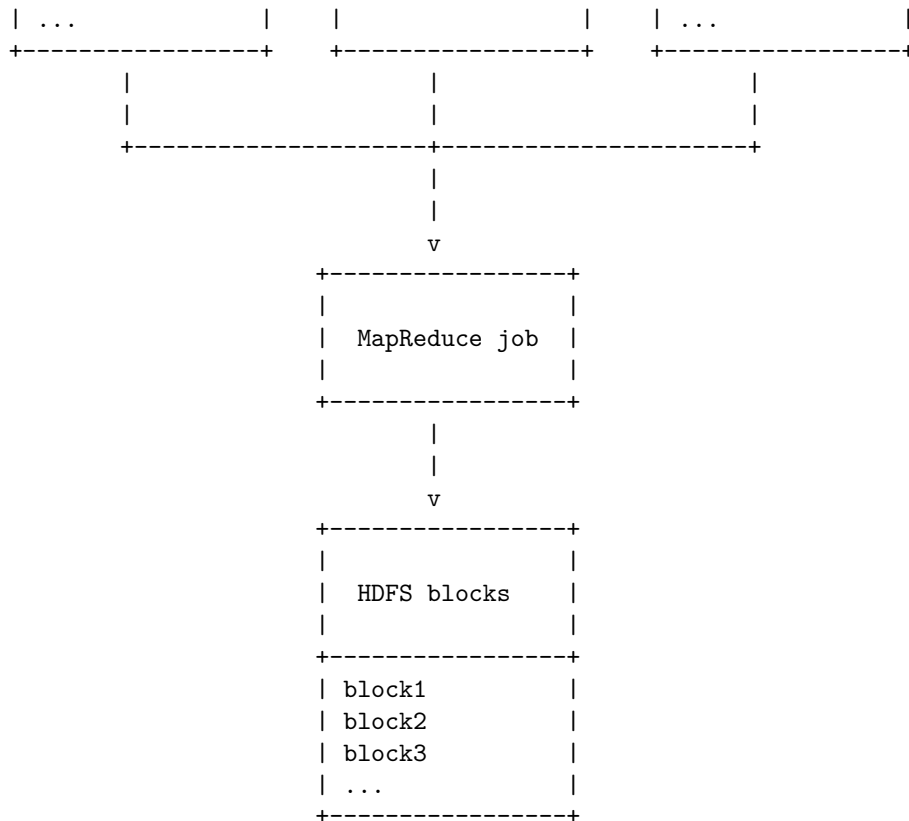
|

|

|

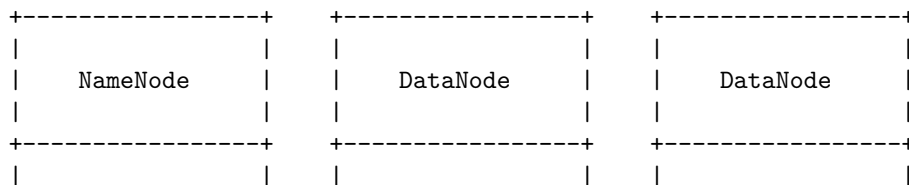
Hadoop archives (HAR) are a way of compressing and storing multiple small files in HDFS more efficiently, reducing the memory usage of the NameNode and allowing transparent access to the files. HAR files are created by running a MapReduce job that takes a collection of files as input and produces an archive file as output. The archive file consists of an index file, a master index file, and one or more part files that contain the compressed data. The index file maps the original file names and sizes to the part files and offsets. The master index file maps the part files to their HDFS block locations. The part files are stored as regular HDFS files and can be accessed using a special `har://` URI scheme.

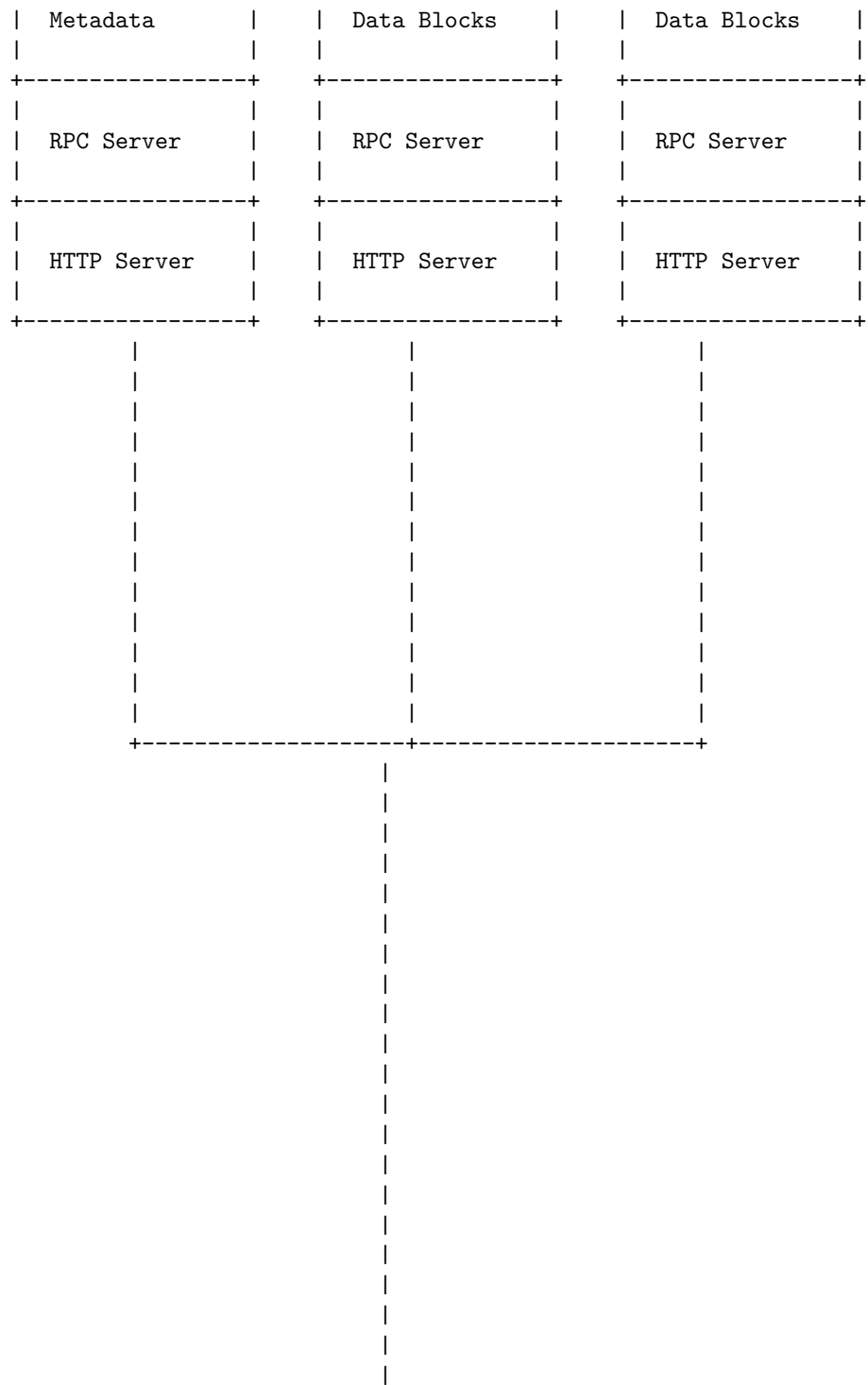
Input files	HAR file	Part files
file1.txt	index	part-0
file2.txt	masterindex	part-1
file3.txt		part-2



Hadoop I/O is the process of reading and writing data from and to the Hadoop Distributed File System (HDFS), which is the storage layer of the Hadoop framework. HDFS is designed to store large amounts of data in a distributed and fault-tolerant manner across multiple nodes in a cluster. HDFS consists of two types of nodes: NameNode and DataNode. NameNode is the master node that manages the metadata of the file system, such as the file names, locations, permissions, etc. DataNode is the worker node that stores the actual data blocks of the files. Each file in HDFS is divided into fixed-size blocks (typically 64 MB or 128 MB) and replicated across multiple DataNodes for reliability. The default replication factor is 3, which means each block has three copies on different DataNodes.

The following diagram illustrates the basic architecture of HDFS:



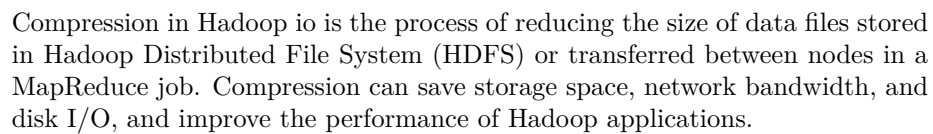


|

|

|

|

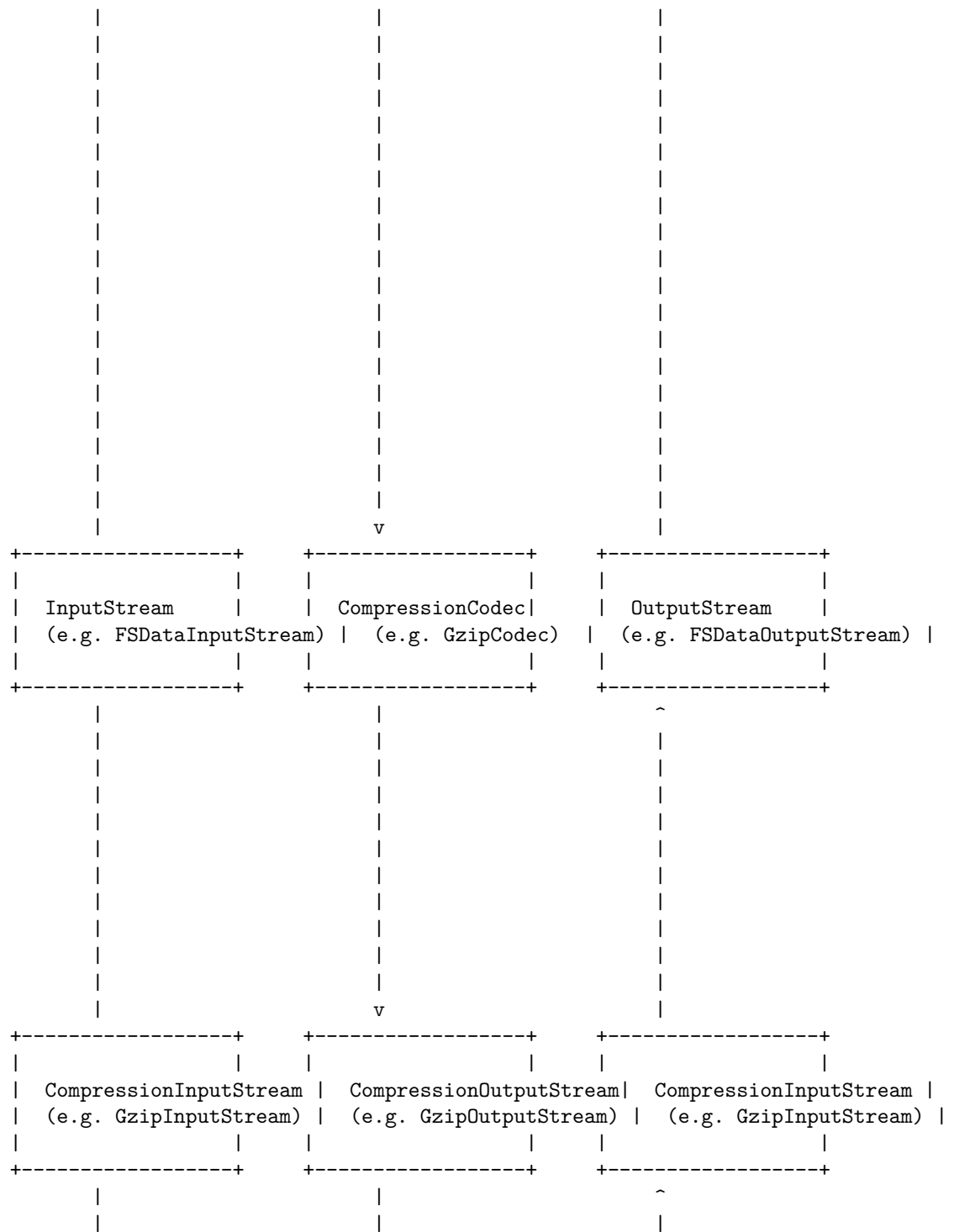


Hadoop supports various compression codecs, such as DEFLATE, gzip, bzip2, LZO, LZ4, and Snappy. Each codec has different characteristics in terms of compression ratio, speed, and splittability. Splittability means that a compressed file can be split into smaller chunks and processed by multiple map tasks in parallel. Only bzip2 is splittable among the standard codecs, but some third-party codecs like LZO can also be made splittable with the help of index files.

Hadoop provides a `CodecFactory` class that can detect the compression format of an input file based on its extension and return the appropriate `CompressionCodec` object. A `CompressionCodec` can be used to create an `InputStream` or an `OutputStream` that can read or write compressed data. Hadoop also provides a `CompressionInputStream` and a `CompressionOutputStream` class that can handle direct byte buffers for faster compression and decompression.

The following diagram illustrates the basic architecture of compression in Hadoop io using ASCII characters:





disk	object	Java objects, Writables, etc.
-----+	-----+	-----+

Avro is a data serialization framework that is widely used in Hadoop and its ecosystem. It stores data along with its schema in a binary format that is compact and efficient. Avro data files are line-oriented, meaning that each row in the file is stored consecutively. Avro data files support compression and are splittable, which makes them suitable for MapReduce data input format.

File-based data structures in Hadoop are data structures that are stored as files in HDFS. They are used to store and process large amounts of data in a distributed manner. Some examples of file-based data structures are sequential files, map files, and Avro data files.

The following diagram illustrates the basic architecture of a file-based data structure in Hadoop:

File System (HDFS)	File System (HDFS)	File System (HDFS)
Data Node	Data Node	Data Node
File Block	File Block	File Block
File Record	File Record	File Record
File Format (Avro, etc.)	File Format (Avro, etc.)	File Format (Avro, etc.)
Schema	Schema	Schema
Data	Data	Data
-----+	-----+	-----+

The following diagram illustrates the basic architecture of an Avro data file:

Avro Data File
-----+
Metadata (Schema, etc.)

```

+-----+
| Data Block 1 |
| (Sync Marker, |
| Length, Data) |
+-----+
| Data Block 2 |
| (Sync Marker, |
| Length, Data) |
+-----+
| ...          |
+-----+
| Data Block N |
| (Sync Marker, |
| Length, Data) |
+-----+

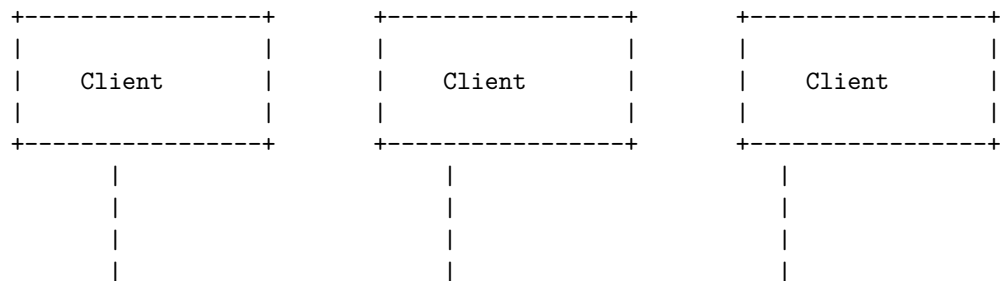
```

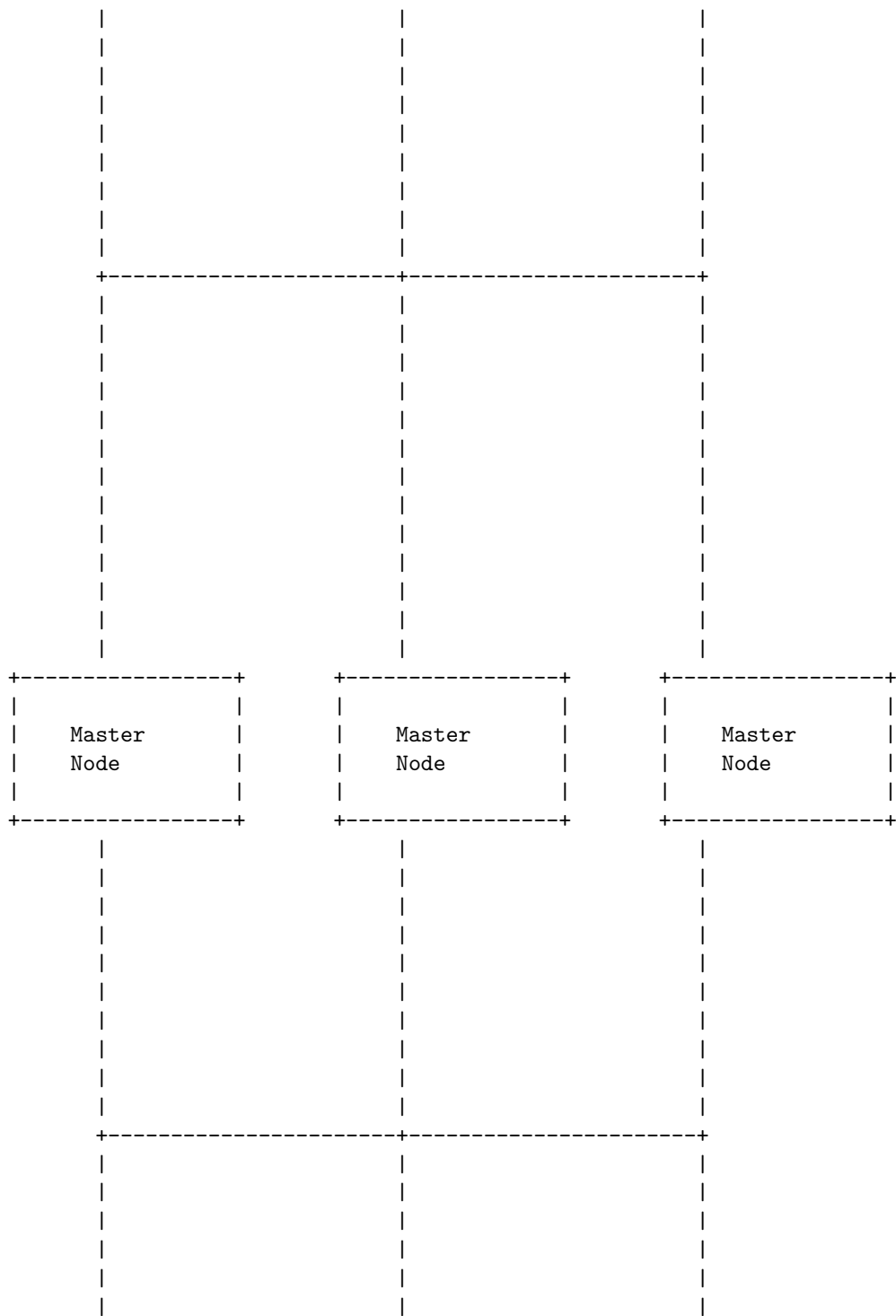
A Hadoop environment is a distributed computing environment that uses Apache Hadoop software to process large data sets across clusters of commodity computers. A Hadoop environment consists of several components, such as:

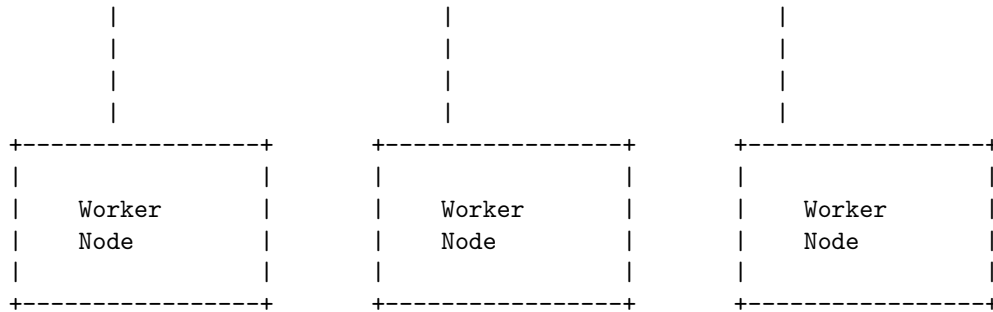
- Hadoop Distributed File System (HDFS): A distributed file system that stores data on the cluster nodes and provides high-throughput access to the data.
- Hadoop MapReduce: A programming model and software framework for writing applications that process large amounts of data in parallel on the cluster nodes.
- Hadoop YARN: A resource management system that allocates and schedules the cluster resources for running applications.
- Hadoop Common: A set of common utilities and libraries that support the other Hadoop components.
- Hadoop Ecosystem: A collection of other software projects that extend the functionality of Hadoop, such as Hive, Pig, HBase, Spark, etc.

Hadoop Environment

The following diagram illustrates the basic architecture of a Hadoop environment using ASCII art:





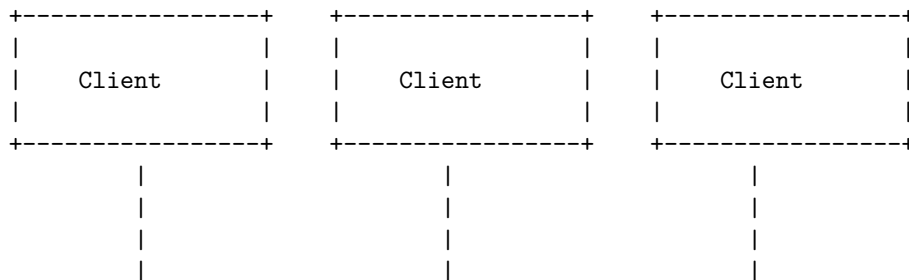


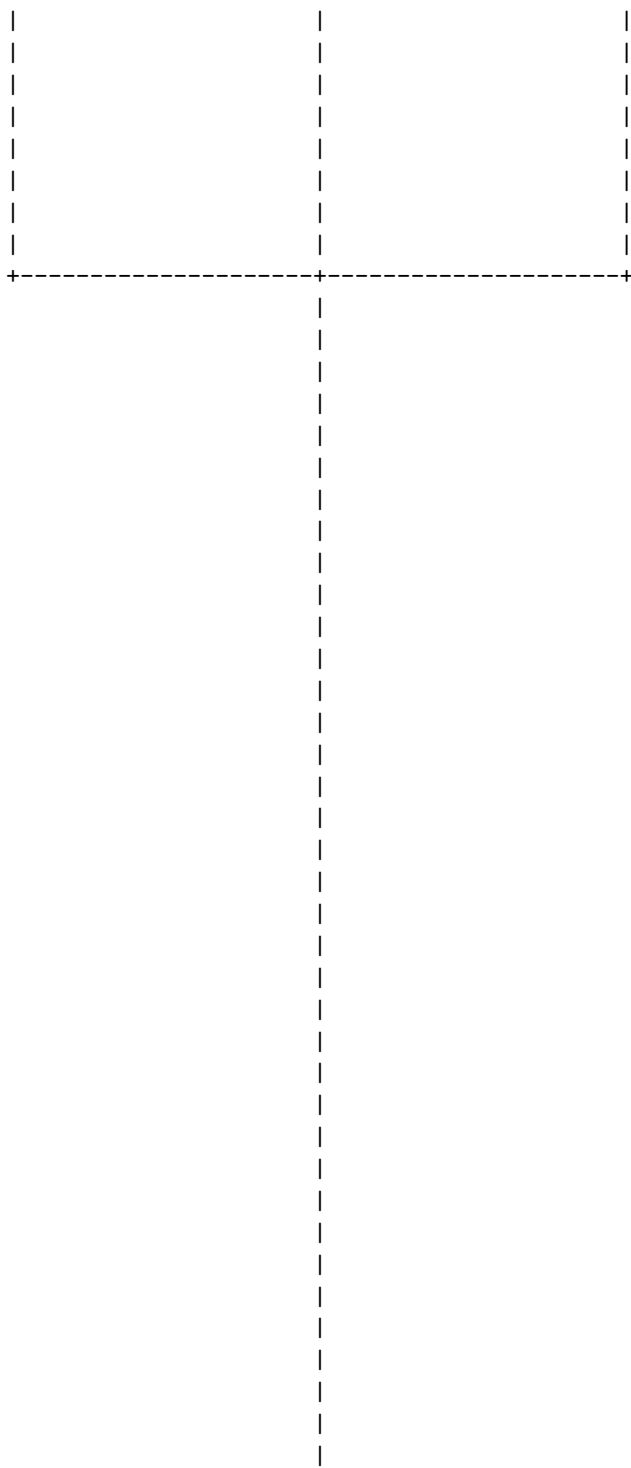
In this diagram, the client nodes are the machines that submit the applications to the Hadoop environment. The master nodes are the machines that coordinate the execution of the applications and manage the cluster resources. The worker nodes are the machines that store the data and run the tasks of the applications. The master nodes and the worker nodes communicate with each other through the Hadoop Common component. The HDFS component provides the distributed file system for storing and accessing the data. The MapReduce component provides the programming model and the software framework for processing the data. The YARN component provides the resource management system for allocating and scheduling the cluster resources. The Hadoop Ecosystem component provides the other software projects that extend the functionality of Hadoop.

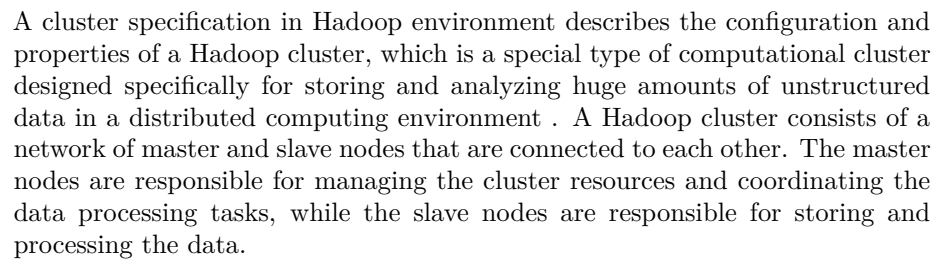
A Hadoop cluster is a collection of computers, known as nodes, that are networked together to store and analyze large amounts of data in a distributed computing environment. A Hadoop cluster follows a master-slave architecture, where the master node coordinates the tasks and the slave nodes execute them. The master node consists of a NameNode, a Secondary NameNode, and a JobTracker, while the slave nodes consist of DataNodes and TaskTrackers. The client node is the interface between the user and the cluster, where the user submits the jobs and monitors their progress.

Setting up a Hadoop cluster in Hadoop Environment

The following diagram illustrates the basic architecture of a Hadoop cluster in a Hadoop environment using ASCII art:

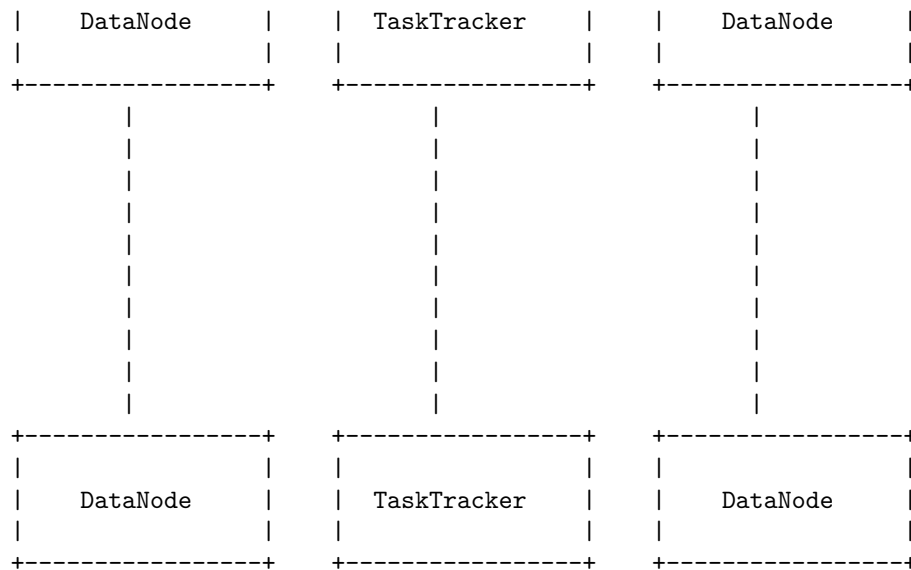






The following diagram illustrates the basic architecture of a Hadoop cluster using ASCII art:





The master nodes are:

- **NameNode:** The NameNode is the central authority that manages the file system namespace and regulates access to files by clients. It also keeps track of the location of data blocks on the DataNodes.
- **JobTracker:** The JobTracker is the central authority that manages the execution of MapReduce jobs on the cluster. It assigns tasks to the TaskTrackers and monitors their progress and status.
- **Secondary NameNode:** The Secondary NameNode is a backup node that periodically merges the namespace image with the edit log to prevent the edit log from becoming too large. It also provides a checkpoint for the NameNode in case of failure.

The slave nodes are:

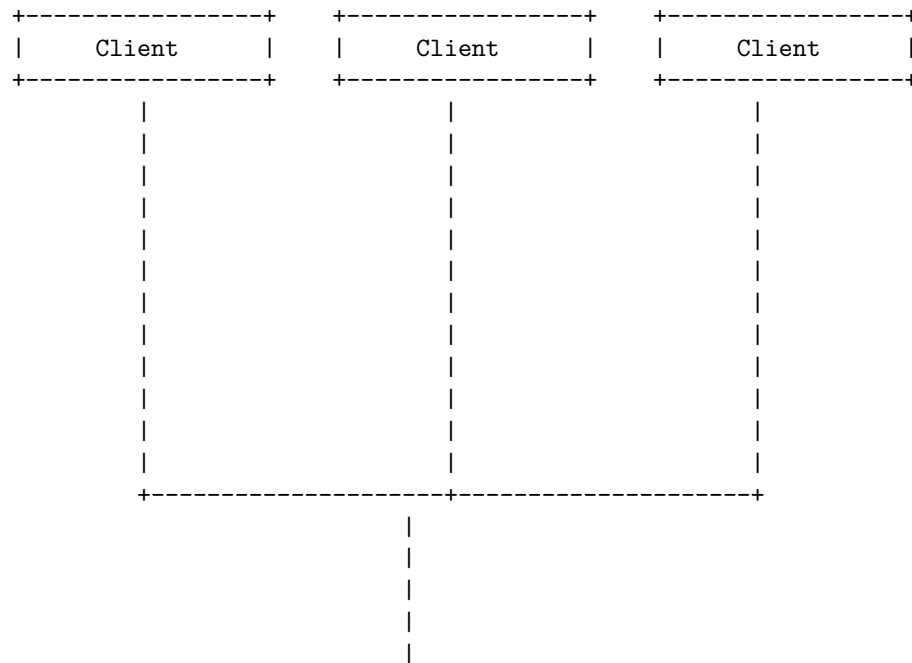
- **DataNode:** The DataNode is the node that stores and serves the data blocks to the clients and the NameNode. It also performs data operations such as replication, deletion, and rebalancing as instructed by the NameNode.
- **TaskTracker:** The TaskTracker is the node that runs the MapReduce tasks assigned by the JobTracker. It also reports the task status and progress to the JobTracker.

The cluster specification in Hadoop environment can be configured by editing the configuration files in the Hadoop installation directory, such as `core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, and `yarn-site.xml`. These files contain the parameters for the Hadoop daemons, such as the hostnames, ports, directories, memory, and CPU settings. The cluster specification can also be modified by using the Hadoop command-line interface or the web-based user interface.

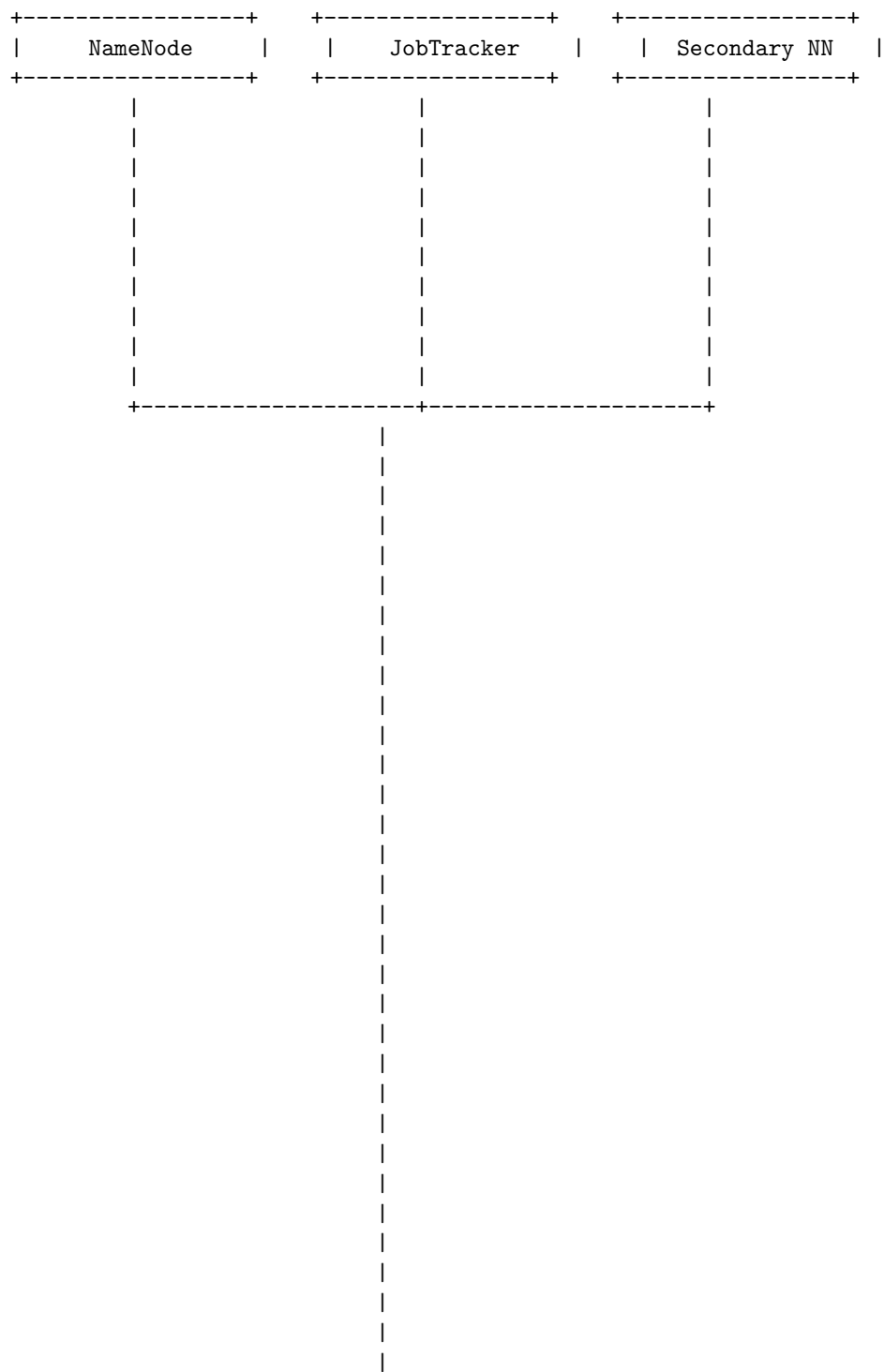
A cluster setup and installation in Hadoop environment involves the following steps:

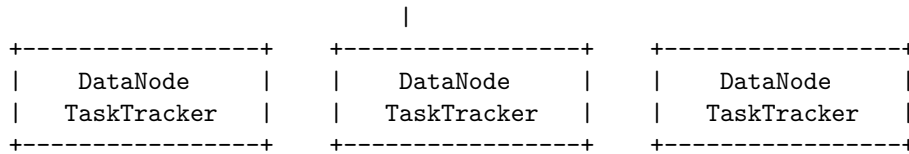
- Installing the required software on all the nodes in the cluster, such as Java, SSH, and Hadoop.
- Unpacking the Hadoop software on all the nodes or installing it via a packaging system as appropriate for your operating system.
- Dividing up the hardware into functions, such as NameNode, DataNode, JobTracker, and TaskTracker. Typically, one machine in the cluster is designated as the NameNode and another machine as the JobTracker, exclusively. These are the masters. The rest of the machines in the cluster act as both DataNode and TaskTracker. These are the workers.
- Configuring the environment variables, such as JAVA_HOME, HADOOP_HOME, HADOOP_CONF_DIR, etc.
- Configuring the Hadoop configuration files, such as core-site.xml, hdfs-site.xml, mapred-site.xml, etc. These files specify the properties of the cluster, such as the location of the NameNode, the replication factor, the memory and CPU allocation, etc.
- Formatting the Hadoop file system (HDFS) on the NameNode.
- Starting the Hadoop daemons on all the nodes, such as NameNode, DataNode, JobTracker, and TaskTracker.
- Verifying the status of the cluster using web interfaces or command-line tools, such as jps, hadoop dfsadmin, hadoop job, etc.

The following diagram illustrates the basic architecture of a Hadoop cluster:



|





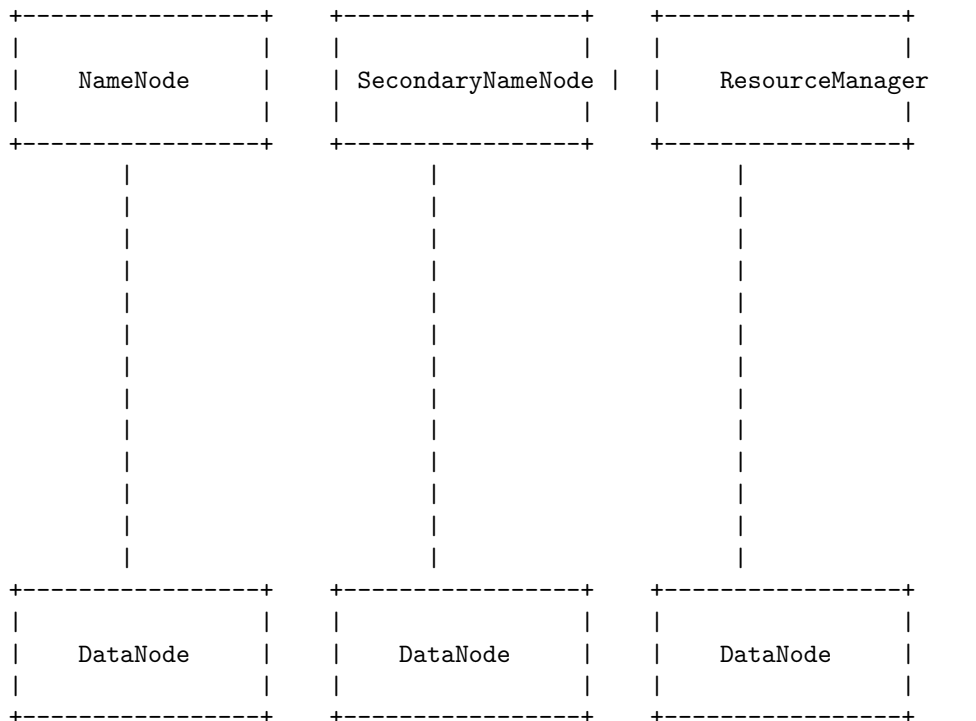
Hadoop configuration in Hadoop Environment

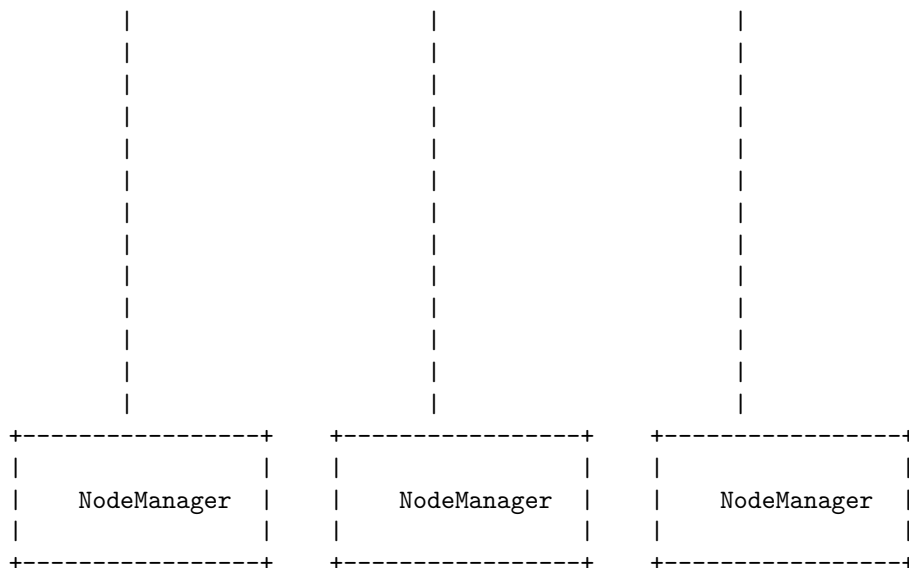
Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers. Hadoop consists of two main components: Hadoop Distributed File System (HDFS) and Yet Another Resource Negotiator (YARN).

HDFS is a distributed file system that provides high-throughput access to data stored on the cluster nodes. HDFS splits the data into blocks and replicates them across multiple nodes for fault tolerance. HDFS also maintains the meta-data of the files, such as the location of the blocks, the size of the files, and the permissions.

YARN is a resource management system that allocates resources (such as CPU and memory) to the applications running on the cluster. YARN also schedules the execution of the tasks and monitors their progress. YARN supports various types of applications, such as MapReduce, Spark, Hive, and Pig.

The following diagram illustrates the basic architecture of a Hadoop cluster:





The NameNode is the master node that manages the namespace and the meta-data of the files stored on HDFS. It also coordinates the replication and the recovery of the blocks. The NameNode communicates with the DataNodes, which are the slave nodes that store the actual data blocks. The NameNode also communicates with the clients, which are the applications that read and write data to HDFS.

The SecondaryNameNode is a helper node that periodically merges the edits log and the fsimage file of the NameNode. It also acts as a backup for the NameNode in case of failure.

The ResourceManager is the master node that manages the resources and the applications running on the cluster. It consists of two main components: the Scheduler and the ApplicationsManager. The Scheduler allocates resources to the applications based on various criteria, such as capacity, fairness, and priority. The ApplicationsManager accepts the application submissions, negotiates the first container for the application, and monitors the application status.

The NodeManager is the slave node that monitors the resource usage and the health of the node. It also communicates with the ResourceManager and the ApplicationMaster. The ApplicationMaster is the process that runs on a container and coordinates the execution of the tasks for a specific application. It requests resources from the ResourceManager and launches containers on the NodeManager. It also reports the progress and the status of the application to the ResourceManager and the client.

Security in Hadoop consists of four main components: authentication, authorization, auditing, and encryption. Authentication is the process of verifying the identity of the users and services that interact with Hadoop. Authorization

is the process of granting or denying access to the resources and operations in Hadoop based on the user's role and privileges. Auditing is the process of recording and monitoring the activities and events that occur in Hadoop. Encryption is the process of protecting the data in transit and at rest from unauthorized access or modification.

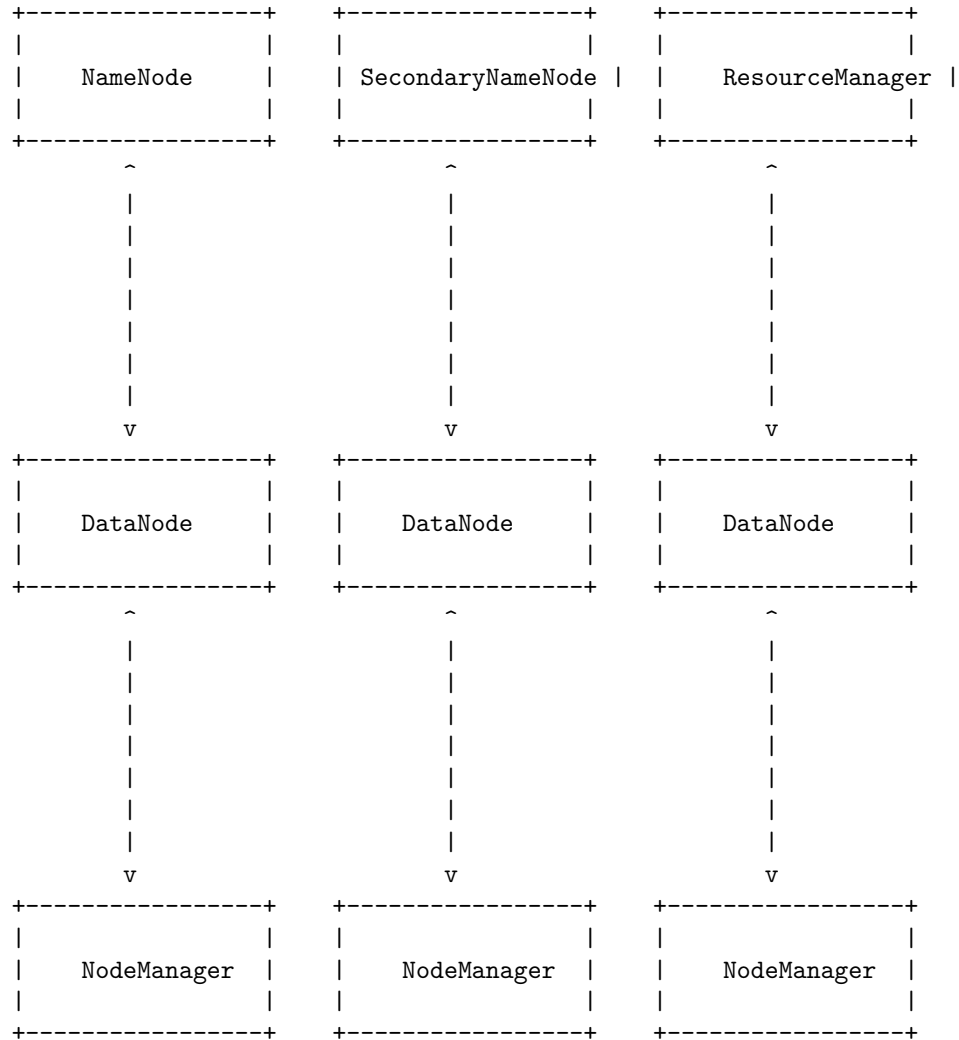
The following diagram illustrates the basic architecture of security in Hadoop in a Hadoop environment:

Client	NameNode	DataNode
Kerberos Authentication	Kerberos Authentication	Kerberos Authentication
Hadoop RPC Encryption	Hadoop RPC Encryption	Hadoop RPC Encryption
HDFS Authorization	HDFS Authorization	HDFS Authorization Data Encryption
HDFS Auditing	HDFS Auditing	HDFS Auditing

Administering Hadoop in Hadoop Environment involves managing the Hadoop clusters and other resources in the Hadoop ecosystem. A Hadoop administrator is responsible for installing, configuring, monitoring, and troubleshooting the Hadoop daemons and services. The basic architecture of a Hadoop cluster consists of a master node and multiple worker nodes. The master node runs the NameNode and the ResourceManager services, which are responsible for managing the metadata and the resources of the cluster. The worker nodes run the DataNode and the NodeManager services, which are responsible for storing the data and executing the tasks. The Hadoop administrator can use the Hadoop shell commands and the web interfaces to interact with the cluster and perform various operations.

The following diagram illustrates the basic architecture of a Hadoop cluster

using ASCII art:



HDFS monitoring & maintenance in Hadoop Environment

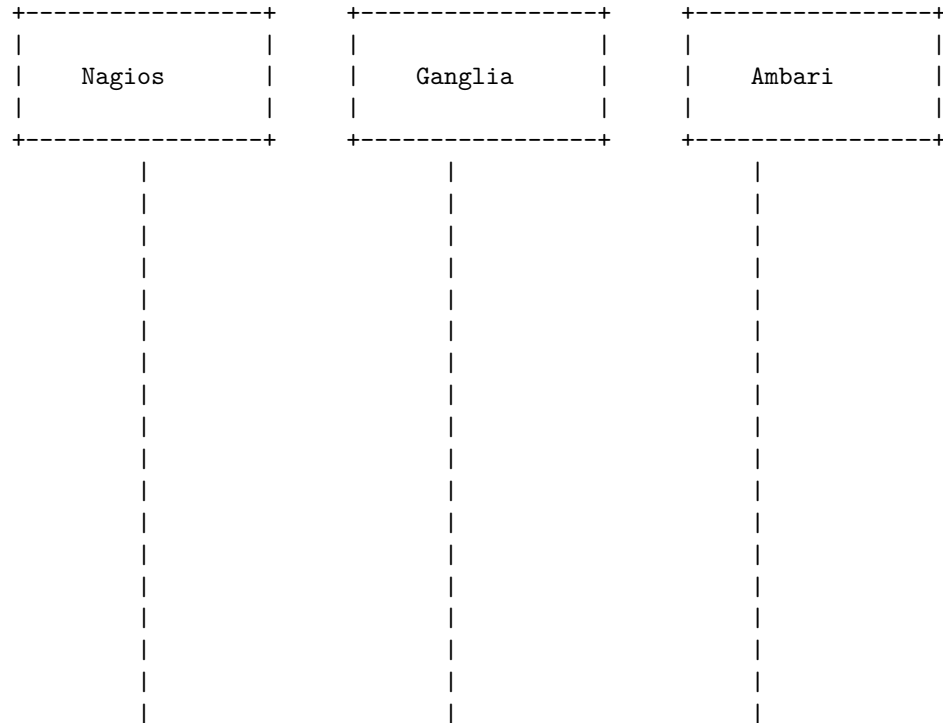
HDFS (Hadoop Distributed File System) is a distributed file system that stores large data sets across multiple nodes and maintains the metadata in the form of log files. HDFS consists of two core components: NameNode and DataNode. NameNode is the master node that manages the file system namespace, the access control, and the block mapping. DataNode is the worker node that stores the actual data blocks and sends heartbeat and block reports to the NameNode. HDFS also supports secondary NameNode and standby NameNode for high availability and fault tolerance.

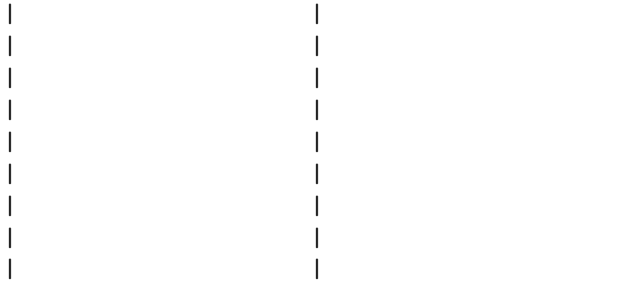
HDFS monitoring & maintenance in Hadoop Environment involves checking the

status, performance, and health of the HDFS cluster, the NameNode, and the DataNodes. Some of the common metrics and tools for HDFS monitoring are:

- **HDFS metrics:** These include NameNode metrics (such as heap memory usage, file system operations, block reports, etc.) and DataNode metrics (such as disk space usage, data transfer, block verification, etc.). These metrics can be accessed through the web UI of the NameNode and the DataNodes, or through the JMX interface.
- **HDFS commands:** These are the command-line tools for interacting with the HDFS file system, such as put, get, ls, du, df, fsck, etc. These commands can be used to upload, download, list, check, and manage files and directories in HDFS. They can also be used to diagnose and repair issues in the HDFS cluster, such as corrupted or missing blocks, under-replicated blocks, etc.
- **HDFS tools:** These are the external tools that can be integrated with HDFS for monitoring and maintenance purposes, such as Nagios, Ganglia, Ambari, Cloudera Manager, etc. These tools can provide a graphical dashboard, alerts, reports, and analytics for the HDFS cluster, the NameNode, and the DataNodes. They can also help with configuration, backup, recovery, and optimization of the HDFS cluster.

The following diagram illustrates the basic architecture of a HDFS cluster and the monitoring and maintenance tools:





Hadoop benchmarks are tests that measure the performance of a Hadoop cluster in various aspects, such as HDFS read and write, MapReduce sorting, and data generation. Hadoop provides some built-in benchmark applications that can be run on a Hadoop cluster using the command-line interface. Some of the common Hadoop benchmarks are:

- **TestDFSIO:** This benchmark tests the read and write performance of HDFS by using one map task per file. It can be run with the following commands:

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-client-jobclient-*-tests.jar  
TestDFSIO -write -nrFiles <number of files> -fileSize <size of  
each file>
```

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-client-jobclient-*-tests.jar  
TestDFSIO -read -nrFiles <number of files> -fileSize <size of  
each file>
```

- **TeraSort:** This benchmark tests the sorting performance of MapReduce by using a custom partitioner and a custom output format. It consists of three components: TeraGen, TeraSort, and TeraValidate. TeraGen generates random data, TeraSort sorts the data using MapReduce, and TeraValidate validates the output. They can be run with the following commands:

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar  
teragen <number of 100-byte rows> <output directory>
```

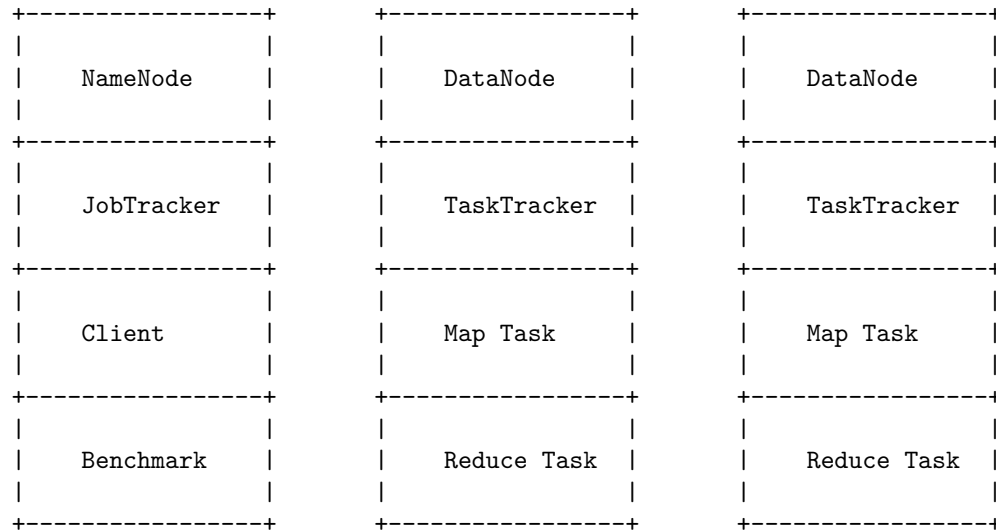
```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar  
terasort <input directory> <output directory>
```

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar  
teravalidate <output directory> <report directory>
```

- **Pi:** This benchmark estimates the value of pi by using a Monte Carlo method. It can be run with the following command:

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar  
pi <number of maps> <number of samples per map>
```

The following diagram illustrates the basic architecture of a Hadoop cluster and how the benchmark applications interact with it:

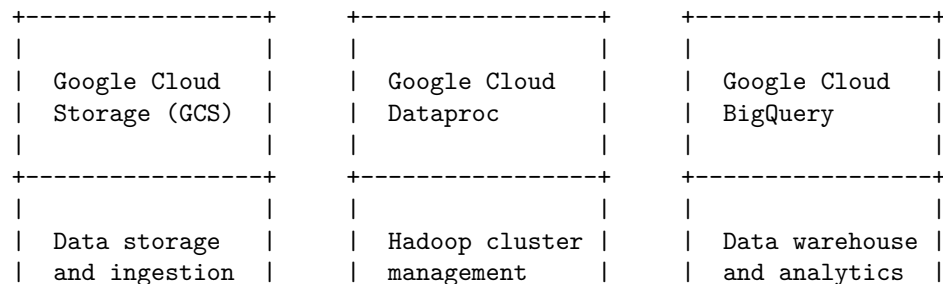


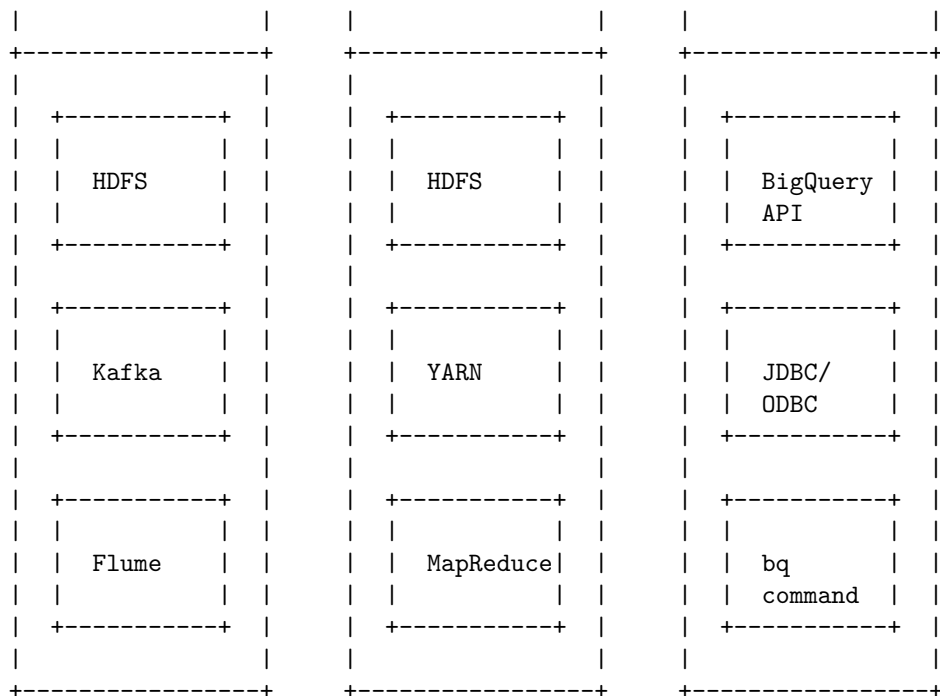
The client node runs the benchmark application, which submits a job to the JobTracker on the NameNode. The JobTracker assigns map and reduce tasks to the TaskTrackers on the DataNodes, which execute them in parallel. The map tasks read data from HDFS and process it, while the reduce tasks aggregate the results and write them back to HDFS. The benchmark application collects the output and reports the performance metrics.

Hadoop in the cloud is a way of running Hadoop clusters on cloud platforms such as Google Cloud, Amazon Web Services, or Microsoft Azure. Hadoop in the cloud can offer benefits such as scalability, elasticity, cost-effectiveness, and data locality. However, it also requires some changes in the architecture and security of Hadoop compared to running it on-premises.

Hadoop in the cloud

The following is a simplified ASCII diagram of a possible Hadoop in the cloud architecture, using Google Cloud as an example:





In this diagram, Google Cloud Storage (GCS) is used as the primary data storage and ingestion layer, which can store structured, semi-structured, or unstructured data in a scalable and durable way. GCS can also integrate with other data sources such as Kafka or Flume for streaming data ingestion.

Google Cloud Dataproc is used as the Hadoop cluster management service, which can create, configure, and delete Hadoop clusters on demand. Dataproc supports Hadoop components such as HDFS, YARN, MapReduce, Hive, Spark, Pig, and more. Dataproc can also leverage GCS as the underlying file system for HDFS, which can improve performance and reduce costs.

Google BigQuery is used as the data warehouse and analytics service, which can run SQL queries over large datasets stored in GCS or other sources. BigQuery can also integrate with Hadoop components such as BigQuery API, JDBC/ODBC drivers, or bq command line tool for data access and manipulation.

This is just one example of how Hadoop in the cloud can be implemented. Different cloud providers may have different services and features that can be used for Hadoop

Unit 3 - Hadoop Eco System and YARN , no SQL databases , MongoDB , Spark , Scala

The following is a detailed ascii diagram for Unit 3 - Hadoop Eco System and YARN , no SQL databases , MongoDB , Spark , Scala:

Hadoop	Spark	Scala	MongoDB
HDFS	Spark Core	Scala Compiler	BSON
MapReduce	Spark SQL	Scala Library	Indexing
YARN	Spark Streaming	Scala REPL	Aggregation
Hive	Spark MLlib	Scala IDE	Replication
Pig	Spark GraphX	Scala Test	Sharding
Oozie			CRUD Operations
Zookeeper			Capped Collections
Mahout			

The diagram illustrates the basic architecture of a Hadoop Eco System and YARN , no SQL databases , MongoDB , Spark , Scala. The Hadoop Eco System consists of HDFS, MapReduce, YARN, Hive, Pig, Oozie, Zookeeper and Mahout.

HDFS is the distributed file system that stores the data. MapReduce is the programming model that processes the data in parallel. YARN is the resource manager that allocates the resources for the applications. Hive is the data warehouse that provides SQL-like queries. Pig is the scripting language that simplifies the data analysis. Oozie is the workflow scheduler that coordinates the jobs. Zookeeper is the service that maintains the configuration and coordination of the cluster. Mahout is the machine learning library that provides scalable algorithms.

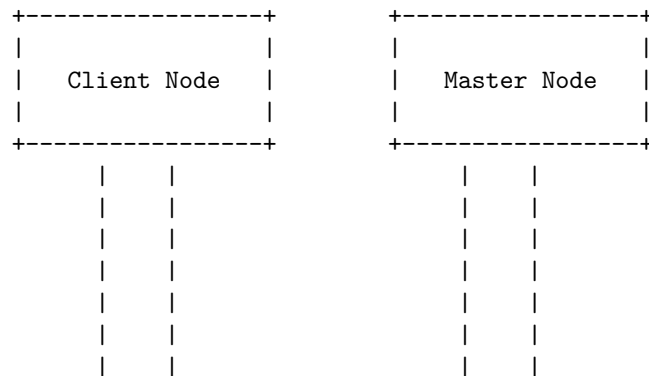
Spark is a fast and general engine for large-scale data processing. It consists of Spark Core, Spark SQL, Spark Streaming, Spark MLlib and Spark GraphX. Spark Core is the foundation that provides the distributed memory abstraction and the basic operations. Spark SQL is the module that supports structured and semi-structured data processing. Spark Streaming is the module that enables real-time data processing. Spark MLlib is the module that provides

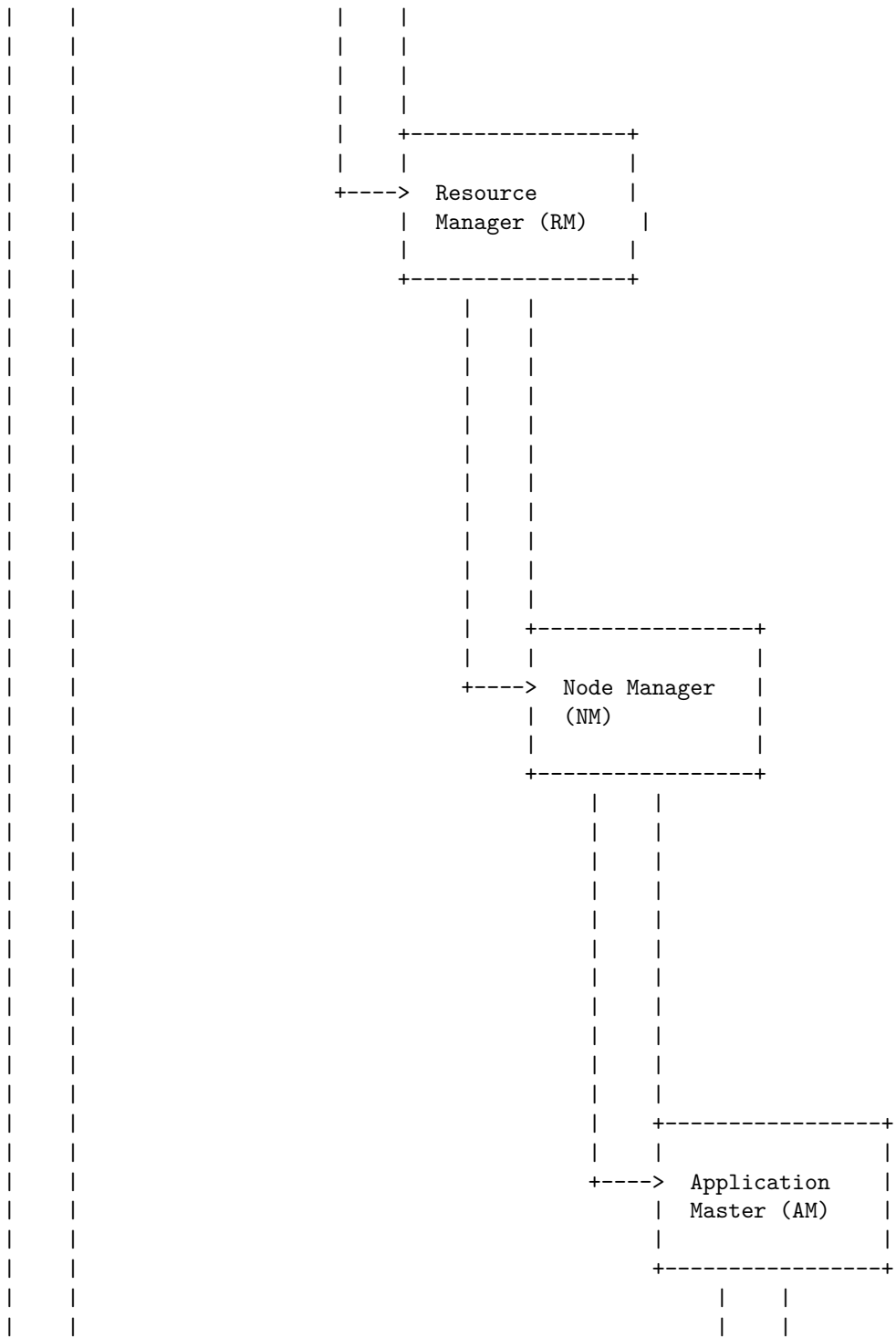
Hadoop Eco System and YARN

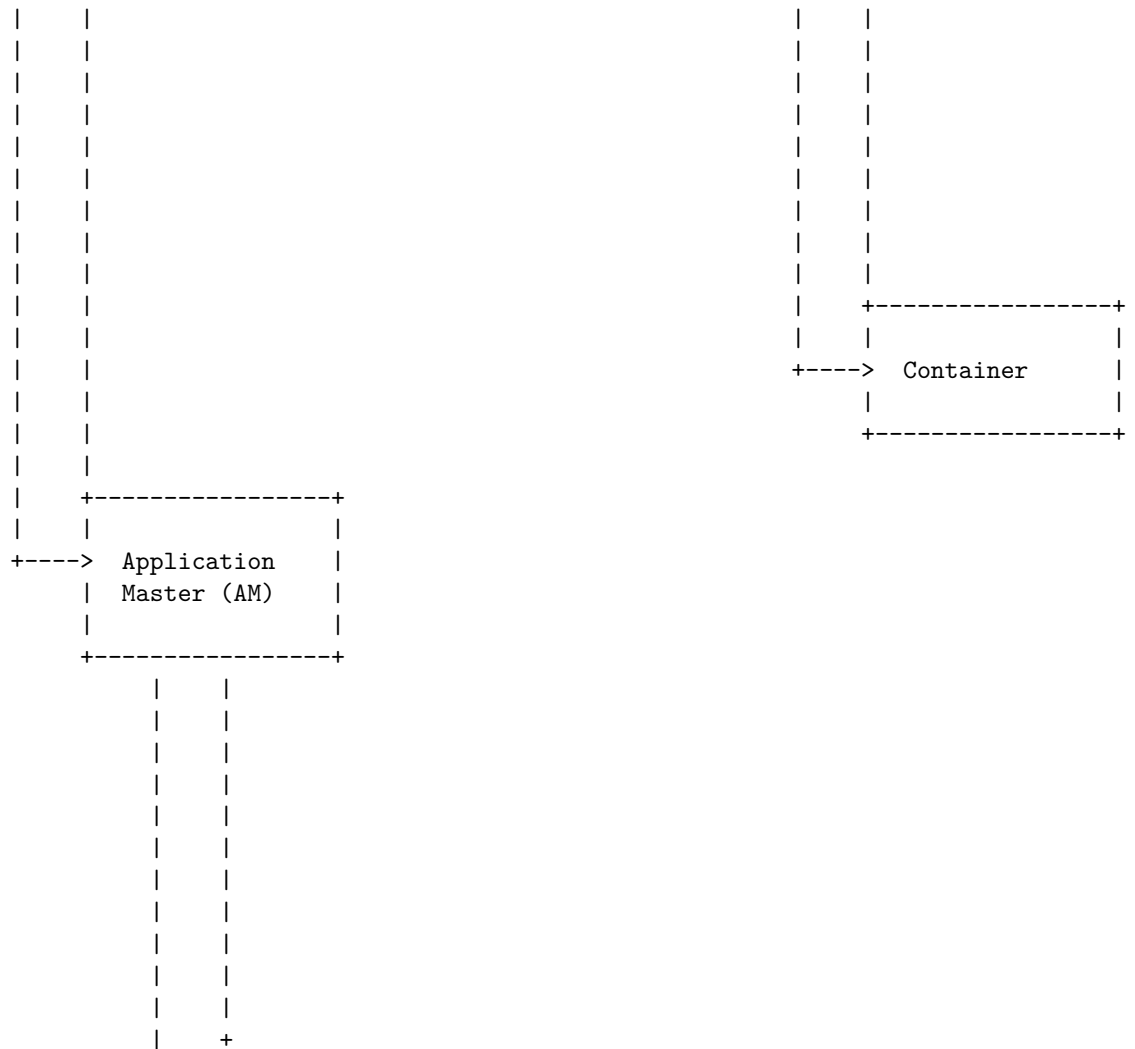
Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers. Hadoop consists of several components, such as Hadoop Distributed File System (HDFS), MapReduce, Hive, Pig, HBase, Oozie, Sqoop, Zookeeper, etc. These components work together to provide various functionalities, such as data storage, data processing, data analysis, data ingestion, data management, etc. The Hadoop ecosystem is the collection of all these components and the tools that interact with them.

YARN stands for Yet Another Resource Negotiator. It is a sub-project of Hadoop that provides a platform for managing and scheduling resources in a Hadoop cluster. YARN was introduced in Hadoop 2.0 to overcome the limitations of MapReduce, such as scalability, resource utilization, and application diversity. YARN separates the resource management and job scheduling functions from the data processing logic, allowing multiple types of applications to run on the same Hadoop cluster.

The following diagram illustrates the basic architecture of YARN:

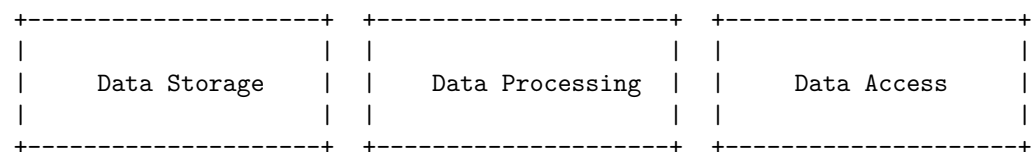






The Hadoop ecosystem is a collection of software components and tools that enable large-scale data processing and analysis using the Hadoop framework. The Hadoop ecosystem consists of four main layers: data storage, data processing, data access, and data management.

The following is a detailed ASCII diagram for the Hadoop ecosystem components:



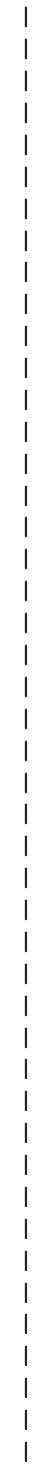
HDFS (Hadoop Distributed File System)	MapReduce (Batch processing)	Hive (SQL-like query language)
HBase (NoSQL database)	Spark (In-memory processing)	Pig (Scripting language)
Kudu (Columnar storage)	Flink (Stream processing)	Sqoop (Data transfer)
Kafka (Message broker)	Storm (Real-time processing)	Flume (Data ingestion)
Data Management		
Zookeeper (Coordination)		
Oozie (Workflow)		
Ambari (Monitoring)		

Schedulers in Hadoop ecosystem are responsible for allocating resources and

The following ASCII diagram illustrates the basic architecture of a Hadoop scheduler:

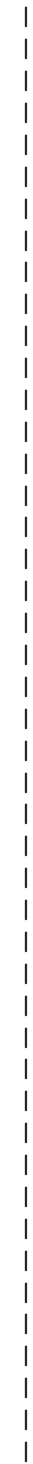


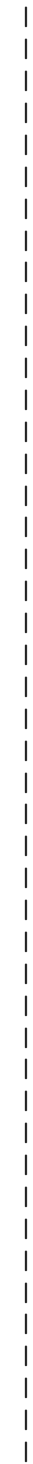




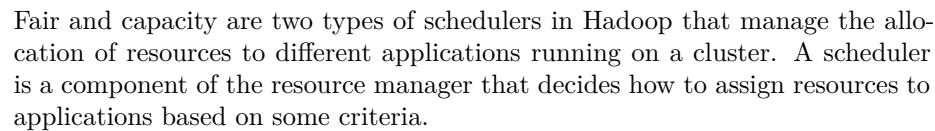






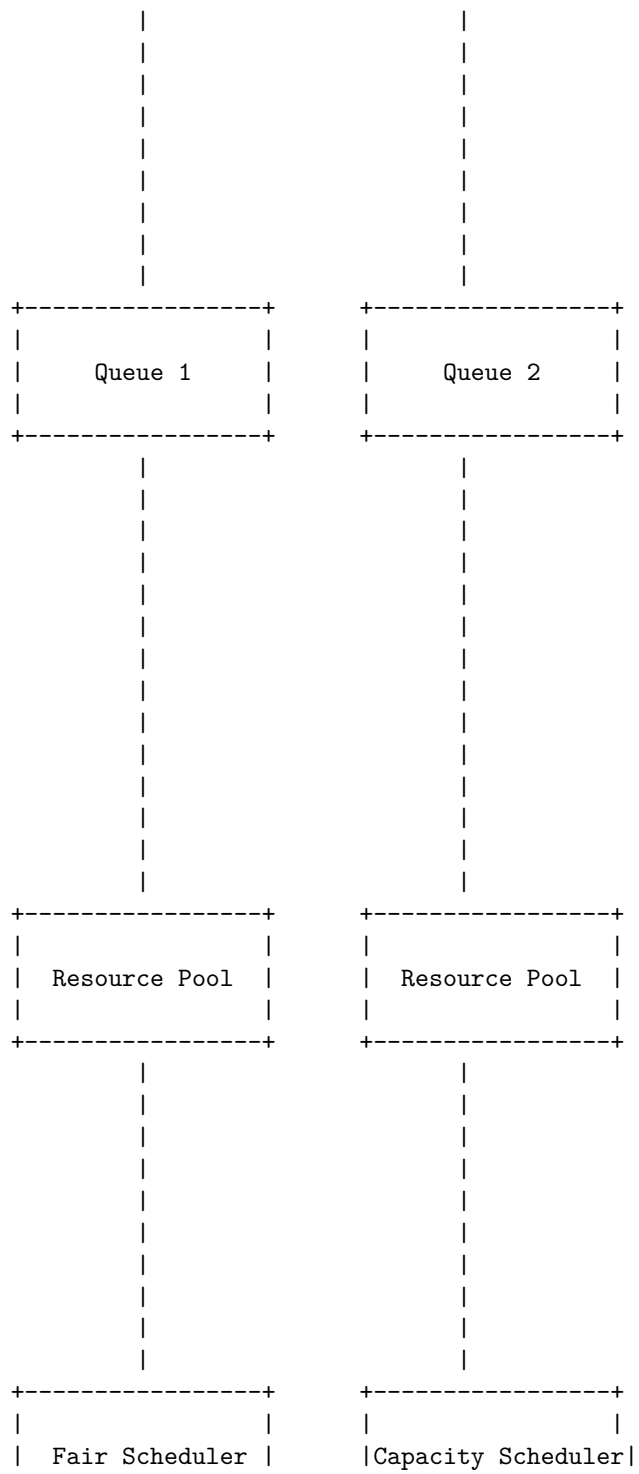


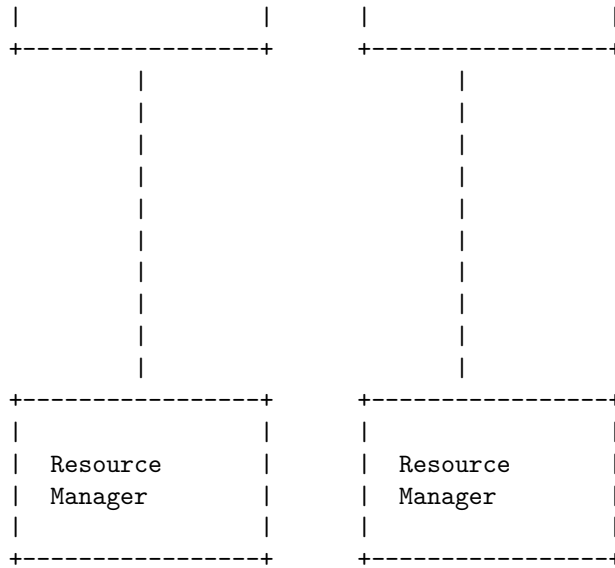




The capacity scheduler allows applications to be grouped into queues, each with a fixed percentage of the cluster capacity. The capacity scheduler can also support hierarchical queues, minimum and maximum capacities, and preemption to handle different priorities and demands of applications.

The diagram illustrates two separate applications, Application 1 and Application 2, each enclosed in a dashed rectangular box. Below each box, a vertical dashed line extends downwards, representing a connection or interface to a common bus or system. The boxes are positioned side-by-side, and the vertical lines are centered under each box.

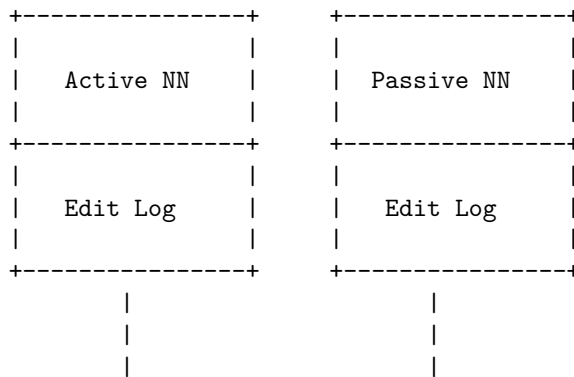


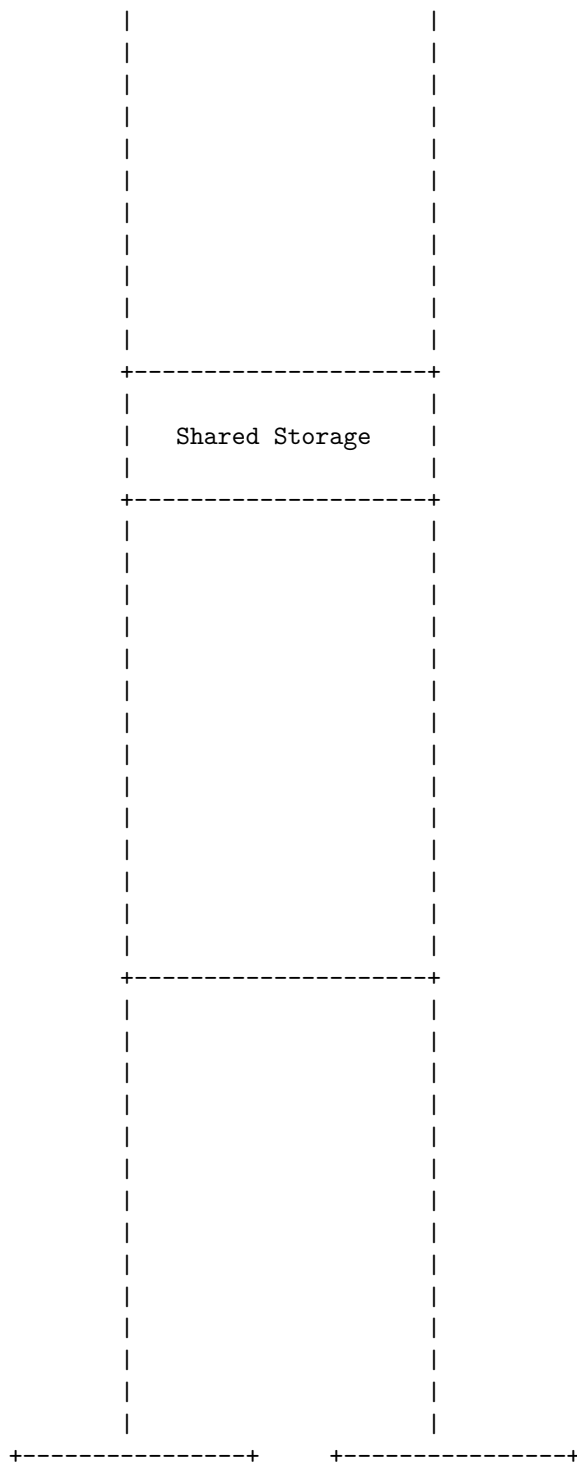


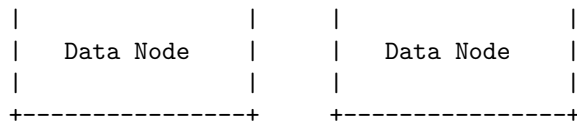
Hadoop 2.0 New Features - NameNode high availability

Hadoop 2.0 introduced the feature of NameNode high availability to overcome the single point of failure (SPOF) problem in the older versions of Hadoop. In Hadoop 2.0, there are two NameNodes in the same cluster, one active and one passive (standby). The active NameNode is responsible for managing the file system namespace and coordinating the data nodes. The passive NameNode is a hot standby that maintains enough state to provide a fast failover if the active NameNode fails. The two NameNodes use a shared storage (such as NFS or Quorum Journal Manager) to store the edit log, which is a persistent record of changes made to the file system metadata. The data nodes send block reports and heartbeats to both NameNodes to keep them updated about the cluster state.

The following diagram illustrates the basic architecture of a Hadoop 2.0 cluster with NameNode high availability:

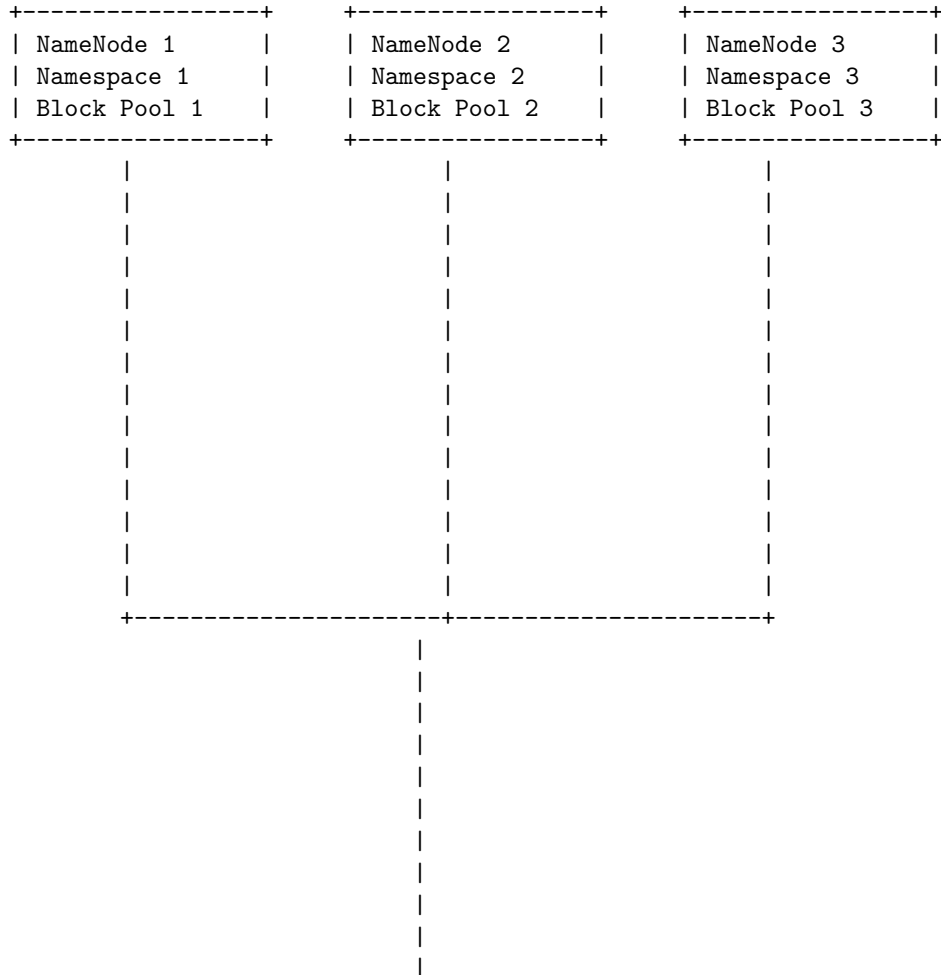






HDFS federation is a feature of Hadoop 2 that allows the use of more than one NameNode/namespace in a cluster. Each NameNode manages a separate namespace volume, which consists of a block pool and a directory tree. The DataNodes store the blocks for multiple namespaces and report them to the respective NameNodes. The clients can access any namespace by contacting the corresponding NameNode. This improves the scalability, performance and isolation of the HDFS architecture.

HDFS federation in Hadoop ecosystem



|

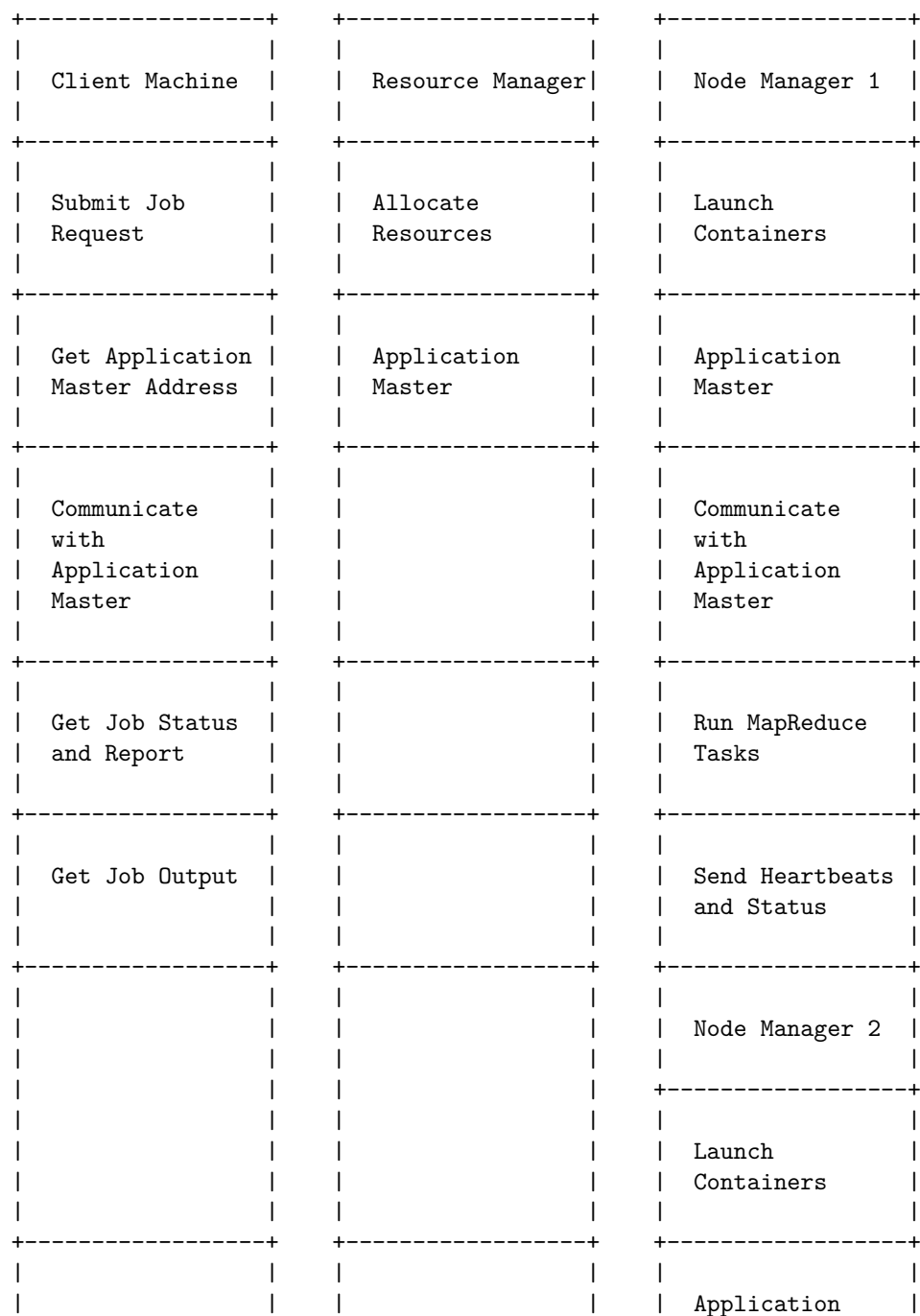
|

|

|

MRv2 is an application framework that runs within YARN, which is a resource management layer in Hadoop 2. MRv2 separates the resource management and scheduling tasks from the MapReduce logic, allowing other applications to run

on YARN as well. The following is a detailed ASCII diagram for MRv2 in Hadoop ecosystem:



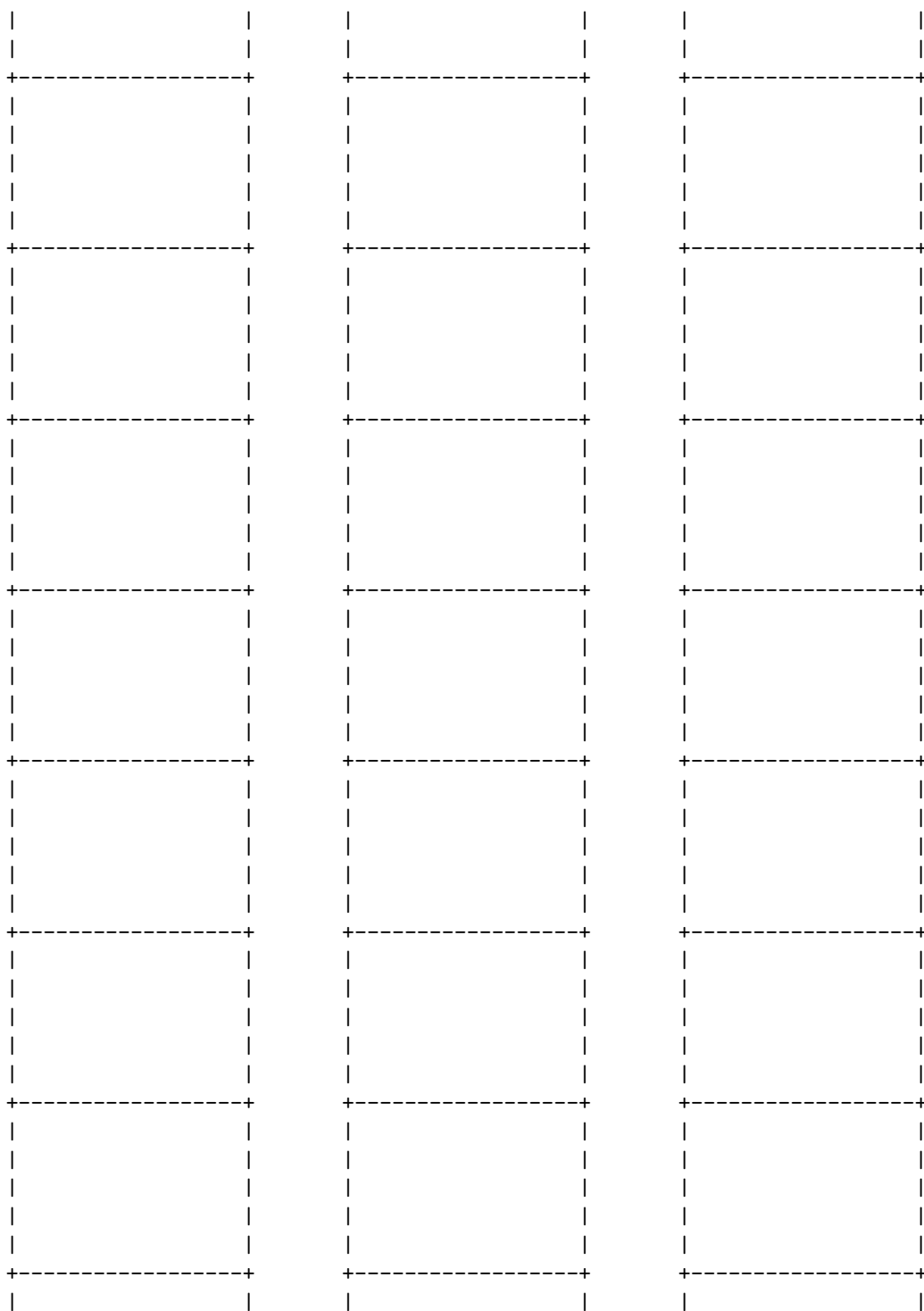
		Master
		Communicate with Application Master
		Run MapReduce Tasks
		Send Heartbeats and Status

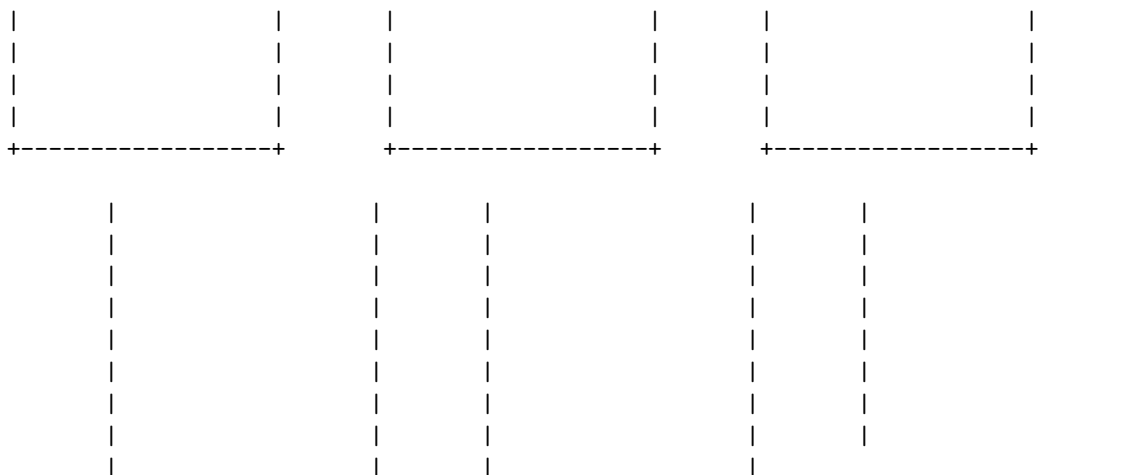
YARN is the acronym for Yet Another Resource Negotiator. It is a resource management framework for Hadoop that separates the processing engine and the management function of MapReduce. YARN consists of multiple components such as Resource Manager, Node Manager, Containers, and Application Master. These components work together to allocate and execute applications on the cluster.

YARN

The following diagram illustrates the basic architecture of YARN using ASCII characters.

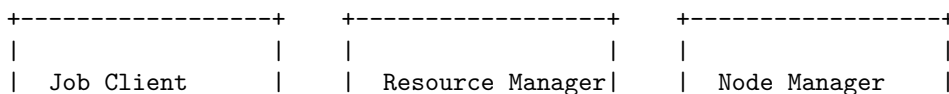
Client Node	Resource Manager (RM)	Node Manager
Submit/monitor applications	Cluster resource allocation	Manage containers

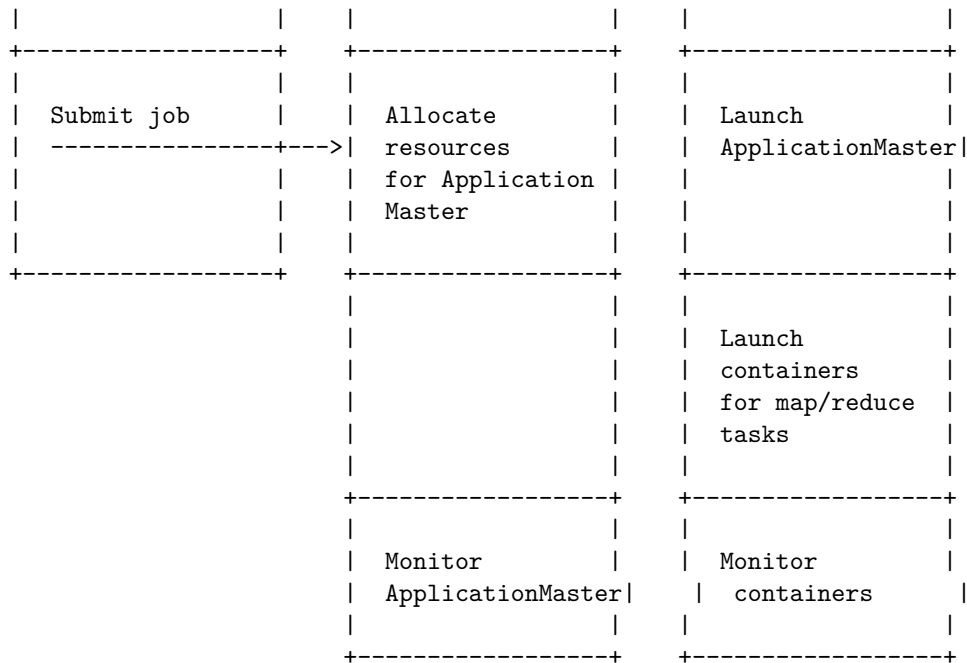




MRv1 is the original MapReduce framework that was part of Hadoop 1.x. It consists of a JobTracker that coordinates the execution of MapReduce jobs, and a number of TaskTrackers that run the map and reduce tasks on the cluster nodes. MRv1 can run on YARN, which is the resource management layer introduced in Hadoop 2.x. YARN provides a more flexible and scalable platform for running various types of applications, not just MapReduce.

- `mapreduce.framework.name`: This should be set to `yarn` to indicate that YARN is the execution framework for MapReduce jobs.
- `yarn.app.mapreduce.am.staging-dir`: This specifies the staging directory for the MapReduce ApplicationMaster, which is the process that manages the lifecycle of a MapReduce job on YARN. The default value is `/user`.
- `yarn.app.mapreduce.am.env`: This sets the environment variables for the MapReduce ApplicationMaster. You can use this to specify the Java options, such as heap size and garbage collection settings, for the ApplicationMaster process.
- `mapreduce.map.env` and `mapreduce.reduce.env`: These set the environment variables for the map and reduce tasks. You can use these to specify the Java options, such as heap size and garbage collection settings, for the task processes.





To submit a MapReduce job using MRv1 on YARN, you can use the yarn command in the Hadoop-YARN bin folder, rather than the hadoop command. For example, to run the wordcount example, you can use the following command:

```
yarn jar /usr/lib/hadoop-mapreduce/hadoop-mapreduce-examples.jar
wordcount /user/hadoop/input /user/hadoop/output
```

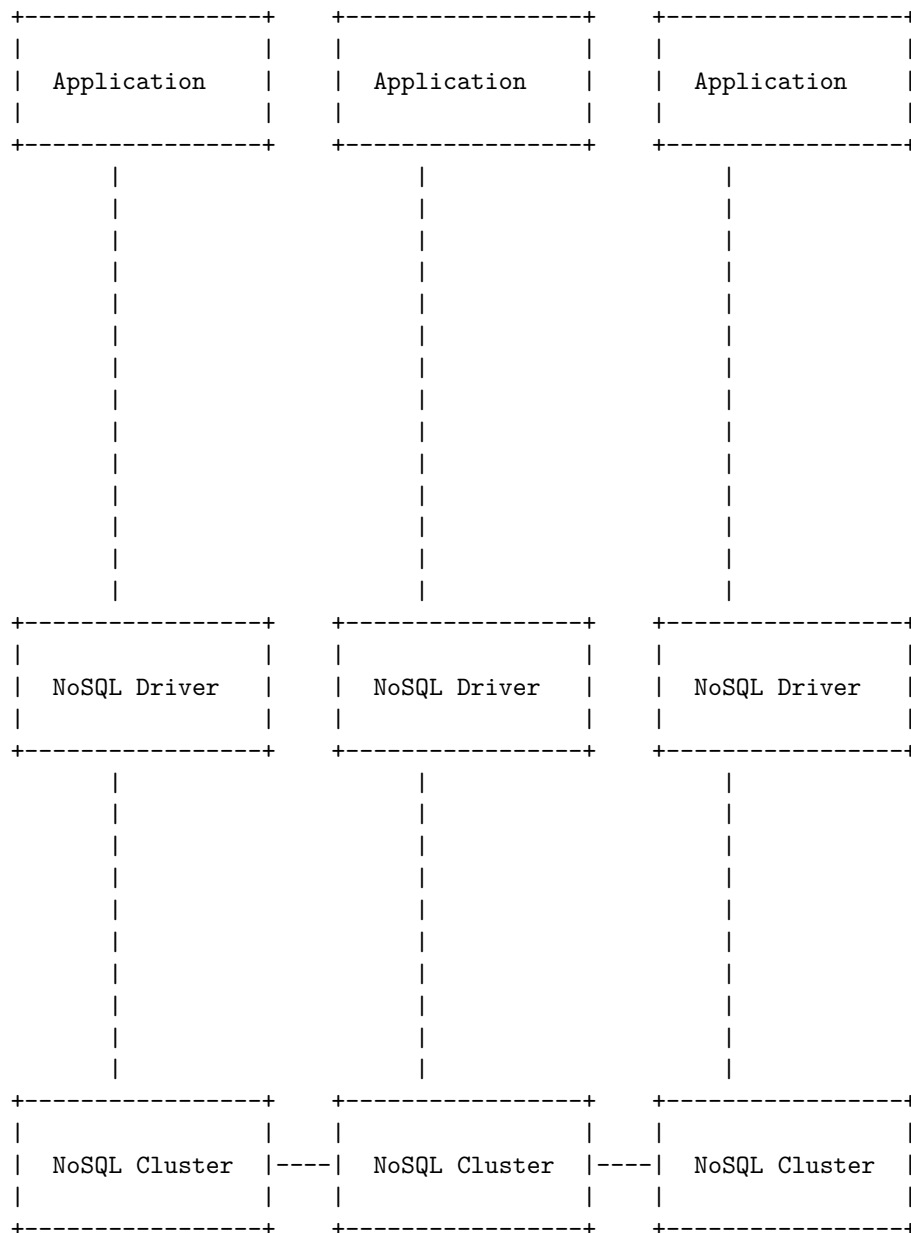
To monitor the MapReduce job, you can use the ResourceManager web interface, which shows the basic cluster metrics, list of applications, and nodes associated with the cluster. You can also use the ApplicationMaster web interface, which shows the details of the MapReduce job, such as the job configuration, counters, map and reduce tasks, and logs. You can access these web interfaces by using the following URLs:

- ResourceManager web interface: <http://:8088>
- ApplicationMaster web interface: <http://:>

NoSQL databases are non-relational databases that can handle large volumes of unstructured or semi-structured data. They provide flexible schemas and scalability, and support different data models, such as document, key-value, wide-column, and graph. NoSQL databases are often used for applications that require real-time processing, high availability, and distributed architectures.

NoSQL Databases

The following diagram illustrates the basic architecture of a NoSQL database:



The diagram shows the following components:

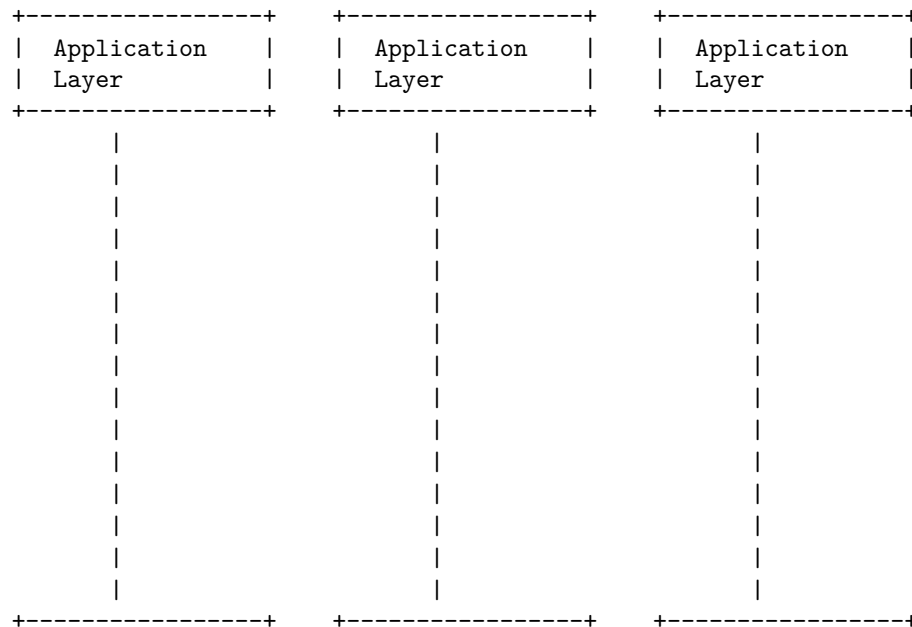
- **Application:** This is the software that interacts with the NoSQL database. It can be a web application, a mobile application, a desktop application, or any other type of software that requires data storage and retrieval. The application can use any programming language or framework that supports the NoSQL driver.

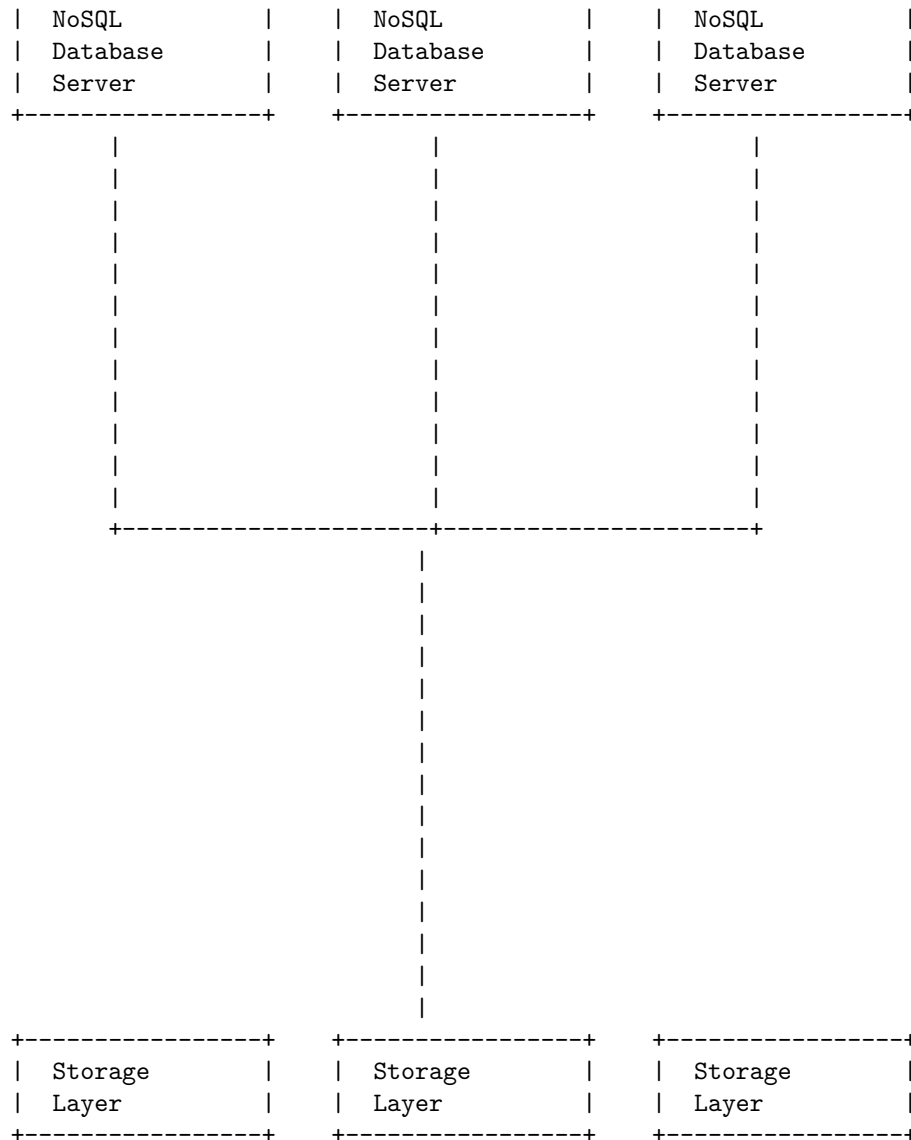
- **NoSQL Driver:** This is the software that provides the interface between the application and the NoSQL database. It handles the communication, serialization, deserialization, and query execution of the data. The NoSQL driver can be a library, a module, a plugin, or a package that is compatible with the application's programming language and framework. The NoSQL driver can also provide additional features, such as caching, connection pooling, load balancing, and encryption.
- **NoSQL Cluster:** This is the collection of servers that store and process the data. The NoSQL cluster can consist of one or more nodes, depending on the size and performance requirements of the application. The NoSQL cluster can also provide features such as replication, sharding, partitioning, and fault tolerance. The NoSQL cluster can use different data models, such as document, key-value, wide-column, or graph, depending on the type and structure of the data. The NoSQL cluster can also use different storage engines, such as memory, disk, or cloud, depending on the speed and durability of the data.

Introduction to NoSQL databases

NoSQL databases are a type of database management system that store and query data in a non-relational way. They are designed to handle large amounts of unstructured or semi-structured data and can handle dynamic changes to the data model. NoSQL databases come in a variety of types based on their data model. The main types are document, key-value, wide-column, and graph.

The following diagram illustrates the basic architecture of a NoSQL database:





The application layer is where the user interacts with the database through an application programming interface (API) or a query language. The NoSQL database server is where the data is stored and processed according to the data model and the query logic. The storage layer is where the data is physically stored on disks or in memory.

Some of the advantages of NoSQL databases are:

- They can handle large volumes of data with high performance and scalability.

- They can store and process different types of data, such as text, images, videos, etc.
- They can adapt to changing data requirements and schemas without affecting the existing data.
- They can support distributed and parallel processing of data across multiple nodes or clusters.

Some of the disadvantages of NoSQL databases are:

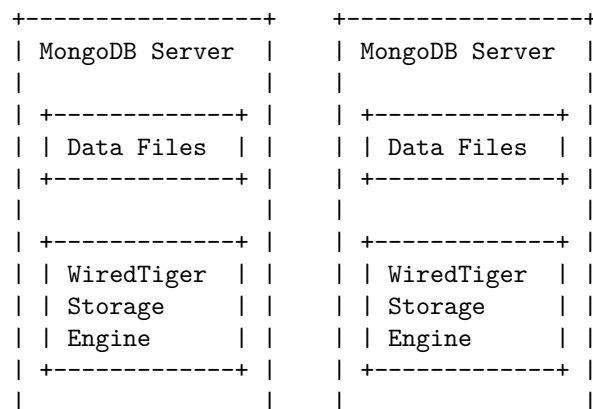
- They may not provide full support for ACID (atomicity, consistency, isolation, durability) properties, which ensure data integrity and reliability.
- They may not support complex queries or joins, which require more processing and logic.
- They may not have a standard query language or API, which makes them less interoperable and portable.
- They may require more expertise and skills to design and maintain.

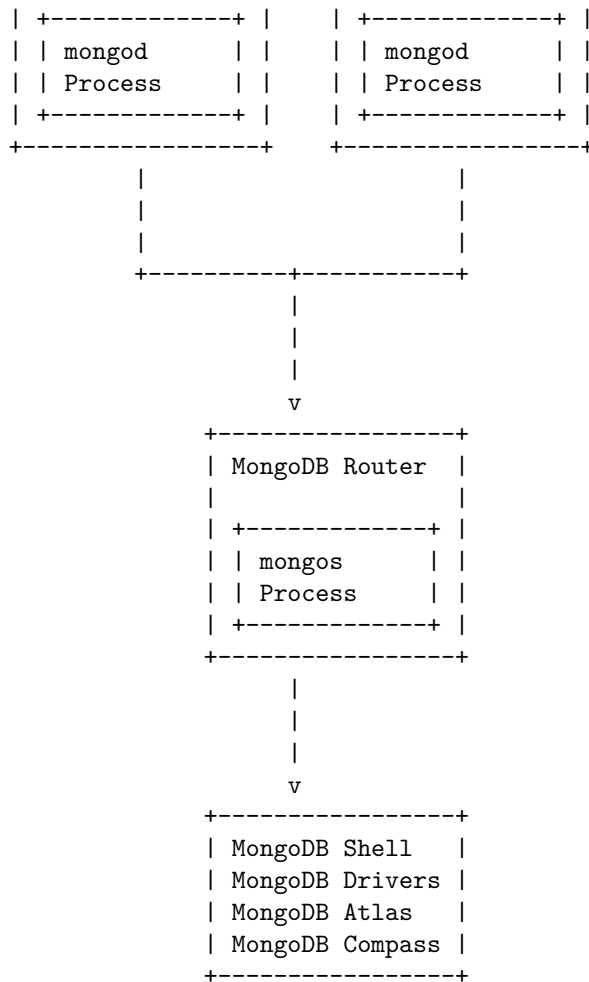
MongoDB is a document-oriented database that stores data in JSON-like format. It is composed of several components, such as:

- MongoDB Server: The core component that handles data operations, queries, and commands.
- MongoDB Shell: A command-line interface that allows users to interact with MongoDB servers.
- MongoDB Drivers: Libraries that provide APIs for various programming languages to connect and communicate with MongoDB servers.
- MongoDB Atlas: A cloud-based service that offers managed MongoDB deployments, backups, monitoring, and scaling.
- MongoDB Compass: A graphical user interface that allows users to explore and manipulate data in MongoDB servers.

The following diagram illustrates the basic architecture of a MongoDB server:

MongoDB





The MongoDB server consists of a mongod process, which is responsible for managing the data files and executing the queries and commands. The mongod process uses the WiredTiger storage engine, which is a high-performance and scalable engine that supports compression, encryption, and transactions. The data files are stored in a binary format called BSON (Binary JSON), which is an extension of JSON that supports additional data types.

The MongoDB router, or mongos, is a process that acts as a query router for a cluster of MongoDB servers. The mongos process distributes the queries and commands to the appropriate mongod processes, and aggregates the results. The mongos process also handles the sharding and replication of the data across the cluster.

The MongoDB shell, or mongo, is a command-line interface that allows users to interact with MongoDB servers. The mongo shell provides a JavaScript

environment that supports various commands and operations. The mongo shell can also be used to run scripts and perform administrative tasks.

The MongoDB drivers are libraries that provide APIs for various programming languages to connect and communicate with MongoDB servers. The MongoDB drivers support a consistent and idiomatic interface for different languages, such as Java, Python, C#, Ruby, and Node.js. The MongoDB drivers also handle the serialization and deserialization of the BSON data.

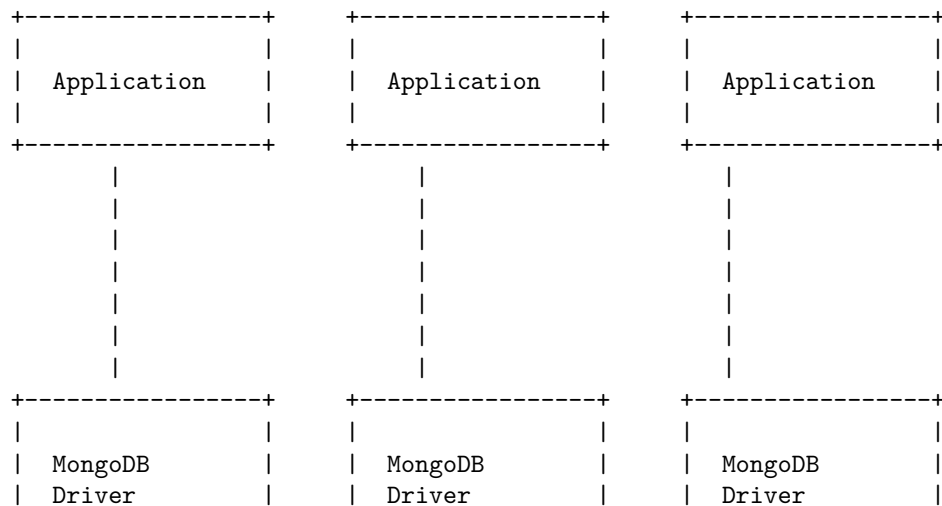
The MongoDB Atlas is a cloud-based service that offers managed MongoDB deployments, backups, monitoring, and scaling. The MongoDB Atlas allows users to create and configure MongoDB clusters in various regions and cloud providers, such as AWS, Azure, and Google Cloud. The MongoDB Atlas also provides security features, such as encryption, authentication, and authorization.

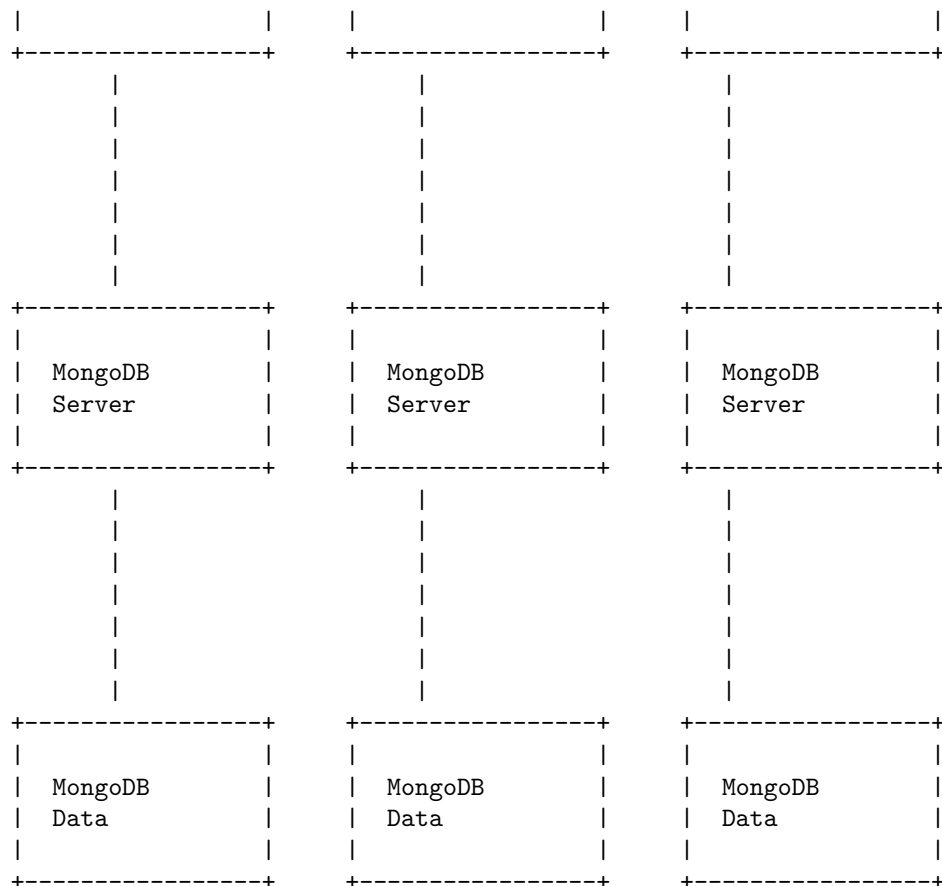
The MongoDB Compass is a graphical user interface that allows users to explore and manipulate data in MongoDB servers. The MongoDB Compass provides a visual representation of the data, schema, indexes, and performance metrics. The MongoDB Compass also allows users to run queries, edit documents, and perform administrative tasks.

Introduction to MongoDB

MongoDB is a document database designed for ease of development and scaling. It is a general-purpose database platform that can handle different types of data sets and applications. MongoDB stores data in flexible, JSON-like documents, which allow for dynamic schemas and rich data structures. MongoDB also supports horizontal scaling through sharding, replication, and load balancing.

The following diagram illustrates the basic architecture of a MongoDB system:



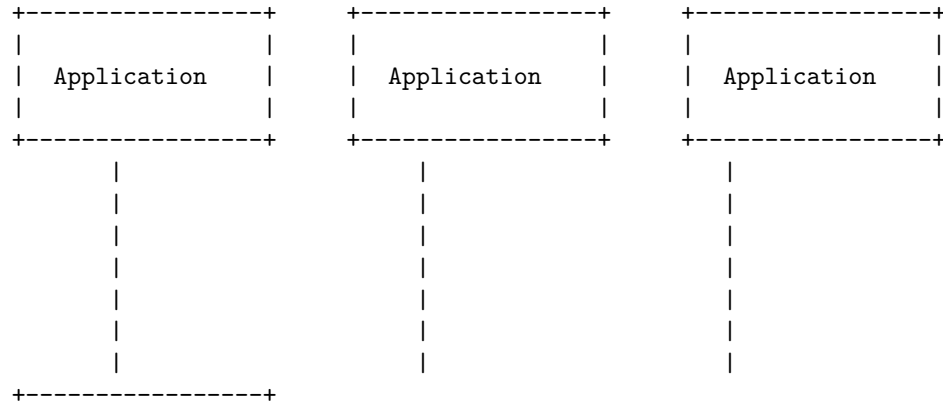


In this diagram, the application layer communicates with the MongoDB server through the MongoDB driver, which provides an API for various programming languages. The MongoDB server manages the data operations, such as queries, updates, aggregations, and transactions. The MongoDB data layer stores the data in documents, which are organized into collections and databases. Each MongoDB server can host multiple databases, and each database can have multiple collections. A collection is a group of documents that share a similar structure or purpose. A document is a record of data that consists of key-value pairs. The values can be simple types, such as strings, numbers, booleans, or dates, or complex types, such as arrays, subdocuments, or binary data.

MongoDB also supports sharding, which is a method of distributing data across multiple servers or clusters. Sharding allows MongoDB to scale horizontally and handle large amounts of data and high throughput. Sharding involves splitting a collection into smaller chunks, and assigning each chunk to a different shard. A shard is a logical group of one or more MongoDB servers that hold a subset of the data. MongoDB uses a shard key, which is a field or a combination of fields in the documents, to determine how to partition the data. MongoDB also uses a

config server, which stores the metadata about the sharding configuration, and a mongos, which is a query router that directs the requests from the application to the appropriate shard.

The following diagram illustrates the basic architecture of a sharded MongoDB system:



Data types in MongoDB are the different kinds of values that can be stored in the documents of a MongoDB collection. MongoDB uses BSON (Binary JSON) format to store the data, which supports various data types, such as strings, numbers, booleans, arrays, objects, dates, object ids, binary data, etc.

The following diagram shows some examples of data types in MongoDB, using the ASCII art syntax for drawing diagrams in markdown:

Data types in MongoDB

Data type	Example	BSON type
String	"Hello world"	0x02
Integer	42	0x10 or 0x12
Double	3.14	0x01
Boolean	true or false	0x08
Array	[1, 2, 3]	0x04
Object	{"name": "Alice", "age": 25}	0x03
Date	ISODate("2022-01-01T00:00:00Z")	0x09

```

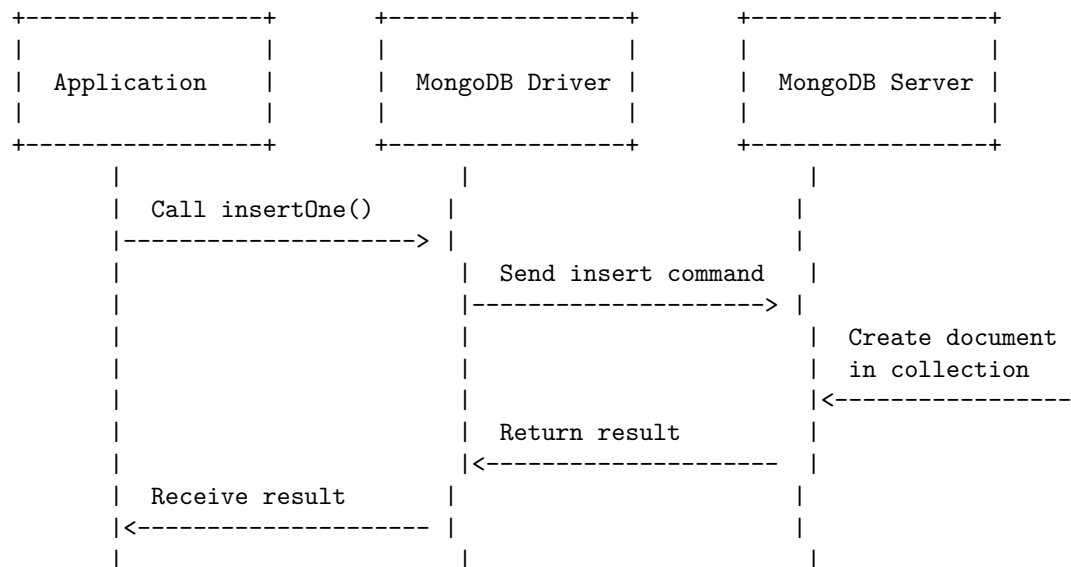
| ObjectId          | ObjectId("507f1f77bcf86cd79943") | 0x07          |
+-----+-----+-----+
| Binary data      | BinData(0, "YmluYXJ5ZGF0YQ==") | 0x05          |
+-----+-----+-----+

```

To create documents in MongoDB, you can use the `insertOne()` or `insertMany()` methods, which insert one or many documents into a collection, respectively. A collection is a group of documents that share a common schema. A document is a JSON-like object that contains key-value pairs. A key is a string that identifies a field in the document, and a value is any valid BSON data type, such as string, number, array, object, etc.

Creating documents in MongoDB

The following diagram illustrates the basic process of creating documents in MongoDB using the `insertOne()` method:



The `insertOne()` method takes a document as a parameter and returns a result object that contains information about the operation, such as the `_id` field of the inserted document, the number of documents inserted, and any errors that occurred. The `_id` field is a unique identifier for each document in a collection. If the document does not specify an `_id` field, the MongoDB driver automatically generates an `ObjectId` value for it.

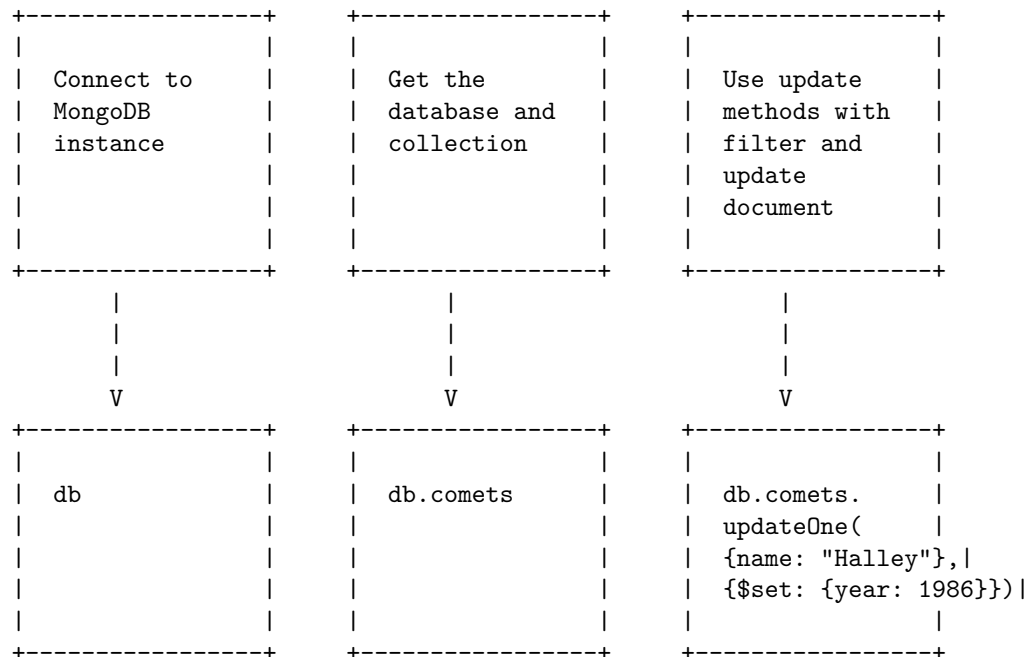
The `insertMany()` method works similarly, but it takes an array of documents as a parameter and inserts them all into the collection. The result object contains an array of `_id` values for the inserted documents, as well as the number of documents inserted and any errors that occurred.

To create documents in MongoDB, you need to have a connection to a MongoDB server and a database name. You can use MongoDB Compass, a graphical user interface for MongoDB, to create and manage databases, collections, and documents. You can also use MongoDB for VS Code, a plugin that allows you to run MongoDB commands and queries in a code editor. Alternatively, you can use the mongo shell, a command-line interface for MongoDB, to interact with the database server.

To update documents in MongoDB, you can use different methods depending on whether you want to update a single document, multiple documents, or replace a document. You also need to specify a filter condition to match the documents you want to update, and an update document that contains the new values or update operators.

Updating documents in MongoDB

The following diagram illustrates the basic steps of updating documents in MongoDB using the MongoDB shell:



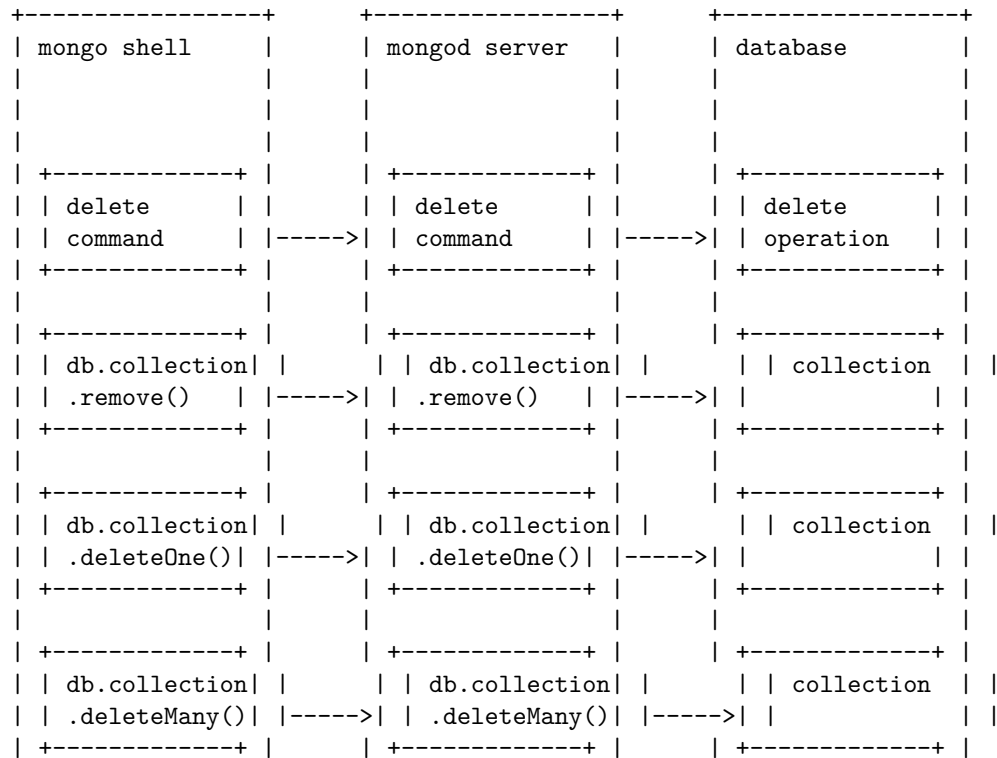
The above example updates one document in the comets collection where the name field is “Halley” and sets the year field to 1986. You can use other update methods, such as updateMany() or replaceOne(), to update multiple documents or replace a document respectively. You can also use other update operators, such as \$inc, \$push, \$rename, etc., to modify the fields in different ways. For more details, please refer to the MongoDB documentation.

Deleting documents in MongoDB

MongoDB provides several methods to delete documents from a collection. The most common ones are:

- `db.collection.remove()` : This method deletes one or more documents that match a given filter condition. It can take an optional parameter to specify whether to delete just one document or all matching documents. It returns a write result object that contains the number of deleted documents and other information.
- `db.collection.deleteOne()` : This method deletes a single document that matches a given filter condition. It returns a delete result object that contains the number of deleted documents and other information.
- `db.collection.deleteMany()` : This method deletes all documents that match a given filter condition. It returns a delete result object that contains the number of deleted documents and other information.
- `delete` : This is a command that can be used in the mongo shell to delete documents from a collection. It takes a query document as a parameter and deletes all matching documents. It returns a command result object that contains the number of deleted documents and other information.

The following diagram illustrates the basic architecture of deleting documents in MongoDB using ASCII art:

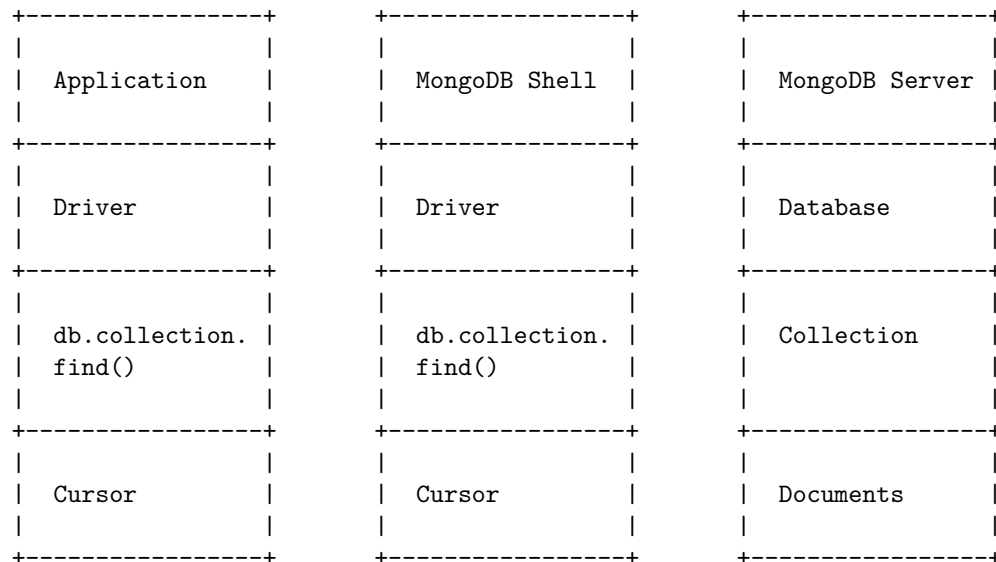


--	--	--

Querying documents in MongoDB is the process of retrieving data from a collection using the `db.collection.find()` method. This method takes two parameters: a filter object that specifies the criteria for selecting documents, and an optional projection object that specifies the fields to return. The method returns a cursor object that can be iterated to access the matching documents.

Querying documents in MongoDB

The following diagram illustrates the basic architecture of querying documents in MongoDB:



The diagram shows the following steps:

- The application or the MongoDB shell uses a driver to connect to the MongoDB server and issue a `db.collection.find()` query.
- The driver sends the query to the MongoDB server and receives a cursor object that points to the first matching document in the collection.
- The application or the MongoDB shell iterates through the cursor to access the documents that satisfy the query criteria. The cursor can also apply additional methods such as `sort()`, `limit()`, or `skip()` to modify the result set.

Indexing in MongoDB is a technique that allows the database to efficiently process queries by using special data structures that store a subset of the document's data in a sorted order. Indexes can improve the performance of queries that match on the indexed fields or sort on them. Indexes can also support

unique constraints, text search, geospatial queries, and other features.

Indexing in MongoDB

The following diagram illustrates the basic architecture of indexing in MongoDB:



The collection is a logical grouping of documents that can be queried by the query engine. Each document has a unique identifier (`_id`) and a set of fields and values.

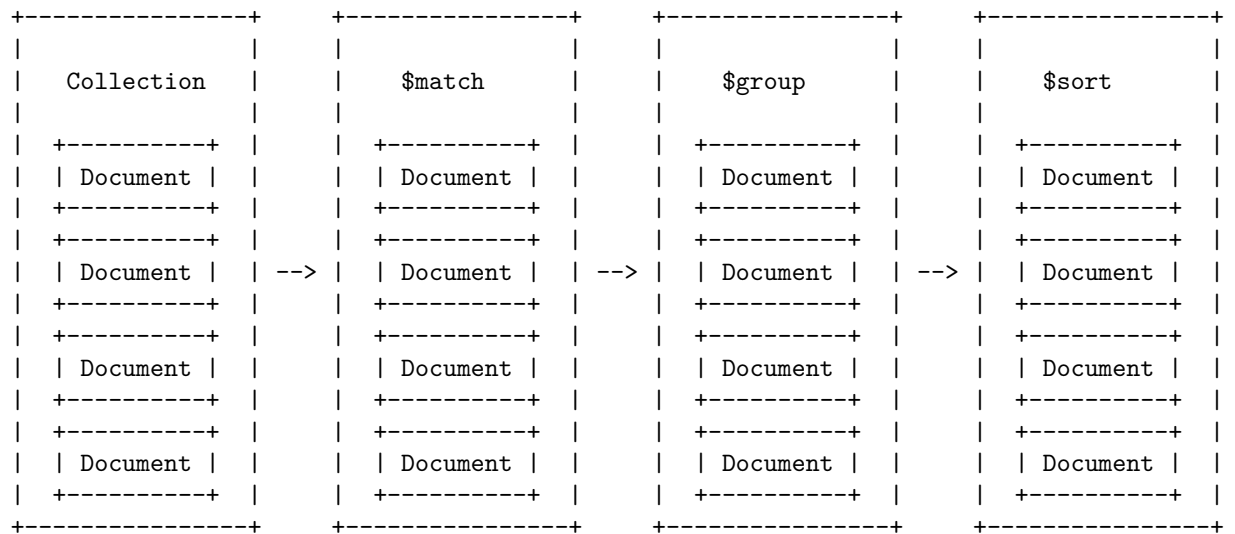
The index is a data structure that stores a subset of the document's fields and their values, along with a pointer (loc) to the location of the document in the data file. The index is sorted by the key, which is the field or combination of fields that the index is based on. The index can be created using the `createIndex()` method and dropped using the `dropIndex()` method.

The data file is a physical file that stores the documents in a binary format (BSON). The data file is managed by the storage engine, which handles the allocation, compression, and encryption of the data. The data file can be accessed by the query engine using the pointers from the index.

The query engine is the component that processes the queries from the clients and returns the results. The query engine can use the index to quickly find the matching documents or sort them by the key. The query engine can also perform other operations, such as aggregation, projection, filtering, and joining. The query engine can use various query operators, such as `$match`, `$sort`, `$project`, `$lookup`, and others.

Aggregation in MongoDB is the process of selecting data from a collection and performing various operations on the data to produce a computed result. Aggregation can be done using two methods: single-purpose aggregation and aggregation pipeline. Single-purpose aggregation consists of helper methods that apply a specific operation to a collection, such as `count`, `distinct`, or `group`. Aggregation pipeline consists of one or more stages that process documents in a sequence. Each stage performs an operation on the input documents and outputs modified documents to the next stage. Some of the common stages are `match`, `group`, `sort`, `project`, and `unwind`.

The following diagram illustrates the basic architecture of an aggregation pipeline in MongoDB using ASCII art:



A capped collection in MongoDB is a fixed-size collection that supports high-throughput operations that insert and retrieve documents based on insertion order. Capped collections work in a way similar to circular buffers: once a collection fills its allocated space, it makes room for new documents by overwriting the oldest documents in the collection .

You must create capped collections explicitly using the `db.createCollection()` method, which is a mongosh helper for the `create` command. When creating a capped collection you must specify the maximum size of the collection in bytes, which MongoDB will pre-allocate for the collection .

Capped collections are basically used to store log information, the high volume of data, and cache information .

The following diagram illustrates the basic architecture of a capped collection in MongoDB:

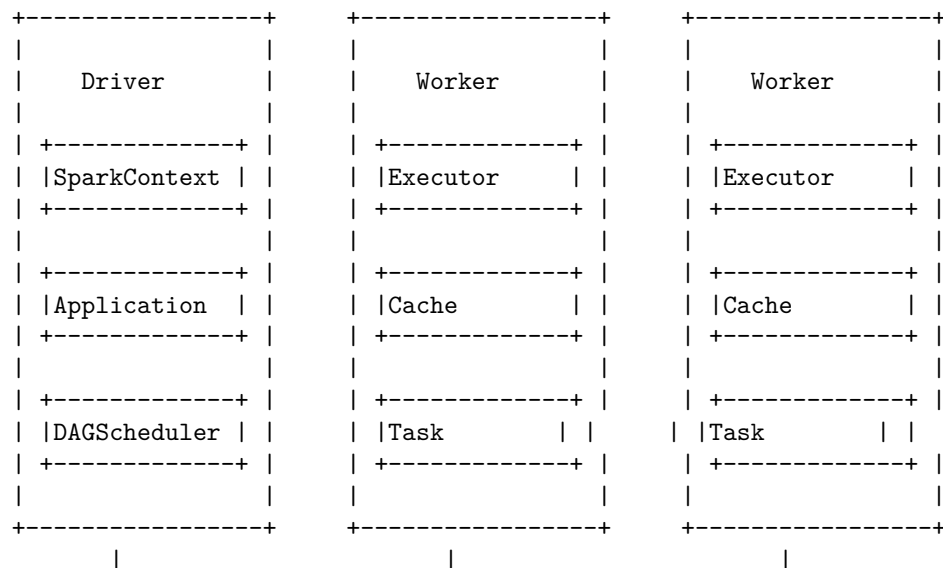
Document 1	Document 2	Document 3	Document 4
Document 5	Document 6	Document 7	Document 8
Document 9	Document 10	Document 11	Document 12
Document 13	Document 14	Document 15	Document 16
Document 17	Document 18	Document 19	Document 20
Document 21	Document 22	Document 23	Document 24
Document 25	Document 26	Document 27	Document 28
Document 29	Document 30	Document 31	Document 32
Document 33	Document 34	Document 35	Document 36
Document 37	Document 38	Document 39	Document 40

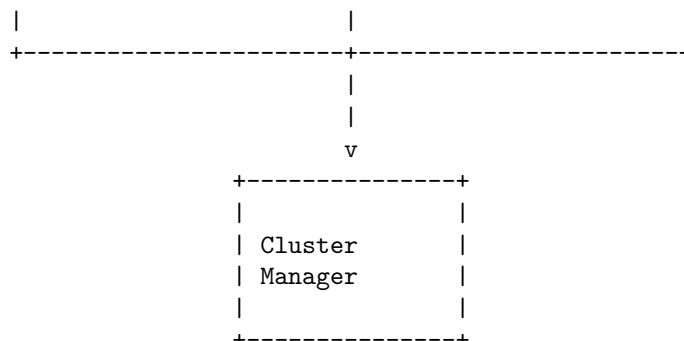
Document 41	Document 42	Document 43	Document 44
Document 45	Document 46	Document 47	Document 48
Document 49	Document 50	Document 51	Document 52
Document 53			

Spark is an open-source framework for large-scale data processing. It consists of four main components: Spark Core, Spark SQL, Spark Streaming, and Spark MLlib. Spark Core is the foundation of the Spark architecture, which provides distributed task scheduling, memory management, fault recovery, and data access. Spark SQL is a module that supports structured and semi-structured data processing using SQL or a DataFrame API. Spark Streaming is a module that enables scalable and fault-tolerant stream processing of live data streams. Spark MLlib is a module that provides machine learning algorithms and utilities for data analysis.

The following diagram illustrates the basic architecture of Spark:

Spark





The driver is the central coordinator of all Spark executions. It creates a Spark-Context object that connects to a cluster manager, which allocates resources across applications. The driver also creates a DAGScheduler, which splits the logical execution plan into stages of tasks and submits them to the cluster manager. The cluster manager can be Spark’s own standalone cluster manager, Mesos, YARN, or Kubernetes.

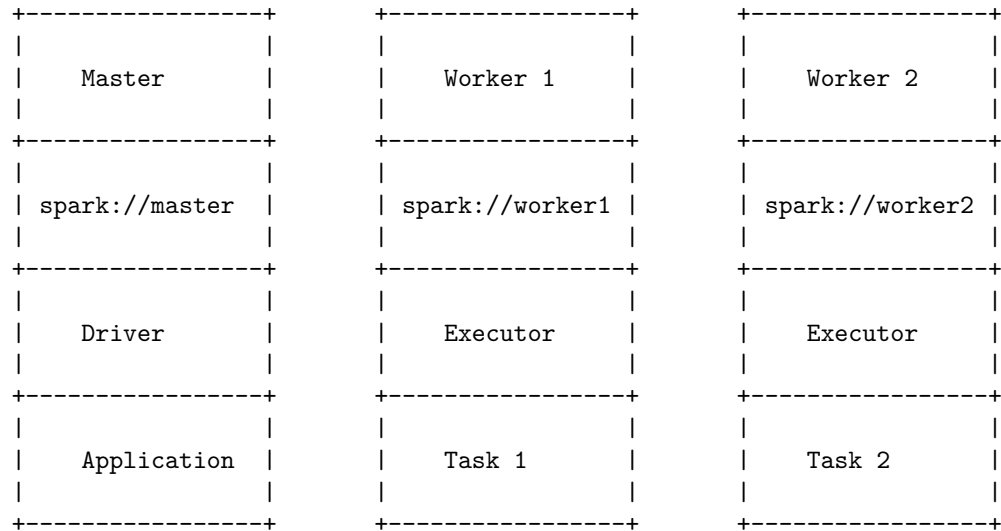
The workers are the nodes that run the tasks assigned by the driver. Each worker has one or more executors, which are processes that run the tasks and store the data in memory or disk. The executors communicate with the driver and the cluster manager to coordinate the execution and report the status of the tasks. The workers also have a cache, which is a local storage for intermediate data that can be reused by other tasks or queries.

Installing spark depends on the operating system and the mode of deployment. Spark can run on Windows, Linux, or Mac OS, and can be deployed in standalone mode, on a cluster, or in the cloud. The basic steps for installing spark are:

- Verify the Java installation on the system. Spark requires Java 8 or higher. You can check the Java version by running the command `java -version` in the terminal.
- Install Scala, the programming language used by Spark. You can download Scala from <https://www.scala-lang.org/download/> or use a package manager such as Homebrew or apt-get.
- Download the latest version of Apache Spark from <https://spark.apache.org/downloads.html>. Choose the package type, the Spark version, and the Hadoop version according to your needs.
- Extract the downloaded file to the desired location. For example, you can create a folder named Spark in the root of your C: drive and extract the file there.
- Set the environment variables for Spark and Java. You need to add the paths of Spark and Java to the system PATH variable, and also set the SPARK_HOME and JAVA_HOME variables. You can do this by editing the .bashrc file on Linux or Mac OS, or the system properties on Windows.
- Verify the Spark installation by running the command `spark-shell` in

the terminal. This will launch the interactive Spark shell, where you can run Spark commands and queries.

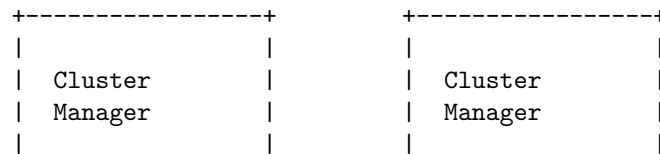
The following diagram illustrates the basic architecture of a Spark installation in standalone mode:

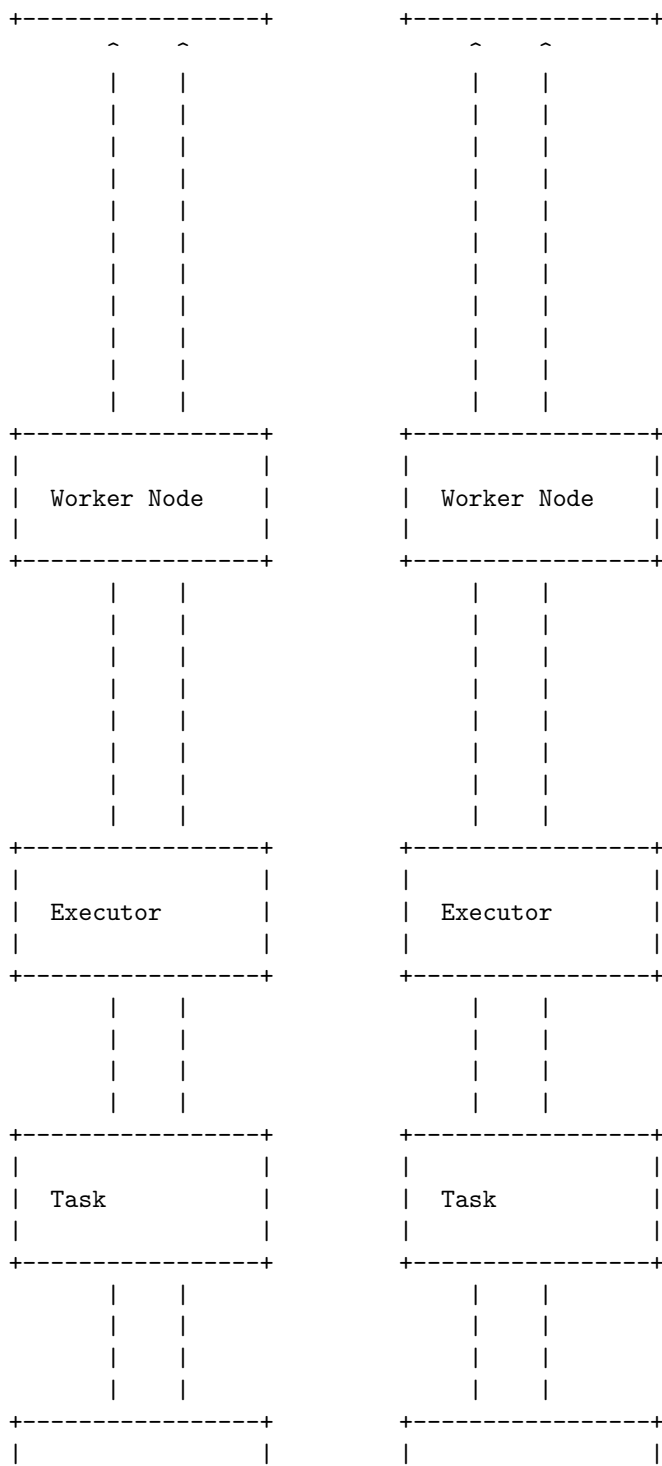


The master node is responsible for coordinating the cluster and scheduling the tasks. The worker nodes are responsible for running the tasks assigned by the master. The driver node is responsible for running the application code and creating the Spark session. The executor nodes are responsible for running the tasks in parallel. The tasks are the units of work that perform the computation on the data.

A spark application is a distributed program that runs on a cluster of nodes. It consists of four main components: the driver, the executors, the cluster manager, and the worker nodes. The driver is the process that coordinates the execution of the application and communicates with the cluster manager. The executors are the processes that run the tasks assigned by the driver and store the data in memory or disk. The cluster manager is the service that allocates resources to the spark application and manages the worker nodes. The worker nodes are the machines that host the executors and provide the computing and storage resources.

The following diagram illustrates the basic architecture of a spark application using ASCII art:

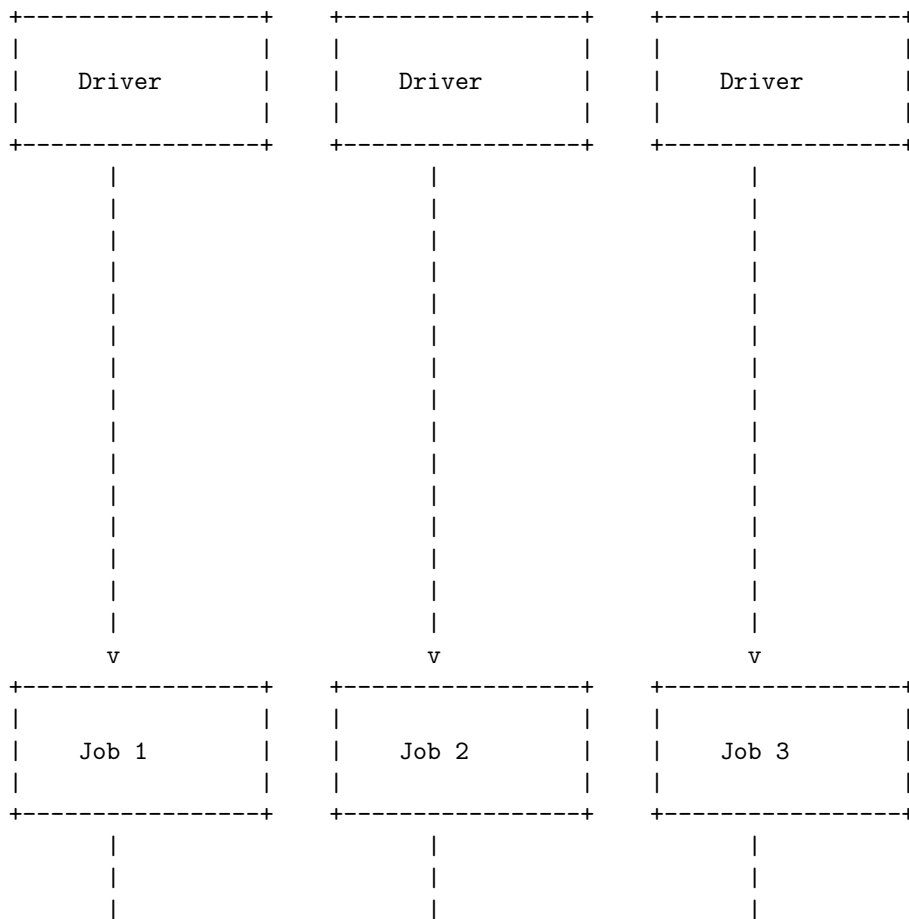


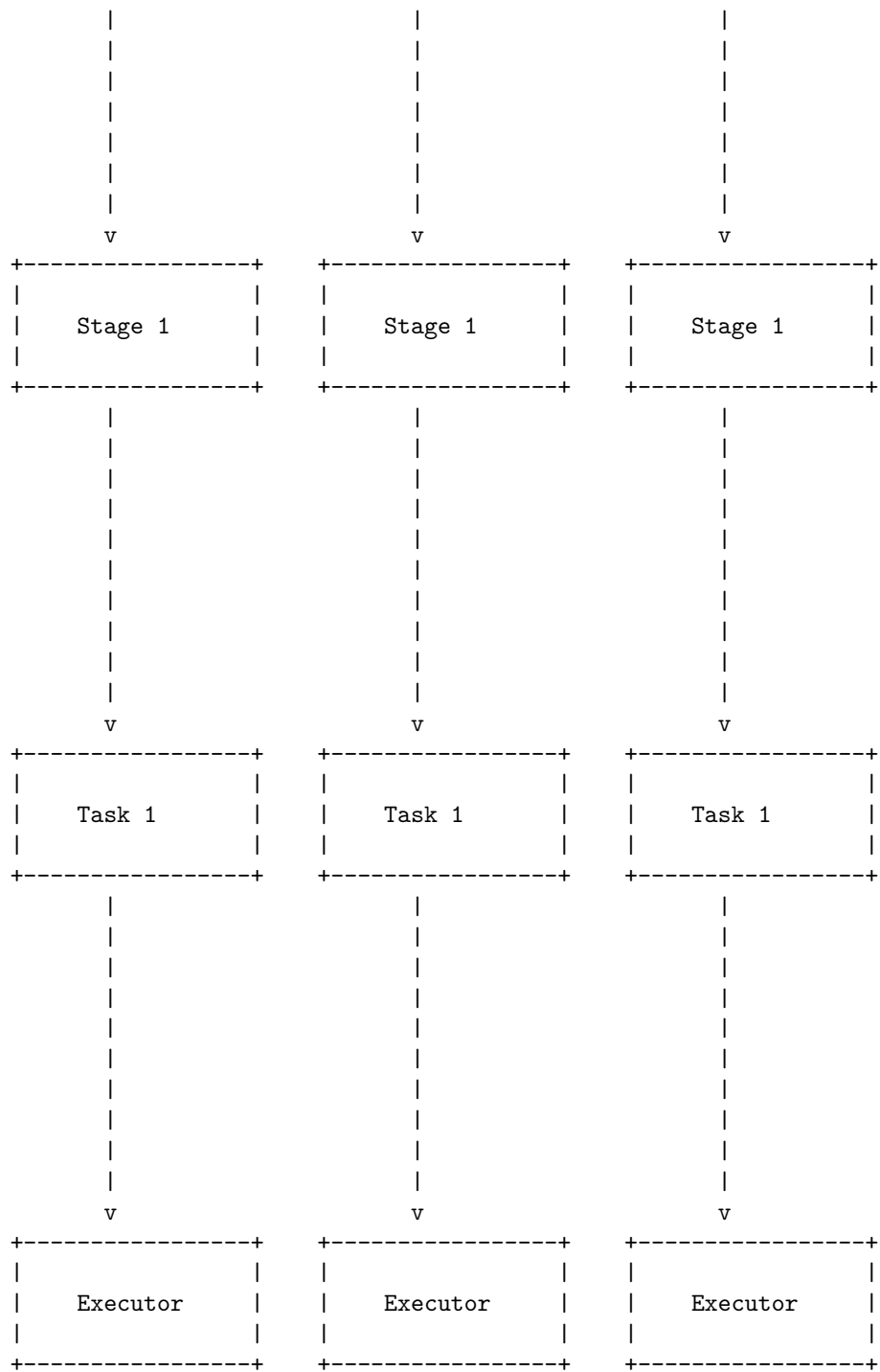




The driver and the executors can run in different modes: local, standalone, Mesos, YARN, or Kubernetes. The cluster manager can be Spark's own standalone cluster manager, Mesos, YARN, or Kubernetes. The worker nodes can be physical machines, virtual machines, or containers. The data can be stored in memory, disk, or external sources such as HDFS, S3, or Cassandra.

Jobs in Spark are parallel computations of tasks that are triggered by actions such as count, collect, read or write. Each job is divided into one or more stages, which are further divided into one or more tasks. Each task is a unit of work that is executed by an executor on a worker node. A stage is a collection of tasks that have the same shuffle dependency. A job can have multiple stages if there are wide transformations that require data to be shuffled across the cluster. The following diagram illustrates the basic architecture of a job in Spark:

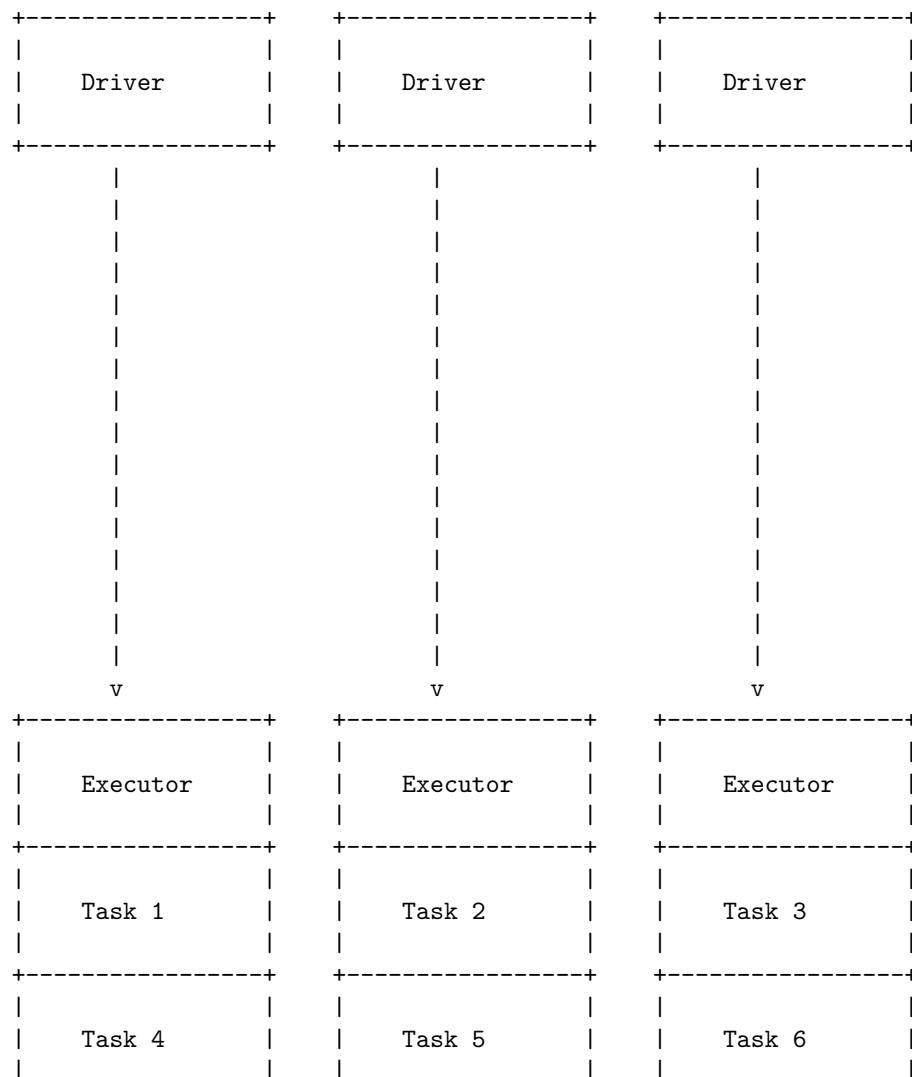


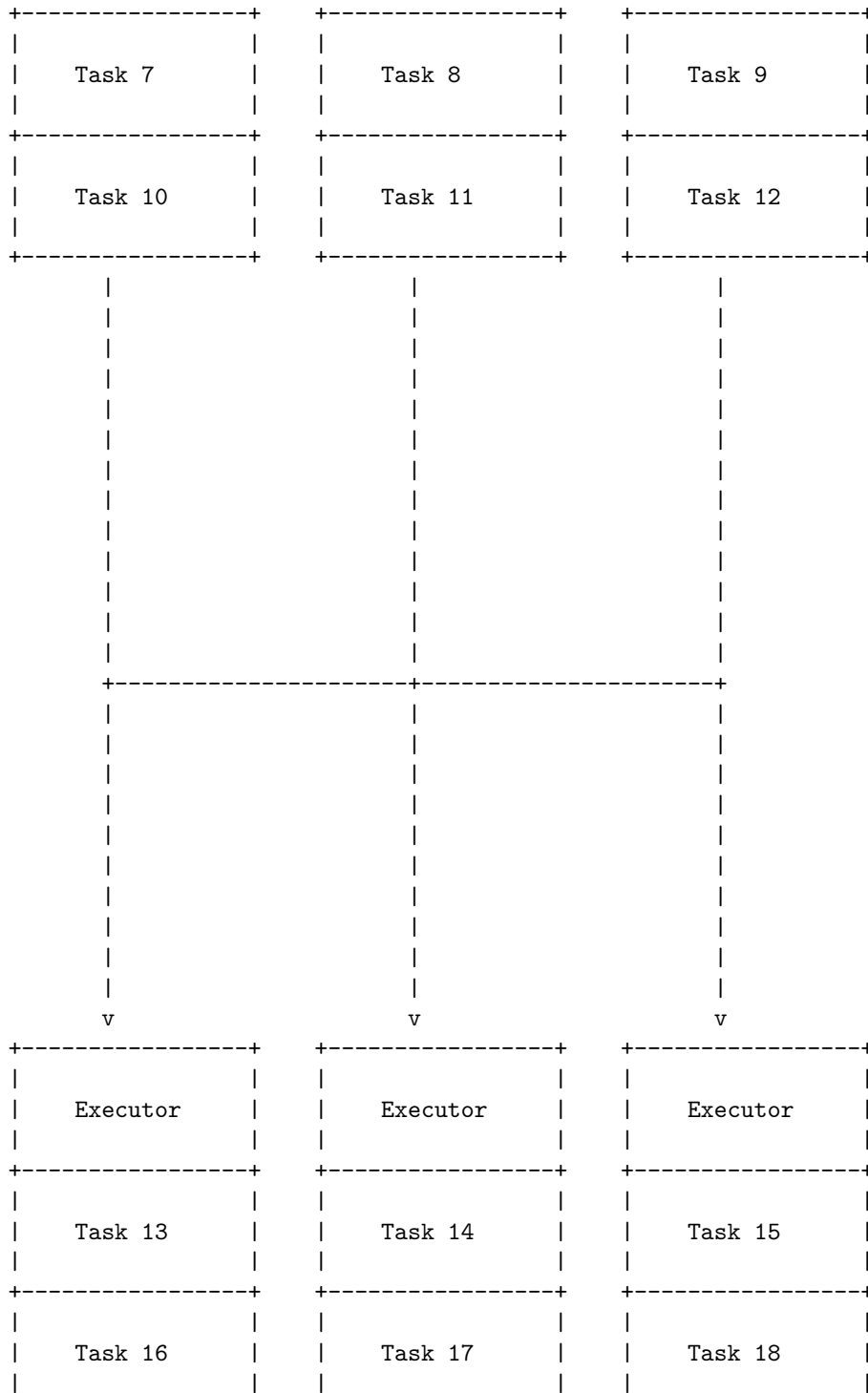


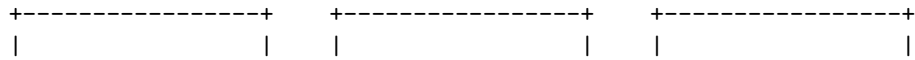
Stages and tasks are the basic units of execution in Spark. A stage is a set of parallel tasks that operate on a subset of the data, and a task is a unit operation that processes a partition of the data. Stages are divided by shuffle boundaries, which are transformations that require data to be redistributed across the cluster, such as `reduceByKey` or `join`. Tasks within a stage are independent of each other and can be executed in parallel on different nodes in the Spark cluster.

Stages and tasks in Spark

The following diagram illustrates the basic architecture of a Spark application, showing the relationship between stages and tasks.

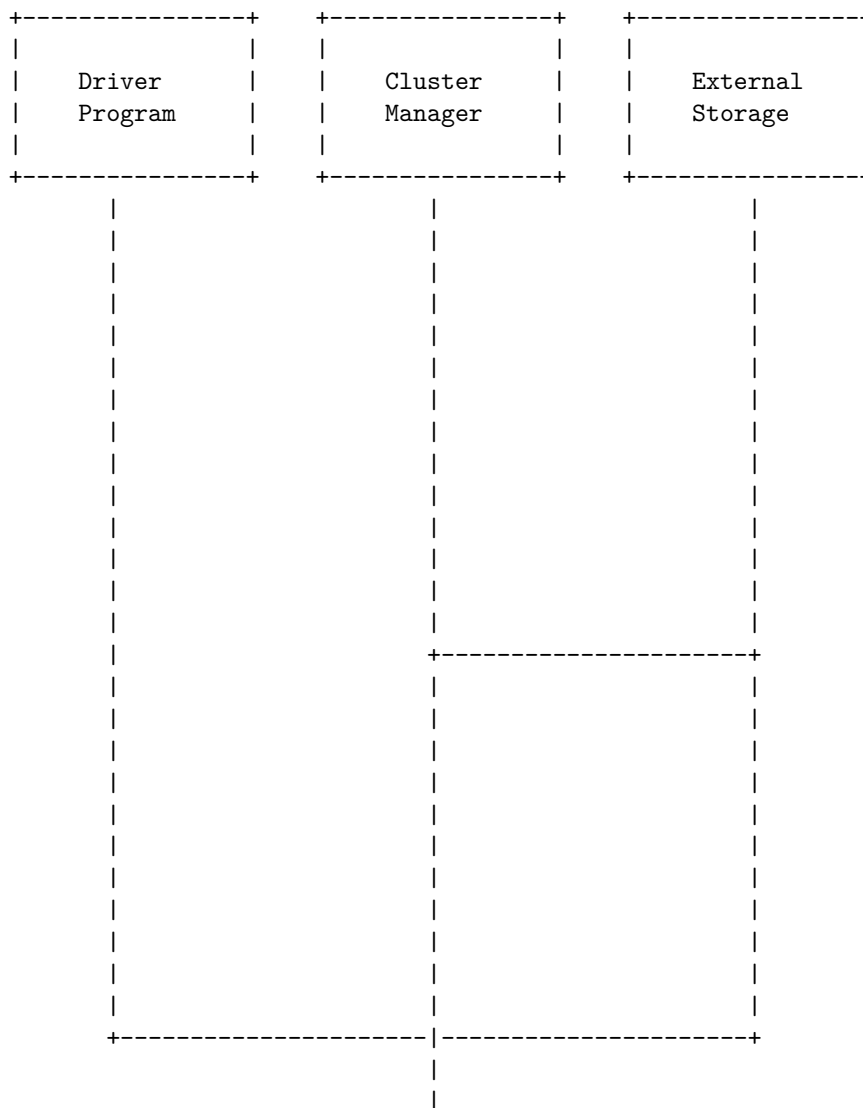






Resilient Distributed Datasets (RDDs) are the fundamental data structure of Spark . They are immutable distributed collections of objects that can be operated on in parallel . Each dataset in RDD is divided into logical partitions, which may be computed on different nodes of the cluster . RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes .

Resilient Distributed Datasets in Spark



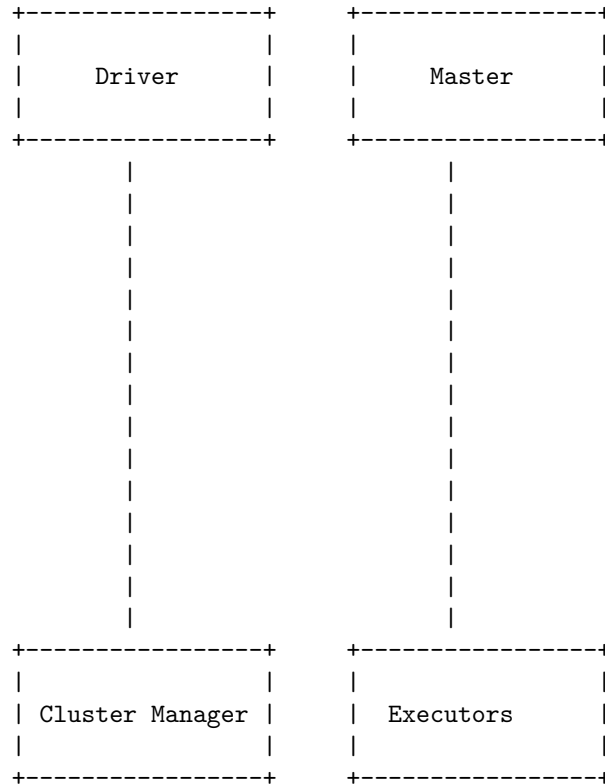


A Spark job run is the execution of a Spark application, which consists of several components that interact with each other to process data. The components of a Spark job run are:

- Driver: The program that runs the `main()` method of the application and creates the `SparkContext`. The driver coordinates the tasks and stages of the job and communicates with the cluster manager and the executors.
- Master: The process that runs on the master node of the cluster and assigns tasks to the workers. The master also monitors the status and health of the cluster and handles failures and recovery.
- Cluster Manager: The service that manages the resources and nodes of the cluster. The cluster manager can be a standalone service, or a third-party service such as YARN or Mesos.

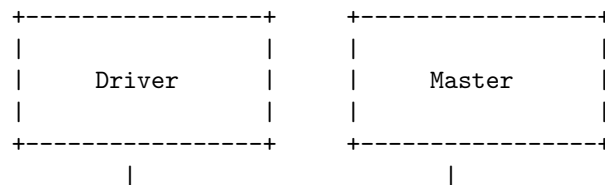
- **Executors:** The processes that run on the worker nodes of the cluster and execute the tasks assigned by the master. The executors also store the data partitions in memory or disk and communicate with the driver and other executors.

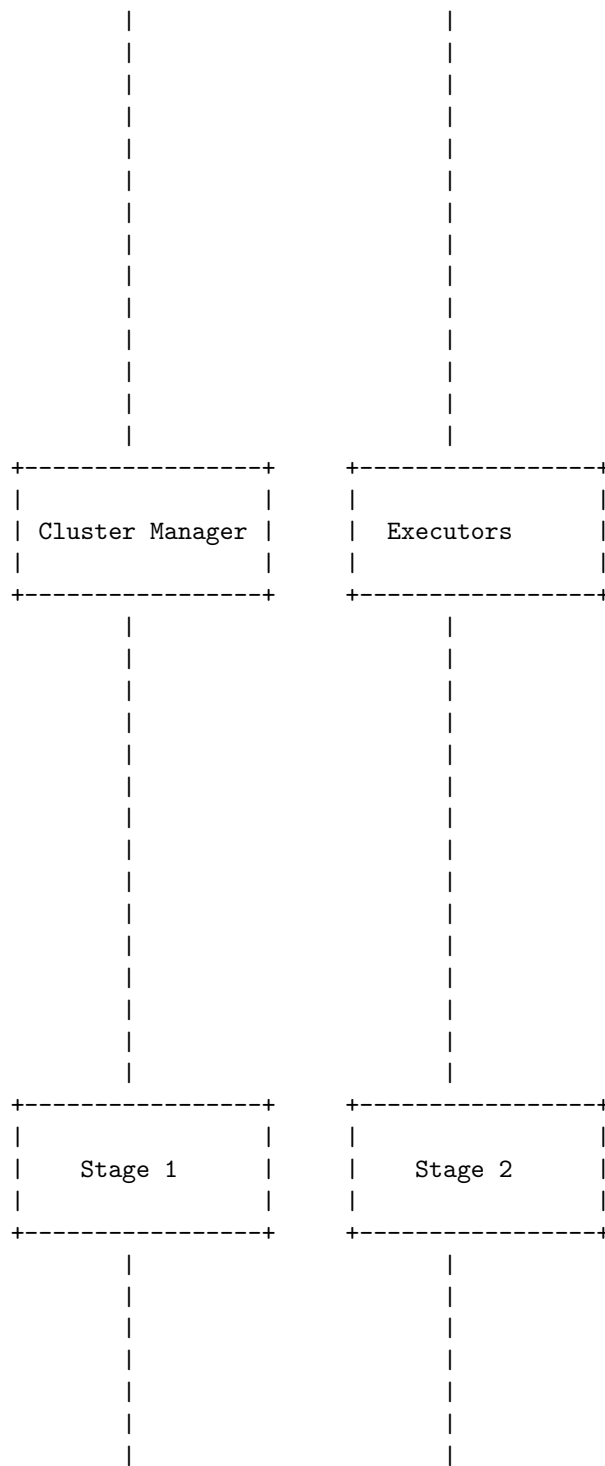
The following diagram illustrates the basic architecture of a Spark job run:

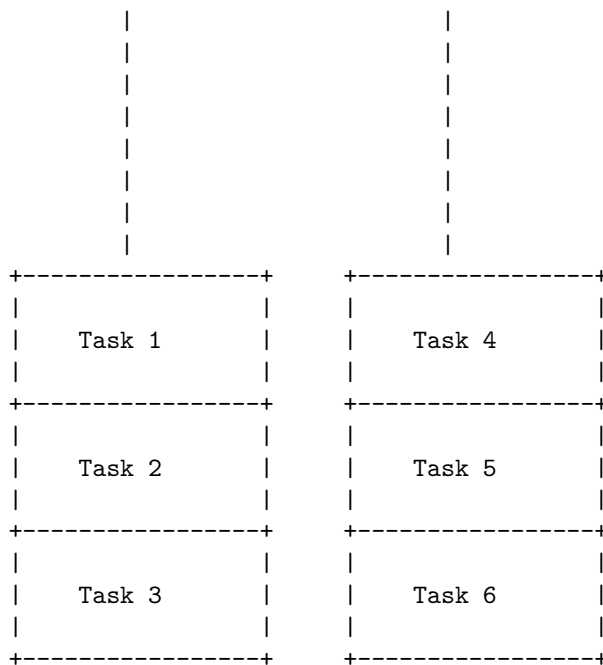


A Spark job run is divided into stages, which are collections of tasks that perform the same computation on different data partitions. A stage is created when a shuffle operation occurs, which redistributes the data across the cluster based on a key. A task is the smallest unit of work in a Spark job run, which processes a single data partition and produces an output partition. A task can be executed in parallel by different executors.

The following diagram illustrates the stages and tasks of a Spark job run:



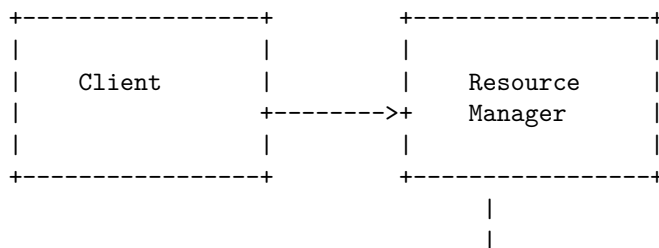


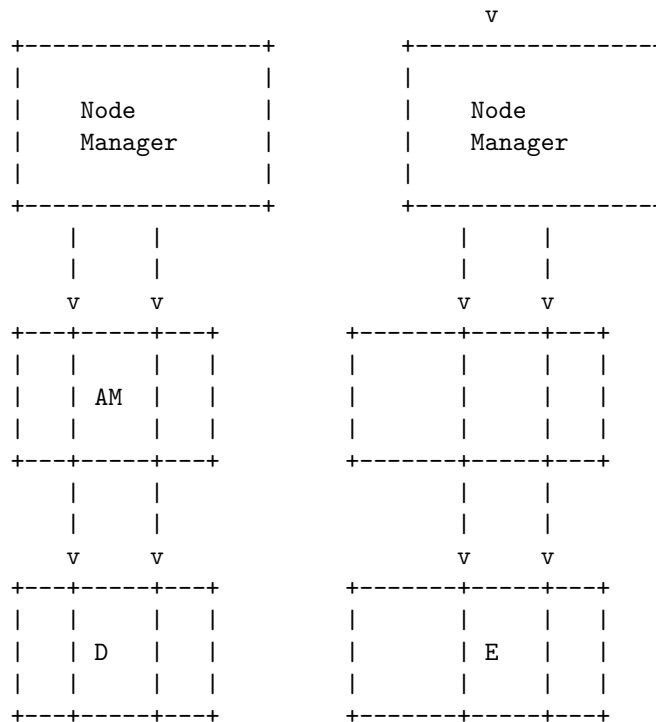


A Spark job run is represented by a directed acyclic graph (DAG) of stages and tasks, which shows the dependencies and order of execution of the computations. The DAG is generated by the driver based on the transformations and actions applied on the data

Spark on YARN is a way of running Spark applications on a Hadoop cluster that uses YARN as the resource manager. YARN is responsible for allocating resources such as CPU and memory to the Spark application, and launching and monitoring the Spark processes on the cluster nodes. Spark on YARN can run in two modes: cluster mode and client mode. In cluster mode, the Spark driver runs inside an application master process that is managed by YARN, and the client can disconnect after initiating the application. In client mode, the driver runs in the client process, and the application master is only used for requesting resources from YARN.

The following diagram illustrates the basic architecture of Spark on YARN in cluster mode:





AM: Application Master
D: Driver
E: Executor

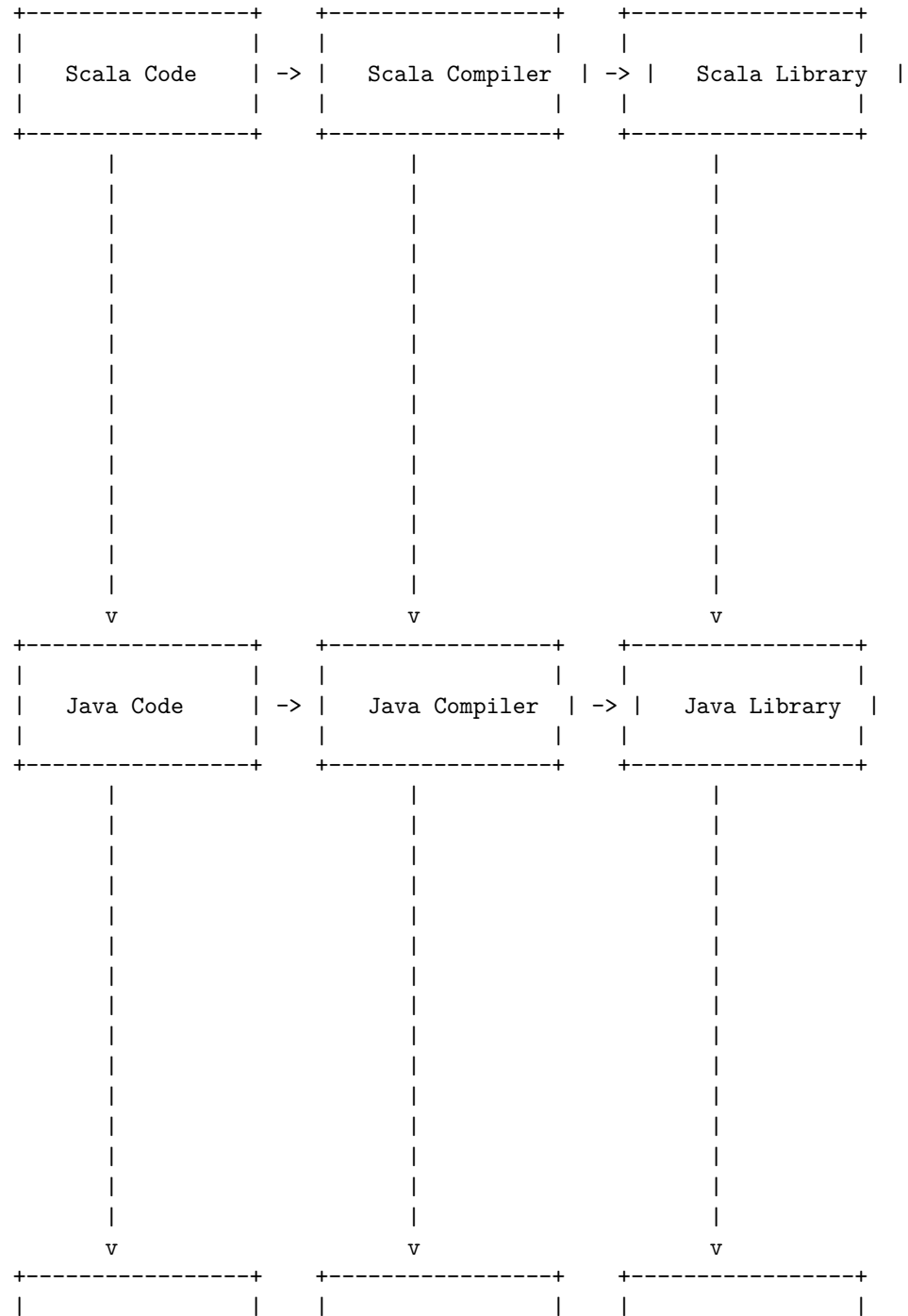
Scala is a general-purpose programming language that supports multiple paradigms, such as object-oriented, functional, concurrent, and reactive programming. Scala runs on the Java Virtual Machine (JVM) and interoperates with Java code and libraries.

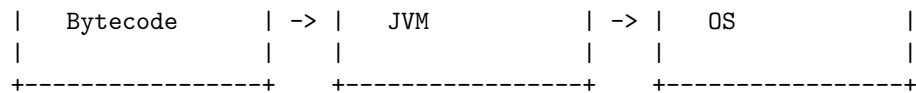
The architecture of Scala is based on the following principles:

- Scala is designed to be expressive and concise, allowing programmers to write less code and achieve more functionality.
- Scala is statically typed, meaning that the types of variables and expressions are checked at compile time, preventing many runtime errors and improving performance.
- Scala is extensible, meaning that programmers can define new types, operators, and syntax using features such as implicit conversions, type classes, and macros.
- Scala is scalable, meaning that it can handle both small and large programs, from scripting to enterprise applications, using features such as traits, pattern matching, and futures.

The following diagram illustrates the basic architecture of a Scala program:

SCALA



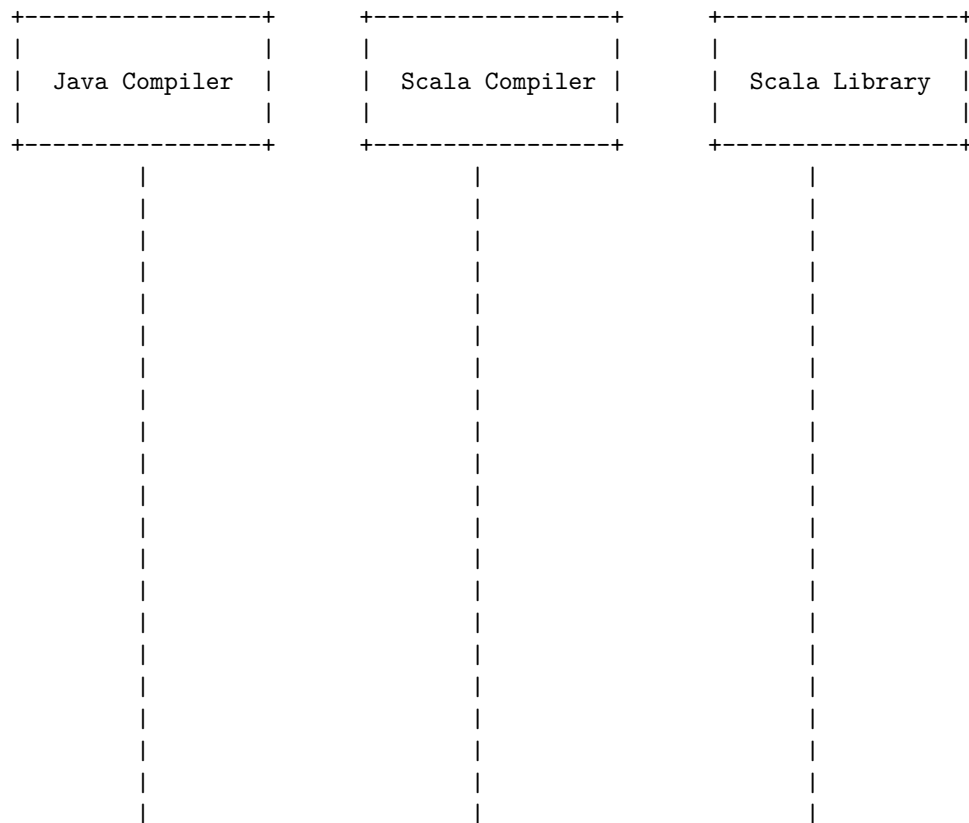


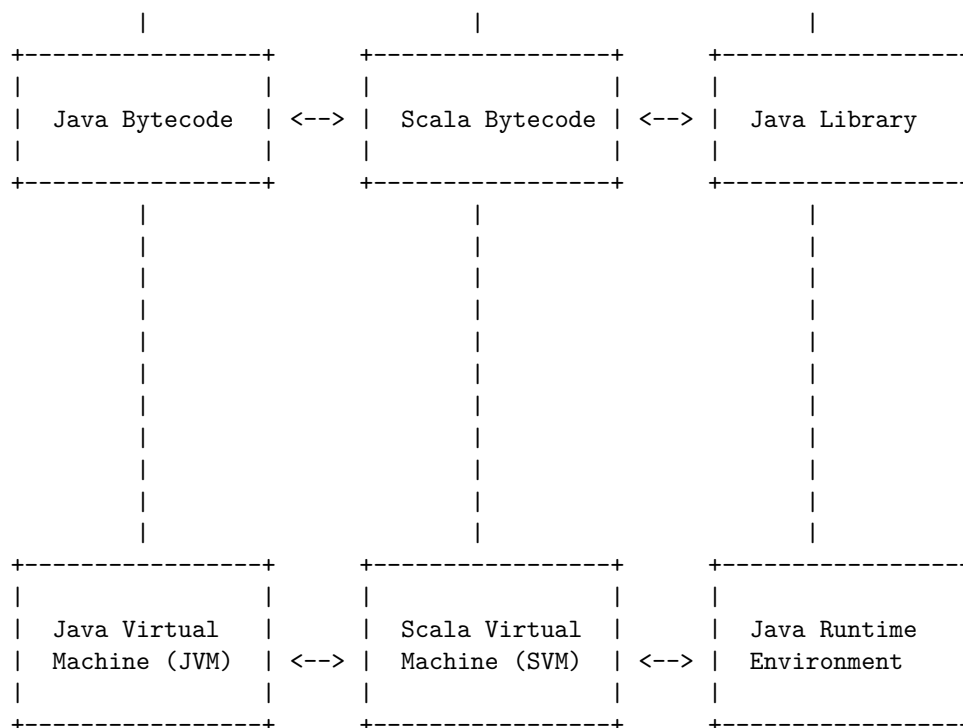
The diagram shows that Scala code is first compiled by the Scala compiler into Java code, which is then compiled by the Java compiler into bytecode, which is then executed by the JVM on the operating system. The Scala library provides a rich set of data structures and functions that are compatible with the Java library. The Scala compiler and library are themselves written in Scala, demonstrating the expressiveness and scalability of the language.

Scala is a general-purpose, high-level, multi-paradigm programming language that seamlessly integrates features of object-oriented and functional languages. It is designed to express common programming patterns in a concise, elegant, and type-safe way. Scala is also capable of working with distributed data and supports immutable data and higher-order functions.

Introduction to Scala

The following diagram illustrates the basic architecture of Scala:





The diagram shows that Scala code is compiled into Java bytecode by the Scala compiler, and can run on the Java Virtual Machine (JVM) or the Scala Virtual Machine (SVM). Scala code can also interact with Java code and libraries, and vice versa. Scala also has its own library that provides additional features and functionality. Scala code can be executed by the Java Runtime Environment (JRE) or the Scala Runtime Environment (SRE).

Classes and objects in Scala are the basic building blocks of object-oriented programming. A class is a blueprint for creating objects, which are instances of the class. A class can contain methods, values, variables, types, objects, traits, and classes as its members. An object is a singleton instance of its own class and can be used to define static members or utility methods.

To define a class in Scala, you use the keyword `class` followed by an identifier and an optional list of parameters. For example:

```
class Person(name: String, age: Int) {
  // class body
}
```

To create an object of a class, you use the keyword `new` followed by the class name and the arguments for the parameters. For example:

```
val alice = new Person("Alice", 25)
```

To define an object in Scala, you use the keyword `object` followed by an identifier. An object can have the same members as a class, except for parameters. For example:

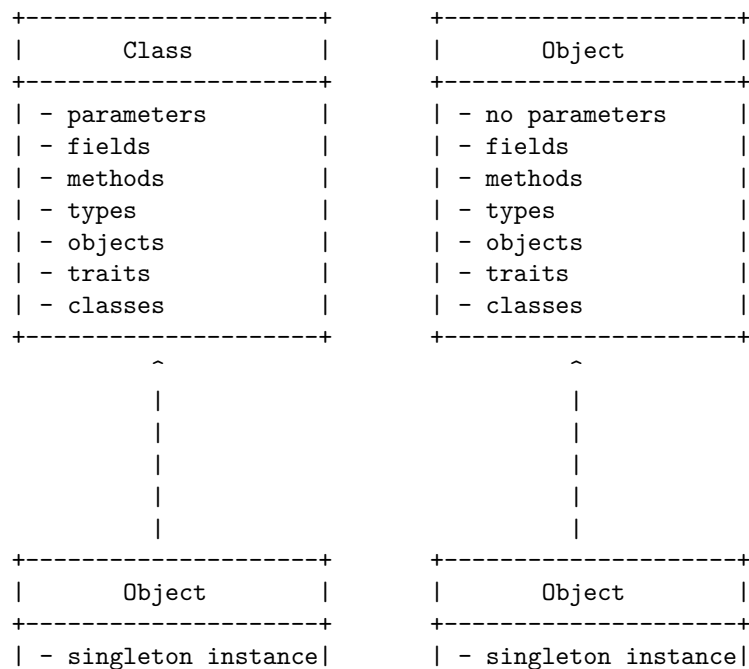
```
object Math {
  // object body
}
```

An object can also be defined as a companion object of a class, which means that it has the same name and is defined in the same file as the class. A companion object can access the private members of the class and vice versa. A companion object is useful for defining factory methods, constants, or implicit conversions. For example:

```
class Circle(radius: Double) {
  // class body
}

object Circle {
  // object body
  val Pi = 3.14
  def area(radius: Double): Double = Pi * radius * radius
}
```

The following diagram illustrates the basic architecture of classes and objects in Scala using ASCII art:

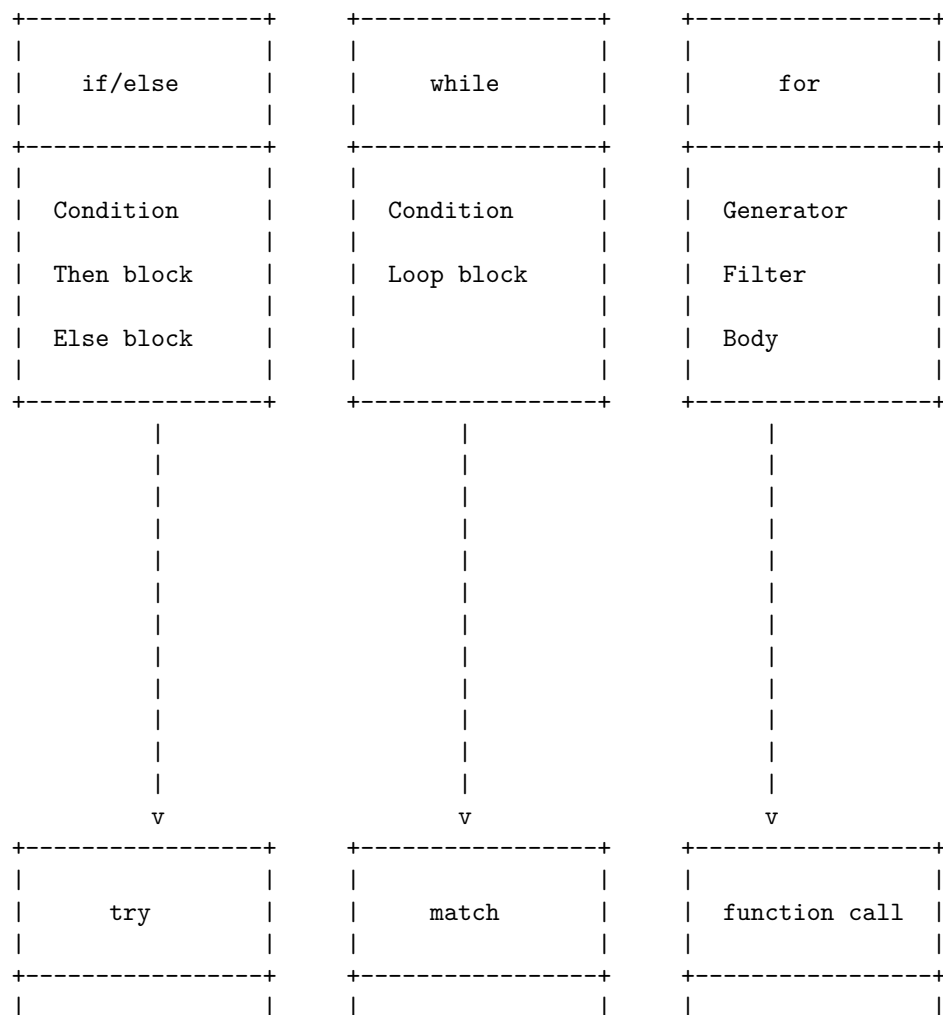


	!	&&		^				

According to the search results, Scala has only a handful of built-in control structures, which are if, while, for, try, match, and function calls . The reason Scala has so few is that it has included function literals since its inception . A function literal is a function that is not defined by a name, but by its parameters and body, such as `(a:Int, b:Int) => a + b`.

The following diagram illustrates the basic architecture of a built-in control structure in Scala:

Built-in control structures in Scala



Try block	Expression	Function name
Catch block	Case clauses	Arguments
Finally block		
-----	-----	-----

A function is a piece of code that takes some input and produces some output. A closure is a special kind of function that can access variables that are defined outside of its scope. For example, in Scala, we can define a function that adds a constant value to its argument:

```
def addConstant(x: Int) = x + 10
```

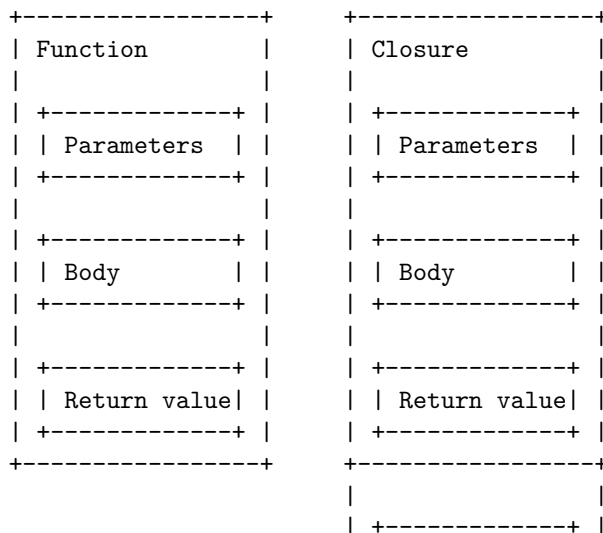
This function is not a closure, because it does not use any free variables. A free variable is a variable that is not defined in the function or passed as a parameter. Now, suppose we want to make the constant value configurable. We can use a closure to achieve this:

```
val constant = 10 // a free variable
val addConstant = (x: Int) => x + constant // a closure
```

This function is a closure, because it uses the variable `constant` that is defined outside of its scope. The closure captures the value of `constant` and uses it in its body. The value of the closure depends on the value of the free variable.

Functions and closures in Scala

The following diagram illustrates the basic architecture of a function and a closure in Scala:



```

| | Free      | |
| | variables | |
| +-----+ |
+-----+

```

A function has parameters, a body, and a return value. A closure has the same components, but also has access to free variables that are defined outside of its scope. The closure can read and write the free variables, and the value of the closure may change if the free variables change.

Inheritance is a mechanism in Scala by which one class can inherit the features (fields and methods) of another class. There are different types of inheritance in Scala, such as single, multilevel, multiple, and hybrid. Multiple and hybrid inheritance can only be achieved by using traits, which are abstract types that can contain both abstract and concrete members.

Inheritance in Scala

The following diagram illustrates the basic concept of inheritance in Scala using ASCII art. The diagram shows a superclass called `Animal`, which has two fields (name and sound) and one method (`makeSound`). The `Animal` class has two subclasses, `Cat` and `Dog`, which inherit the fields and method of `Animal` and also have their own fields and methods. The `Cat` class has a field called `color` and a method called `purr`, while the `Dog` class has a field called `breed` and a method called `fetch`. The subclasses can override the inherited members of the superclass by using the `override` keyword.

```

+-----+
|   Animal   |
+-----+
| - name: String |
| - sound: String |
+-----+
| + makeSound(): Unit |
+-----+
      / \
     /   \
    /       \
+-----+   +-----+
|   Cat   |   |   Dog   |
+-----+   +-----+
| - color: String |   | - breed: String |
+-----+   +-----+
| + makeSound(): Unit |   | + makeSound(): Unit |
| + purr(): Unit      |   | + fetch(): Unit      |
+-----+   +-----+

```

The diagram can be interpreted as follows:

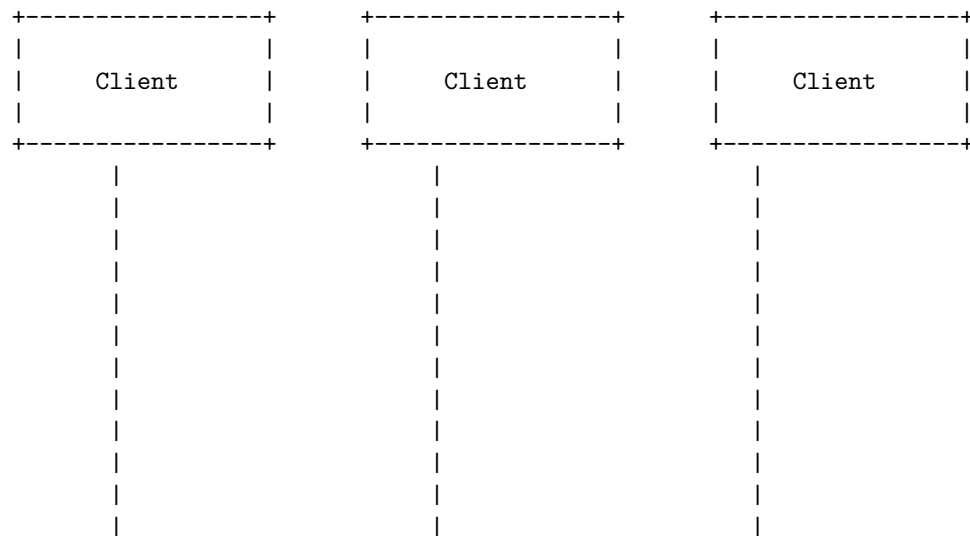
- The Animal class is the superclass of both Cat and Dog classes.
- The Cat and Dog classes are subclasses of the Animal class and inherit its fields and methods.
- The Cat and Dog classes can access the name and sound fields of the Animal class and call the makeSound method of the Animal class.
- The Cat and Dog classes can also define their own fields and methods, such as color, breed, purr, and fetch.
- The Cat and Dog classes can override the makeSound method of the Animal class by providing their own implementation of the method using the override keyword. For example, the Cat class can override the makeSound method to print “Meow” instead of the sound field of the Animal class.

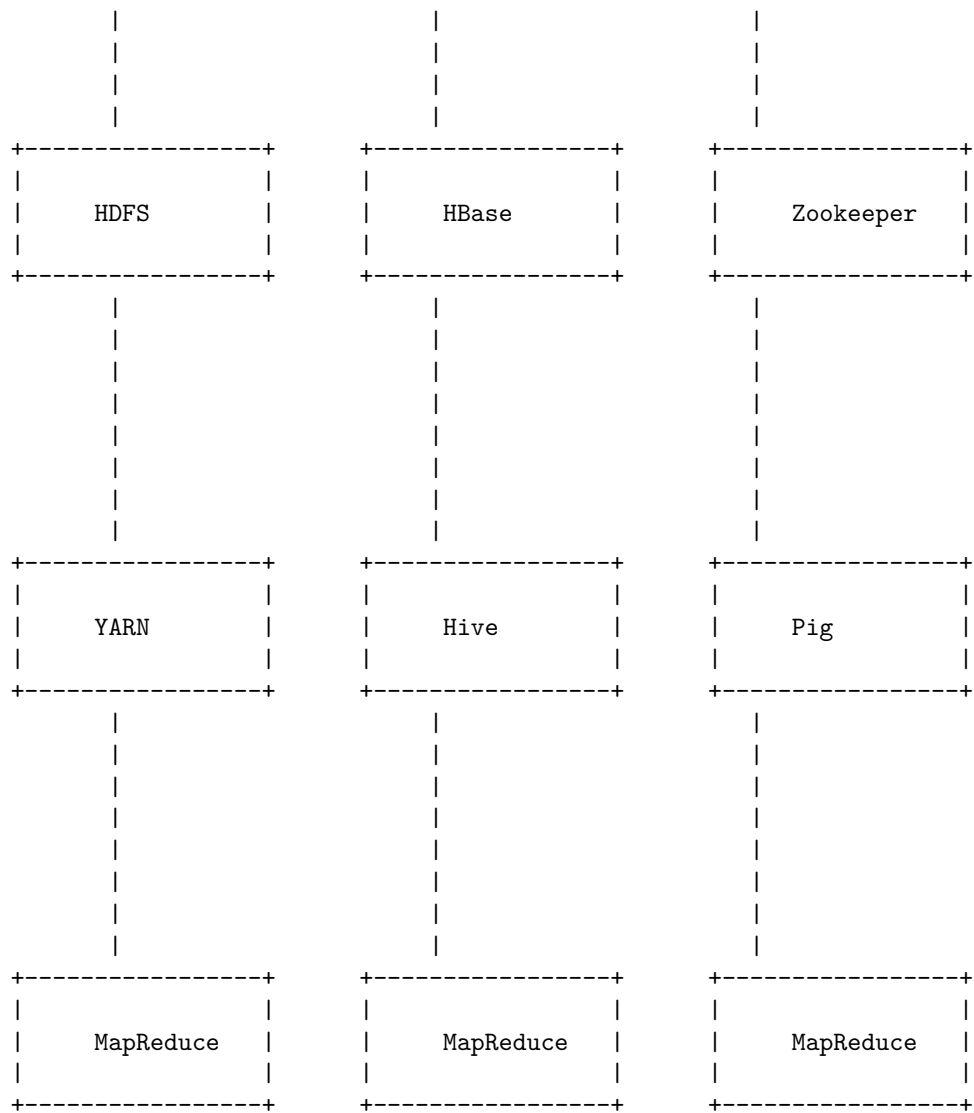
Hadoop is a framework for distributed processing of large-scale data sets across clusters of computers. It consists of several components, such as HDFS (Hadoop Distributed File System), MapReduce (a programming model for parallel processing), and YARN (a resource management platform).

Hadoop also includes several additional modules that provide additional functionality, such as:

- Pig: a high-level platform for creating MapReduce programs using a scripting language called Pig Latin.
- Hive: a data warehouse infrastructure that provides data summarization and ad-hoc querying using a SQL-like query language called HiveQL.
- HBase: a non-relational, distributed database that supports structured data storage for large tables.
- Zookeeper: a service for coordinating and managing distributed systems.

The following diagram illustrates the basic architecture of a Hadoop ecosystem, including Pig, Hive, and HBase:





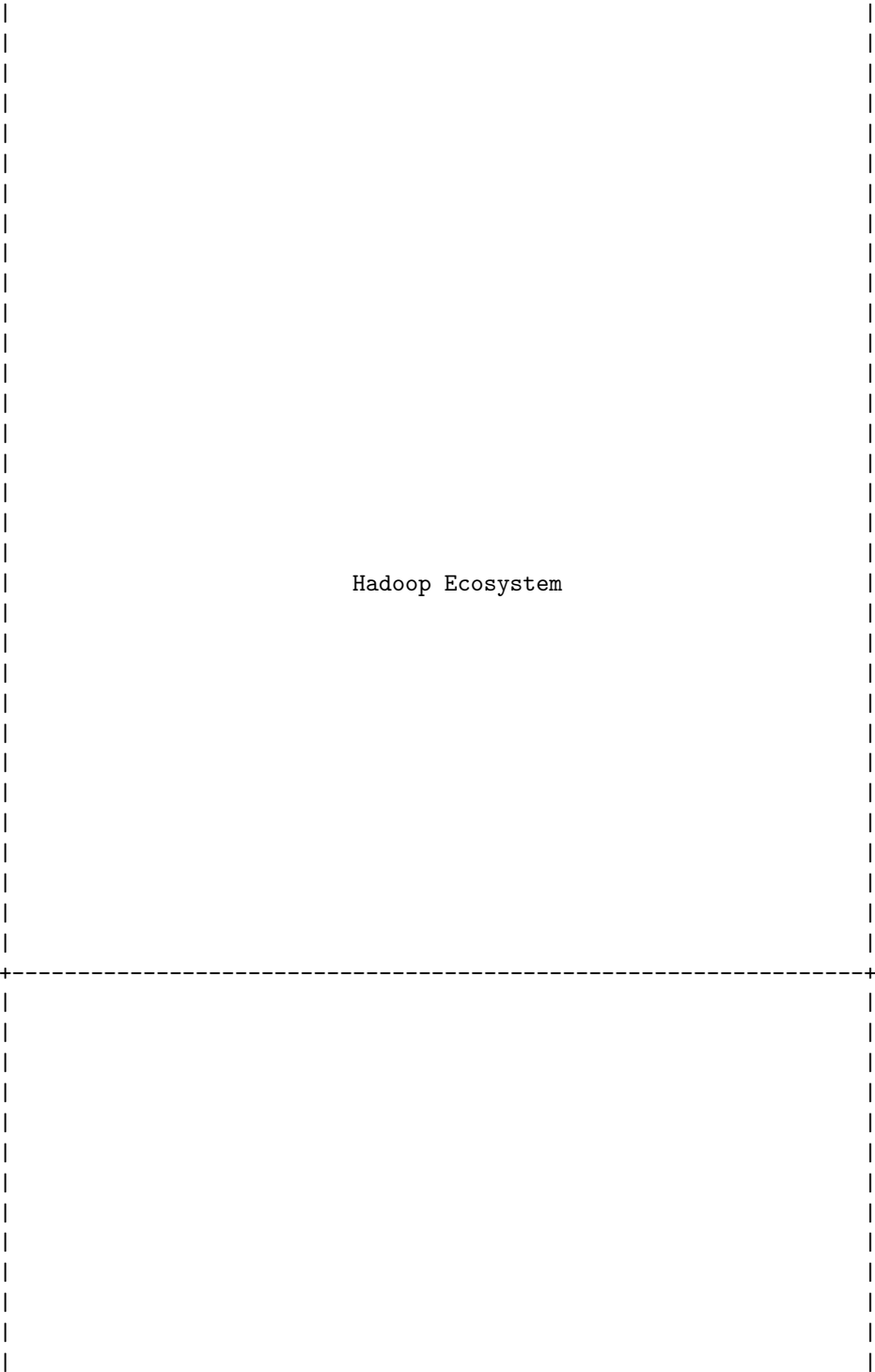
Hadoop Eco System Frameworks are a set of tools, libraries, and frameworks that help you build applications on top of Apache Hadoop. Hadoop is a framework that enables processing of large data sets which reside in the form of clusters. The core component of the Hadoop ecosystem is a Hadoop distributed file system (HDFS). HDFS is a distributed file system that has the capability to store a large stack of data sets. There are four major elements of Hadoop i.e. HDFS, MapReduce, YARN, and Hadoop Common .

The following is a detailed ASCII diagram for Hadoop Eco System Frameworks:

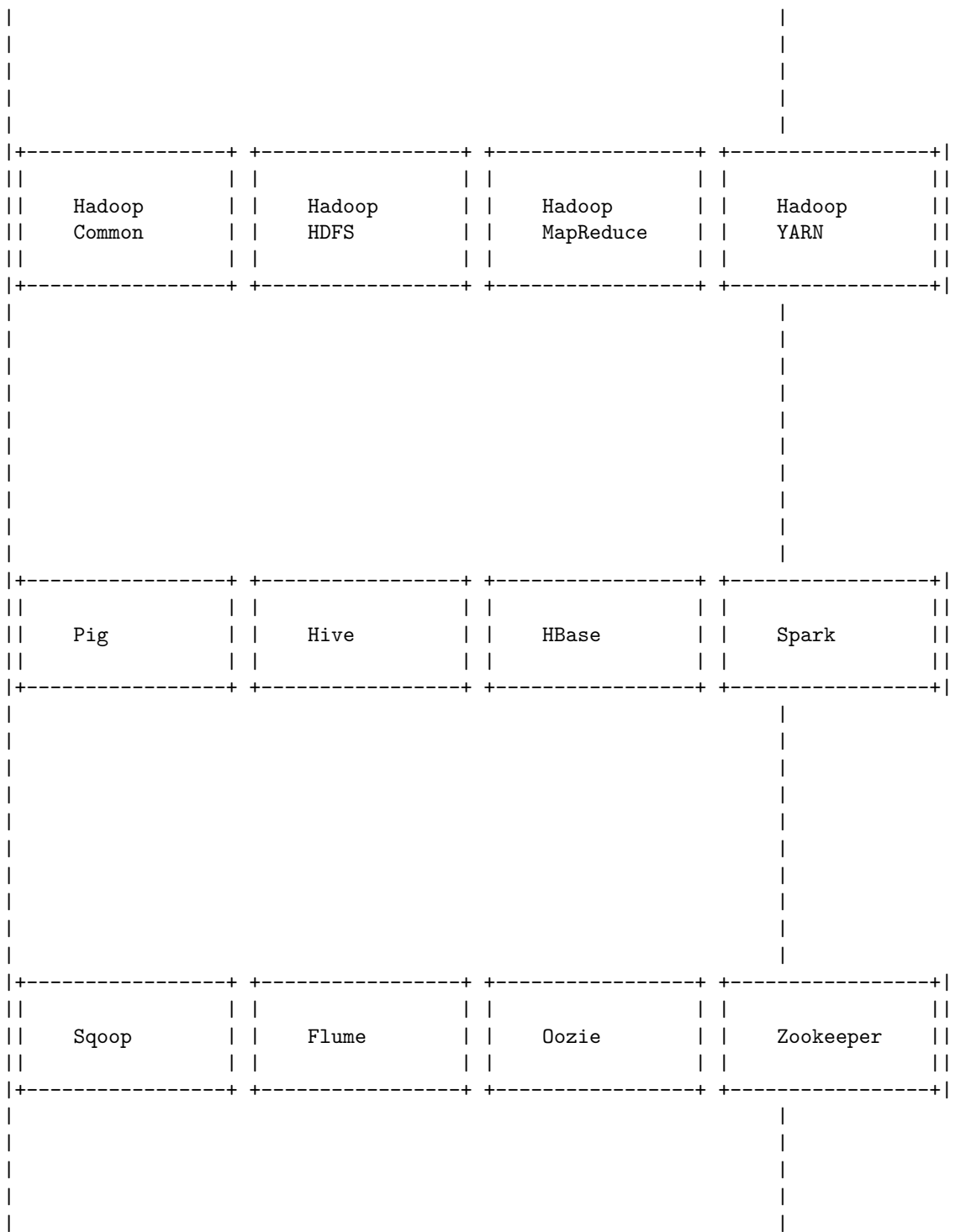
```

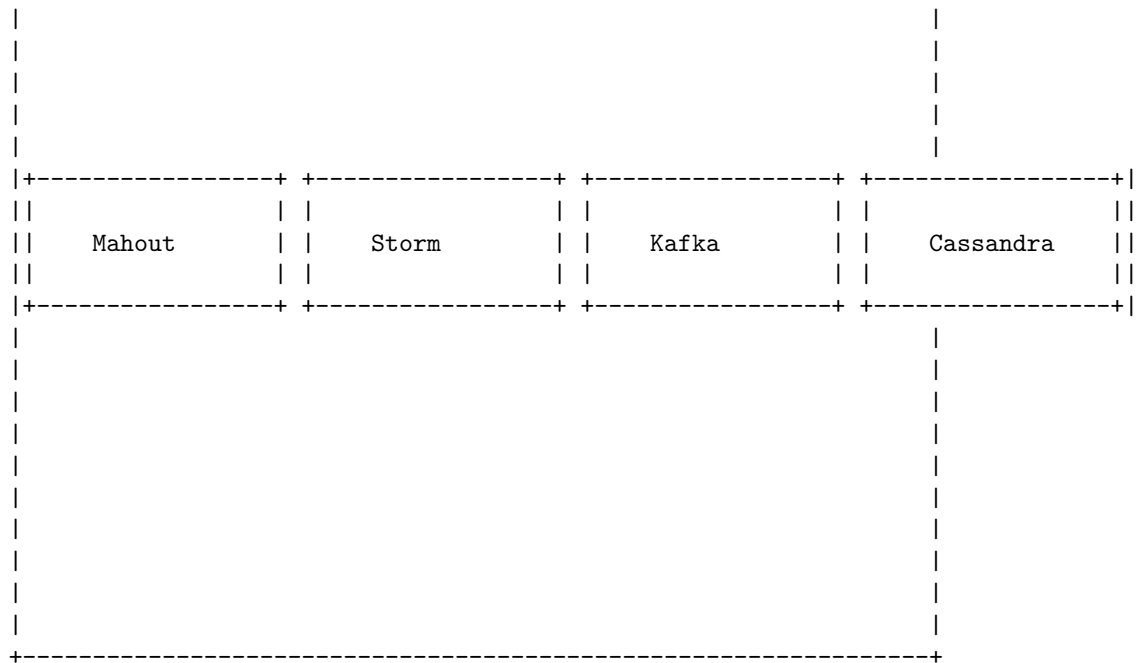
+-----+

```



Hadoop Ecosystem





The diagram shows the four core components of Hadoop and some of the most popular tools and frameworks that are built on top of them or work with them. Each of these components and tools has a specific function and role in the Hadoop ecosystem. For example:

- Hadoop Common: It provides the common utilities and libraries that are used by other Hadoop modules.
- Hadoop HDFS: It is the distributed file system that stores the data in a fault-tolerant and scalable manner across the cluster nodes.
- Hadoop MapReduce: It is the programming model that allows for parallel processing of large data sets using key-value pairs.
- Hadoop YARN: It is the resource management layer that allocates and manages the compute resources for the applications running

Applications on Big Data using Pig

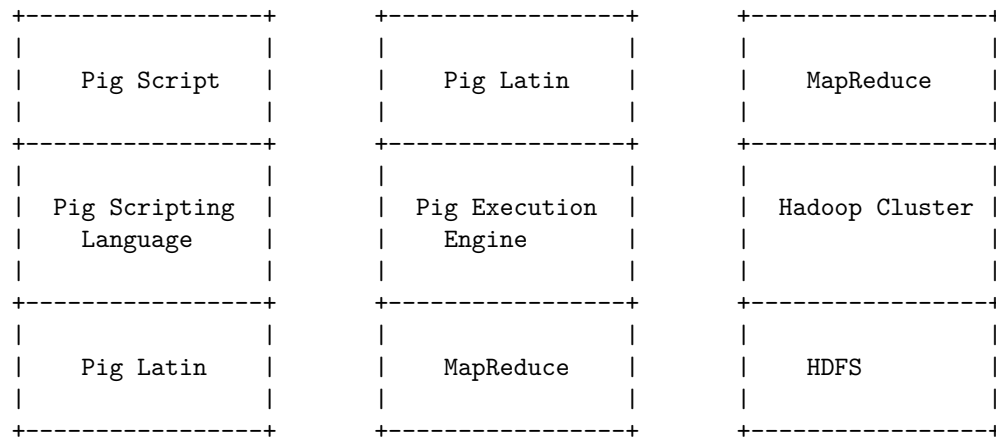
Pig is a high-level platform or tool which is used to process large datasets. It provides a high level of abstraction for processing over MapReduce. It provides a high-level scripting language, known as Pig Latin which is used to develop the data analysis codes.

Some of the applications of Pig in big data are:

- For exploring large datasets Pig Scripting is used .
- Provides supports across large data sets for Ad-hoc queries .
- In the prototyping of large data-sets processing algorithms.
- Required to process the time-sensitive data loads.

- For collecting large amounts of datasets in form of search logs and web crawls.
- Used where the analytical insights are needed using the sampling.
- Utilized by telecom organizations to de-identify the customer call data information.
- Handles a wide range of data, both unstructured as well as structured.
- Provides the ability to create user-defined functions in other programming languages like Java and embed or invoke them in Pig Scripts.

The following diagram illustrates the basic architecture of a Pig application:



The Pig Scripting Language is used to write the Pig Script, which is a sequence of data transformations and operations. The Pig Script is then compiled into Pig Latin, which is an intermediate representation of the script. The Pig Execution Engine then converts the Pig Latin into MapReduce jobs, which are executed on the Hadoop Cluster. The Hadoop Cluster consists of the HDFS, which is the distributed file system that stores the data, and the MapReduce framework, which is the parallel processing engine that performs the data analysis. The output of the MapReduce jobs is then stored back in the HDFS or returned to the user.

Applications on Big Data using Hive

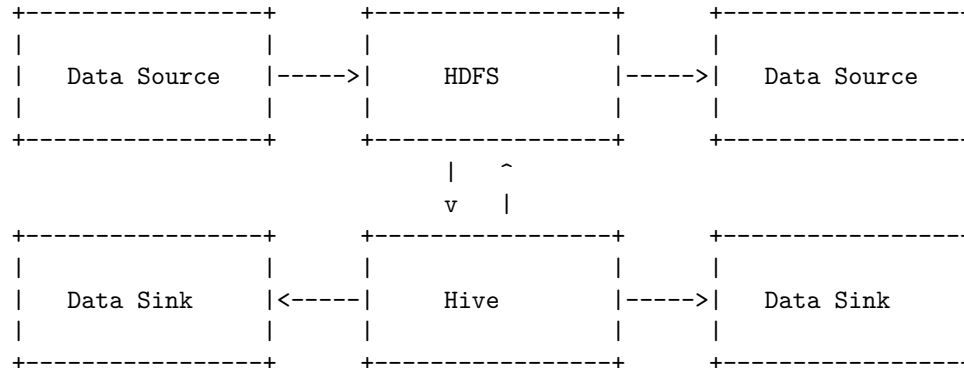
Hive is a data warehouse system that allows users to query and analyze large-scale data stored in Hadoop Distributed File System (HDFS) using a SQL-like language called HiveQL. Hive can also interact with other data sources, such as relational databases, NoSQL databases, and cloud storage services.

Hive data is predominantly used in the following applications:

- Big Data Analytics, running analytics reports on transaction behavior, activity, volume, and more
- Tracking fraudulent activity and generating reports on this activity

- Creating dashboards based on the data
- Auditing purposes and a store for historical data
- Feeding data for Machine learning and building intelligence around it

The following diagram illustrates the basic architecture of a Hive application on Big Data:



In this diagram, the data source can be any system that produces or collects data, such as web servers, sensors, logs, etc. The data sink can be any system that consumes or stores data, such as databases, dashboards, reports, etc. HDFS is the distributed file system that stores the data in a scalable and fault-tolerant manner. Hive is the data warehouse system that provides a SQL-like interface to query and analyze the data stored in HDFS.

Some examples of Hive applications on Big Data are:

- FINRA, a financial regulatory authority, uses Hive on Amazon EMR clusters to process and analyze trade data of up to 90 billion events using SQL.
- Netflix, a streaming service provider, uses Hive to perform ETL (extract, transform, load) operations on data from various sources, such as user behavior, ratings, recommendations, etc. and store them in HDFS for further analysis.
- Facebook, a social media platform, uses Hive to store and query data from its 300 PB data warehouse, which contains data from user profiles, messages, likes, comments, etc. Hive also supports Facebook's machine learning and data mining applications.

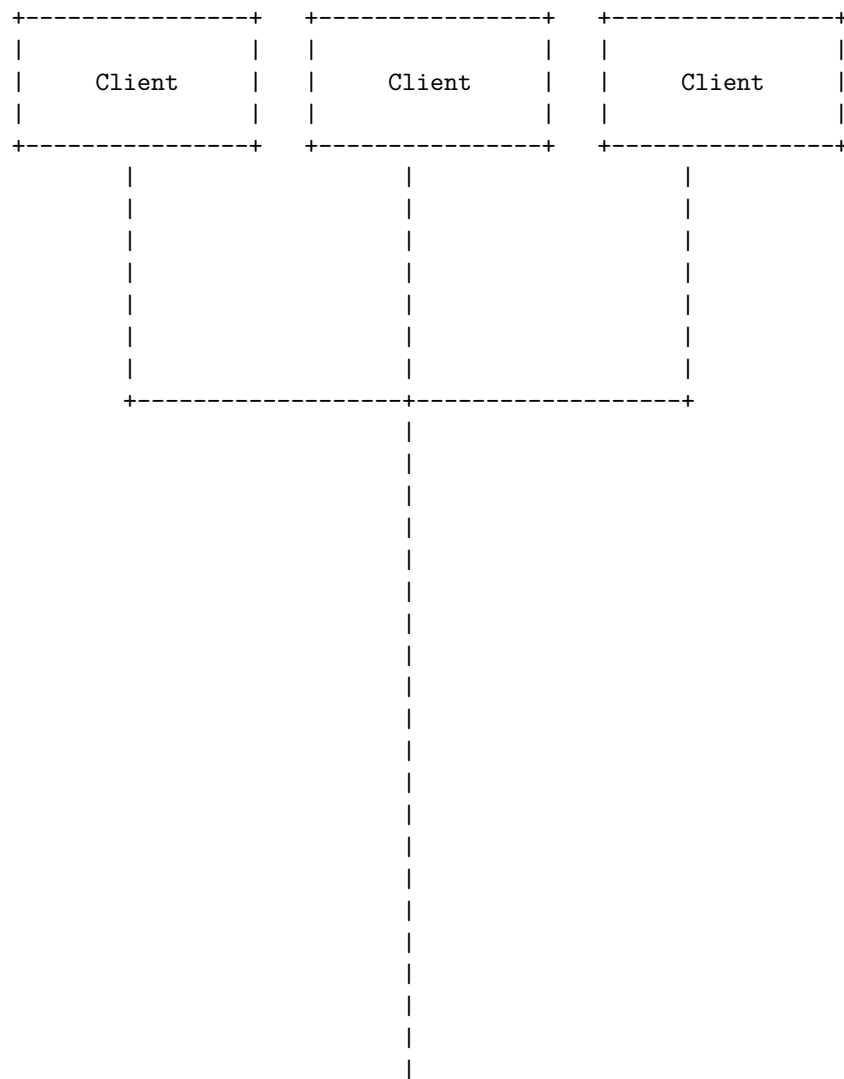
Applications on Big Data using HBase

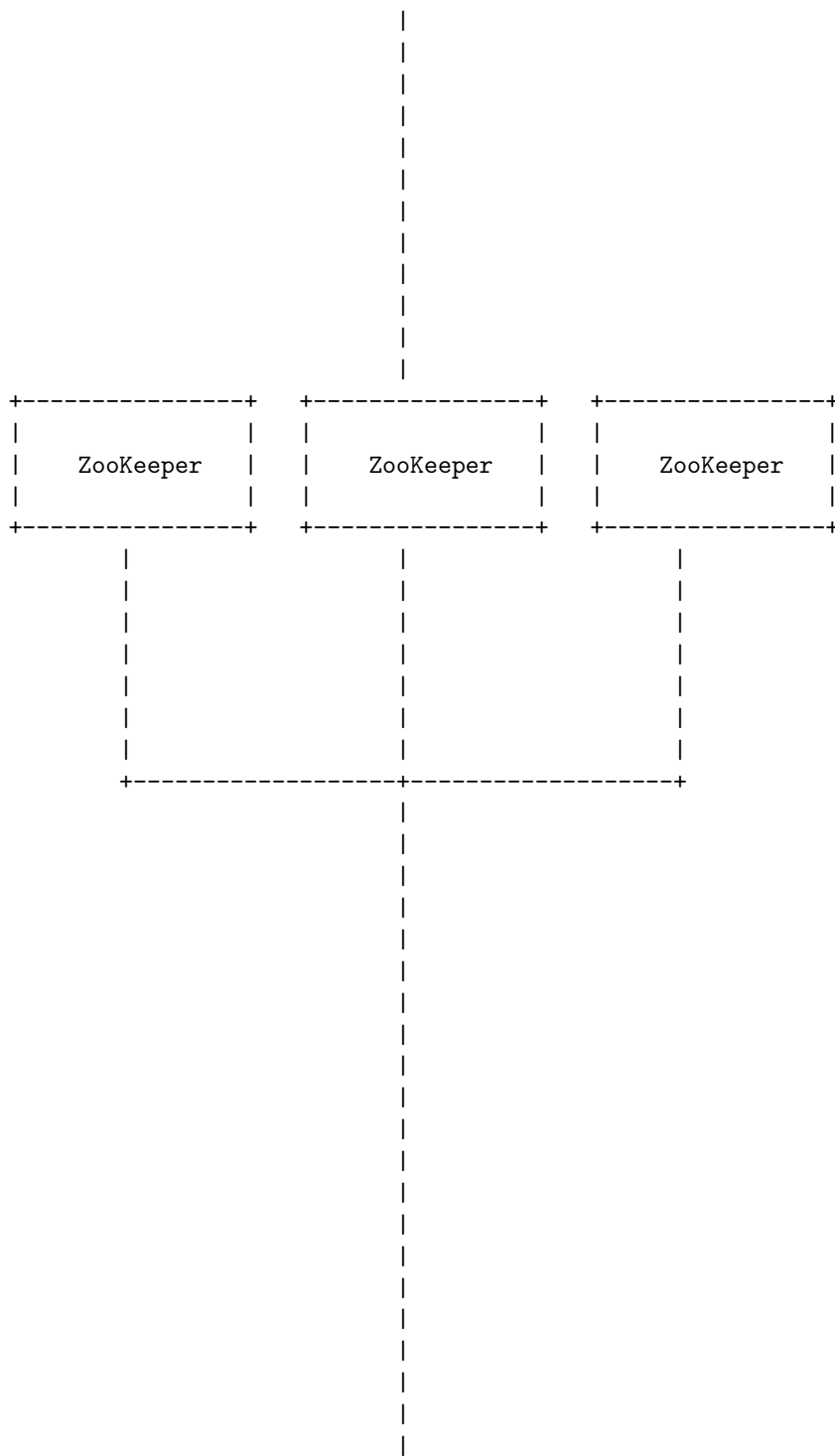
HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS). HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases. It is well suited for real-time data processing or random read/write access to large volumes of data.

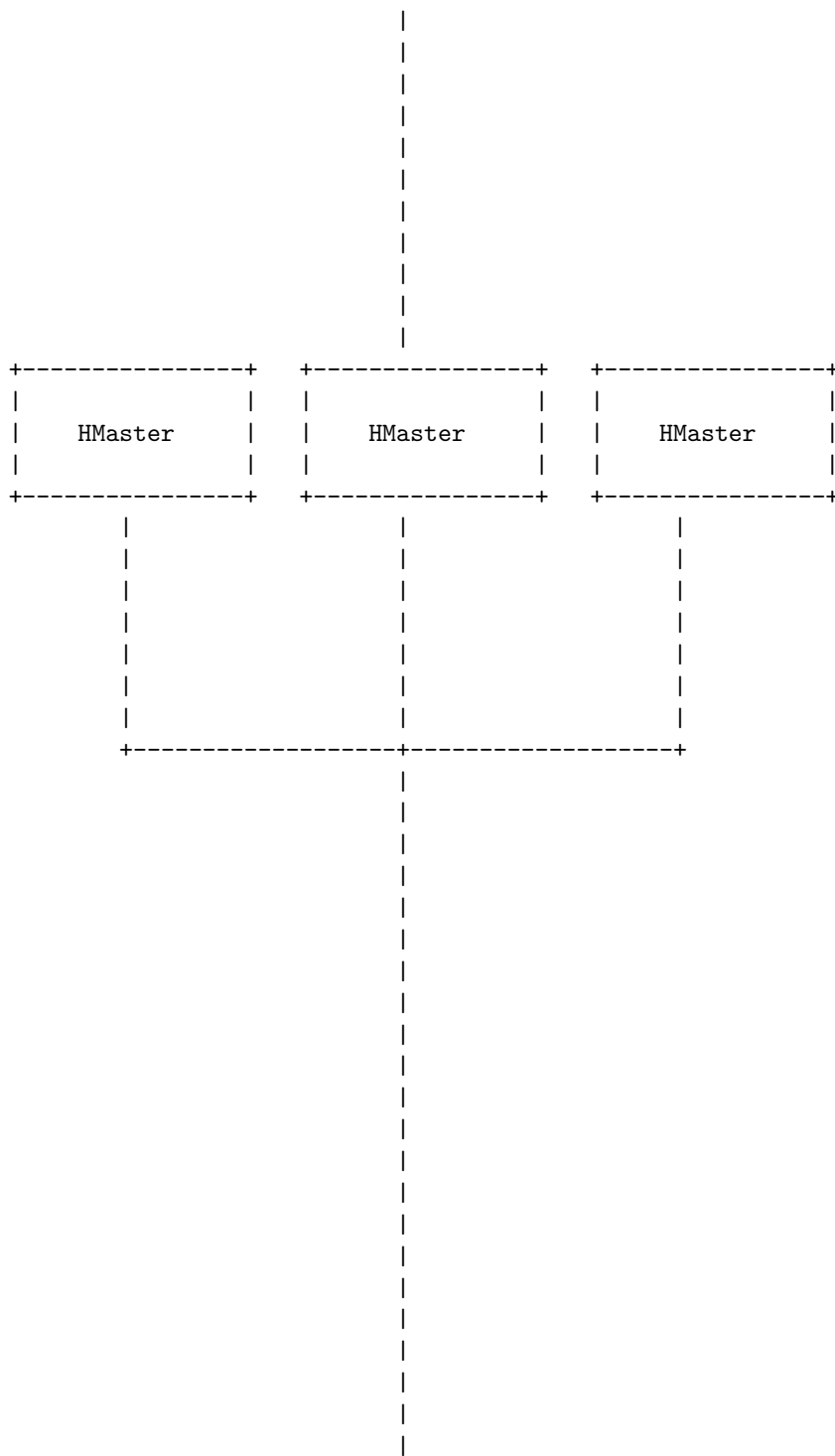
HBase works well with Hive, a query engine for batch processing of big data, to enable fault-tolerant big data applications. An example of HBase and Hive integration is Facebook’s messaging platform, which uses HBase for storing messages and metadata, and Hive for analytics and reporting.

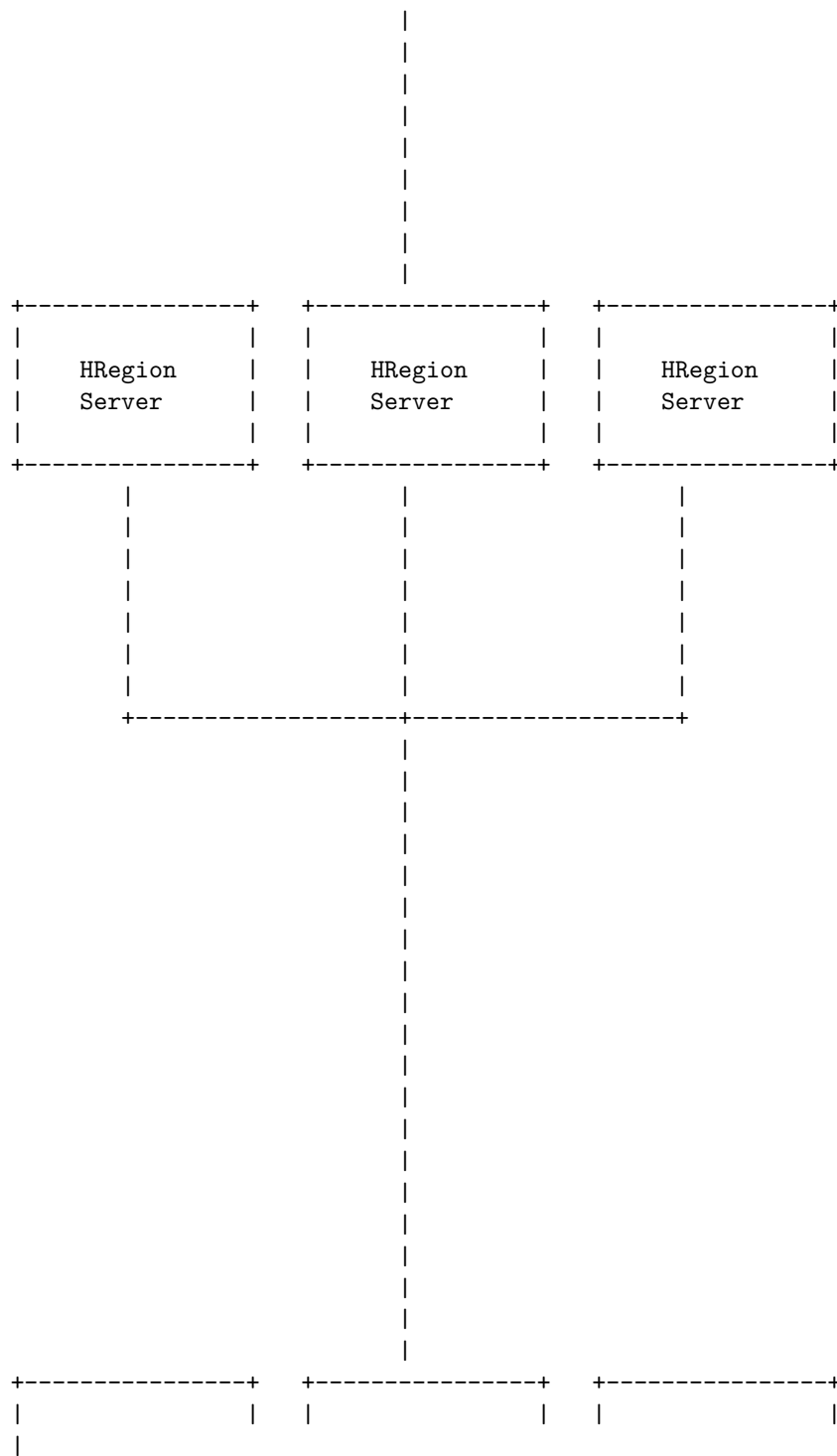
HBase can also be used with other Hadoop ecosystem components, such as MapReduce, Spark, Pig, Flume, and Sqoop, to perform various data processing and ingestion tasks. HBase can also be accessed through Java API, REST API, Thrift API, or shell commands.

The following diagram illustrates the basic architecture of a HBase application on big data:





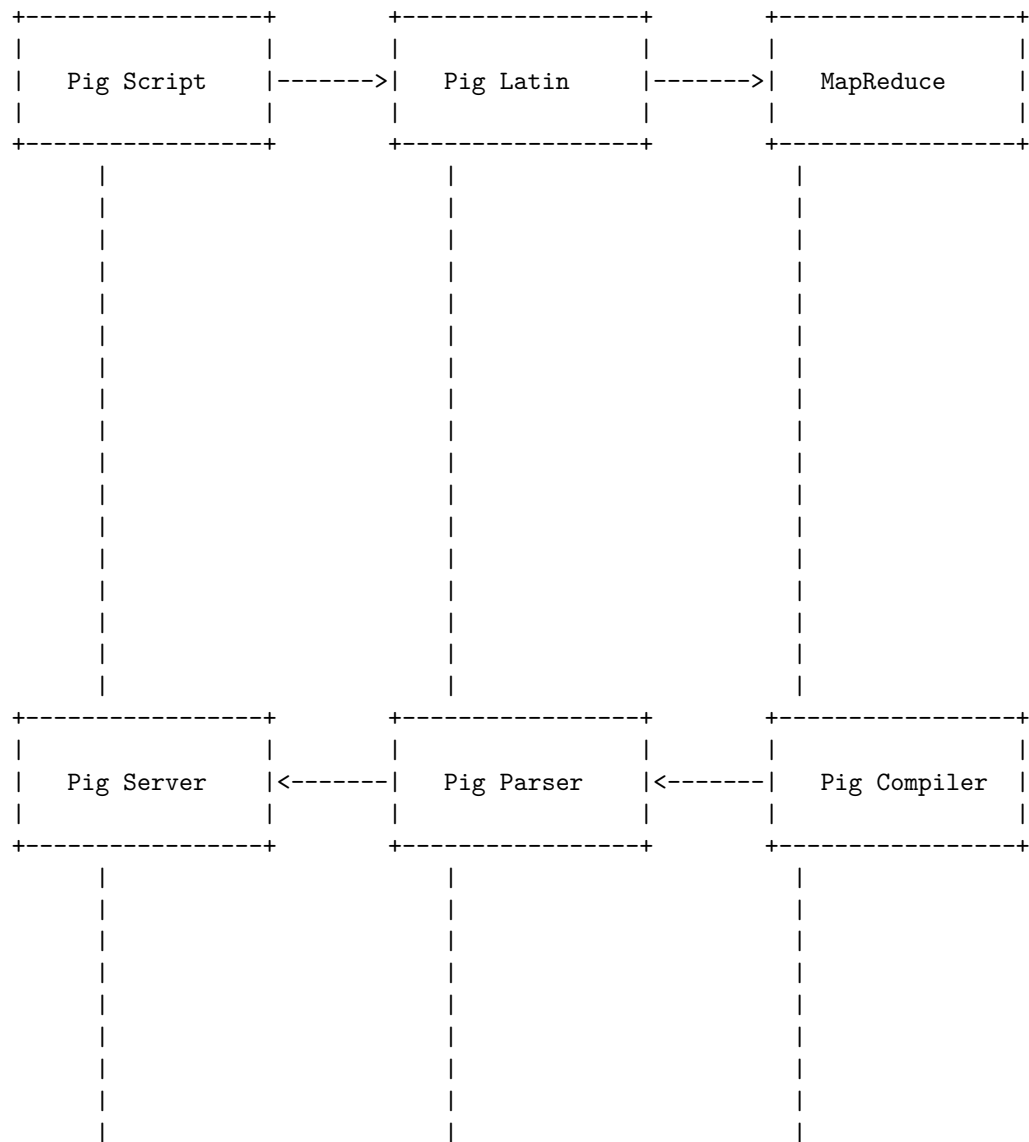


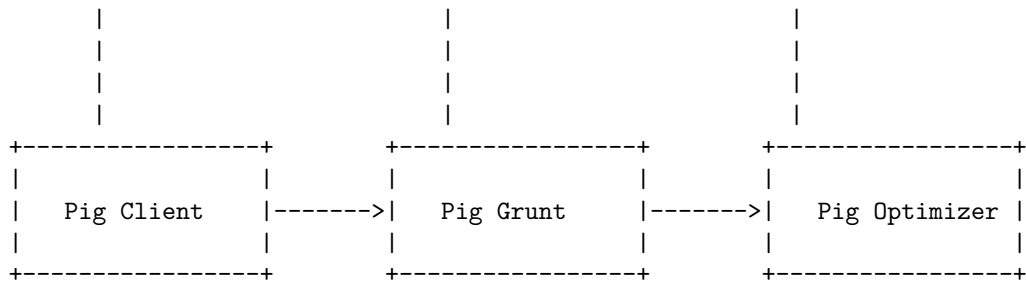


Pig is a high-level platform or tool which is used to process large datasets on Hadoop using a scripting language called Pig Latin. Pig Latin consists of a series of operations or transformations which are applied to the input data to produce output. Pig has a runtime environment that executes Pig Latin programs on Hadoop clusters.

The following diagram illustrates the basic architecture of Pig:

Pig





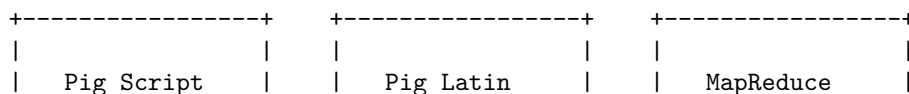
The Pig architecture consists of the following components:

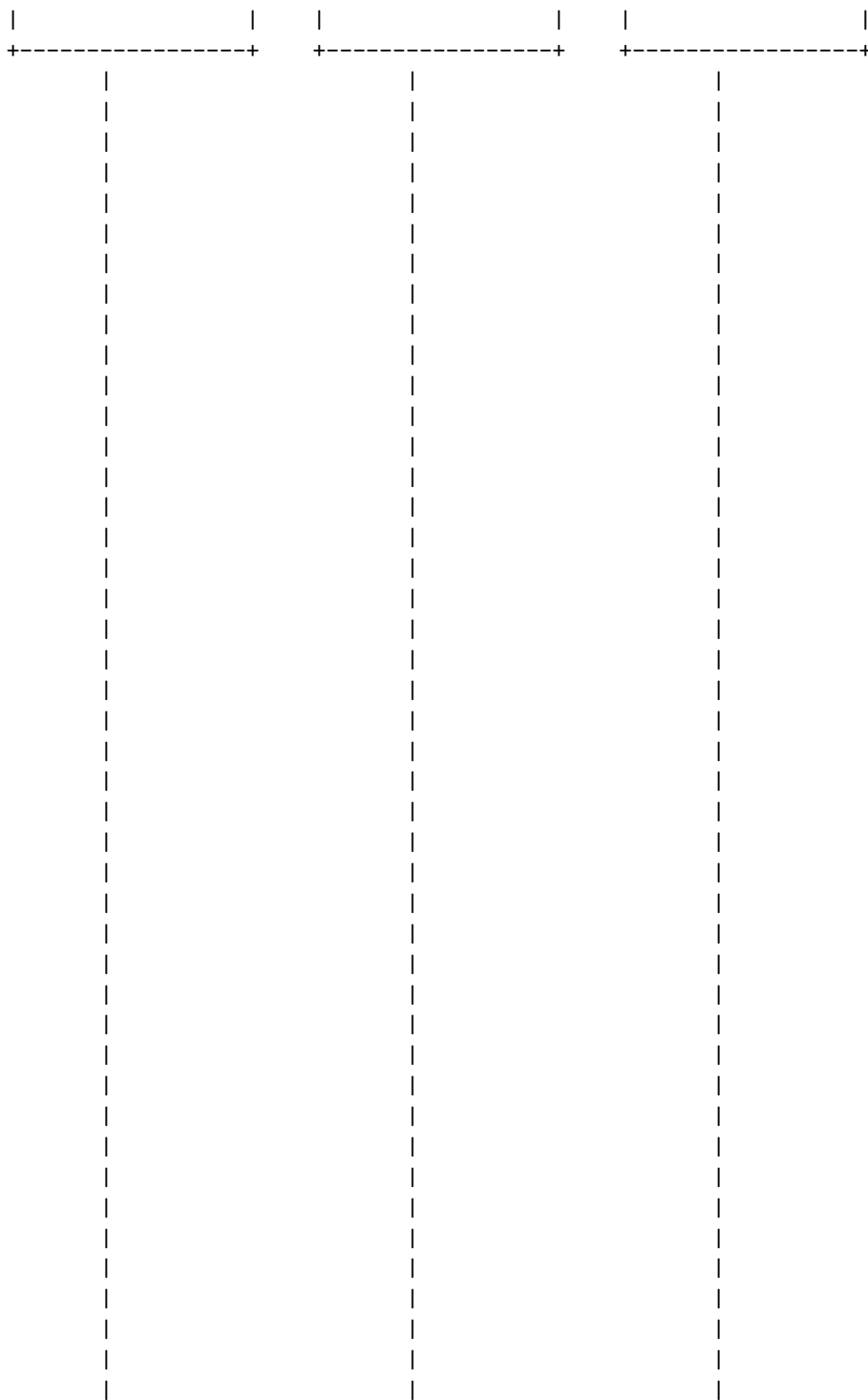
- Pig Script: This is the user-written script in Pig Latin that specifies the data analysis tasks.
- Pig Latin: This is the high-level data processing language that provides a rich set of data types and operators to perform various operations on the data.
- Pig Parser: This is the component that checks the syntax and semantics of the Pig Latin script and converts it into a logical plan.
- Pig Optimizer: This is the component that applies various optimization techniques to the logical plan to improve the performance of the Pig Latin script.
- Pig Compiler: This is the component that converts the optimized logical plan into a series of MapReduce jobs that can be executed on Hadoop clusters.
- Pig Server: This is the component that manages the execution of the MapReduce jobs on Hadoop clusters and returns the results to the Pig Client.
- Pig Client: This is the component that interacts with the Pig Server and provides the user interface to run Pig Latin scripts.
- Pig Grunt: This is the interactive shell that allows the user to run Pig Latin commands and scripts interactively.

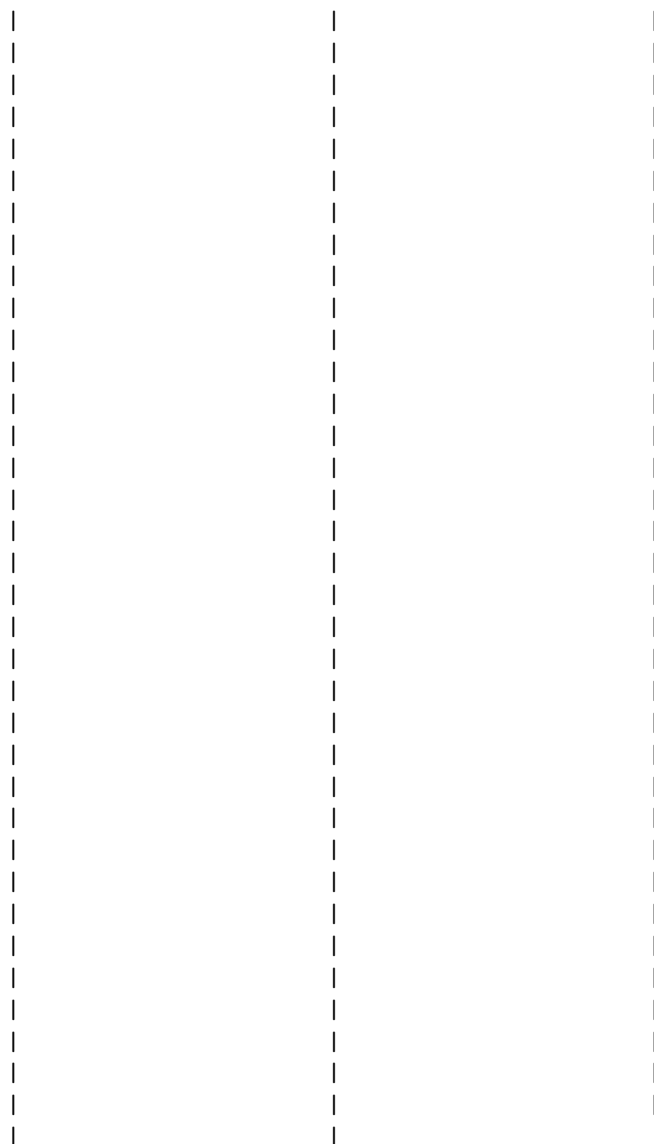
Pig is a platform for analyzing large data sets that consists of a high-level language for expressing data analysis programs, coupled with infrastructure for evaluating these programs. Pig is generally used with Hadoop; we can perform all the data manipulation operations in Hadoop using Apache Pig. To write data analysis programs, Pig provides a high-level language known as Pig Latin. This language provides various operators using which programmers can develop their own functions for reading, writing, and processing data.

Pig - Introduction to PIG

The following diagram illustrates the basic architecture of Pig:



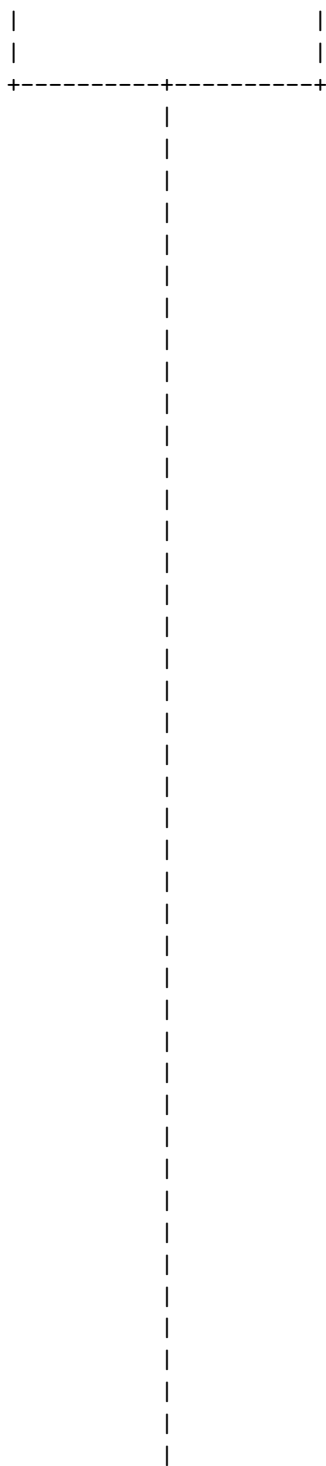




Execution Modes of Pig are the ways of running a Pig program on different environments. Pig has two main execution modes: local mode and MapReduce mode. Local mode runs on a single JVM and is used for development, experimenting and prototyping. MapReduce mode runs on a Hadoop cluster and is used for processing large-scale data. Pig also has other execution modes such as Tez mode, Spark mode and Embedded mode. Tez mode runs on a Tez engine and is used for faster and more efficient execution. Spark mode runs on a Spark engine and is used for in-memory processing. Embedded mode runs on a Java program and is used for defining custom functions.

Execution Modes of Pig

Local Mode	MapReduce Mode	Tez Mode
Runs on a single JVM	Runs on a Hadoop cluster	Runs on a Tez engine
Used for development, experimenting and prototyping	Used for processing large-scale data	Used for faster and more efficient execution



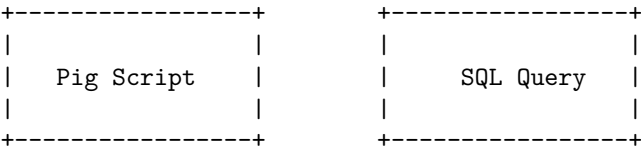


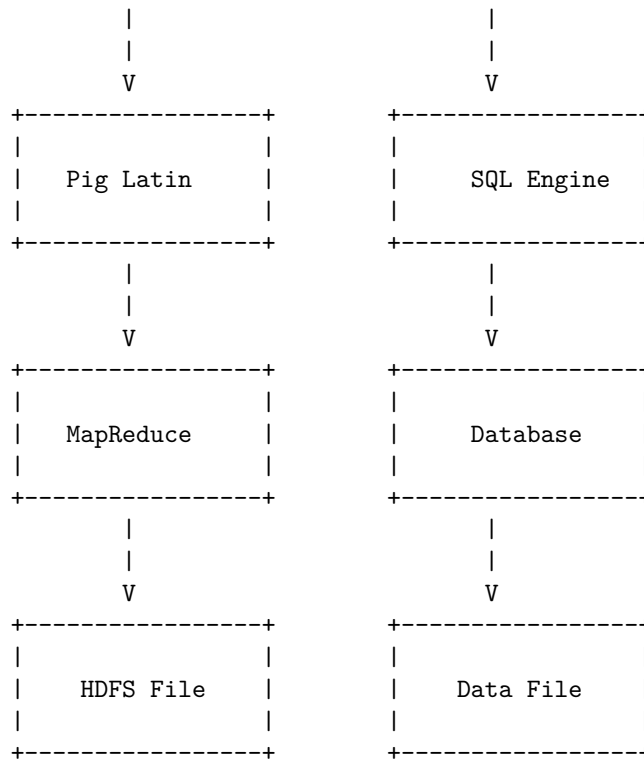
Comparison of Pig with Databases

Pig is a high-level scripting platform that runs on top of Hadoop and allows users to process and analyze large datasets using a language called Pig Latin. Pig Latin is similar to SQL, but it also supports complex data types, user-defined functions, and nested data structures. Pig can work with structured and semi-structured data, and it can perform transformations, aggregations, joins, and other operations on the data.

Databases are systems that store and manage structured data in tables, rows, and columns. Databases use SQL as the standard query language to manipulate and retrieve data. Databases can also enforce constraints, indexes, and transactions on the data. Databases are designed for fast and reliable access to data, but they may not scale well for very large datasets or complex queries.

The following diagram illustrates the basic architecture of Pig and a typical database system:





Some of the advantages of Pig over databases are:

- Pig can handle very large datasets that may not fit in a single database server.
- Pig can process unstructured and semi-structured data, such as JSON, XML, or logs, without requiring a predefined schema.
- Pig can leverage the parallelism and fault-tolerance of Hadoop to run distributed computations on the data.
- Pig can easily integrate with other tools and frameworks in the Hadoop ecosystem, such as Hive, Spark, or HBase.

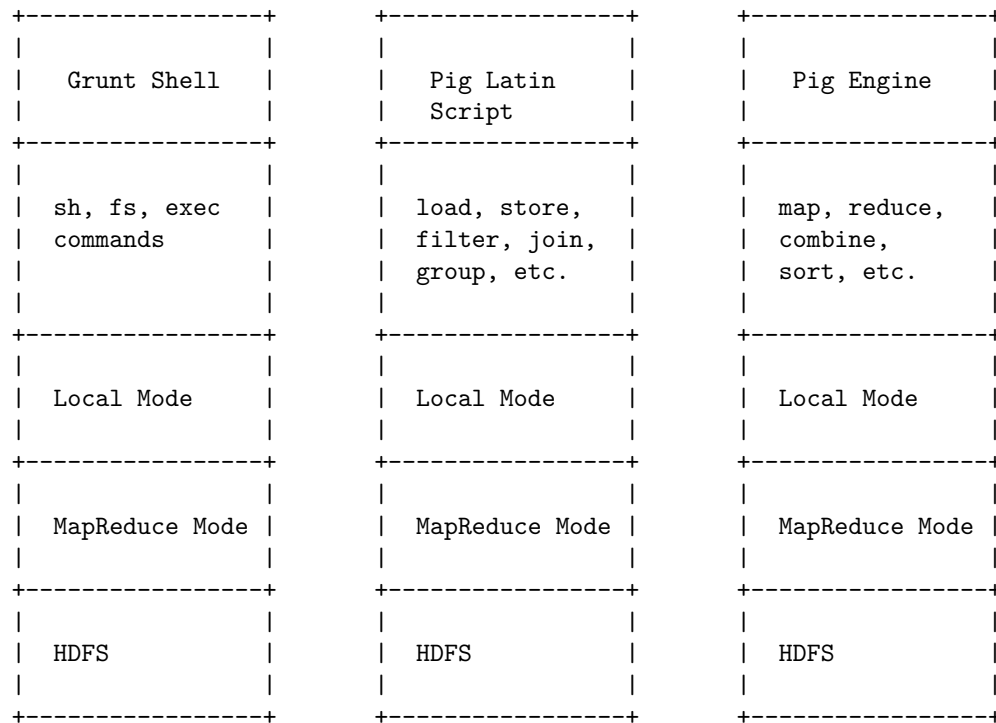
Some of the advantages of databases over Pig are:

- Databases can provide faster and more consistent performance for queries that involve simple filtering, sorting, or aggregation of data.
- Databases can support transactions, concurrency control, and data integrity features that ensure the consistency and reliability of the data.
- Databases can offer more advanced features, such as views, triggers, stored procedures, or security mechanisms, that may not be available in Pig.
- Databases can use SQL, which is a widely used and standardized query language that is easier to learn and use than Pig Latin.

Grunt is an interactive shell for Apache Pig, which is a platform for analyzing

large data sets using a high-level language called Pig Latin. Grunt can be used to write Pig Latin scripts, execute shell commands, and interact with the Hadoop Distributed File System (HDFS).

The following diagram illustrates the basic architecture of Grunt in Pig:



In this diagram, the Grunt shell is the interface for the user to interact with Pig. The user can write Pig Latin scripts in the shell, or execute them from a file using the exec command. The user can also use the sh command to invoke any shell commands from the Grunt shell, or the fs command to interact with the HDFS.

The Pig Latin script is the high-level language that describes the data analysis tasks. The script consists of a series of statements that perform operations on the data, such as loading, storing, filtering, joining, grouping, etc. The script can also use user-defined functions (UDFs) to extend the functionality of Pig.

The Pig engine is the component that translates the Pig Latin script into a series of MapReduce jobs that run on the Hadoop cluster. The Pig engine optimizes the execution plan by applying various rules and techniques, such as logical and physical optimization, combiners, secondary sort, etc. The Pig engine also handles the data types, schemas, and serialization of the data.

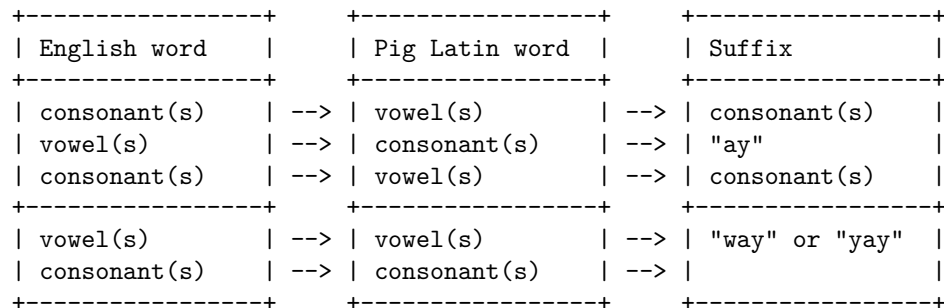
The user can run Pig in two modes: local mode and MapReduce mode. In local mode, Pig runs on a single machine without using Hadoop. In MapReduce

mode, Pig runs on a Hadoop cluster and uses the HDFS as the data source and destination. The user can switch between the modes using the -x option when launching Pig.

Pig Latin is a language game in which words in English are altered by moving the initial consonant or consonant cluster of a word to the end of the word and adding a suffix, usually “ay”. For example, “pig” becomes “igpay” and “latin” becomes “atinlay”. If a word begins with a vowel, the suffix “way” or “yay” is added. For example, “out” becomes “outway” or “outyay”.

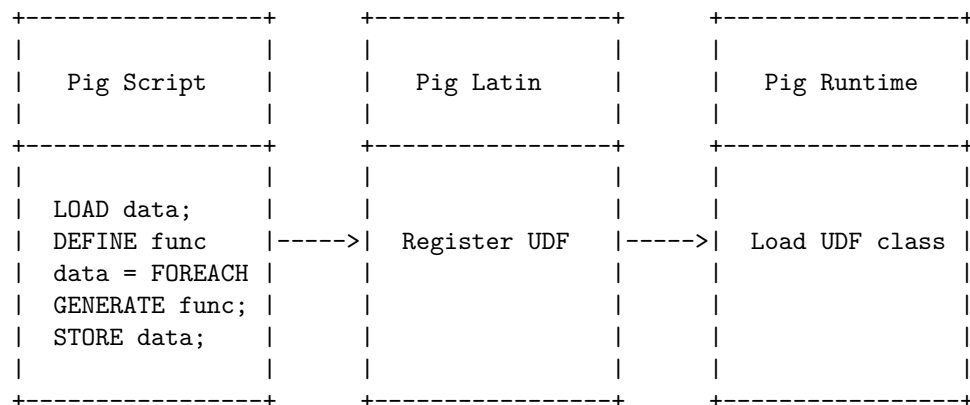
The following diagram illustrates the basic architecture of a Pig Latin word:

Pig Latin



User Defined Functions (UDFs) are custom functions that can be used to perform specific processing in Apache Pig. UDFs can be implemented in six languages: Java, Jython, Python, JavaScript, Ruby and Groovy. UDFs are different from built-in functions, which are predefined and do not need to be registered.

The following diagram illustrates the basic architecture of a User Defined Function in Pig:



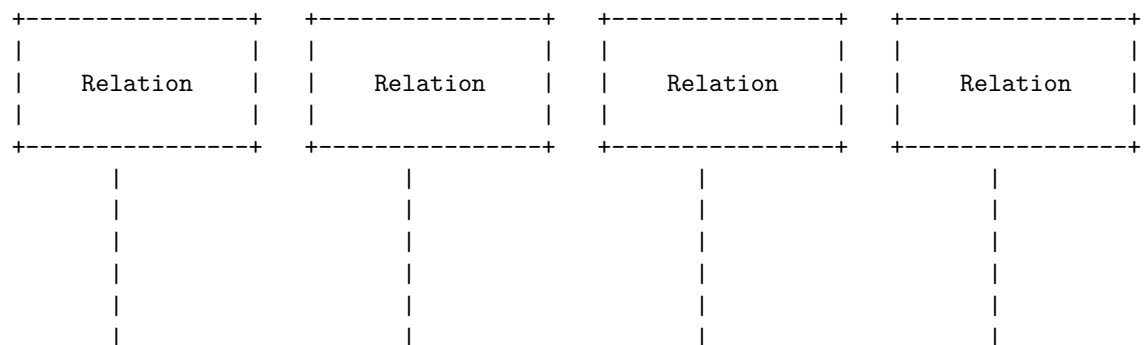
The steps involved in using a UDF are:

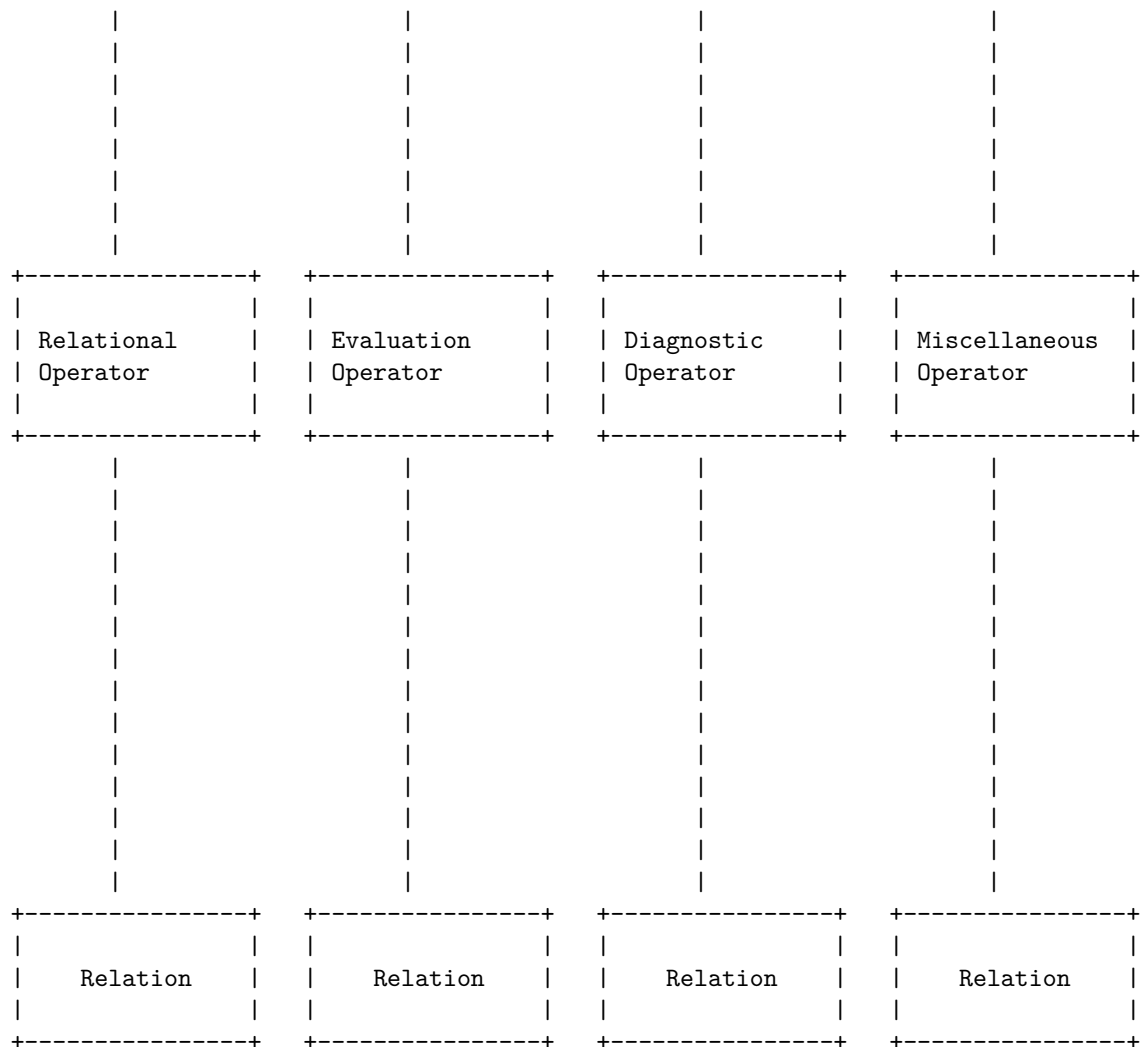
- Write the UDF code in one of the supported languages and compile it into a JAR file (for Java) or a script file (for other languages).
- Load the data to be processed using the LOAD statement in the Pig script.
- Define the UDF using the DEFINE statement and provide the name, path and arguments of the UDF.
- Register the UDF using the REGISTER statement and provide the JAR file or script file name.
- Apply the UDF to the data using the FOREACH or FILTER statement and generate the output.
- Store the output using the STORE statement in the Pig script.

Data Processing Operators are the main tools that Pig Latin provides to operate on the data. They allow you to transform the data by sorting, grouping, joining, projecting, and filtering. There are four types of Data Processing Operators in Pig:

- Relational Operators: These operators take one or more relations as input and produce another relation as output. They are used to perform common data operations such as loading, storing, filtering, grouping, joining, etc. Examples of relational operators are LOAD, STORE, FILTER, GROUP, JOIN, etc.
- Evaluation Operators: These operators are used to manipulate or generate values from the input data. They are usually embedded within relational operators. Examples of evaluation operators are arithmetic operators, comparison operators, string operators, etc.
- Diagnostic Operators: These operators are used to display information about the data or the execution of the Pig script. They are useful for debugging and testing purposes. Examples of diagnostic operators are DUMP, DESCRIBE, EXPLAIN, ILLUSTRATE, etc.
- Miscellaneous Operators: These operators are used to perform some additional tasks that are not covered by the other types of operators. Examples of miscellaneous operators are ORDER BY, LIMIT, DISTINCT, UNION, SPLIT, etc.

The following diagram illustrates the basic architecture of a Pig script using Data Processing Operators:





Hive is a data warehouse system that provides a SQL-like interface to query and analyze large-scale data stored in Hadoop. Hive architecture consists of the following components:

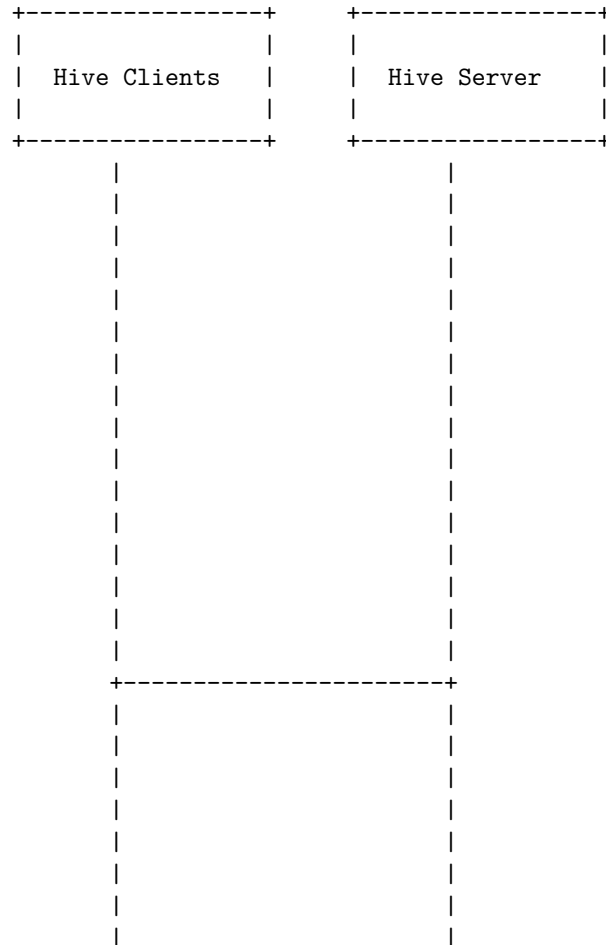
- **Hive Clients:** These are the applications that interact with Hive through various interfaces, such as JDBC, ODBC, Thrift, or command line. They can submit queries, view results, and perform other operations on Hive.
- **Hive Server:** This is the service that handles the requests from the Hive clients. It parses, optimizes, and executes the queries using the Hive execution engine. It also communicates with the Hive metastore and the Hadoop cluster.
- **Hive Metastore:** This is the component that stores the metadata of the tables, partitions, columns, and other schema information. It also main-

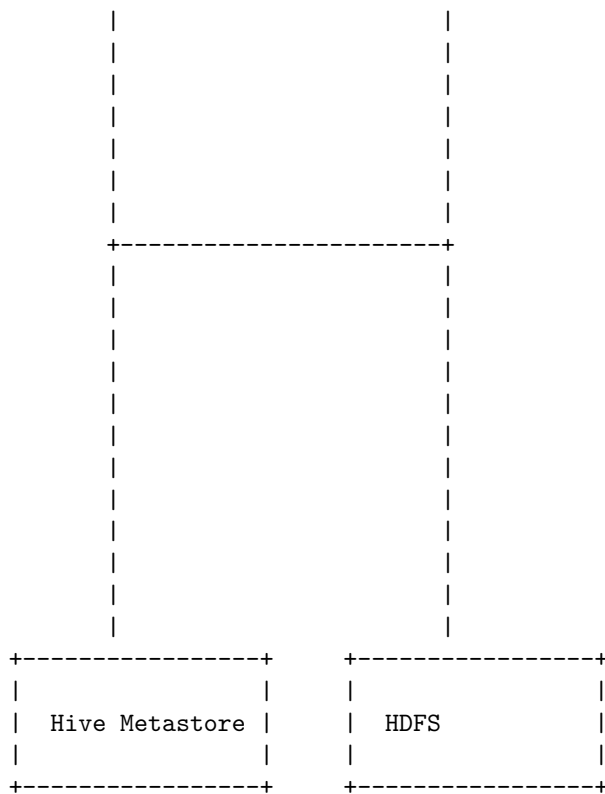
tains the statistics of the data and the location of the data files on HDFS. The Hive metastore can use different backends, such as Derby, MySQL, PostgreSQL, etc.

- **Hive Execution Engine:** This is the component that executes the queries using the MapReduce framework. It converts the SQL-like queries into a series of MapReduce jobs and submits them to the Hadoop cluster. It also performs optimizations, such as partition pruning, predicate pushdown, join reordering, etc.
- **HDFS:** This is the distributed file system that stores the actual data files of the tables and partitions. Hive supports various file formats, such as text, RCFile, ORC, Parquet, etc.

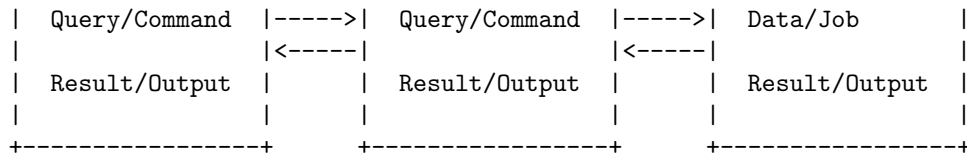
The following diagram illustrates the basic architecture of Hive:

Hive





The following is a detailed ASCII diagram for Apache Hive architecture:



The diagram shows the major components of Hive and its interactions with Hadoop. As shown in the diagram, the main components of Hive are:

- **Hive Clients:** These are the interfaces that allow users to interact with Hive. They include command-line tools, web-based tools, and APIs for different languages.
- **Hive Services:** These are the components that process the queries and commands from the clients. They include HiveServer2, which is the main service that accepts requests and creates execution plans, Hive Thrift, which is the protocol for communication between clients and services, Hive JDBC/ODBC, which are the drivers for connecting to Hive from external applications, Hive Metastore, which is the central repository of metadata about the tables, partitions, columns, etc., Hive Compiler, which is the component that parses, analyzes, and optimizes the queries, and Hive Executor, which is the component that executes the queries using the underlying processing framework.
- **Hadoop Cluster:** This is the distributed storage and computation platform that Hive relies on. It includes HDFS, which is the file system that stores the data, MapReduce, which is the original processing framework for batch processing, YARN, which is the resource manager that allocates resources for the jobs, Tez, which is an alternative processing framework for interactive and complex queries, and Spark, which is another alternative processing framework for fast and in-memory processing.

Hive is a data warehouse software that facilitates querying and managing large datasets residing in distributed storage. To install Hive on Ubuntu, you need to follow these steps:

1. Download and untar Hive from the official website or use the command `wget http://archive.apache.org/dist/hive/hive-2.1.0/apache-hive-2.1.0-bin.tar.gz` in the terminal.
2. Configure Hive environment variables by editing the `.bashrc` file in your home directory. You need to set the `$HIVE_HOME` variable to point to the Hive installation directory and add `$HIVE_HOME/bin` to the `$PATH` variable. You also need to set the `$HADOOP_HOME` variable to point to the Hadoop installation directory if you have not done so already.
3. Edit the `hive-conf.sh` file in the `$HIVE_HOME/bin` directory and add the following lines:

```
export HADOOP_HOME=/path/to/hadoop
export HIVE_CONF_DIR=$HIVE_HOME/conf
```

4. Create a `hive-site.xml` file in the `$HIVE_HOME/conf` directory and add

the following configuration properties:

```
<configuration>
  <property>
    <name>javax.jdo.option.ConnectionURL</name>
    <value>jdbc:derby:;databaseName=/path/to/metastore_db;create=true</value>
    <description>JDBC connect string for a JDBC metastore</description>
  </property>
  <property>
    <name>javax.jdo.option.ConnectionDriverName</name>
    <value>org.apache.derby.jdbc.EmbeddedDriver</value>
    <description>Driver class name for a JDBC metastore</description>
  </property>
  <property>
    <name>hive.metastore.warehouse.dir</name>
    <value>/user/hive/warehouse</value>
    <description>location of default database for the warehouse</description>
  </property>
</configuration>
```

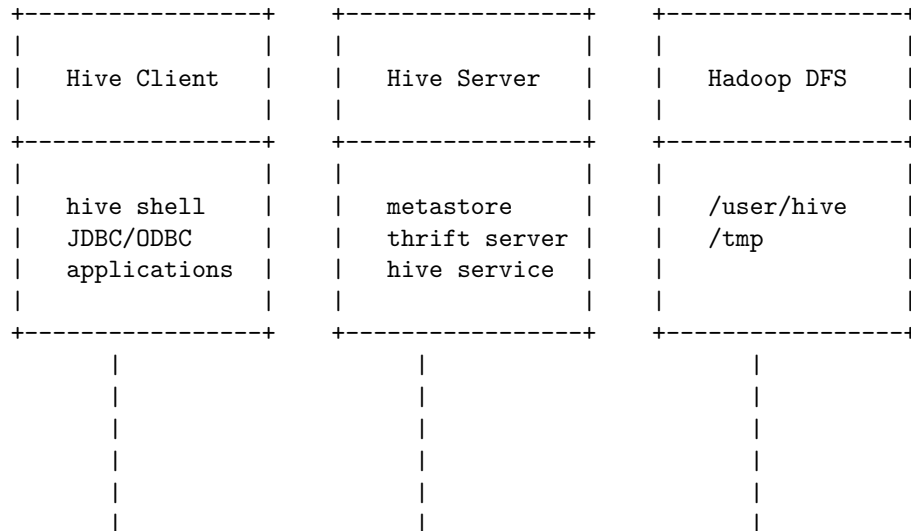
5. Start the Hive shell by running the command `hive` in the terminal. You should see a prompt like this:

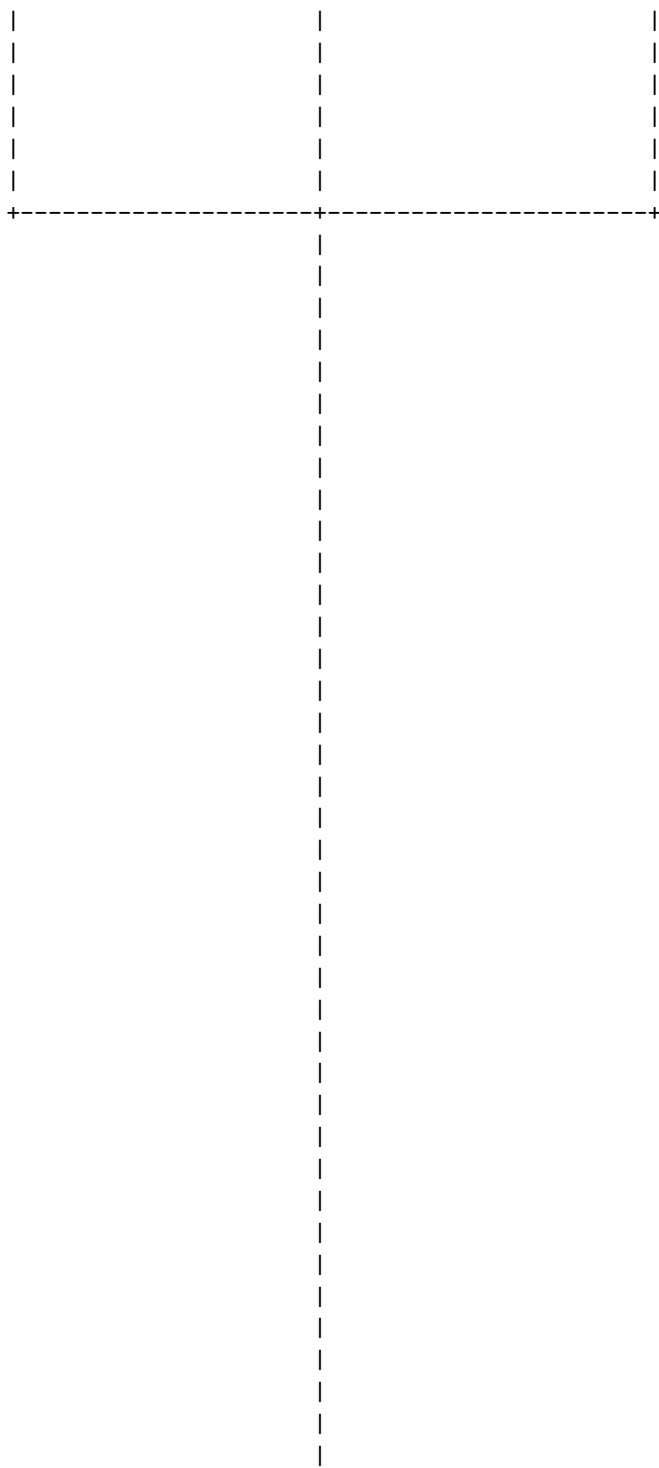
```
hive>
```

You can now run Hive queries and commands in the shell.

Installing Hive

The following diagram illustrates the basic architecture of a Hive installation:

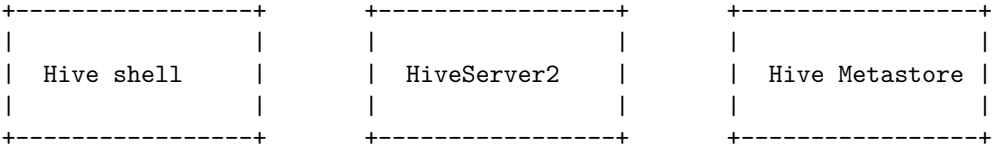


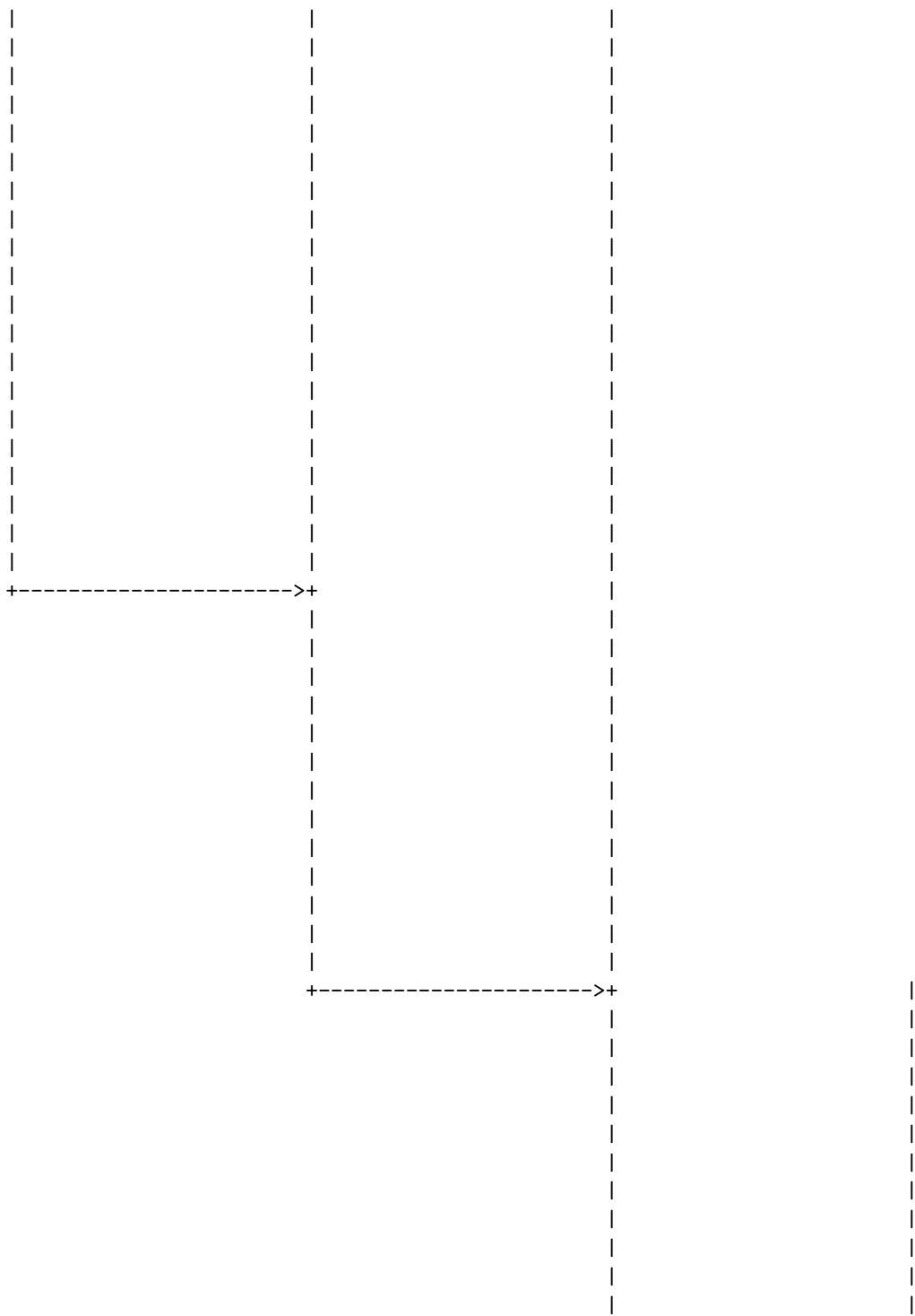


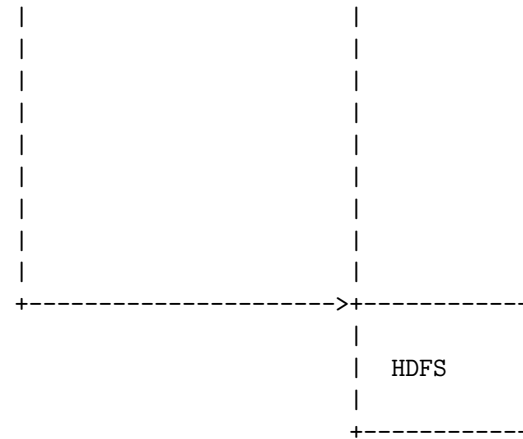


Hive shell is a command-line interface that allows users to interact with Hive and run Hive queries. It can be used in interactive or batch mode. Hive shell communicates with HiveServer2, which is a service that provides access to Hive via JDBC or ODBC drivers. HiveServer2 executes the queries on the Hadoop cluster using MapReduce, Tez, or Spark as the execution engine. Hive stores the metadata of the tables, partitions, columns, etc. in a relational database called Hive Metastore. Hive also uses HDFS or other compatible file systems to store the actual data.

The following diagram illustrates the basic architecture of Hive shell using ASCII characters:



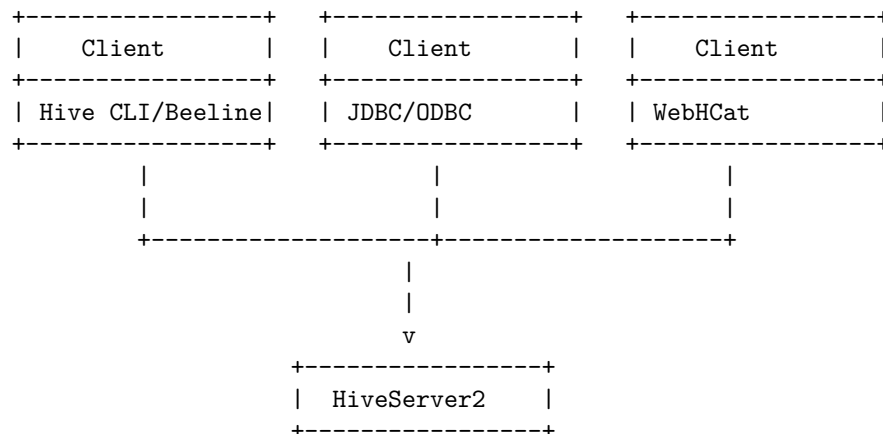


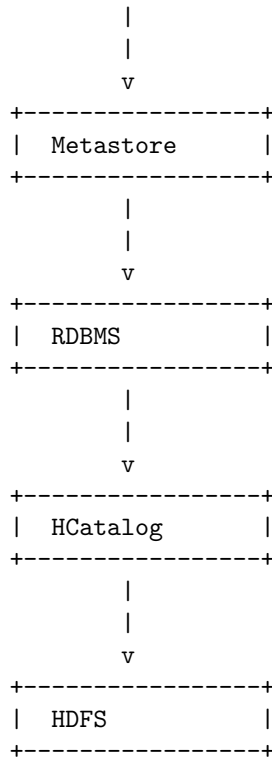


Hive services are the components that perform client interactions with Hive. They include the following:

- Hive CLI: A command-line interface that allows users to submit Hive queries and commands.
- HiveServer2: A service that provides a JDBC/ODBC server and a Thrift server for remote clients to access Hive.
- Beeline: A command shell that connects to HiveServer2 and allows users to submit queries and commands using HiveQL.
- WebHCat: A REST API service that provides metadata and job execution access to Hive, Pig, and MapReduce.
- Metastore: A service that stores the metadata of Hive tables, partitions, columns, etc. in a relational database.
- HCatalog: A service that provides a table abstraction layer for data stored in HDFS and other storage systems.

The following diagram illustrates the basic architecture of Hive services using ASCII characters:

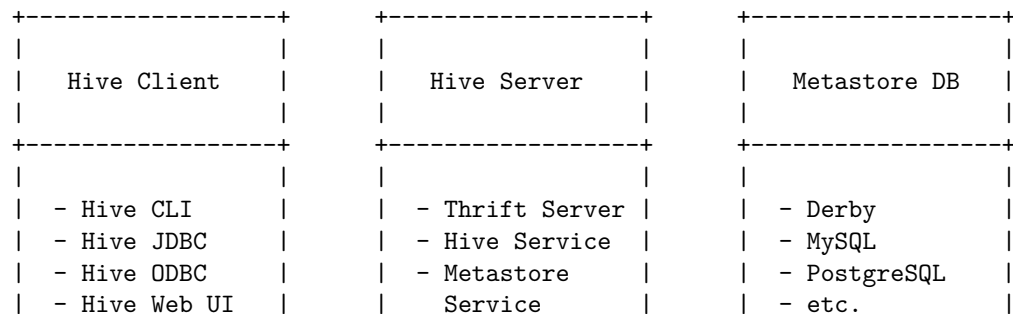


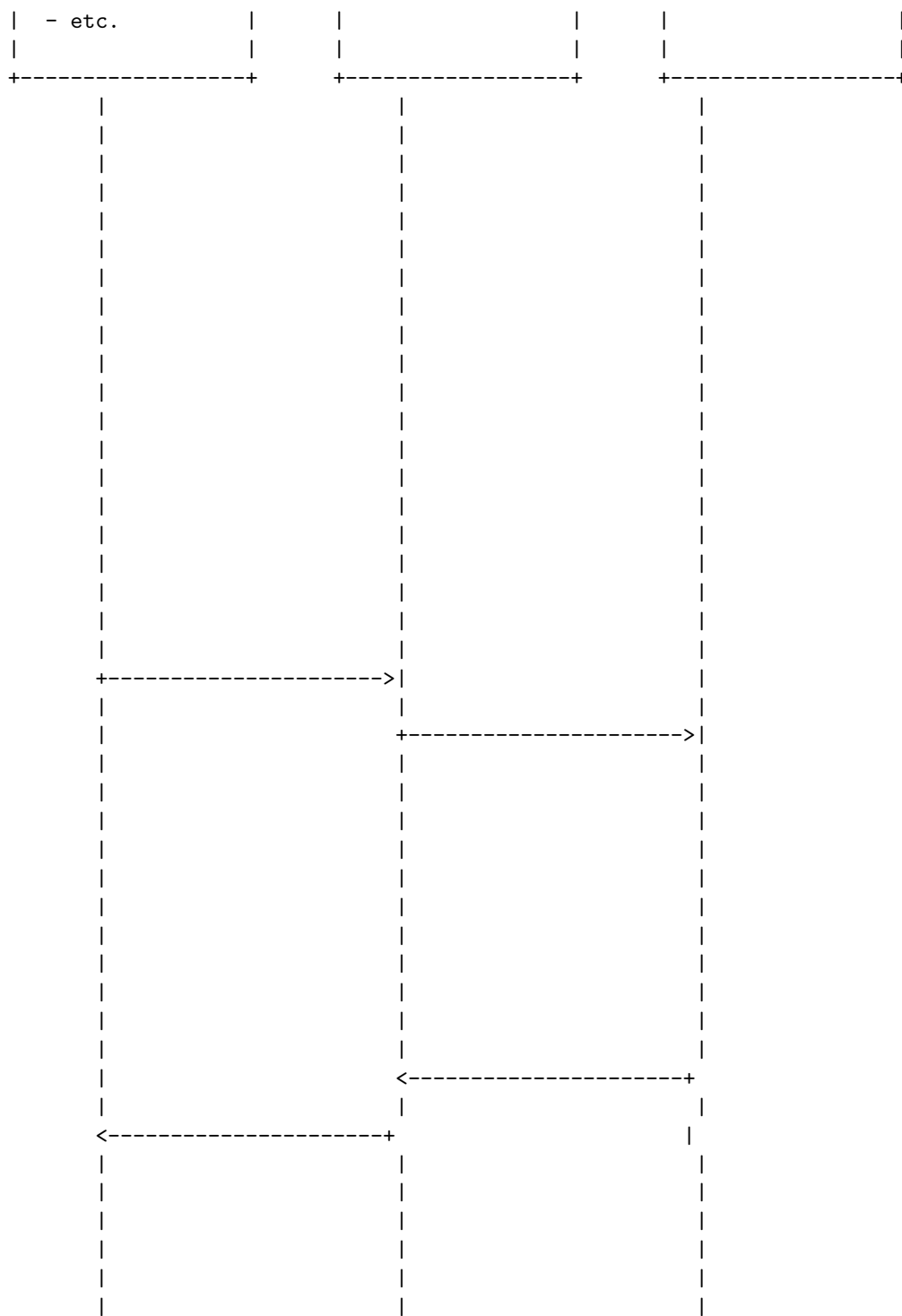


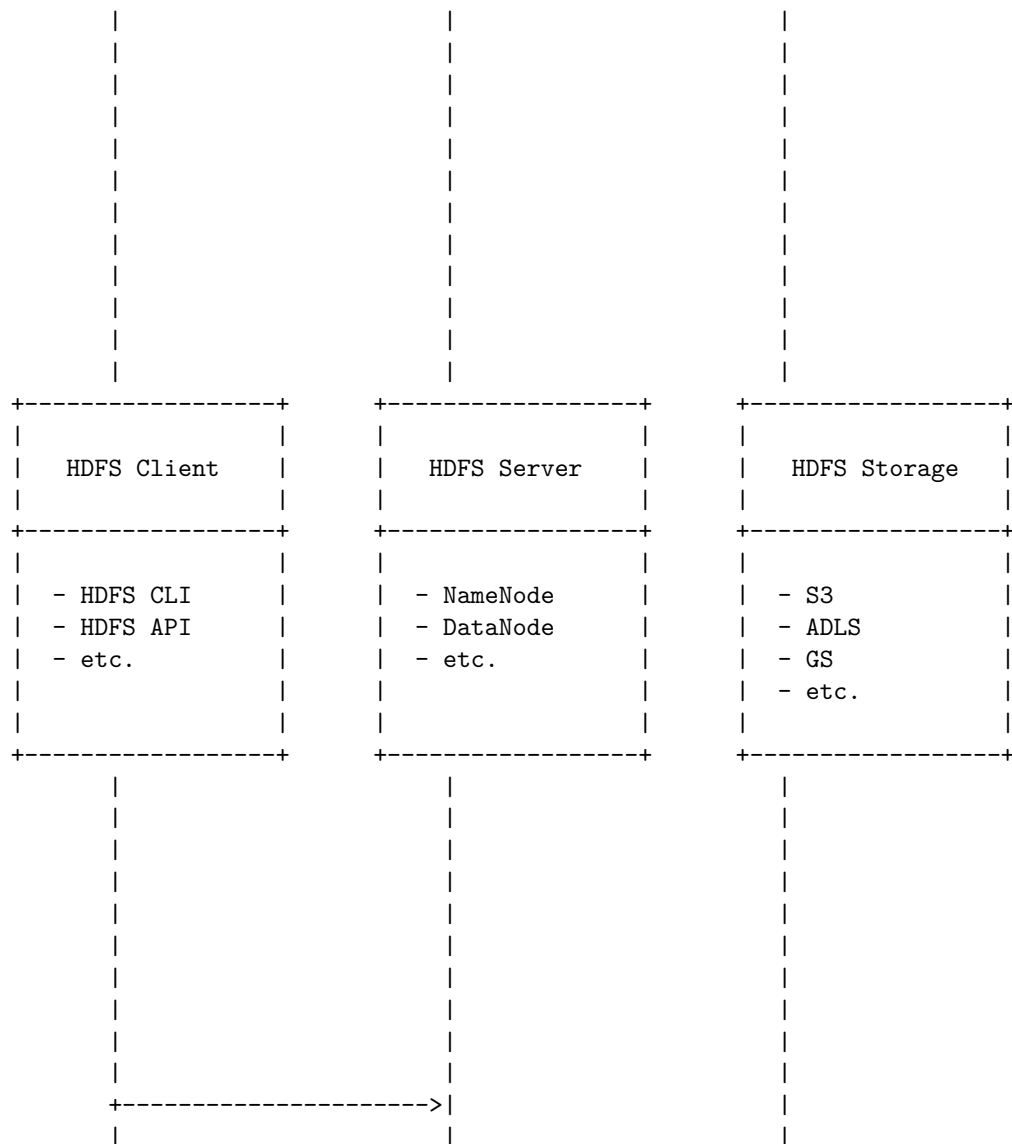
Hive metastore is a central repository of metadata that can easily be analyzed to make informed, data driven decisions, and therefore it is a critical component of many data lake architectures. It stores information about the tables, partitions, columns, data types, locations, and other properties of the data stored in Hive. It also supports storage on various file systems such as S3, ADLS, GS, etc. through HDFS. Hive metastore can be configured to use different backends such as Derby, MySQL, PostgreSQL, etc. to store the metadata.

Hive metastore

The following diagram illustrates the basic architecture of a Hive metastore:





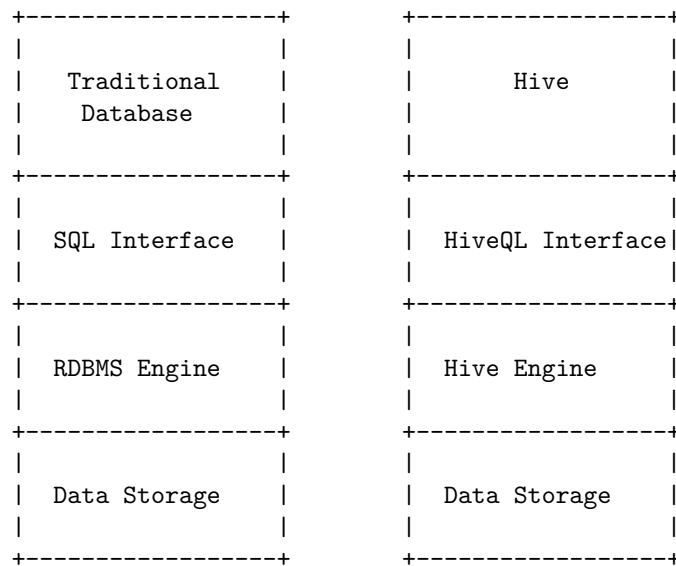


Comparison of Hive with traditional databases

Hive is a data warehouse software system that provides data query and analysis on large datasets stored in Hadoop. Hive supports a SQL-like interface called HiveQL, but it is not a full database. Hive enforces schema on read time, which means it does not verify the data when it is loaded, but only when it is queried. Hive also does not support record-level updates, insertions, and deletions. Hive is designed for batch processing and analytical queries, not for real-time transactions.

Traditional databases, such as MySQL, PostgreSQL, Oracle, and MS SQL Server, are relational database management systems (RDBMS) that store data in tables with predefined schemas. Traditional databases enforce schema on write time, which means they check the data for consistency and integrity when it is inserted or updated. Traditional databases support record-level updates, insertions, and deletions, as well as transactions and concurrency control. Traditional databases are designed for real-time operations and transactional queries, not for large-scale data analysis.

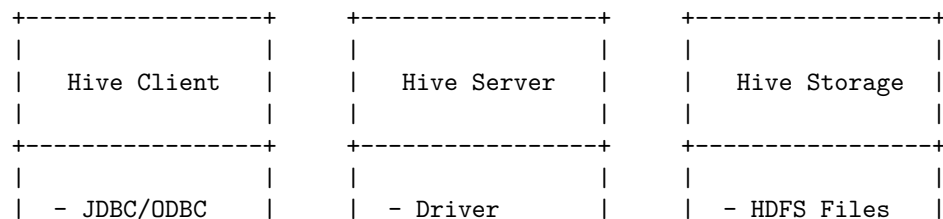
The following diagram illustrates the basic architecture of a traditional database and Hive:

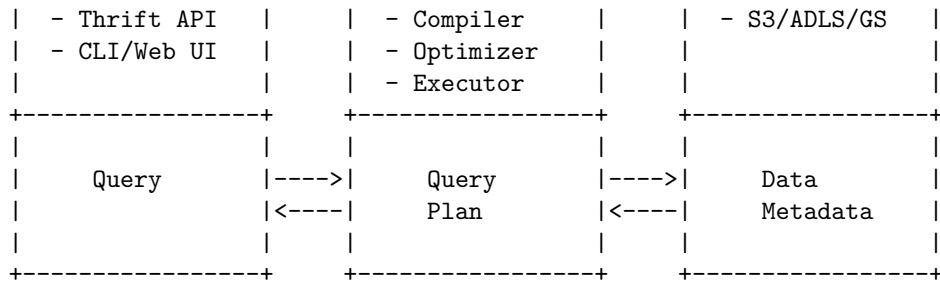


HiveQL is a query language for Apache Hive, a data warehouse system that runs on top of Hadoop. HiveQL allows users to perform SQL-like operations on structured and semi-structured data stored in Hadoop. HiveQL also supports user-defined functions, map-reduce scripts, and custom serializers and deserializers.

HiveQL Architecture

The following diagram illustrates the basic architecture of HiveQL:





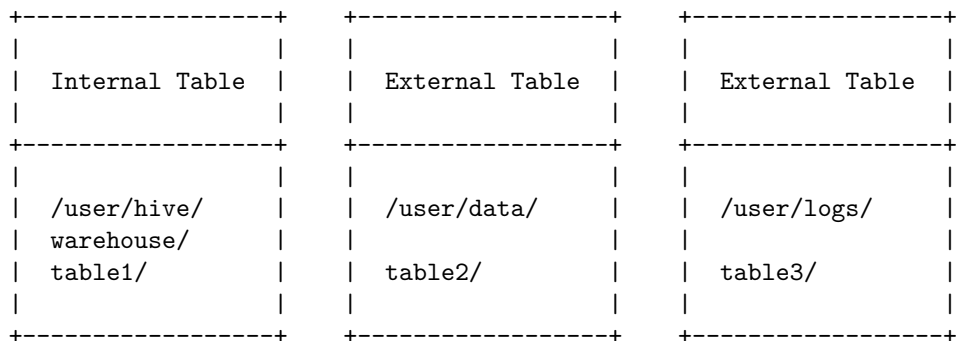
The main components of HiveQL architecture are:

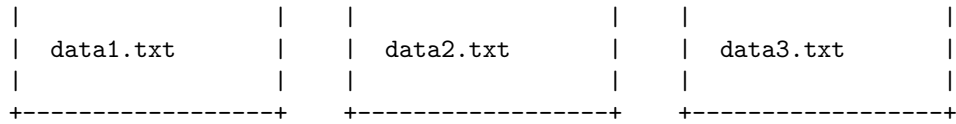
- **Hive Client:** The user interface for users to submit queries and other operations to the system. It can be a JDBC/ODBC driver, a Thrift API, a command line interface, or a web-based GUI.
- **Hive Server:** The component that receives the queries from the client, parses them, compiles them into a logical plan, optimizes the plan, and executes it on Hadoop. It also communicates with the Hive Storage to access the data and metadata.
- **Hive Storage:** The component that stores the data and metadata for the tables and partitions in the warehouse. It can be HDFS files, S3 buckets, ADLS containers, GS buckets, or other storage systems. It also provides serializers and deserializers to read and write data in different formats.

Tables in Hive are analogous to tables in a relational database management system. Each table belongs to a directory in HDFS. By default, it is /user/hive/warehouse directory. There are two types of tables that you can create with Hive: internal and external .

Internal tables store data in the Hive data warehouse. The data is managed by Hive and deleted when the table is dropped. Internal tables are also called managed tables. External tables store data outside the data warehouse. The data is not managed by Hive and remains even when the table is dropped. External tables are also called unmanaged tables.

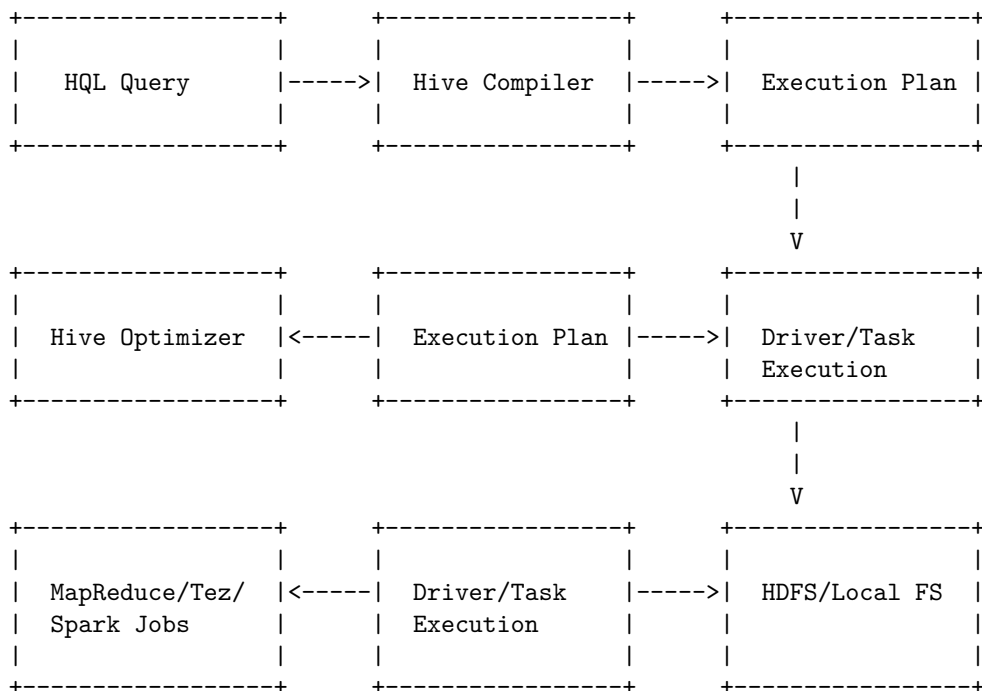
The following diagram illustrates the basic architecture of tables in Hive using ASCII characters:





Querying data in Hive is done using Hive Query Language (HQL), which is a declarative language similar to SQL. HQL allows users to process and analyze structured and semi-structured data stored in Hadoop Distributed File System (HDFS) or other data sources. HQL converts the queries into MapReduce, Tez, or Spark jobs that run on the Hadoop cluster.

The following diagram illustrates the basic architecture of a Hive query:



The basic way to query data in Hive is using SELECT statement, which has the following syntax:

```
SELECT [ALL | DISTINCT] select_expr, select_expr, ...
FROM table_reference
[WHERE where_condition]
[GROUP BY col_list]
[HAVING having_condition]
[ORDER BY col_list]
[LIMIT number]
[CLUSTER BY col_list | [DISTRIBUTE BY col_list] [SORT BY col_list]]
```

Some examples of Hive queries are:

- Select all records from a table:

```
SELECT * FROM table_name;
```

- Select specific columns from a table:

```
SELECT col1, col2, col3 FROM table_name;
```

- Select records with a filter condition:

```
SELECT * FROM table_name WHERE col1 = 'value';
```

- Select records with aggregation and grouping:

```
SELECT col1, COUNT(*) AS count FROM table_name GROUP BY col1;
```

- Select records with sorting and limiting:

```
SELECT * FROM table_name ORDER BY col1 DESC LIMIT 10;
```

- Select records with join operation:

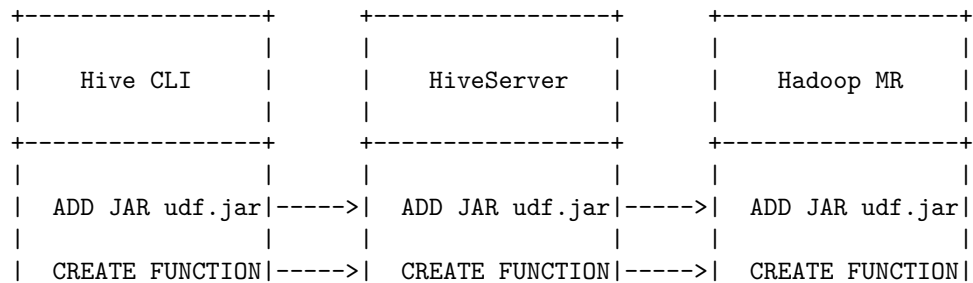
```
SELECT t1.col1, t2.col2 FROM table1 t1 JOIN table2 t2 ON t1.col3 = t2.col3;
```

User defined functions (UDFs) in Hive are custom functions that can be developed in Java, integrated with Hive, and built on top of a Hadoop cluster to allow for efficient and complex computation that would not otherwise be possible with simple SQL. They can be useful and very powerful, and yet online documentation is pretty weak.

There are three types of UDFs in Hive:

- UDF: This type of UDF takes one or more columns as input and returns a single value as output. For example, a UDF that converts a string to uppercase.
- UDAF: This type of UDF takes multiple rows as input and returns a single value as output. For example, a UDAF that calculates the average of a column.
- UDTF: This type of UDF takes one or more columns as input and returns multiple rows as output. For example, a UDTF that splits a string into words.

The following diagram illustrates the basic architecture of a UDF in Hive using ASCII art:



my_udf AS	my_udf AS	my_udf AS
'com.example.	'com.example.	'com.example.
MyUDF'	MyUDF'	MyUDF'
SELECT my_udf(	SELECT my_udf(	SELECT my_udf(
col) FROM table	col) FROM table	col) FROM table

The steps involved in creating and using a UDF in Hive are :

- Write a Java class that extends the UDF, UDAF or UDTF abstract class and implements the evaluate method with the desired logic.
- Compile the Java class and package it into a JAR file.
- Copy the JAR file to a location accessible by Hive, such as HDFS or a local directory.
- Use the ADD JAR command on the Hive CLI or HiveServer to register the JAR file with Hive.
- Use the CREATE FUNCTION command to create a function name and associate it with the fully qualified class name of the UDF.
- Use the function name in a SELECT statement to apply the UDF to the input columns.

Sorting and aggregating in Hive can be achieved by using different clauses and functions, such as ORDER BY, SORT BY, DISTRIBUTE BY, GROUP BY, and aggregate functions. However, each of these clauses and functions has different effects on the data and the execution plan. The following diagram illustrates the basic architecture of a sorting and aggregating query in Hive using MapReduce:

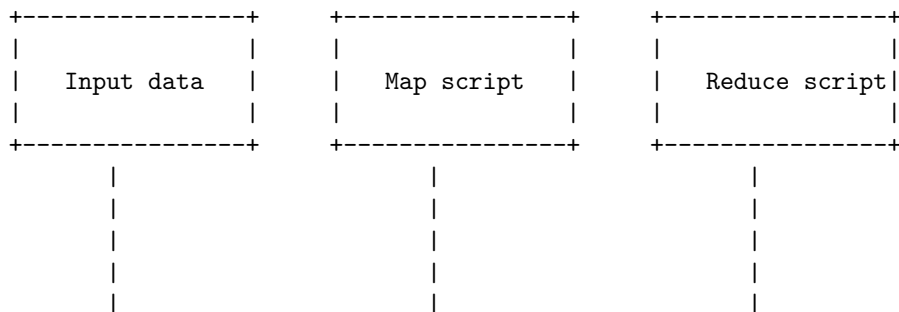
Input Data	Map Phase	Reduce Phase	Output Data
key1, value1	key1, value1	key1, value1	key1, value1
key2, value2	key2, value2	key2, value2	key2, value2
key3, value3	key3, value3	key3, value3	key3, value3

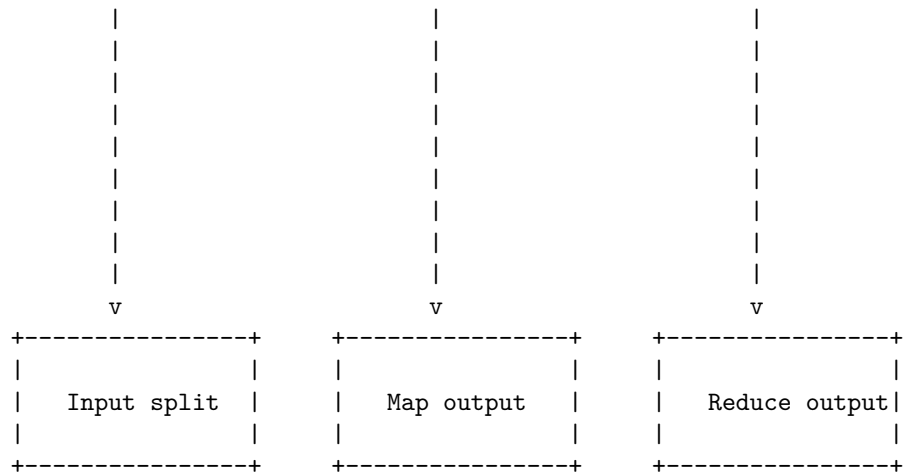
key4, value4	key4, value4	key4, value4	key4, value4
+-----+	+-----+	+-----+	+-----+
key5, value5	key5, value5	key5, value5	key5, value5
+-----+	+-----+	+-----+	+-----+
key6, value6	key6, value6	key6, value6	key6, value6
+-----+	+-----+	+-----+	+-----+
key7, value7	key7, value7	key7, value7	key7, value7
+-----+	+-----+	+-----+	+-----+
key8, value8	key8, value8	key8, value8	key8, value8
+-----+	+-----+	+-----+	+-----+
key9, value9	key9, value9	key9, value9	key9, value9
+-----+	+-----+	+-----+	+-----+
key10, value10	key10,		

Map Reduce scripts in Hive are a way to use custom mappers and reducers in the data stream by using the TRANSFORM clause in the Hive language. The TRANSFORM clause allows the user to specify an executable script that can process the input data and output the transformed data. The script can be written in any language that can read from standard input and write to standard output, such as Python, Ruby, Perl, etc.

The basic architecture of a Map Reduce script in Hive is as follows:

Map Reduce scripts in Hive



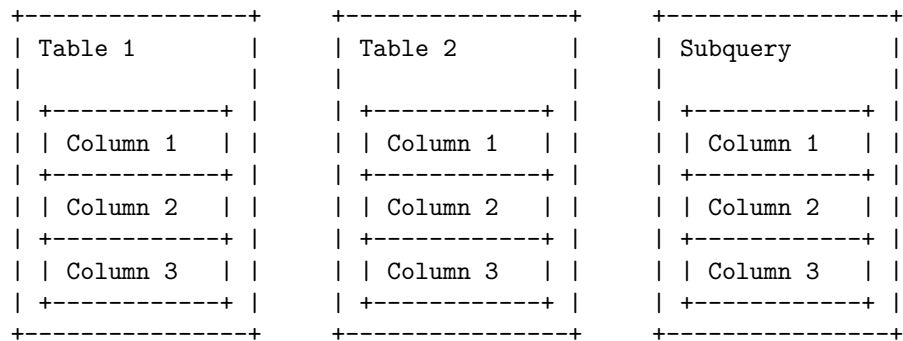


The input data is split into multiple chunks and fed into the map script, which performs some transformation on each chunk and outputs the intermediate results. The map output is then shuffled and sorted by key and fed into the reduce script, which performs some aggregation on the values corresponding to each key and outputs the final results. The reduce output is then stored in the output location specified by the user.

Joins and subqueries in Hive are used to combine data from different tables or sources based on some common criteria. Joins can be performed on two or more tables using different types of join conditions, such as inner join, left outer join, right outer join, full outer join, cross join, etc. Subqueries are queries that are nested inside another query, usually in the FROM clause. Subqueries can also use UNION to combine the results of multiple queries. Subqueries must have a name and unique column names.

The following diagram illustrates the basic syntax of joins and subqueries in Hive using ASCII characters:

Joins and subqueries in Hive



```

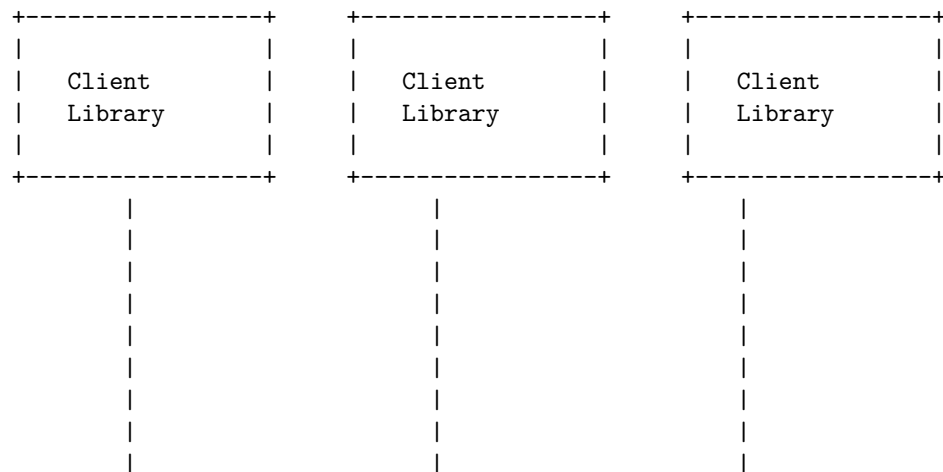
SELECT ... FROM Table 1 JOIN Table 2 ON join_condition
SELECT ... FROM Table 1 LEFT OUTER JOIN Table 2 ON join_condition
SELECT ... FROM Table 1 RIGHT OUTER JOIN Table 2 ON join_condition
SELECT ... FROM Table 1 FULL OUTER JOIN Table 2 ON join_condition
SELECT ... FROM Table 1 CROSS JOIN Table 2
SELECT ... FROM (subquery) AS name
SELECT ... FROM (subquery1) AS name1 UNION (subquery2) AS name2

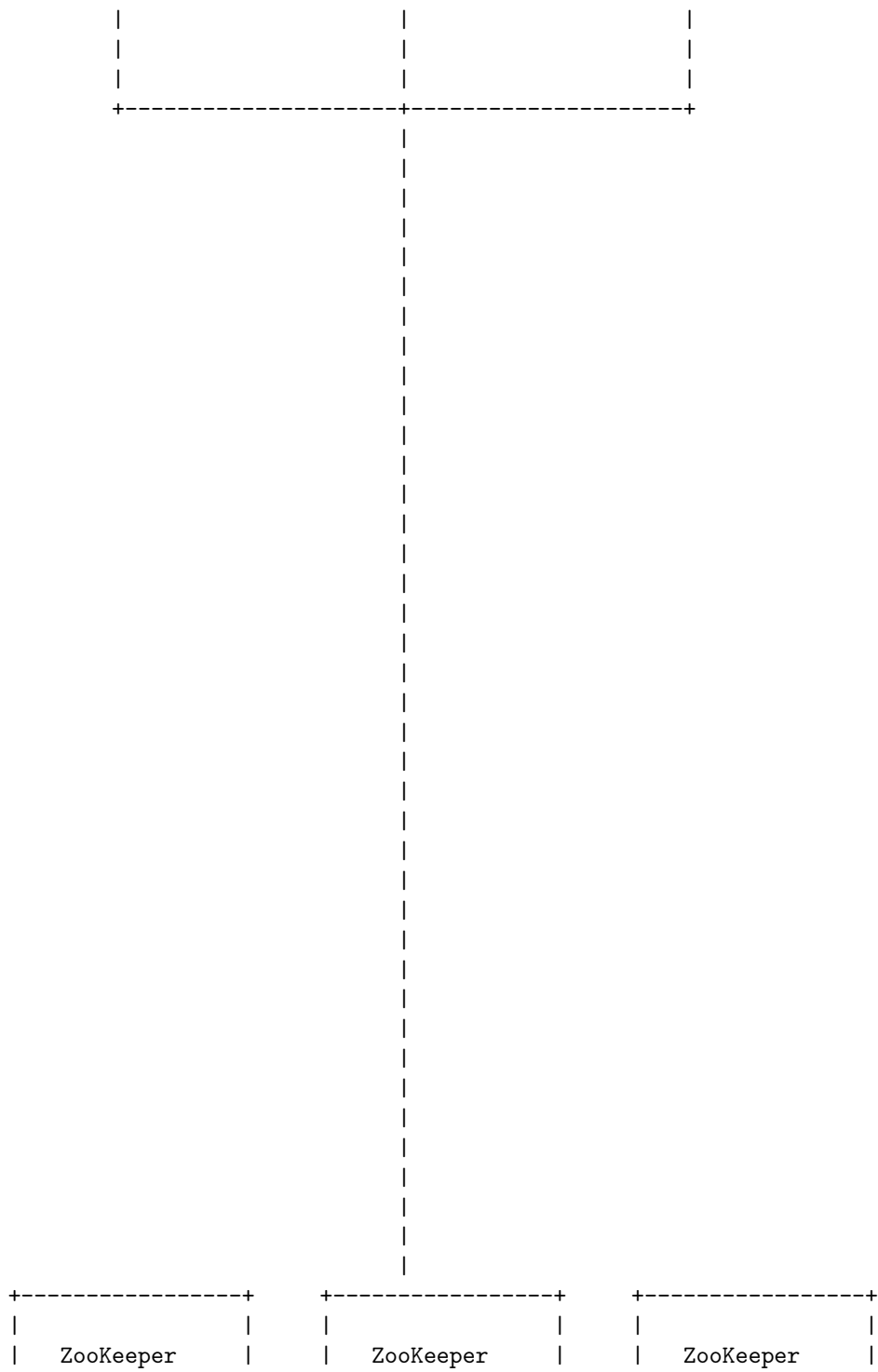
```

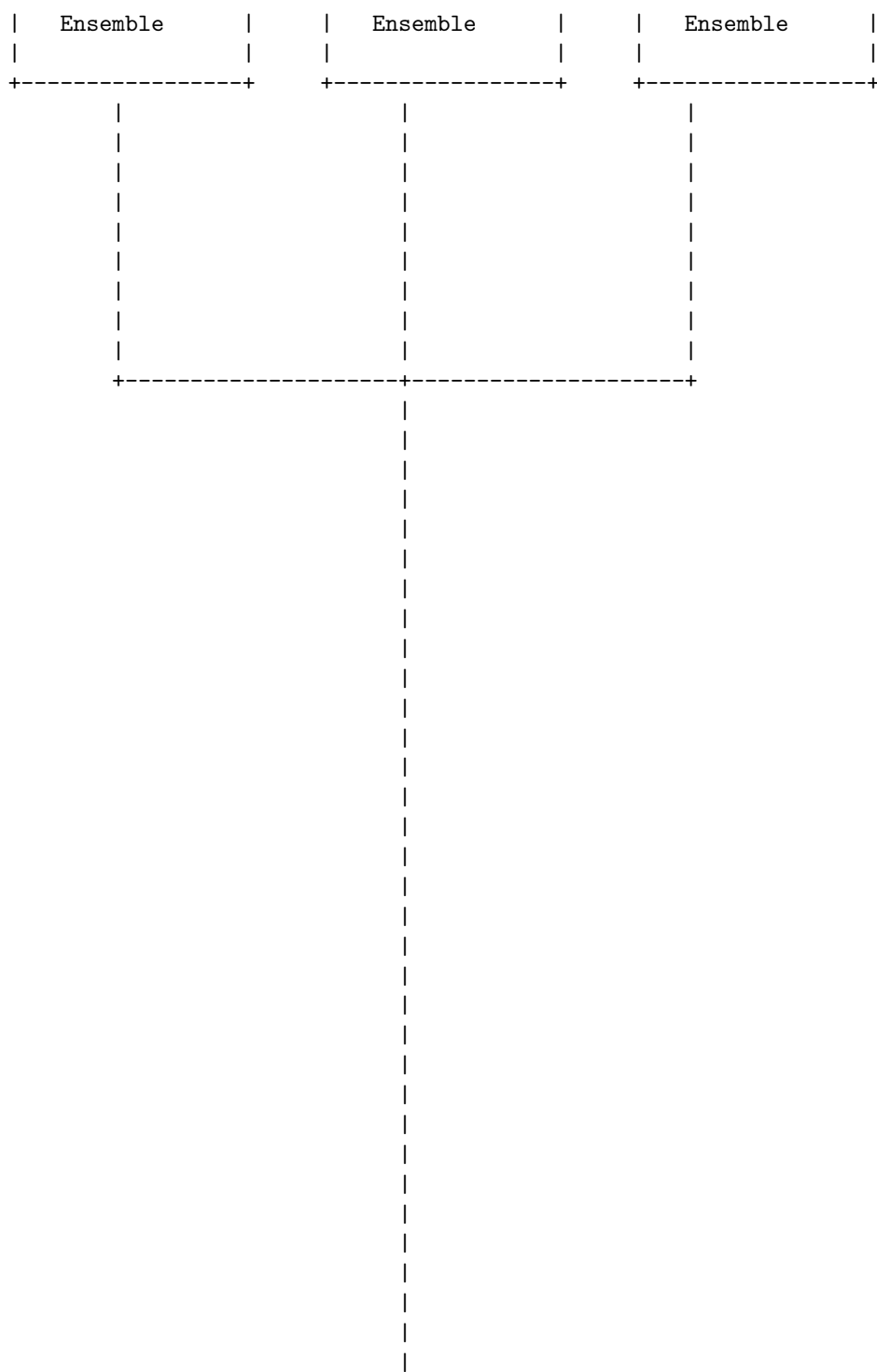
HBase is a column-oriented data storage system that runs on top of HDFS and provides low-latency random access to large amounts of data. HBase has three main components: the client library, the master server, and the region servers. The client library provides the API for interacting with HBase. The master server manages the cluster metadata, such as the table schema, the region assignments, and the load balancing. The region servers host the regions, which are the horizontal partitions of a table. Each region server can serve multiple regions, and each region can store multiple column families. A column family is a logical grouping of columns that share the same compression, encoding, and storage options. Each column family consists of one or more columns, which are identified by a qualifier. Each column can store multiple versions of a value, which are distinguished by a timestamp. A row in HBase is identified by a unique row key, and it can have any number of columns from any column family. HBase stores the data in HDFS as files called HFiles, which are sorted by row key and column. HBase also uses a write-ahead log (WAL) to ensure durability of writes. The WAL records all the changes made to the regions in a region server, and it is also stored in HDFS. HBase also uses ZooKeeper, a distributed coordination service, to maintain the cluster state and handle failover.

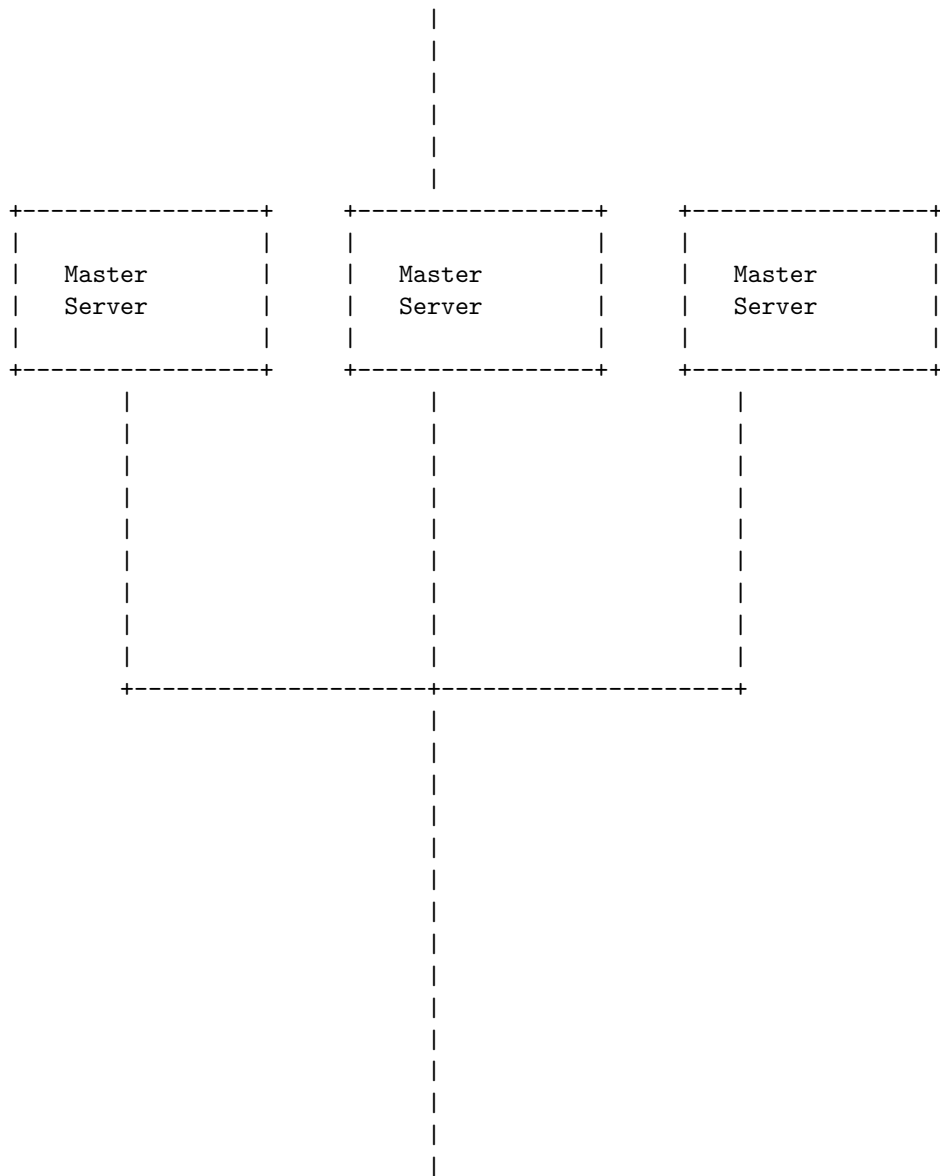
The following diagram illustrates the basic architecture of HBase:

HBase









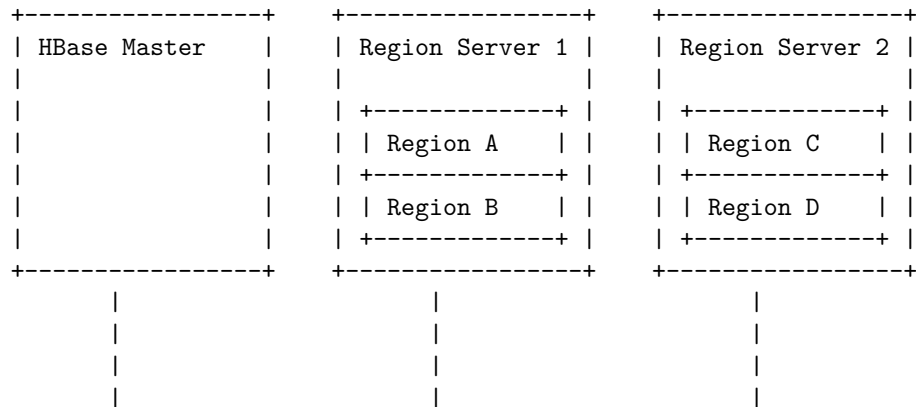
HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS). It is based on the Google Bigtable data model and provides fast and random access to large amounts of structured data. HBase is a part of the Hadoop ecosystem and can integrate with other Hadoop components such as MapReduce, Spark, Hive, and Pig.

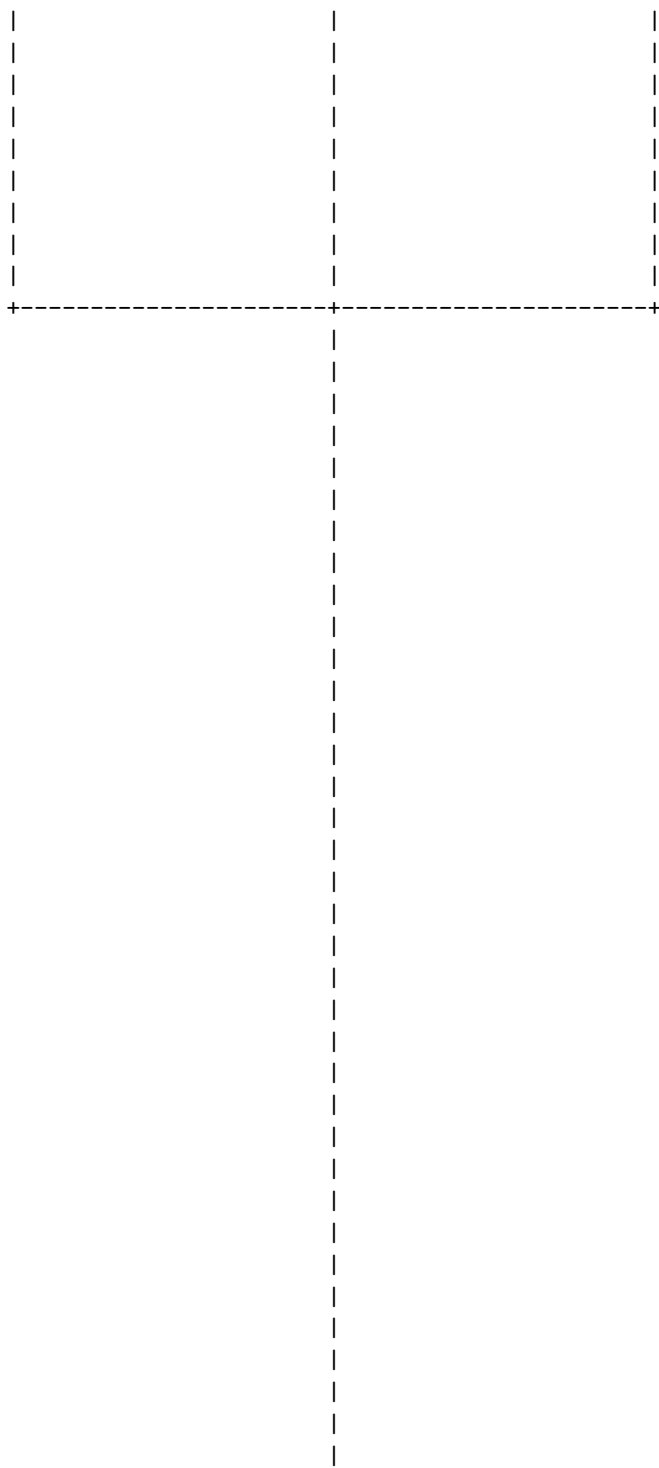
HBase concepts

Some of the core concepts of HBase are:

- **Table:** A table is a collection of rows that are organized into columns. Each table has a name and can have one or more column families.
- **Row:** A row is a unit of data that is identified by a unique row key. Rows are sorted lexicographically by their row keys. A row can have multiple versions, which are distinguished by timestamps.
- **Column family:** A column family is a group of columns that share a common prefix and have the same configuration and storage properties. A column family is stored as a separate file on HDFS and can have one or more columns.
- **Column qualifier:** A column qualifier is the suffix of a column name that distinguishes it from other columns in the same column family. A column qualifier can be any arbitrary byte array.
- **Cell:** A cell is the intersection of a row and a column. It stores a single value and a timestamp. A cell can have multiple versions, which are ordered by their timestamps in descending order.
- **Region:** A region is a contiguous range of rows that are stored together on a region server. A region is the basic unit of data distribution and load balancing in HBase. A region can be split into smaller regions when it grows too large.
- **Region server:** A region server is a process that runs on a Hadoop node and serves one or more regions. A region server is responsible for handling read and write requests, performing compactions, and communicating with the HBase master.
- **HBase master:** The HBase master is a process that runs on a Hadoop node and coordinates the cluster operations. The HBase master assigns regions to region servers, monitors their health and load, handles region server failures, and performs administrative tasks such as creating and deleting tables.

The following diagram illustrates the basic architecture of HBase using ASCII art:





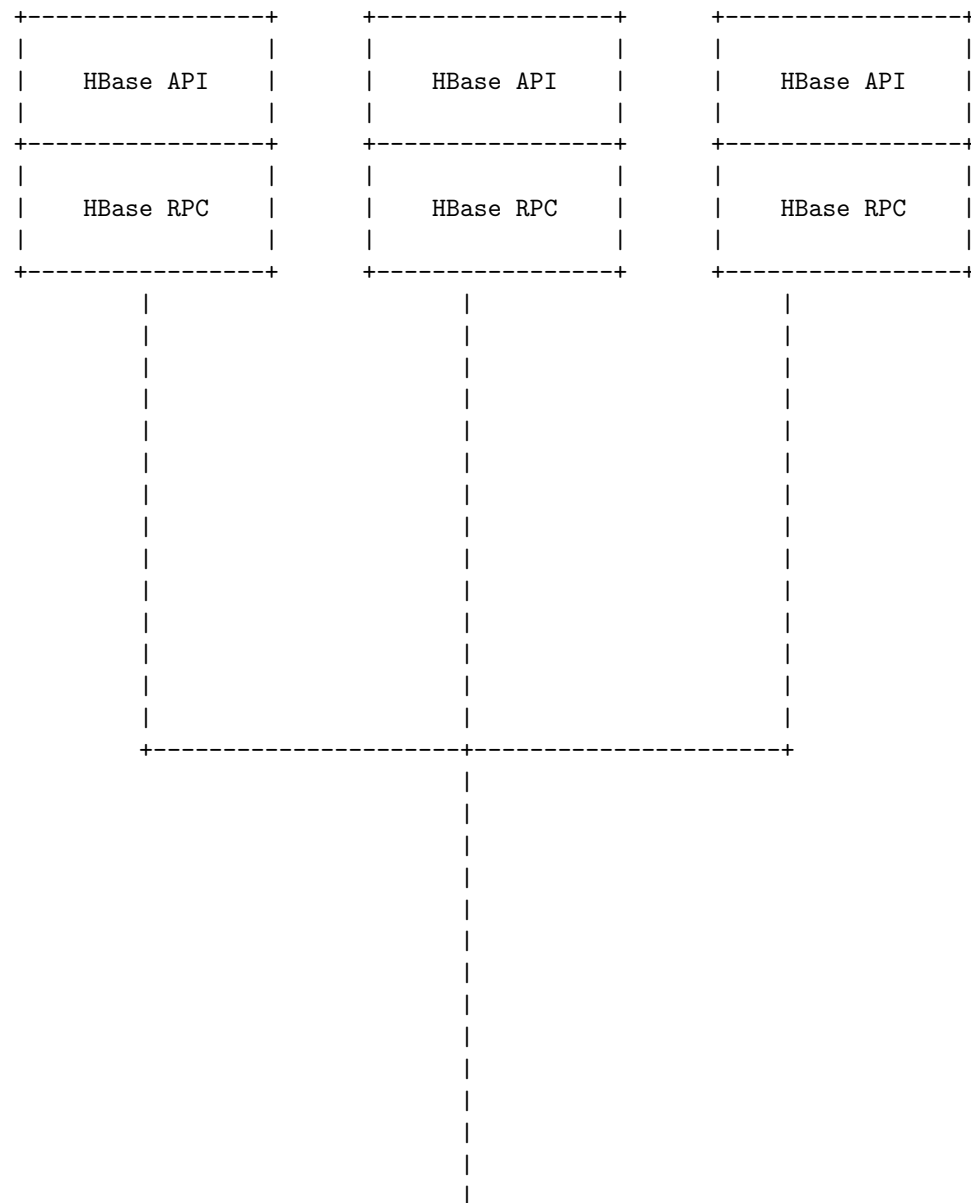
|

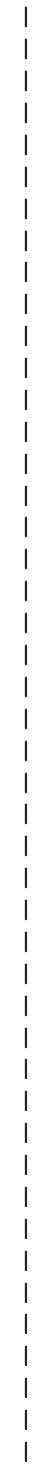
HBase clients are the applications that use the HBase API to interact with the HBase cluster. They can perform operations such as creating, deleting, updat-

ing, and scanning tables, as well as reading and writing data. HBase clients communicate with the HBase master and region servers using RPC (remote procedure call) mechanism.

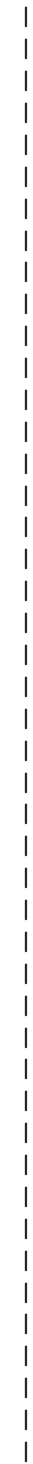
HBase clients

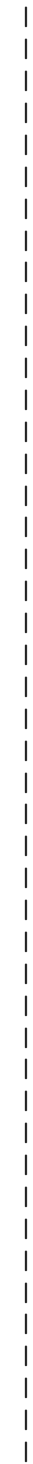
The following is a simplified ASCII diagram of the HBase clients architecture:

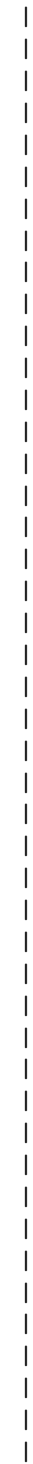














HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System (HDFS). HBase provides a fault-tolerant way of storing sparse data sets, which are common in many big data use cases. It is well suited for real-time data processing or random read/write access to large volumes of data.

An example of HBase is to store diagnostic logs from servers in your environment, where each row is a log record, and each column is an attribute of the log record, such as timestamp, server name, message, etc. HBase can also store multiple versions of the same column, which can be useful for tracking changes over time.

HBase example

The following diagram illustrates the basic architecture of a HBase table, using the server log example:

+-----+-----+-----+-----+

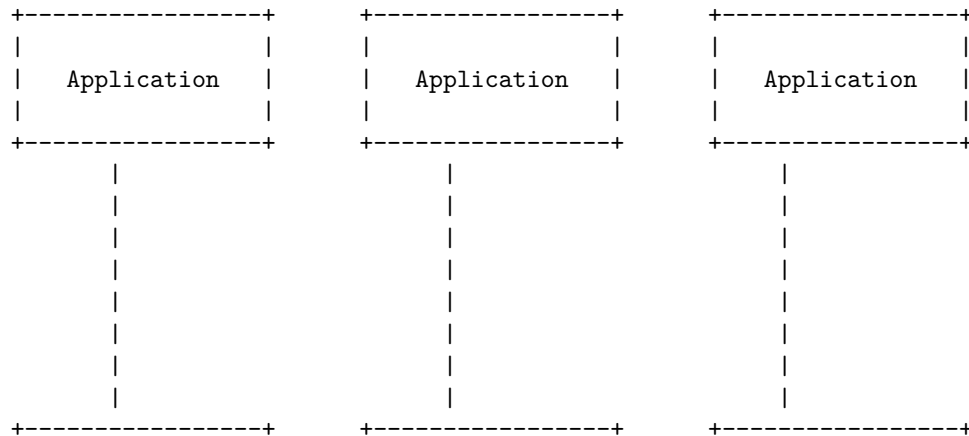
Row Key	Column Family 1	Column Family 2	Column Family 3
row1	timestamp:1	server:1	message:1
	timestamp:2	server:2	message:2
	timestamp:3	server:3	message:3
row2	timestamp:4	server:4	message:4
	timestamp:5	server:5	message:5
row3	timestamp:6	server:6	message:6

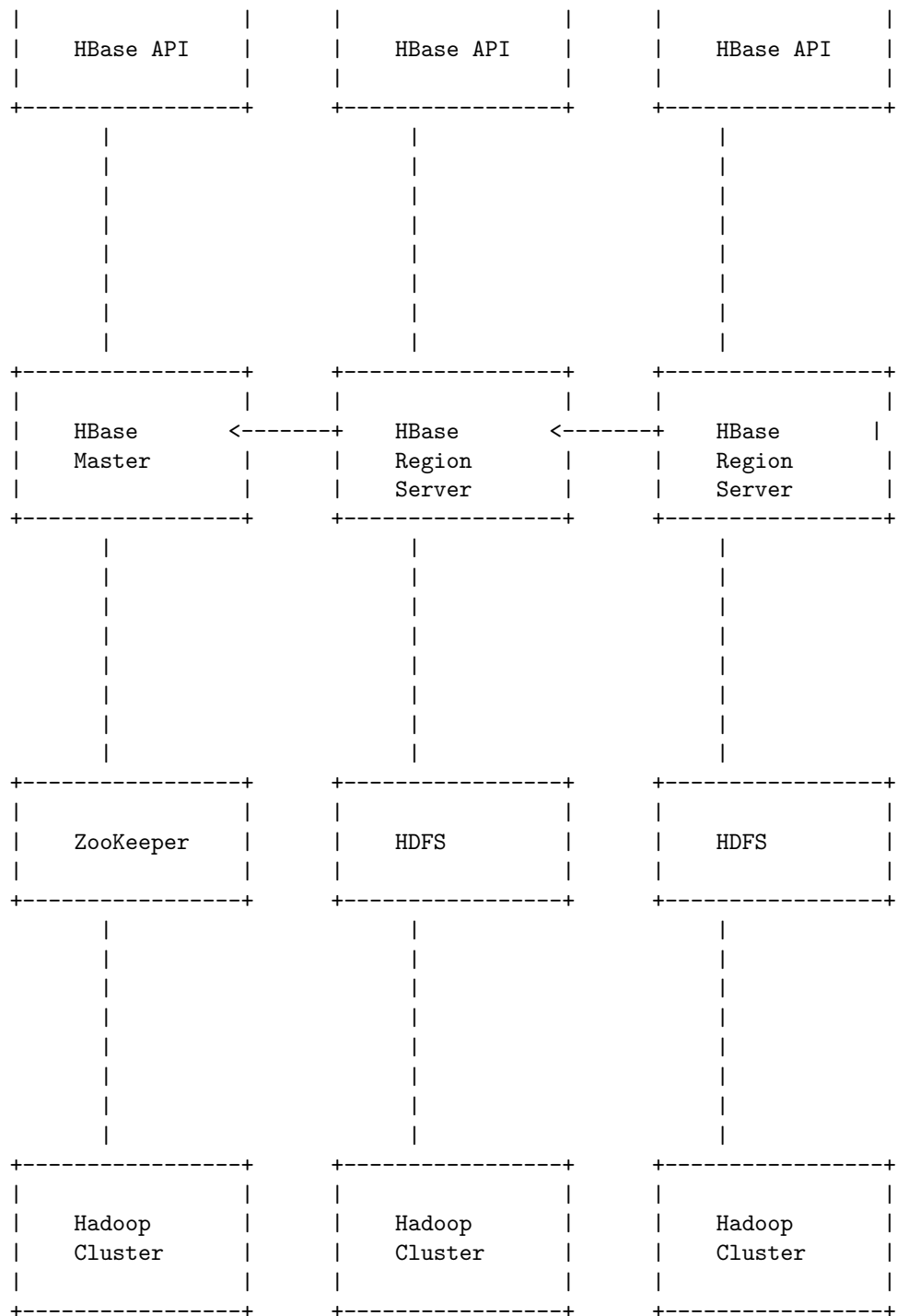
Each row has a unique row key, which is used to identify and locate the row in the HBase cluster. Each row can have one or more column families, which are groups of columns that share some common characteristics, such as compression, encoding, or versioning. Each column family can have one or more columns, which are identified by a qualifier, such as timestamp, server, or message. Each column can have one or more values, which are stored as byte arrays and can be of any data type. Each value also has a timestamp, which is used to order the values within a column.

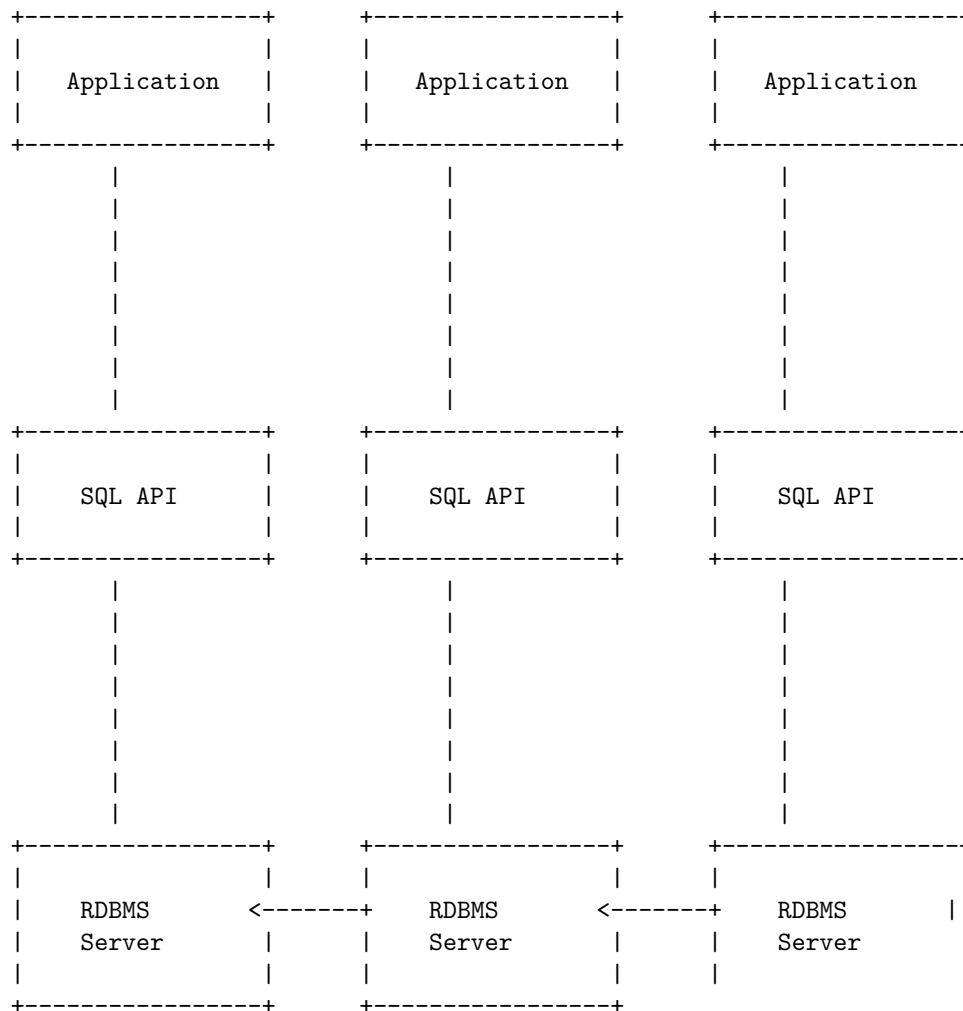
HBase tables are distributed and replicated across multiple nodes in the Hadoop cluster, which provides high availability and scalability. HBase also supports various operations on the tables, such as create, drop, alter, scan, get, put, delete, etc. HBase also provides a shell command interface, a Java API, and a REST API for interacting with the tables.

HBase vs RDBMS

HBase and RDBMS are both column-oriented database management systems, but they differ in several ways. The following diagram illustrates the basic architecture of each system using ASCII art:







HBase is a distributed, scalable, and column-oriented database that runs on top of the Hadoop Distributed File System (HDFS). It is designed to handle large amounts of data and provide fast and random access to them. HBase can also integrate with other Hadoop components, such as MapReduce, Hive, and Pig, to perform various data processing and analysis tasks.

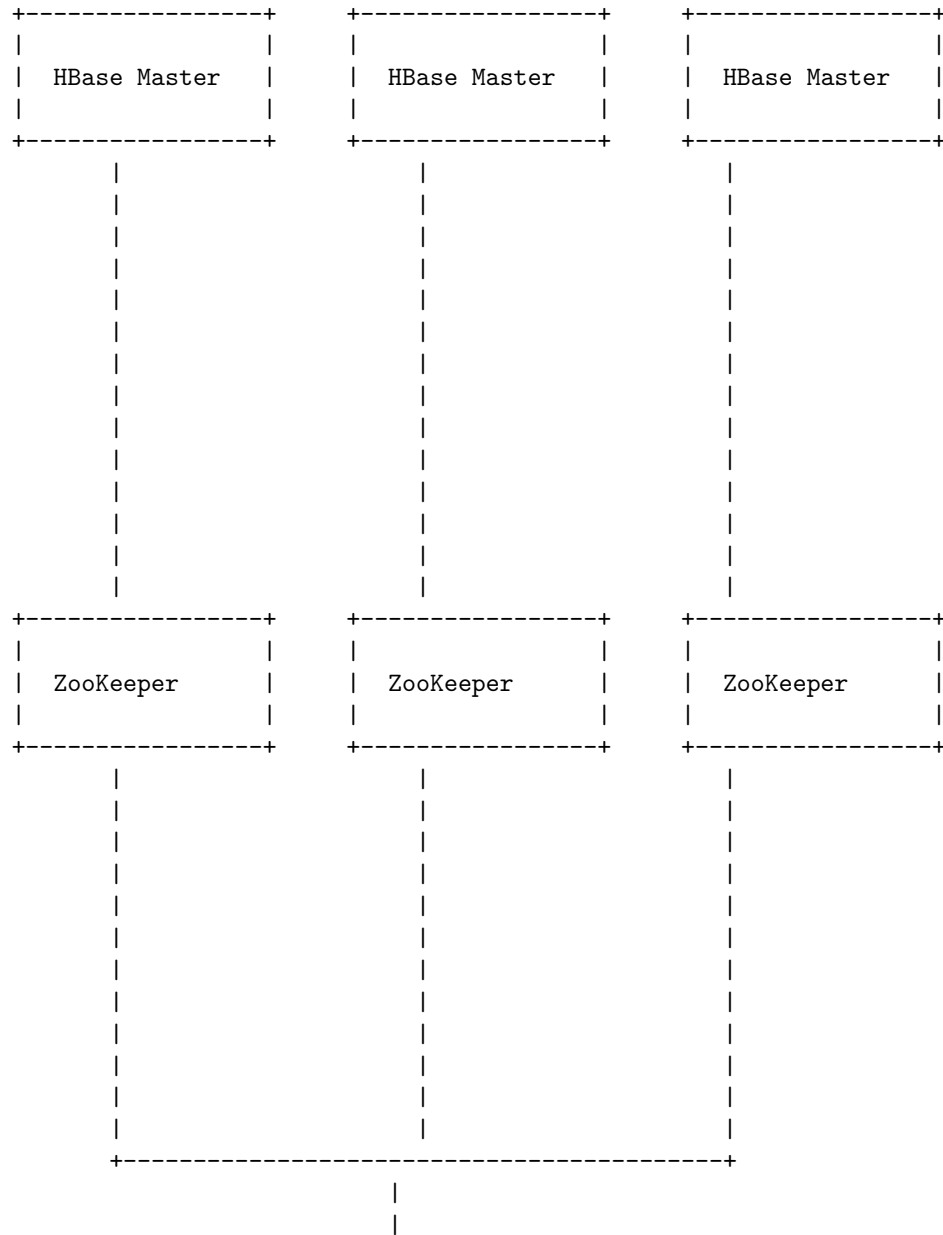
Advanced usage of HBase

HBase can be used for various advanced use cases, such as:

- Storing and querying genome sequences and disease history in the medical field
- Storing and analyzing customer search history and target advertisement in the e-commerce field

- Storing and predicting match outcomes and statistics in the sports field
- Storing and processing large-scale graph data, such as social networks, web graphs, and recommendation systems
- Storing and aggregating time series data, such as sensor data, stock prices, and web logs

The following diagram illustrates the basic architecture of a HBase cluster:



|

|

|



Schema design in HBase is very different from relational database schema design. HBase does not support joins, but it provides single-indexing on the row key. HBase also supports denormalization with nested entities, which are columns whose names are unique identifiers for the nested entity and whose values are the entire record mashed together. HBase allows dynamic column definition, so there is no problem with adding new attributes.

The following ASCII diagram illustrates the basic architecture of a schema design in HBase:

Row Key	Column Family 1	Column Family 2	Column Family 3
row1	cf1:col1=val1	cf2:col1=val2	cf3:col1=val3
	cf1:col2=val4	cf2:col2=val5	cf3:col2=val6
	cf1:col3=val7	cf2:col3=val8	cf3:col3=val9
row2	cf1:col1=val10	cf2:col1=val11	cf3:col1=val12
	cf1:col2=val13	cf2:col2=val14	cf3:col2=val15
	cf1:col3=val16	cf2:col3=val17	cf3:col3=val18

row3	cf1:col1=val19	cf2:col1=val20	cf3:col1=val21	
	cf1:col2=val22	cf2:col2=val23	cf3:col2=val24	
	cf1:col3=val25	cf2:col3=val26	cf3:col3=val27	
+-----+-----+-----+-----+				

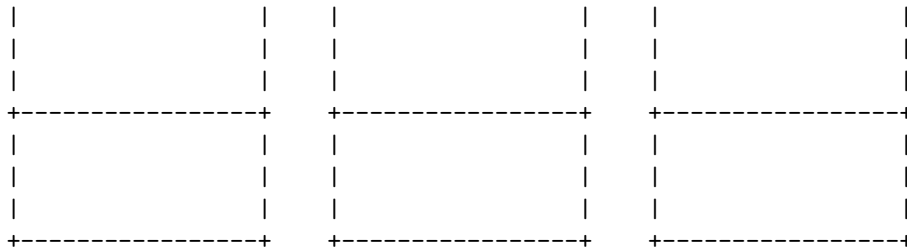
In this diagram, each row has a row key and three column families (cf1, cf2, cf3). Each column family has three columns (col1, col2, col3) and each column has a value. The values are stored as byte arrays and can be any type of data. The column names are prefixed with the column family name and a colon. The column families are defined at the table creation time, but the columns can be added dynamically. The row key is the only index for the table and the data is sorted lexicographically by the row key. HBase also supports versioning and timestamps for each cell, but they are not shown in this diagram.

Advanced indexing in HBase is a technique to create and maintain secondary indexes on HBase tables, which can improve the performance of queries that do not use the primary row key. There are different approaches to implement advanced indexing in HBase, such as using coprocessors, Phoenix, Lily HBase Indexer, or manual indexing.

The following diagram illustrates the basic architecture of a coprocessor-based indexing solution, which uses a custom observer class to intercept the put and delete operations on the main table and update the index table accordingly .

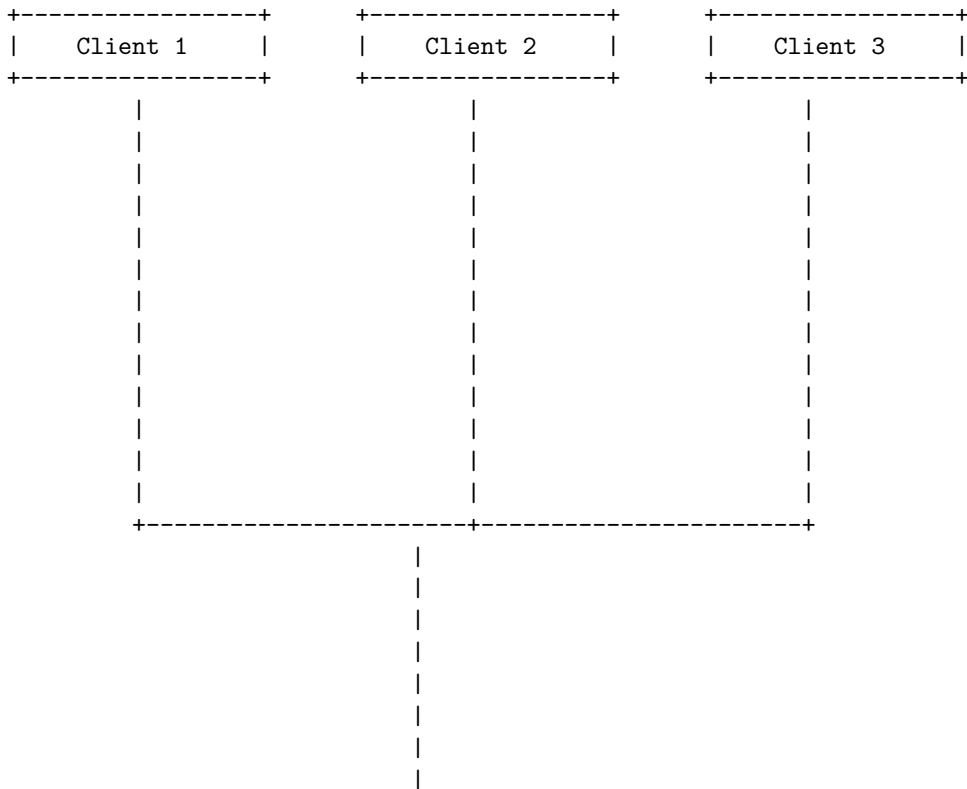
Advanced indexing in HBase

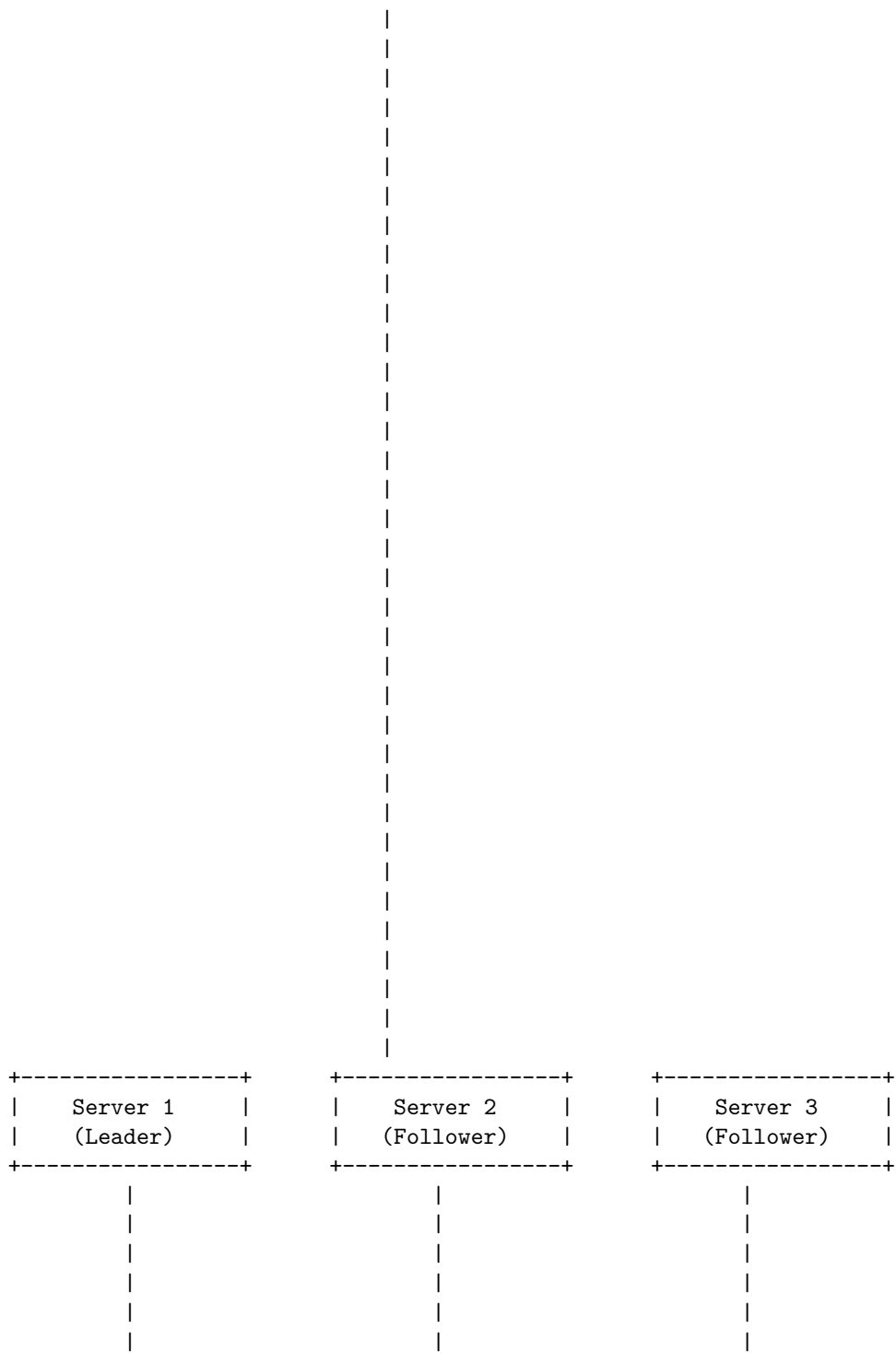
+-----+		+-----+		+-----+	
Main Table		Index Table		Observer Class	
+-----+		+-----+		+-----+	
Row Key		Row Key		prePut()	
Column Family		Column Family		preDelete()	
Column Qualifier		Column Qualifier			
Value		Value			
+-----+		+-----+		+-----+	
Put/Delete		Put/Delete		postPut()	
Operation		Operation		postDelete()	
+-----+		+-----+		+-----+	
+-----+		+-----+		+-----+	



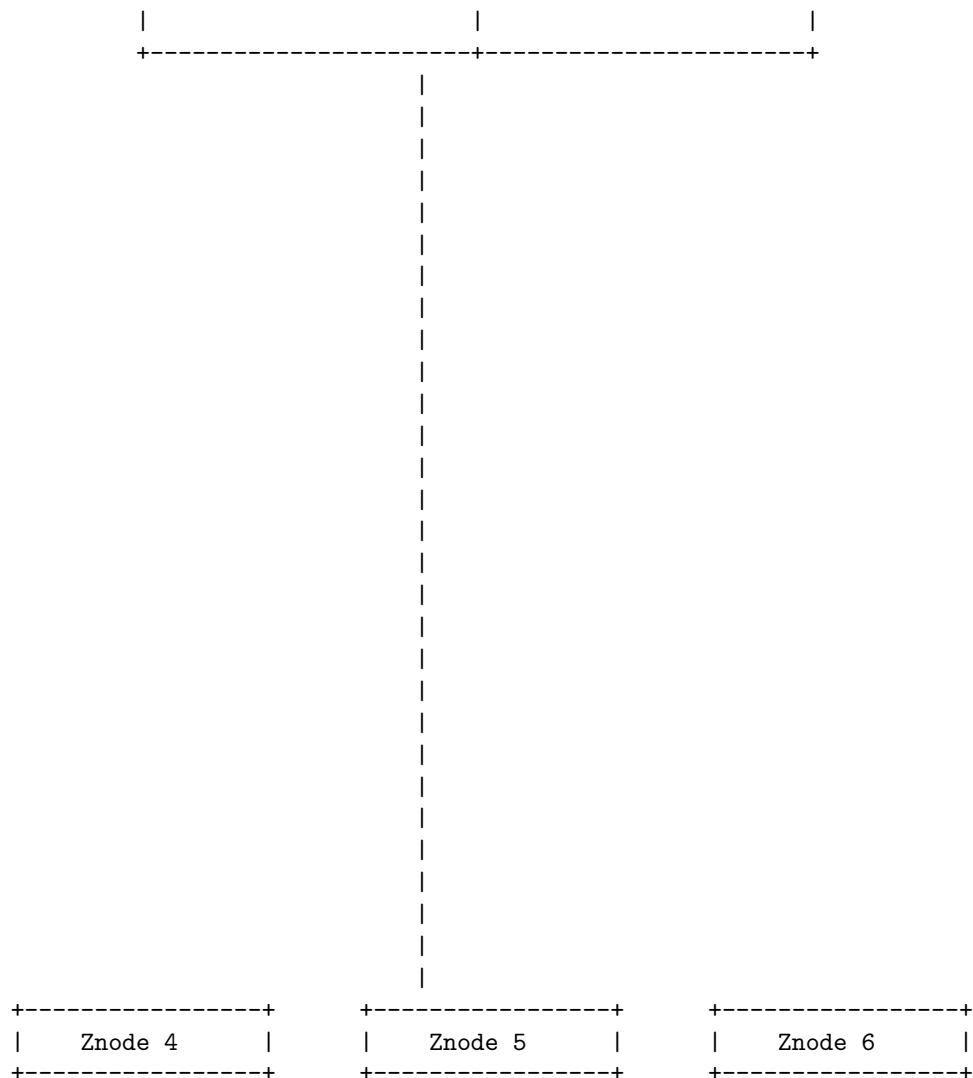
Zookeeper is a distributed coordination service for distributed systems. It has a simple client-server model in which clients are nodes (i.e. machines) and servers are nodes. Zookeeper maintains a hierarchical namespace of znodes, which are data nodes that can store data and have permissions. Clients can read and write data from znodes, and also set watches on them to get notified of changes. Servers form a quorum, which is a majority of servers that agree on the state of the system. One of the servers acts as a leader, which handles write requests and coordinates the followers. The followers handle read requests and sync with the leader.

Zookeeper







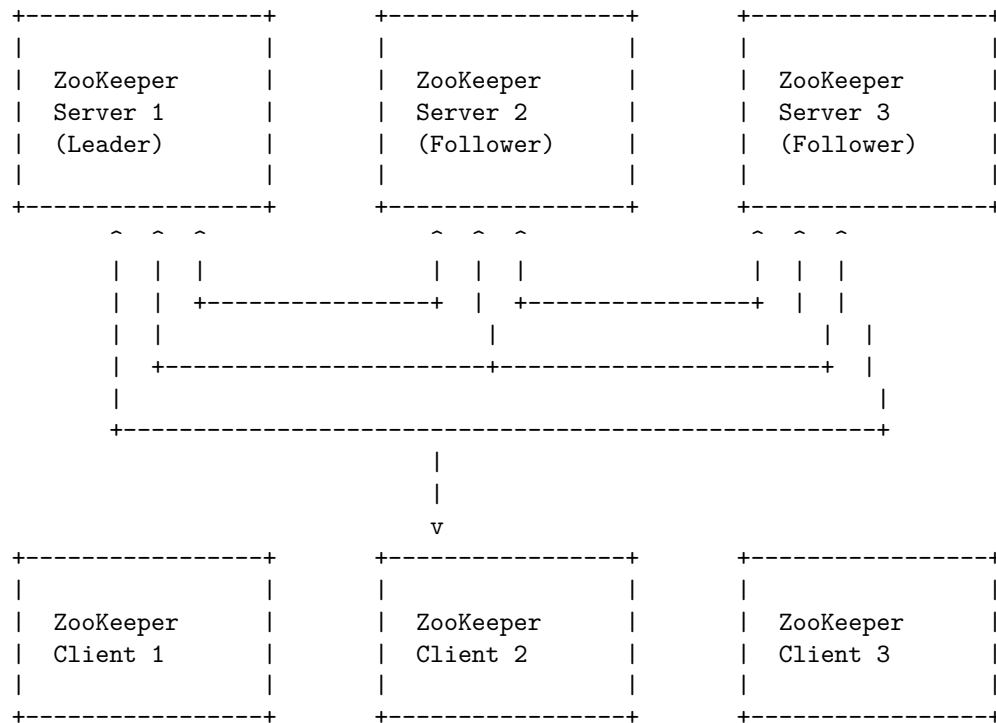


ZooKeeper is a distributed coordination service for distributed systems. It provides common services such as naming, configuration management, synchronization, and group services in a simple and reliable way. ZooKeeper has a client-server architecture, where clients are the nodes that use the services and servers are the nodes that provide the services. ZooKeeper servers form a cluster called an ensemble, which elects a leader to handle write requests and synchronizes the state of the data across the servers. ZooKeeper clients connect to one of the servers in the ensemble and send requests to read or write data. ZooKeeper data is organized in a hierarchical namespace, similar to a file system, where each node is called a znode and can store some data and have children znodes. ZooKeeper guarantees that the data is consistent, ordered, and durable across

the ensemble.

ZooKeeper concepts

The following diagram illustrates the basic architecture of a ZooKeeper system using ASCII art:



The diagram shows an ensemble of three ZooKeeper servers, one of which is the leader and the other two are followers. The leader is responsible for processing write requests and synchronizing the state of the data with the followers. The followers can process read requests and forward write requests to the leader. The diagram also shows three ZooKeeper clients, each connected to one of the servers. The clients can send requests to read or write data to the ZooKeeper namespace, which is a tree-like structure of znodes. Each znode can store some data and have children znodes. For example, the following diagram shows a possible ZooKeeper namespace:

```
/
|-- config
|   |-- db
|   `-- web
|-- locks
|   |-- lock1
```



```

|   |-- lock2
|-- workers
    |-- worker1
    |-- worker2

```

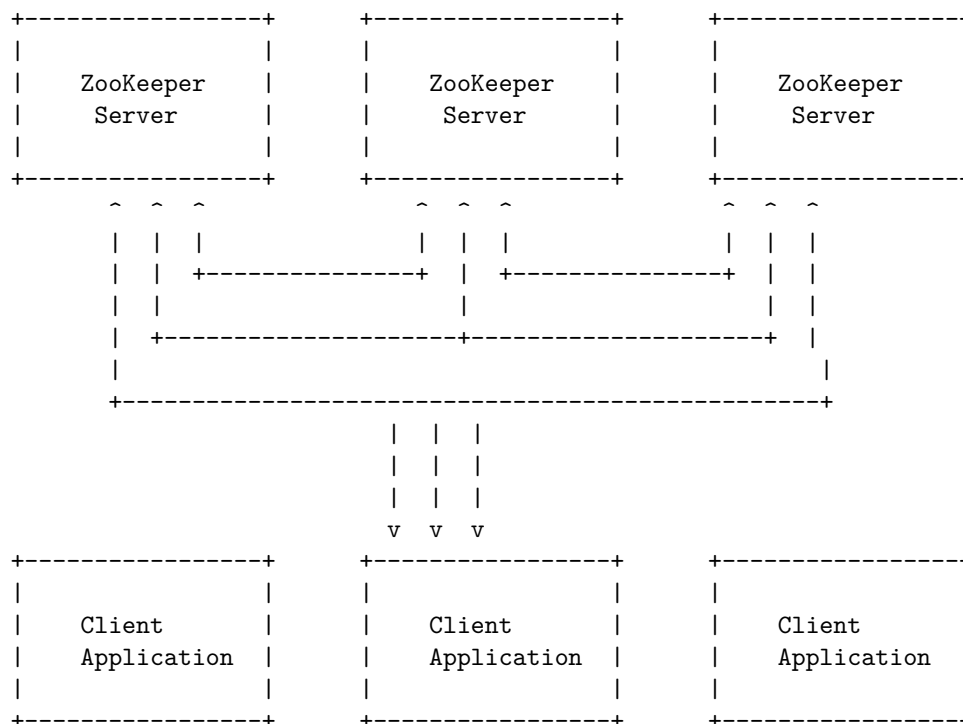
The diagram shows that the root znode (/) has three children: config, locks, and workers. The config znode has two children: db and web, which can store some configuration data for a database and a web server, respectively. The locks znode has two children: lock1 and lock2, which can be used for implementing distributed locks by the clients. The workers znode has two children: worker1 and worker2, which can store some information about the workers in a distributed system. ZooKeeper ensures that the data in the namespace is consistent, ordered, and durable across the ensemble. ZooKeeper also provides some features such as watches, ephemeral nodes, and sequential nodes to facilitate the implementation of common coordination patterns.

ZooKeeper is a distributed coordination service that helps to manage configuration information, naming, group services, and synchronization for distributed applications. It implements different protocols on the cluster so that the applications do not have to implement them on their own. It provides a single coherent view of multiple machines.

ZooKeeper helps in monitoring a cluster by providing the following features:

- **Leader election:** ZooKeeper can elect a leader among a group of nodes that need to coordinate with each other. The leader can perform tasks that require global coordination, such as assigning work, managing configuration, or monitoring health. ZooKeeper ensures that there is always one and only one leader at any given time, and that the leader can be replaced quickly if it fails or leaves the cluster.
- **Configuration management:** ZooKeeper can store and distribute configuration data across the cluster. The configuration data is stored as znodes, which are hierarchical data structures that resemble a file system. ZooKeeper ensures that the configuration data is consistent and up-to-date on all nodes, and that any changes are propagated atomically and reliably.
- **Group membership:** ZooKeeper can keep track of the nodes that belong to a certain group or service. The group membership is also stored as znodes, and ZooKeeper notifies the nodes of any changes in the group, such as nodes joining or leaving. ZooKeeper can also assign unique identifiers to the nodes, which can be used for coordination or load balancing.
- **Locking and synchronization:** ZooKeeper can provide distributed locking and synchronization primitives, such as mutexes, barriers, queues, and counters. These primitives can be used to implement coordination and concurrency control among the nodes. ZooKeeper guarantees that the locking and synchronization operations are atomic, consistent, and fault-tolerant.

The following diagram illustrates the basic architecture of a ZooKeeper cluster:



In this diagram, there are three ZooKeeper servers and three client applications. The ZooKeeper servers form a quorum, which is a majority of servers that can agree on the state of the cluster. The quorum elects one of the servers as the leader, and the other servers are followers. The leader is responsible for processing all write requests from the clients, and replicating them to the followers. The followers process read requests from the clients, and forward write requests to the leader. The clients connect to any of the ZooKeeper servers, and use the ZooKeeper API to perform operations on the znodes. The clients can also watch the znodes for changes, and receive notifications from the ZooKeeper servers. The ZooKeeper servers use a consensus protocol, such as Zab, to ensure that the state of the znodes is consistent and durable across the cluster. The ZooKeeper servers also use heartbeats and timeouts to detect and recover from failures.

ZooKeeper is a distributed system coordinator that provides services such as configuration management, synchronization, naming, and leader election. To build applications with ZooKeeper, you need to install and run a ZooKeeper server, and use a ZooKeeper client to interact with the server.

The following steps describe how to set up a ZooKeeper server in standalone mode, which is suitable for development and testing purposes. For production environments, you need to set up a ZooKeeper ensemble, which is a group of servers that work together to provide high availability and fault tolerance.

1. Download a stable ZooKeeper release from the official website and unpack it to a directory of your choice.
2. Create a configuration file named `zoo.cfg` in the `conf` subdirectory of the ZooKeeper installation directory. The configuration file should contain at least the following parameters:

```
tickTime=2000
dataDir=/var/lib/zookeeper
clientPort=2181
```

The `tickTime` is the basic time unit in milliseconds used by ZooKeeper. The `dataDir` is the directory where ZooKeeper will store its data. The `clientPort` is the port that ZooKeeper will listen for client connections.

3. Start the ZooKeeper server by running the following command from the ZooKeeper installation directory:

```
bin/zkServer.sh start
```

You should see a message like `ZooKeeper JMX enabled by default` and `Using config: ../zoo.cfg`.

4. To verify that the ZooKeeper server is running, you can use the `telnet` command to connect to the `clientPort` and issue the `stat` command. You should see some information about the server status, such as the mode, the number of connections, and the node count.

```
$ telnet localhost 2181
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
stat
ZooKeeper version: 3.7.0-.../3.7.0
Clients:
 /127.0.0.1:...[0] (queued=0,recved=1,sent=0)
```

```
Latency min/avg/max: 0/0/0
Received: 1
Sent: 0
Connections: 1
Outstanding: 0
Zxid: 0x0
Mode: standalone
Node count: 4
Connection closed by foreign host.
```

5. To stop the ZooKeeper server, you can use the following command from the ZooKeeper installation directory:

```
bin/zkServer.sh stop
```

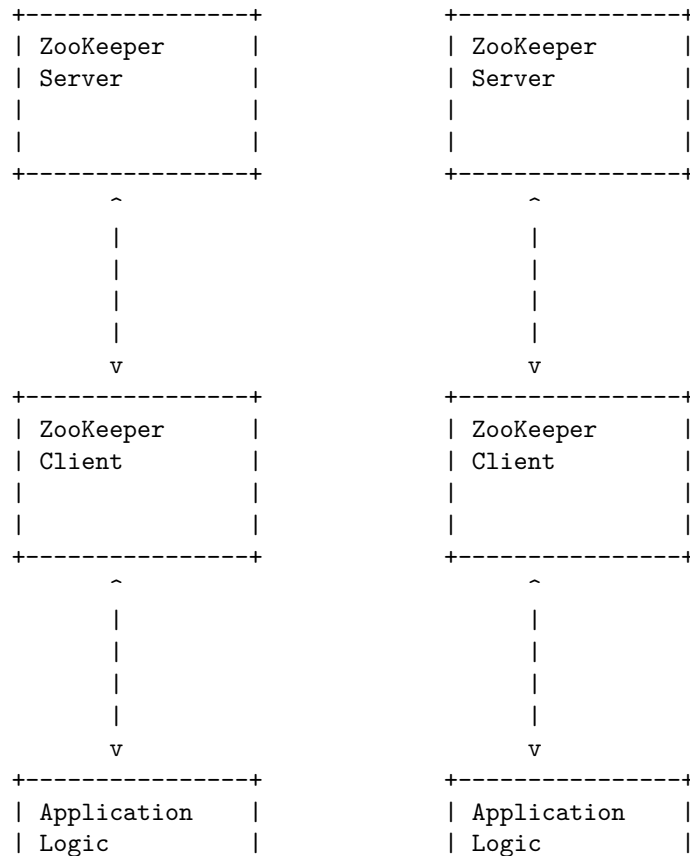
You should see a message like `Stopping zookeeper ... STOPPED`.

How to build applications with ZooKeeper

To build applications with ZooKeeper, you need to use a ZooKeeper client to connect to the ZooKeeper server and perform operations on the data stored in the ZooKeeper tree. The ZooKeeper tree is a hierarchical namespace of nodes, called znodes, that can store data and have children. Each znode has a path that identifies its location in the tree, such as `/app/config` or `/app/locks/lock1`.

The ZooKeeper client provides methods to create, read, update, and delete znodes, as well as to set and get watches on znodes. Watches are callbacks that are triggered when a znode changes, such as when its data or children are modified or deleted. Watches allow applications to react to changes in the ZooKeeper tree and implement features such as configuration management, synchronization, naming, and leader election.

The following diagram illustrates the basic architecture of a ZooKeeper-based application:

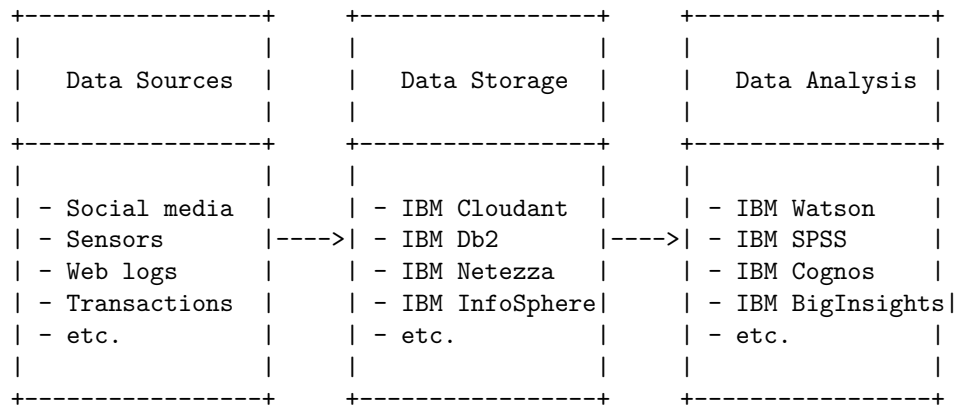


| |

According to the search results, IBM Big Data strategy is a corporate initiative that offers solutions to store, manage, and analyze the huge amounts of data generated daily and equip large and small companies to make informed business decisions. IBM Big Data strategy is based on four dimensions: volume, variety, velocity, and veracity of data. IBM Big Data strategy also aims to create a Smarter Planet, where data is used to achieve economic and sustainable growth and societal progress.

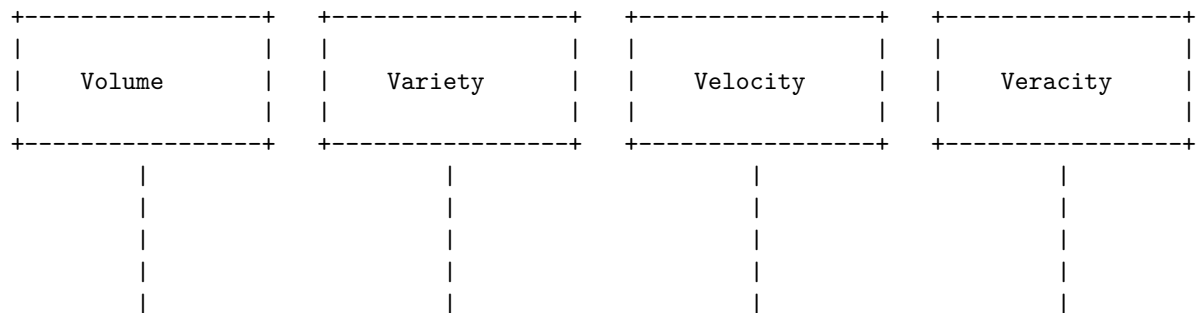
IBM Big Data strategy

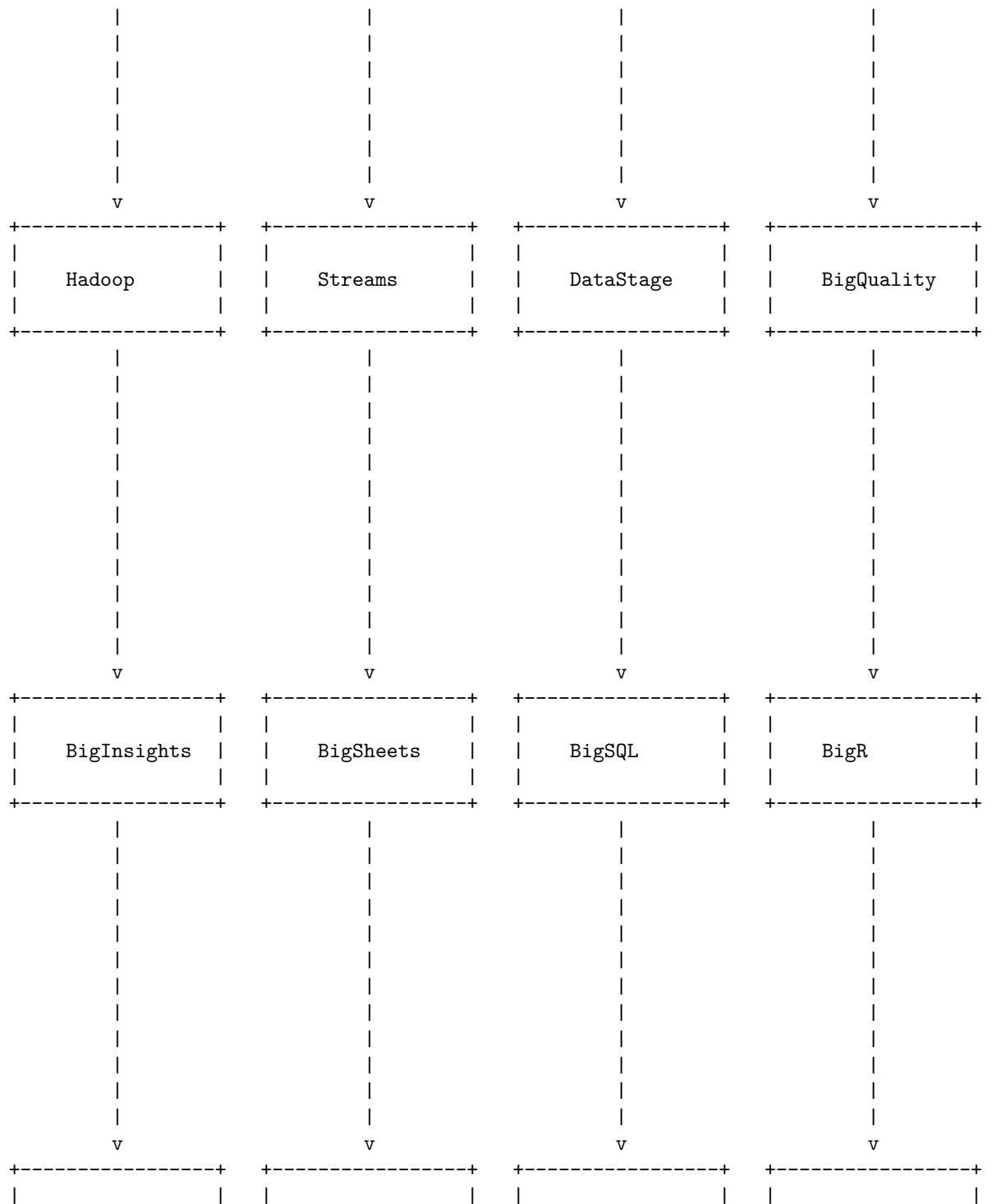
The following is an ASCII diagram of the basic architecture of IBM Big Data strategy, based on the information from the search results:

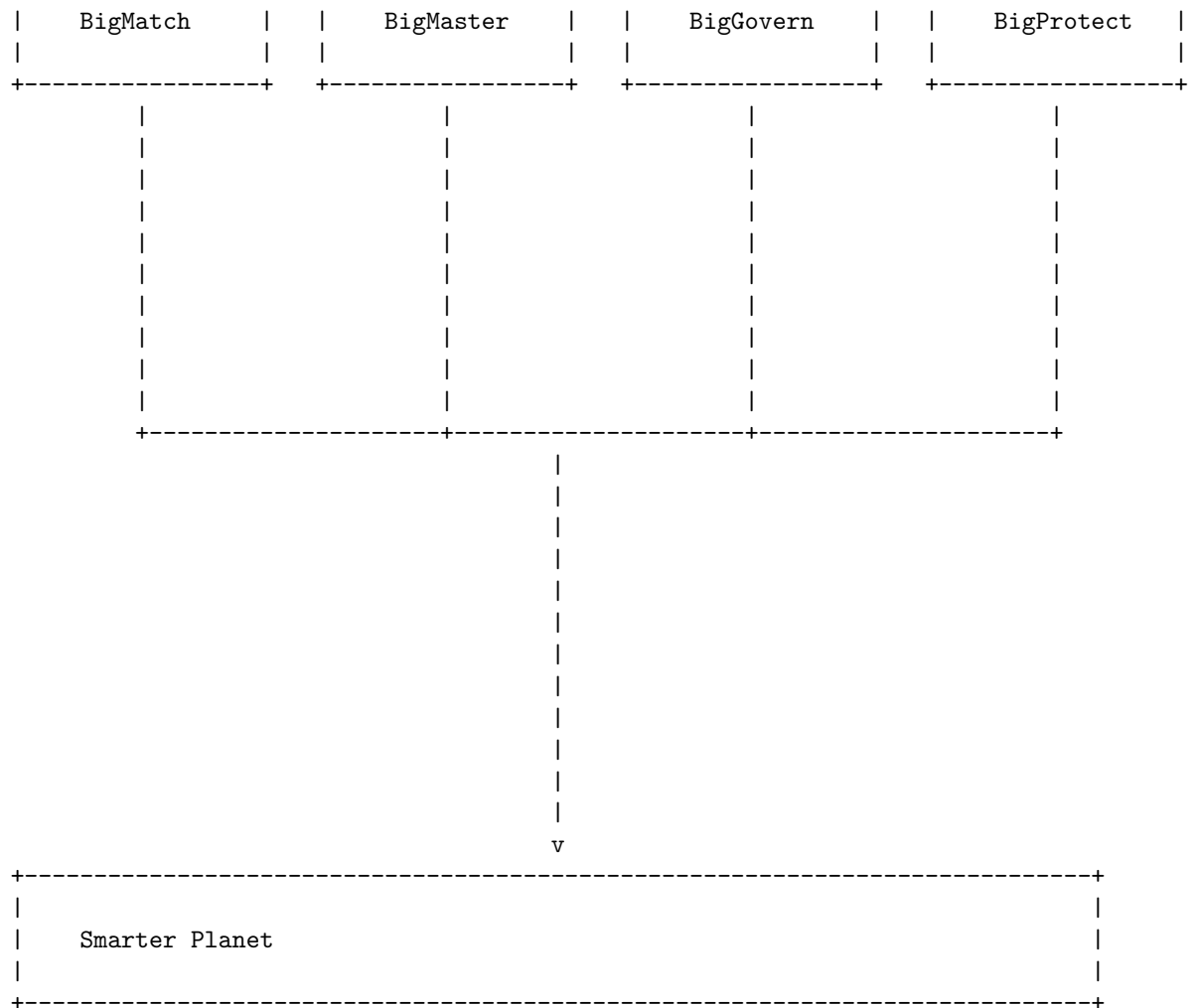


IBM Big Data strategy is a set of solutions and services that help organizations store, manage, and analyze the huge amounts of data generated daily and make informed business decisions. IBM Big Data strategy is based on four dimensions: volume, variety, velocity, and veracity. IBM Big Data strategy aims to create a Smarter Planet, where data is used to improve efficiency, innovation, and sustainability.

The following is a detailed ASCII diagram for IBM Big Data strategy:



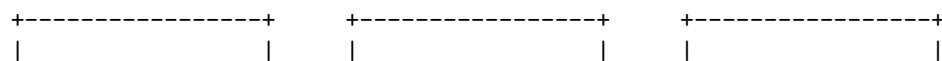




Introduction to Infosphere

Infosphere is a term that refers to the metaphysical realm of information, data, knowledge, and communication, populated by informational entities called inforgs. It is analogous to a biosphere, which is the realm of living organisms. The concept of infosphere was coined by philosopher Luciano Floridi, who argues that the infosphere is the new environment in which humans and other intelligent agents interact.

The following diagram illustrates the basic architecture of an infosphere using ASCII characters:



Inforgs	Inforgs	Inforgs
Data	Data	Data
Metadata	Metadata	Metadata
Services	Services	Services
Platform	Platform	Platform
Hardware	Hardware	Hardware

Each infosphere consists of several layers:

- Hardware: the physical devices and infrastructure that support the infosphere, such as computers, networks, sensors, etc.
- Platform: the software and operating systems that run on the hardware and enable the infosphere to function, such as Windows, Linux, iOS, etc.
- Services: the applications and processes that provide functionality and value to the infosphere, such as web browsers, email, social media, etc.
- Metadata: the information that describes and organizes the data in the infosphere, such as schemas, ontologies, tags, etc.
- Data: the raw and processed information that is stored and exchanged in the infosphere, such as text, images, audio, video, etc.
- Inforgs: the informational entities that inhabit and interact with the infosphere, such as humans, artificial agents, organizations, etc.

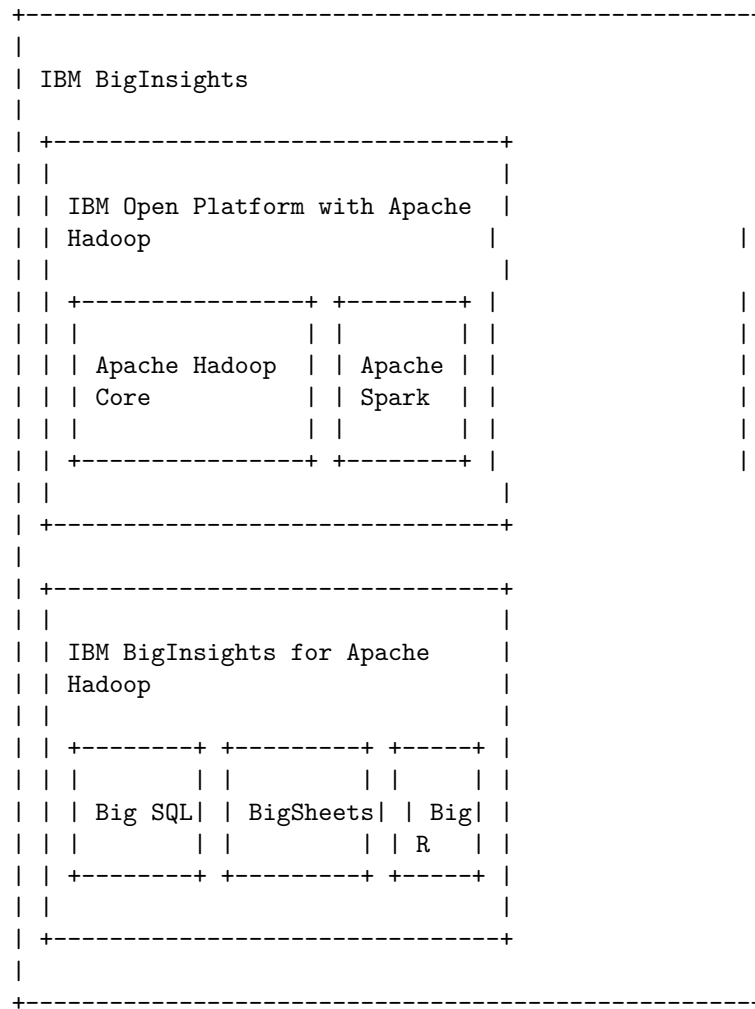
The infosphere is a dynamic and evolving environment that can be influenced by the actions and decisions of the inforgs. The infosphere also affects the inforgs by shaping their identities, values, and behaviors. Therefore, the infosphere poses ethical, social, and political challenges and opportunities for the inforgs and their societies.

BigInsights is an IBM product that helps firms analyze the increasing volume, velocity and veracity of data of their interest. BigInsights does not replace a relational database management system (DBMS) or a traditional data warehouse,

but rather complements them by providing a scalable and flexible platform for processing and analyzing large and complex data sets using Apache Hadoop and its ecosystem of tools.

Introduction to BigInsights

The following diagram illustrates the basic architecture of BigInsights using ASCII art:



BigInsights consists of two main modules:

- **IBM Open Platform with Apache Hadoop:** This module provides the core components of Apache Hadoop, such as HDFS, MapReduce, YARN, ZooKeeper, etc. It also includes Apache Spark, a fast and general engine

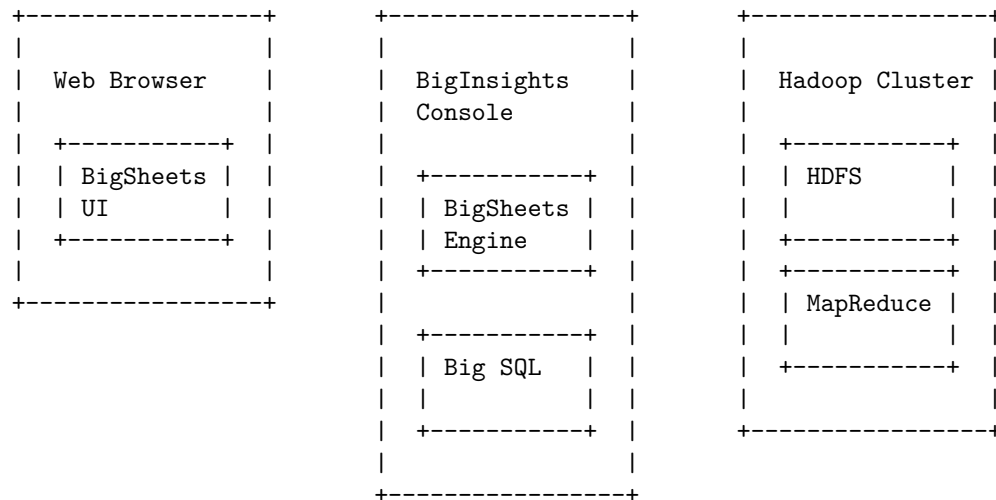
for large-scale data processing that supports SQL, streaming, machine learning, and graph analytics.

- IBM BigInsights for Apache Hadoop: This module provides additional value-added components that enhance the capabilities of Apache Hadoop, such as Big SQL, a SQL engine that allows users to query data stored in Hadoop using standard SQL syntax and JDBC/ODBC drivers; BigSheets, a spreadsheet-like interface that allows users to explore, visualize, and analyze data in Hadoop using a web browser; Big R, a framework that allows users to run R scripts on Hadoop data using a familiar R syntax and environment; and other services such as Text Analytics, Spectrum Scale (GPFS), and Platform Symphony.

Big Sheets is a spreadsheet-style tool for business analysts provided with IBM InfoSphere BigInsights, a platform based on the open source Apache Hadoop project. Big Sheets enables non-programmers to iteratively explore, manipulate, and visualize data stored in your distributed file system.

Introduction to Big Sheets

The following diagram illustrates the basic architecture of Big Sheets:



The diagram shows the following components:

- Web Browser: The user interface for Big Sheets. It allows the user to create, edit, and view worksheets that contain data from the Hadoop cluster.
- BigInsights Console: The web-based management console for IBM InfoSphere BigInsights. It provides access to various tools and services, including Big Sheets.
- Big Sheets Engine: The core component of Big Sheets. It handles the communication between the web browser and the Hadoop cluster. It also

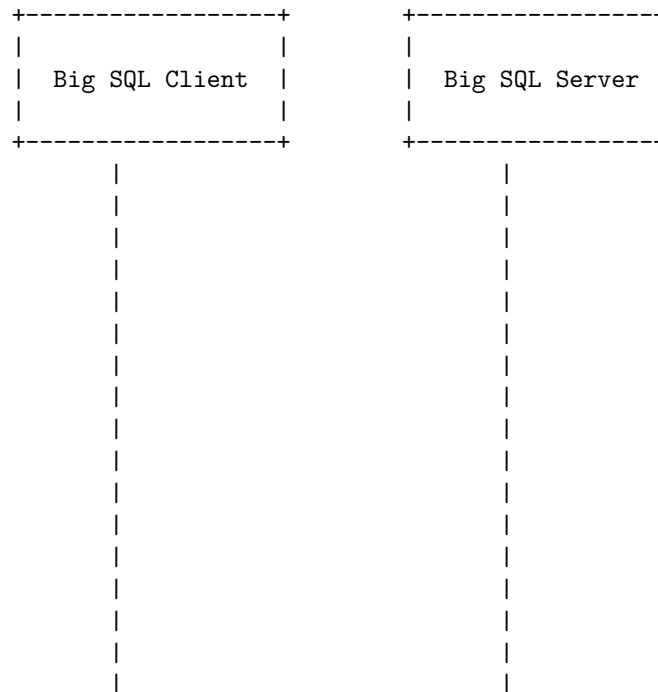
performs data processing and transformation using Big SQL and MapReduce.

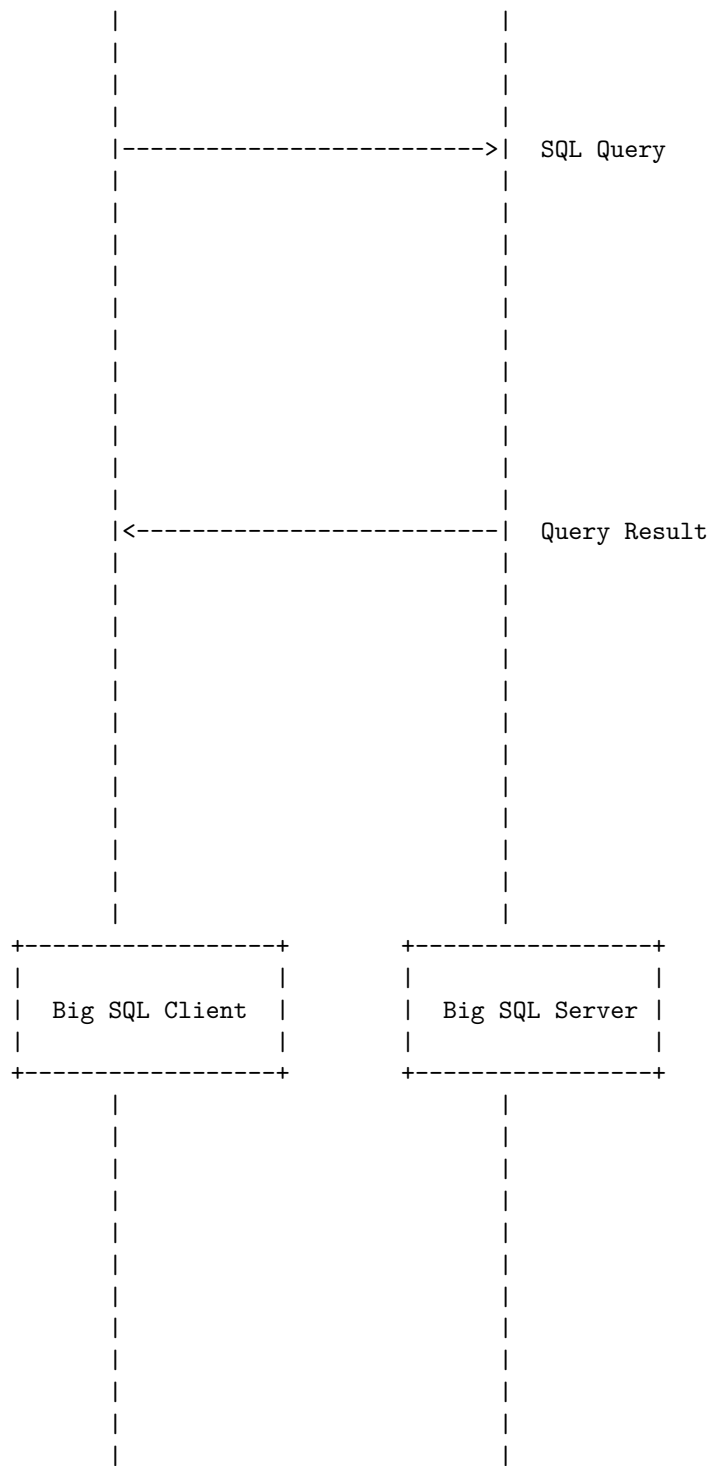
- **Big SQL:** A SQL engine that runs on top of Hadoop. It allows Big Sheets to query and manipulate data using SQL syntax and functions.
- **Hadoop Cluster:** The distributed system that stores and processes large amounts of data. It consists of two main components: HDFS and MapReduce.
- **HDFS:** The Hadoop Distributed File System. It is a scalable and fault-tolerant file system that stores data across multiple nodes in the cluster.
- **MapReduce:** The programming model and framework for parallel processing of data in Hadoop. It divides the data into smaller chunks and assigns them to different nodes for processing.

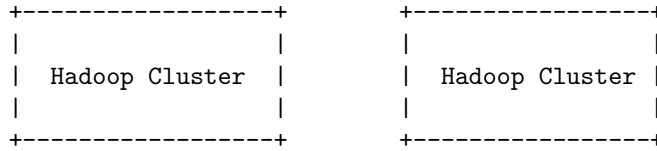
Introduction to Big SQL

Big SQL is a massively parallel processing (MPP) database engine that is built on the IBM common SQL database technology and is optimized to work with the Apache Hadoop ecosystem. Big SQL allows you to query and analyze data stored in Hadoop Distributed File System (HDFS) using the standard SQL syntax and the familiar relational database features. Big SQL also supports accessing data from other sources, such as Apache Hive, Apache HBase, Apache Kafka, and relational databases, through a federated query mechanism.

The following diagram illustrates the basic architecture of Big SQL:







The Big SQL client is a software component that allows you to connect to the Big SQL server and submit SQL queries. The Big SQL client can be a command-line interface (CLI), a graphical user interface (GUI), a JDBC or ODBC application, or a REST API.

The Big SQL server is a software component that runs on one or more nodes of the Hadoop cluster and executes the SQL queries submitted by the Big SQL client. The Big SQL server consists of a head node and one or more worker nodes. The head node is responsible for parsing, optimizing, and coordinating the query execution. The worker nodes are responsible for accessing, processing, and returning the data from the Hadoop cluster.

The Hadoop cluster is a distributed system that stores and processes large volumes of data using the Hadoop components, such as HDFS, Hive, HBase, Kafka, and Spark. The Big SQL server interacts with the Hadoop cluster through the Hadoop connectors, which are software components that enable the Big SQL server to read and write data from and to the Hadoop components. The Big SQL server also leverages the Hadoop resource management system, such as YARN or Kubernetes, to allocate and manage the resources for the query execution.